

ระบบออกแบบทริปการท่องเที่ยวด้วย AI

(AI-Powered Travel Trip Design System)

ผู้จัดทำ

รหัสนักศึกษา 66122250054 นาย มณฑวรรษ เรืองมณียา

อาจารย์ที่ปรึกษาโครงงาน

ผศ.ดร เสถียร จันทร์ปลา

รายงานนี้เป็นส่วนหนึ่งของวิชา CSD3104 โครงงานวิทยาการคอมพิวเตอร์และนวัตกรรมข้อมูล

หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์และนวัตกรรมข้อมูล

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏสวนสุนันทา

ภาคเรียนที่ 2 ปีการศึกษา 2568

สารบัญ

Contents

สารบัญ ..... ก
บทนำ 1
1.1 ที่มาและความสำคัญของโครงงาน 1
1.2 ปัญหาและความท้าทายที่พบ 2
1.3 วัตถุประสงค์ 2
1.4 ขอบเขตของการศึกษา 2
1.5 สมมติฐานหรือคำถามวิจัย 4
1.6ประโยชน์ที่คาดว่าจะได้รับ 4
เอกสารและงานวิจัยที่เกี่ยวข้อง 2
2.1แนวคิดและทฤษฎีที่เกี่ยวข้องกับโมบายแอปพลิเคชัน 2
2.1.1 Web and Mobile Application 2
2.1.2 Client-Server Architecture 26
2.1.3 API (Application Programming Interface) 45
2.1.4 Cross-Platform Application Development 61
2.1.5 Map Integration and Geolocation 82
2.1.6 NoSQL and Cloud Database 97
2.2 เทคโนโลยีและเครื่องมือที่เกี่ยวข้อง 114
2.2.1 Flutter and Dart 114
2.2.2 OpenStreetMap and Geocoding 129
2.2.3 Google Gemini API 143
2.2.4 Firebase Platform 156
2.2.5 Unsplash API 168
2.3 งานวิจัยและผลงานที่เกี่ยวข้องทั้งในและต่างประเทศ 180
2.3.1 การประยุกต์และสังเคราะห์งานวิจัยที่เกี่ยวข้องต่อการพัฒนาระบบออกแบบทริปการท่องเที่ยวด้วย AI 183
2.4 การสังเคราะห์งานที่เกี่ยวข้องและช่องว่างของโครงงาน 185
วิธีการดำเนินโครงงาน 166
3.1 การวิเคราะห์ความต้องการของระบบ 166
3.2 สถาปัตยกรรมของระบบ 189
3.3 เครื่องมือและเทคโนโลยีที่ใช้ 190

3.3.1 Flutter and Dart 191
3.3.2 OpenStreetMap and Geocoding 191
3.3.3 Google Gemini API 191
3.3.4 Firebase Platform 192
3.3.5 Unsplash API 192
3.4 การออกแบบระบบ 192
3.4.1 การออกแบบ Use Case 193
3.4.2 การออกแบบแผนภาพกิจกรรม (Activity Diagram) 194
3.4.3 การออกแบบโครงสร้างข้อมูล/ฐานข้อมูล 198
3.4.4 การออกแบบส่วนติดต่อผู้ใช้ (UI/UX) 201
3.4.5 การวางแผนและออกแบบส่วนเชื่อมต่อโปรแกรมประยุกต์ (API Planning and Design) 215
3.4.6 การวางแผนและออกแบบโครงสร้างส่วนหน้าด้วย Flutter Web (Frontend Application Planning) 216
3.4.7 การวางแผนและออกแบบส่วนติดต่อผู้ใช้งานสำหรับผู้ดูแลระบบ (Admin Web Dashboard Planning) 219
3.5 การหาประสิทธิภาพของระบบออกแบบทริปการท่องเที่ยวด้วย AI 221
3.5.1 รูปแบบและวิธีการทดสอบระบบ (Testing Methodology) 222
3.5.2 เครื่องมือที่ใช้ในการประเมินประสิทธิภาพ (Evaluation Tools) 223
3.5.3 ตัวชี้วัดประสิทธิภาพ (Key Performance Indicators: KPIs) 223
3.5.4 การวิเคราะห์ข้อมูลและสถิติ (Data Analysis and Statistics) 224
ผลการพัฒนาและการทดสอบระบบ 189
4.1 ผลการพัฒนาโมบายแอปพลิเคชัน 189
หน้าแอพพลิเคชั่น 227
หน้า Web Admin 244
4.2 ฟังก์ชันการทำงานหลักของระบบ 254
4.2.1 ฟังก์ชันการสมัครสมาชิกและเข้าสู่ระบบ 254
4.2.2 ฟังก์ชันการสร้างแผนการเดินทางด้วย AI 254
4.2.3 ฟังก์ชันการแสดงรายละเอียดแผนการเดินทาง 254
4.2.4 ฟังก์ชันการแสดงตำแหน่งสถานที่บนแผนที่ 255
4.2.5 ฟังก์ชันการแสดงรูปภาพสถานที่ 255
4.2.6 ฟังก์ชันการบันทึกและเรียกดูแผนการเดินทาง 255
4.2.7 ฟังก์ชันการลบแผนการเดินทาง 255
4.2.8 ฟังก์ชันสำหรับผู้ดูแลระบบ 256

4.3 ผลการทดสอบการทำงานของระบบ 256
4.4 การประเมินประสิทธิภาพและความถูกต้อง 259
4.4.1 ผลการประเมินประสิทธิภาพเชิงเวลา (Time Efficiency) 259
4.4.2 ผลการประเมินคุณภาพของผลลัพธ์ (Output Quality) 261
4.4.3 ผลการตรวจสอบความถูกต้องของพิกัดสถานที่ 262
สรุปผล อภิปรายผล และข้อเสนอแนะ 227
5.1 สรุปผลการดำเนินโครงงาน 227
5.2 อภิปรายผลการพัฒนา 265
5.3 ข้อจำกัดของโครงงาน 271
5.4 ข้อเสนอแนะและแนวทางการพัฒนาต่อยอด 272
บรรณานุกรม 275

บทที่ 1
บทนำ

1.1 ที่มาและความสำคัญของโครงงาน

การท่องเที่ยวเป็นอุตสาหกรรมสำคัญที่มีบทบาทต่อการพัฒนาเศรษฐกิจของประเทศ เนื่องจากประเทศไทยมีทรัพยากรการท่องเที่ยวที่หลากหลายทั้งด้านธรรมชาติ ประวัติศาสตร์ และวัฒนธรรม ส่งผลให้การท่องเที่ยวกลายเป็นกลไกสำคัญในการสร้างรายได้และส่งเสริมการพัฒนาเศรษฐกิจของประเทศ นอกจากนี้การพัฒนาเทคโนโลยีสารสนเทศและระบบดิจิทัลได้เข้ามามีบทบาทในการสนับสนุนการท่องเที่ยวมากยิ่งขึ้น โดยเฉพาะระบบแนะนำข้อมูล (Recommendation System) ที่สามารถช่วยวิเคราะห์ความสนใจของผู้ใช้งานและแนะนำสถานที่ท่องเที่ยวที่เหมาะสมกับแต่ละบุคคล ทำให้นักท่องเที่ยวสามารถตัดสินใจและวางแผนการเดินทางได้อย่างมีประสิทธิภาพมากขึ้น อย่างไรก็ตาม การค้นหาข้อมูลสถานที่ท่องเที่ยวจากเว็บไซต์หรือแหล่งข้อมูลออนไลน์ต่าง ๆ ยังคงต้องใช้เวลาและความพยายามในการรวบรวมข้อมูลจำนวนมาก ดังนั้นการพัฒนาระบบหรือแอปพลิเคชันที่สามารถรวบรวม วิเคราะห์ และแนะนำสถานที่ท่องเที่ยวที่เหมาะสมกับผู้ใช้งานจึงเป็นสิ่งสำคัญที่จะช่วยอำนวยความสะดวกให้กับนักท่องเที่ยวและเพิ่มประสิทธิภาพในการวางแผนการเดินทาง (Suwanee Kulkarineetham & Weeriya Supanich, 2024)

อย่างไรก็ตาม ระบบหรือเว็บไซต์ด้านการท่องเที่ยวที่มีอยู่ในปัจจุบันยังคงมีข้อจำกัดหลายประการ เช่น การจัดเก็บและนำเสนอข้อมูลสถานที่ท่องเที่ยวที่ไม่เป็นระบบ ข้อมูลบางส่วนไม่มีการจัดหมวดหมู่ที่ชัดเจน หรือไม่ได้รับการปรับปรุงให้เป็นปัจจุบัน ส่งผลให้ผู้ใช้งานต้องใช้เวลาในการค้นหาข้อมูลจำนวนมาก อีกทั้งระบบส่วนใหญ่ยังไม่สามารถวิเคราะห์ความต้องการของผู้ใช้งานแต่ละบุคคลได้อย่างมีประสิทธิภาพ ทำให้การแนะนำสถานที่ท่องเที่ยวไม่ตรงกับความสนใจของนักท่องเที่ยว นอกจากนี้ แม้ว่าปัจจุบันเทคโนโลยีปัญญาประดิษฐ์ (Artificial Intelligence: AI) จะถูกนำมาประยุกต์ใช้ในหลายด้าน รวมถึงอุตสาหกรรมการท่องเที่ยวเพื่อช่วยวิเคราะห์ข้อมูลและแนะนำสถานที่ท่องเที่ยวที่เหมาะสมกับผู้ใช้ แต่ระบบบางส่วนยังขาดการประยุกต์ใช้ AI อย่างเต็มประสิทธิภาพในการวิเคราะห์พฤติกรรมหรือความต้องการของผู้ใช้งาน จึงทำให้การให้คำแนะนำยังไม่สามารถตอบสนองต่อความต้องการของนักท่องเที่ยว (ดวงชัย มณีโชติ, 2564; จันทร์ธิดา ระบือธรรม และกนกวรรณ์ นันทะวงค์, 2561)

ด้วยเหตุนี้ ผู้จัดทำจึงมีแนวคิดในการพัฒนา “ระบบออกแบบทริปการท่องเที่ยวด้วย AI (AI-Powered Travel Trip Design System)” โดยนำเทคโนโลยีปัญญาประดิษฐ์ (Artificial Intelligence: AI) มาประยุกต์ใช้เป็นส่วนสำคัญของระบบ เพื่อช่วยวิเคราะห์ข้อมูลความสนใจของผู้ใช้งาน เช่น ประเภทสถานที่ท่องเที่ยว ระยะเวลาในการเดินทาง และงบประมาณ จากนั้นระบบจะ

ทำการประมวลผลข้อมูลเพื่อออกแบบแผนการท่องเที่ยวหรือทริปการเดินทางที่เหมาะสมกับผู้ใช้งานแต่ละบุคคล โดยแสดงลำดับสถานที่ท่องเที่ยวและเส้นทางการเดินทางผ่านโมบายแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ ระบบดังกล่าวจะช่วยเพิ่มความสะดวกในการวางแผนการเดินทาง ลดเวลาในการค้นหาข้อมูล ได้อย่างมีประสิทธิภาพมากยิ่งขึ้น

1.2 ปัญหาและความท้าทายที่พบ

1. ปัญหาด้านการเข้าถึงข้อมูลแหล่งท่องเที่ยว
2. การขาดระบบแนะนำสถานที่ท่องเที่ยวที่ใช้เทคโนโลยีปัญญาประดิษฐ์ (AI) เพื่อวิเคราะห์ความสนใจของผู้ใช้งาน
3. การประชาสัมพันธ์แหล่งท่องเที่ยวท้องถิ่นยังไม่ทั่วถึง
4. ความท้าทายด้านการจัดการและบูรณาการข้อมูลสถานที่ท่องเที่ยวจากหลายแหล่ง
5. ความท้าทายด้านการออกแบบระบบและความปลอดภัยของข้อมูลผู้ใช้งาน

1.3 วัตถุประสงค์

1. เพื่อพัฒนาระบบ ระบบออกแบบทริปการท่องเที่ยวด้วย AI (AI-Powered Travel Trip Design System)
2. เพื่อหาประสิทธิภาพของระบบออกแบบทริปการท่องเที่ยวด้วย AI (AI-Powered Travel Trip Design System)

1.4 ขอบเขตของการศึกษา

ขอบเขตของโครงงาน ระบบออกแบบทริปการท่องเที่ยวด้วย AI (AI-Powered Travel Trip Design System) แบ่งออกเป็นส่วนของผู้ใช้งาน ระบบปัญญาประดิษฐ์ และเครื่องมือเทคโนโลยีที่ใช้ในการพัฒนา ดังนี้

1.4.1 ขอบเขตด้านผู้ใช้งาน (User Scope)

ระบบแบ่งผู้ใช้งานออกเป็น 2 ระดับ ได้แก่

1) ผู้ใช้งานทั่วไป (User)

ผู้ใช้งานทั่วไปสามารถใช้งานระบบเพื่อออกแบบและวางแผนการท่องเที่ยว โดยมีความสามารถดังนี้

- สมัครสมาชิกและเข้าสู่ระบบ
- เลือกความสนใจในการท่องเที่ยว เช่น
    - ธรรมชาติ

    - เมือง
    - วัฒนธรรม
    - อาหาร
    - กีฬา
    - แคมป์ปิ้ง
- กำหนดข้อมูลการเดินทาง เช่น
    - ระยะเวลาทริป
    - จำนวนผู้เดินทาง
    - งบประมาณ
- ใช้ระบบ AI เพื่อออกแบบแผนการท่องเที่ยวอัตโนมัติ
- แสดงรายการสถานที่ท่องเที่ยวที่ระบบแนะนำ
- ดูแผนการเดินทางรายวัน (Daily Trip Plan)
- ดูตำแหน่งสถานที่ผ่านแผนที่
- บันทึก แก้ไข และจัดการทริปที่ออกแบบไว้

2) ผู้ดูแลระบบ (Administrator)

ผู้ดูแลระบบมีหน้าที่จัดการข้อมูลภายในระบบ โดยสามารถ

- เข้าสู่ระบบเพื่อจัดการข้อมูล
- เพิ่ม แก้ไข และลบข้อมูลสถานที่ท่องเที่ยว
- จัดการข้อมูลกิจกรรมและรายละเอียดสถานที่
- จัดการรูปภาพและเนื้อหาที่ใช้ในการแนะนำสถานที่
- ตรวจสอบข้อมูลที่ใช้ในระบบแนะนำทริป

1.4.2 ขอบเขตด้านระบบ AI และอัลกอริทึม

ระบบใช้แนวคิดด้านปัญญาประดิษฐ์เพื่อช่วยออกแบบแผนการท่องเที่ยว โดยประกอบด้วยองค์ประกอบหลักดังนี้

1. User Preference Model

   ใช้สำหรับวิเคราะห์ความสนใจของผู้ใช้งานจากข้อมูลที่ผู้ใช้ระบุ เช่น ประเภทสถานที่ งบประมาณ และระยะเวลาในการเดินทาง

2. Trip Design Model
   ใช้สร้างแผนการท่องเที่ยวอัตโนมัติ โดยระบบจะเลือกสถานที่ที่เหมาะสมและจัดลำดับกิจกรรมรายวัน
3. Route Optimization Algorithm
   ใช้สำหรับจัดลำดับเส้นทางการเดินทางระหว่างสถานที่ต่าง ๆ เพื่อลดระยะเวลาและเพิ่มประสิทธิภาพในการเดินทาง

1.4.3 ขอบเขตด้านเทคโนโลยี (Technology Scope)

เทคโนโลยีที่ใช้ในการพัฒนาระบบประกอบด้วย

- Frontend: React Native สำหรับพัฒนาโมบายแอปพลิเคชัน
- Backend: Node.js หรือระบบ API สำหรับจัดการข้อมูล
- AI Model: พัฒนาโดยใช้ภาษา Python
- Database:SQL สำหรับจัดเก็บข้อมูลผู้ใช้และข้อมูลสถานที่
- Map Service: Google Maps API สำหรับแสดงแผนที่และตำแหน่งสถานที่ท่องเที่ยว
- Development Tools: Visual Studio Code และ Android Studio

1.5 สมมติฐานหรือคำถามวิจัย

โครงงานนี้ตั้งสมมติฐานว่า “การพัฒนาโมบายแอปพลิเคชันสำหรับการออกแบบทริปการท่องเที่ยวโดยใช้เทคโนโลยีปัญญาประดิษฐ์ (AI) ที่สามารถวิเคราะห์ความสนใจ ระยะเวลา และงบประมาณของผู้ใช้งาน จะช่วยให้ผู้ใช้งานสามารถวางแผนการท่องเที่ยวได้สะดวกและมีประสิทธิภาพมากยิ่งขึ้น รวมทั้งช่วยลดเวลาในการค้นหาข้อมูลและการจัดลำดับสถานที่ท่องเที่ยวด้วยตนเอง”

1.6ประโยชน์ที่คาดว่าจะได้รับ

1. ผู้ใช้งานสามารถเข้าถึงระบบและได้รับแผนการท่องเที่ยวที่เหมาะสมกับความต้องการ ความสนใจ และ ระยะเวลาได้อย่างสะดวก
2. ช่วยลดปัญหาการวางแผนการท่องเที่ยวโดยไม่รู้ว่าจะไปที่ใดก่อนหรือเลือกสถานที่ใด ทำให้ผู้ใช้งานสามารถวางแผนการเดินทางได้อย่างมีประสิทธิภาพมากยิ่งขึ้น

แผนการดำเนินงานของโครงงาน

การดำเนินงาน	**ระยะเวลา (สัปดาห์)**															
	เดือน1				**เดือน2**				**เดือน3**				**เดือน4**			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
ศึกษาแนวคิด ทฤษฎี และเอกสารที่เกี่ยวข้อง																
ศึกษาเทคโนโลยีที่จะใช้																
วิเคราะห์ปัญหาและความต้องการของระบบ																
รวบรวมและจัดเตรียมข้อมูลที่เกี่ยวข้องกับการพัฒนาระบบ																
วิเคราะห์ข้อมูลและออกแบบระบบ																
การสร้างตัวแบบหรือการพัฒนาโปรแกรม																
ทดสอบติดตั้งระบบหรือการทดสอบตัวแบบ																
จัดทำเอกสาร																

บทที่ 2
เอกสารและงานวิจัยที่เกี่ยวข้อง

2.1แนวคิดและทฤษฎีที่เกี่ยวข้องกับโมบายแอปพลิเคชัน

การพัฒนาระบบออกแบบทริปการท่องเที่ยวด้วย AI จำเป็นต้องอาศัยแนวคิดและทฤษฎีจากหลายด้าน เพื่อใช้เป็นพื้นฐานในการออกแบบและพัฒนาระบบให้สามารถทำงานได้อย่างมีประสิทธิภาพ ทั้งในด้านการพัฒนาโมบายแอปพลิเคชัน การออกแบบส่วนติดต่อผู้ใช้ การวิเคราะห์ความสนใจของผู้ใช้งาน ระบบแนะนำข้อมูล และการประยุกต์ใช้ปัญญาประดิษฐ์ในการประมวลผลข้อมูลและจัดลำดับเส้นทางการเดินทาง ดังนั้น ผู้จัดทำจึงได้ศึกษาแนวคิดและทฤษฎีที่เกี่ยวข้อง เพื่อนำมาใช้เป็นแนวทางในการพัฒนาระบบให้สามารถแนะนำสถานที่ท่องเที่ยวและวางแผนการเดินทางได้อย่างเหมาะสมและมีประสิทธิภาพ

2.1.1 Web and Mobile Application

ในปัจจุบันเทคโนโลยีดิจิทัลเข้ามามีบทบาทสำคัญต่อการดำเนินชีวิตประจำวัน การดำเนินธุรกิจ การศึกษา การติดต่อสื่อสาร และการให้บริการในรูปแบบต่าง ๆ มากขึ้น โดยเฉพาะการใช้งานระบบผ่านเครือข่ายอินเทอร์เน็ตและอุปกรณ์พกพา เช่น สมาร์ตโฟนและแท็บเล็ต ส่งผลให้เว็บแอปพลิเคชันและโมบายแอปพลิเคชันกลายเป็นเครื่องมือสำคัญสำหรับการให้บริการและการเข้าถึงข้อมูลในยุคปัจจุบัน

จากเอกสาร *Web and Mobile Applications* ระบุว่า อุตสาหกรรมเว็บและโมบายแอปพลิเคชันมีการเติบโตอย่างต่อเนื่อง โดยมีปัจจัยสำคัญจากการเปลี่ยนแปลงทางดิจิทัล การเพิ่มขึ้นของการใช้งานสมาร์ตโฟน และความต้องการซอฟต์แวร์ที่สามารถรองรับการขยายตัวขององค์กรและผู้ใช้งานได้ ปัจจัยดังกล่าวสะท้อนให้เห็นว่าแอปพลิเคชันไม่ได้ถูกใช้เพียงเพื่ออำนวยความสะดวกแก่ผู้ใช้งานเท่านั้น แต่ยังเป็นส่วนสำคัญของโครงสร้างพื้นฐานด้านดิจิทัลขององค์กรในหลายอุตสาหกรรม เว็บแอปพลิเคชันและโมบายแอปพลิเคชันสามารถนำไปประยุกต์ใช้ในหลายด้าน เช่น พาณิชย์อิเล็กทรอนิกส์ การเงิน การแพทย์ การศึกษา การสื่อสาร และการให้บริการออนไลน์ โดยเอกสารได้กล่าวถึงอุตสาหกรรมสำคัญที่มีการใช้งานเว็บและโมบายแอปพลิเคชันในระดับสูง ได้แก่ E-commerce, Fintech, Healthcare และ Education ซึ่งแสดงให้เห็นถึงความยืดหยุ่นของแอปพลิเคชันที่สามารถปรับใช้ให้เหมาะสมกับความต้องการของแต่ละอุตสาหกรรมได้

สำหรับโครงงาน TripAI แนวคิดเกี่ยวกับ Web and Mobile Application ถือเป็นพื้นฐานสำคัญของการพัฒนาระบบ เนื่องจากระบบถูกออกแบบในลักษณะเว็บแอปพลิเคชันที่สามารถใช้งานผ่านเว็บเบราว์เซอร์ และพัฒนาด้วยเทคโนโลยีที่สามารถรองรับการใช้งานบนอุปกรณ์หลายประเภทได้

การศึกษาแนวคิดของเว็บและโมบายแอปพลิเคชันจึงช่วยให้เข้าใจลักษณะการทำงาน ข้อดี ข้อจำกัด และแนวทางในการออกแบบระบบที่เหมาะสมกับผู้ใช้

ความหมายและความสำคัญของ Web and Mobile Application

Web Application หรือเว็บแอปพลิเคชัน เป็นโปรแกรมประยุกต์ที่ทำงานผ่านระบบเครือข่ายและสามารถเข้าถึงได้ด้วยเว็บเบราว์เซอร์ โดยผู้ใช้งานไม่จำเป็นต้องติดตั้งโปรแกรมทั้งหมดลงบนอุปกรณ์เหมือนกับซอฟต์แวร์แบบดั้งเดิม ระบบสามารถประมวลผลข้อมูลร่วมกับบริการหรือระบบที่อยู่บนเครือข่าย ทำให้ผู้ใช้งานสามารถเข้าถึงบริการจากอุปกรณ์และสถานที่ที่แตกต่างกันได้ ตราบใดที่มีการเชื่อมต่ออินเทอร์เน็ตและมีเว็บเบราว์เซอร์ที่รองรับ

ในขณะที่ Mobile Application หรือโมบายแอปพลิเคชัน เป็นซอฟต์แวร์ที่ออกแบบมาเพื่อใช้งานกับอุปกรณ์พกพา เช่น สมาร์ตโฟนและแท็บเล็ต โดยทั่วไปแอปพลิเคชันประเภทนี้ถูกออกแบบให้เหมาะสมกับขนาดหน้าจอ รูปแบบการโต้ตอบผ่านระบบสัมผัส และความสามารถเฉพาะของอุปกรณ์พกพา เช่น กล้อง ระบบระบุตำแหน่ง และการแจ้งเตือน

ความสำคัญของ Web and Mobile Application เพิ่มขึ้นอย่างต่อเนื่องตามพฤติกรรมของผู้ใช้งานที่เปลี่ยนแปลงไป โดยผู้ใช้งานในปัจจุบันคาดหวังว่าจะสามารถเข้าถึงข้อมูลและบริการได้อย่างรวดเร็วผ่านอุปกรณ์ที่ใช้งานอยู่ ไม่ว่าจะเป็นคอมพิวเตอร์ตั้งโต๊ะ โน้ตบุ๊ก สมาร์ตโฟน หรือแท็บเล็ต จึงทำให้ผู้พัฒนาระบบจำเป็นต้องคำนึงถึงความสามารถในการรองรับอุปกรณ์และแพลตฟอร์มที่หลากหลาย

เอกสาร *Web and Mobile Applications* ระบุว่าตลาดโมบายแอปพลิเคชันทั่วโลกมีมูลค่าประมาณ 289.17 พันล้านดอลลาร์สหรัฐในปี ค.ศ. 2024 และคาดว่าจะเพิ่มขึ้นเป็นมากกว่า 1,103.48 พันล้านดอลลาร์สหรัฐภายในปี ค.ศ. 2034 โดยปัจจัยสำคัญมาจากการขยายตัวของบริการที่ทำงานผ่านแอปพลิเคชันและการเพิ่มขึ้นของการใช้งานสมาร์ตโฟน ข้อมูลดังกล่าวแสดงให้เห็นถึงบทบาทที่เพิ่มขึ้นของโมบายแอปพลิเคชันต่อพฤติกรรมของผู้บริโภคและการดำเนินธุรกิจในปัจจุบัน

ในส่วนของเว็บแอปพลิเคชัน เอกสารระบุว่าตลาดเว็บแอปพลิเคชันมีการขยายตัวเช่นเดียวกัน โดยมีปัจจัยสำคัญจากการนำบริการบนระบบคลาวด์และแพลตฟอร์ม Software as a Service หรือ SaaS มาใช้งานมากขึ้น สิ่งนี้สะท้อนให้เห็นว่าแนวทางการใช้งานซอฟต์แวร์ได้เปลี่ยนจากการติดตั้งและบริหารจัดการโปรแกรมภายในอุปกรณ์เพียงอย่างเดียว ไปสู่การใช้ซอฟต์แวร์ที่สามารถเข้าถึงผ่านเครือข่ายและให้บริการจากส่วนกลาง

การเติบโตของ Web and Mobile Application ยังสัมพันธ์กับแนวคิด Digital Transformation หรือการเปลี่ยนแปลงองค์กรและบริการเข้าสู่รูปแบบดิจิทัล องค์กรสามารถนำแอปพลิเคชันมาใช้เพื่อเพิ่มประสิทธิภาพการทำงาน ลดขั้นตอนที่ซ้ำซ้อน และสร้างช่องทางการให้บริการ

ใหม่แก่ผู้ใช้งาน ตัวอย่างจากเอกสารแสดงให้เห็นว่าหลายองค์กรพัฒนาระบบเว็บและโมบายเพื่อสนับสนุนกระบวนการทำงานเฉพาะด้าน ตั้งแต่ระบบบริหารจัดการ ระบบอีคอมเมิร์ซ ระบบด้านสุขภาพ ระบบการศึกษา ไปจนถึงแพลตฟอร์มการเดินทางและการท่องเที่ยว

นอกจากการเพิ่มประสิทธิภาพการทำงานแล้ว การพัฒนาแอปพลิเคชันในปัจจุบันยังให้ความสำคัญกับประสบการณ์ของผู้ใช้งานเป็นอย่างมาก ระบบควรสามารถใช้งานได้ง่าย มีรูปแบบการแสดงผลที่เหมาะสมกับอุปกรณ์ และตอบสนองต่อการใช้งานได้อย่างรวดเร็ว ในเอกสารมีการกล่าวถึงการพัฒนาเว็บและโมบายแอปพลิเคชันที่เน้นส่วนติดต่อผู้ใช้ที่ใช้งานง่าย ความสามารถในการรองรับผู้ใช้งานจำนวนมาก และการรักษาความปลอดภัยของระบบ

แนวคิดดังกล่าวสอดคล้องกับลักษณะของระบบ TripAI ซึ่งต้องให้ผู้ใช้งานกรอกข้อมูลเกี่ยวกับการเดินทาง เช่น จุดหมายปลายทาง จำนวนวัน งบประมาณ จำนวนผู้เดินทาง และรูปแบบการท่องเที่ยว ก่อนที่ระบบและรูปภาพประกอบ ดังนั้น ส่วนติดต่อผู้ใช้งานจึงต้องสามารถนำเสนอข้อมูลจำนวนมากโดยยังคงความเข้าใจง่ายและสะดวกต่อการใช้งานบนอุปกรณ์ที่หลากหลาย

อีกประเด็นหนึ่งที่มีความสำคัญคือความสามารถในการเชื่อมโยงแอปพลิเคชันกับเทคโนโลยีสมัยใหม่ เอกสารแสดงให้เห็นว่า Web and Mobile Application ในปัจจุบันมักถูกพัฒนาร่วมกับบริการด้าน Cloud Computing, Artificial Intelligence, Data Analytics และระบบอัตโนมัติ ตัวอย่างเช่น บริษัทที่พัฒนา Web and Mobile Application หลายแห่งในเอกสารมีการนำ Artificial Intelligence และ Machine Learning มาประยุกต์ใช้ร่วมกับระบบเพื่อช่วยในการวิเคราะห์ข้อมูลและสร้างบริการที่มีความสามารถมากขึ้น

ในกรณีของ TripAI แนวคิดดังกล่าวถูกนำมาประยุกต์ใช้โดยการผสานเว็บแอปพลิเคชันเข้ากับระบบปัญญาประดิษฐ์เพื่อสร้างแผนการเดินทางตามข้อมูลที่ผู้ใช้กำหนด รวมถึงการเชื่อมต่อกับบริการภายนอกเพื่อค้นหาพิกัดและรูปภาพของสถานที่ ซึ่งแสดงให้เห็นถึงรูปแบบของแอปพลิเคชันสมัยใหม่ที่ไม่ได้ทำงานแบบแยกส่วน แต่ทำงานร่วมกับบริการและแหล่งข้อมูลหลายประเภท

เว็บแอปพลิเคชัน (Web Application)

เว็บแอปพลิเคชันเป็นรูปแบบของโปรแกรมประยุกต์ที่ผู้ใช้งานสามารถเข้าถึงผ่านเว็บเบราว์เซอร์ โดยการทำงานของระบบสามารถประกอบด้วยส่วนที่ทำงานบนอุปกรณ์ของผู้ใช้งานและส่วนที่เชื่อมต่อกับบริการภายนอกหรือระบบประมวลผลผ่านเครือข่ายอินเทอร์เน็ต จุดเด่นที่สำคัญของเว็บแอปพลิเคชันคือผู้ใช้งานสามารถเข้าถึงระบบได้โดยไม่จำเป็นต้องติดตั้งโปรแกรมเฉพาะลงบนอุปกรณ์ในลักษณะเดียวกับโปรแกรมแบบดั้งเดิม

เว็บแอปพลิเคชันสามารถนำมาใช้พัฒนาระบบได้หลายลักษณะ ตั้งแต่เว็บไซต์ที่มีการโต้ตอบกับผู้ใช้งาน ระบบจัดการข้อมูล ระบบธุรกิจอิเล็กทรอนิกส์ ไปจนถึงระบบที่มีการประมวลผลที่ซับซ้อน โดยความแตกต่างระหว่างเว็บไซต์ทั่วไปกับเว็บแอปพลิเคชันคือ เว็บแอปพลิเคชันมักมีฟังก์ชันการทำงานและการโต้ตอบที่มากกว่า เช่น การเข้าสู่ระบบ การจัดเก็บข้อมูล การสร้างหรือแก้ไขข้อมูล การประมวลผลข้อมูล และการเชื่อมต่อกับบริการอื่น

เอกสารระบุว่า Web Application มีแนวโน้มเติบโตขึ้นจากการใช้งาน Cloud-based Services และ Web-based SaaS Platform มากขึ้น ส่งผลให้องค์กรสามารถให้บริการซอฟต์แวร์ผ่านระบบอินเทอร์เน็ตแทนการติดตั้งโปรแกรมแยกในแต่ละอุปกรณ์ ซึ่งช่วยลดความยุ่งยากในการดูแลและปรับปรุงระบบ

ข้อดีที่สำคัญของเว็บแอปพลิเคชันคือความสะดวกในการเข้าถึง เนื่องจากผู้ใช้งานสามารถเปิดระบบผ่านเว็บเบราว์เซอร์จากอุปกรณ์ที่รองรับได้ทันที ช่วยลดข้อจำกัดด้านระบบปฏิบัติการและขั้นตอนการติดตั้ง

นอกจากนี้ เมื่อมีการปรับปรุงระบบ ผู้พัฒนาสามารถนำเวอร์ชันใหม่ขึ้นจะประมวลผลและให้บริการจากส่วนกลางได้ ทำให้ผู้ใช้งานสามารถเข้าถึงเวอร์ชันล่าสุดได้โดยไม่จำเป็นต้องดาวน์โหลดโปรแกรมใหม่ด้วยตนเองในทุกครั้ง

อีกคุณสมบัติหนึ่งของเว็บแอปพลิเคชันคือความสามารถในการรองรับการแสดงผลบนอุปกรณ์หลายขนาด ซึ่งเกี่ยวข้องกับแนวคิด Responsive Design ระบบควรสามารถปรับองค์ประกอบของหน้าจอให้เหมาะสมกับอุปกรณ์ที่ผู้ใช้งานใช้ เช่น การจัดตำแหน่งข้อความ ปุ่ม รูปภาพ และส่วนประกอบต่าง ๆ เพื่อให้สามารถใช้งานได้ทั้งจากคอมพิวเตอร์และอุปกรณ์พกพา

เอกสารยกตัวอย่างผู้พัฒนาระบบที่ให้ความสำคัญกับ Responsive Interface, UI/UX Design และระบบ Backend ที่มีความเสถียร โดยคุณสมบัติเหล่านี้มีส่วนช่วยให้แอปพลิเคชันสามารถรองรับการใช้งานที่หลากหลายและขยายระบบได้ในอนาคต นอกจากนี้ยังมีตัวอย่างของระบบเว็บที่เน้น Responsive Design และการเพิ่มประสิทธิภาพประสบการณ์ของผู้ใช้งาน เพื่อให้ระบบสามารถใช้งานบนอุปกรณ์หลายรูปแบบได้อย่างมีประสิทธิภาพ

อย่างไรก็ตาม เว็บแอปพลิเคชันมีข้อจำกัดบางประการ เช่น ประสิทธิภาพการทำงานบางประเภทอาจขึ้นอยู่กับความเร็วของเครือข่ายอินเทอร์เน็ต และความสามารถในการเข้าถึงทรัพยากรของอุปกรณ์อาจแตกต่างจากโมบายแอปพลิเคชันที่ติดตั้งโดยตรง นอกจากนี้ หากระบบมีข้อมูลจำนวนมากหรือมีการประมวลผลที่ซับซ้อน ผู้พัฒนาจำเป็นต้องออกแบบโครงสร้างระบบให้เหมาะสมเพื่อรักษาความเร็วและความเสถียรในการใช้งาน

สำหรับโครงงาน TripAI การเลือกพัฒนาระบบในรูปแบบเว็บแอปพลิเคชันมีความเหมาะสมกับลักษณะการใช้งาน เนื่องจากผู้ใช้งานสามารถเข้าสู่ระบบผ่านเว็บเบราว์เซอร์ได้โดยไม่จำเป็นต้อง

ติดตั้งโปรแกรมเพิ่มเติม อีกทั้งยังช่วยให้การปรับปรุงและนำระบบขึ้นใช้งานสามารถดำเนินการได้จากส่วนกลาง โดยระบบสามารถให้บริการฟังก์ชันต่าง ๆ ได้แก่ การเข้าสู่ระบบ การกรอกข้อมูลทริป การประมวลผลแผนการเดินทาง การแสดงแผนที่ การแสดงรูปภาพ และการจัดเก็บแผนการเดินทางของผู้ใช้งานภายในระบบเดียว

โมบายแอปพลิเคชัน (Mobile Application)

โมบายแอปพลิเคชัน (Mobile Application) คือซอฟต์แวร์ประยุกต์ที่ได้รับการออกแบบสำหรับใช้งานบนอุปกรณ์พกพา เช่น สมาร์ตโฟนและแท็บเล็ต โดยระบบสามารถพัฒนาให้ทำงานร่วมกับระบบปฏิบัติการที่แตกต่างกัน เช่น Android และ iOS แอปพลิเคชันประเภทนี้มีบทบาทสำคัญต่อการใช้งานเทคโนโลยีดิจิทัลในชีวิตประจำวัน เนื่องจากอุปกรณ์พกพาสามารถเชื่อมต่ออินเทอร์เน็ตและเข้าถึงบริการออนไลน์ได้จากแทบทุกสถานที่ ทำให้ผู้ใช้งานสามารถเข้าถึงข้อมูลและบริการต่าง ๆ ได้อย่างสะดวกและรวดเร็ว

การขยายตัวของการใช้งานสมาร์ตโฟนเป็นหนึ่งในปัจจัยสำคัญที่ส่งผลให้โมบายแอปพลิเคชันได้รับความนิยมเพิ่มขึ้น จากเอกสาร Web and Mobile Applications ระบุว่าการเติบโตของตลาดนำเสนอข้อมูลในรูปแบบของโมบายแอปพลิเคชันได้รับแรงผลักดันจากการเพิ่มขึ้นของบริการที่ให้บริการผ่านแอปพลิเคชันและการแพร่หลายของสมาร์ตโฟน โดยตลาดโมบายแอปพลิเคชันทั่วโลกมีมูลค่าประมาณ 289.17 พันล้านดอลลาร์สหรัฐในปี ค.ศ. 2024 และมีแนวโน้มเติบโตอย่างต่อเนื่องในระยะยาว

โมบายแอปพลิเคชันสามารถแบ่งตามลักษณะการพัฒนาได้หลายรูปแบบ เช่น แอปพลิเคชันที่พัฒนาสำหรับระบบปฏิบัติการโดยเฉพาะ แอปพลิเคชันแบบไฮบริด และแอปพลิเคชันแบบข้ามแพลตฟอร์ม อย่างไรก็ตาม ไม่ว่าจะใช้แนวทางการพัฒนาในรูปแบบใด สิ่งสำคัญคือการออกแบบให้เหมาะสมกับลักษณะของอุปกรณ์พกพา ทั้งในด้านขนาดหน้าจอ รูปแบบการควบคุมผ่านการสัมผัส และการจัดวางข้อมูลให้ผู้ใช้งานสามารถเข้าถึงฟังก์ชันสำคัญได้อย่างสะดวก

เอกสารได้ยกตัวอย่างการพัฒนาโมบายแอปพลิเคชันสำหรับทั้ง Android และ iOS โดยให้ความสำคัญกับการสร้างแอปพลิเคชันที่สามารถรองรับการขยายตัวของระบบ มีความง่ายต่อการใช้งาน และช่วยเพิ่มการมีส่วนร่วมของผู้ใช้ จากแนวคิดดังกล่าวสามารถเห็นได้ว่า การพัฒนา Mobile Application ไม่ได้ให้ความสำคัญเพียงความสามารถของระบบเท่านั้น แต่ยังต้องคำนึงถึงประสบการณ์ที่ผู้ใช้งานได้รับจากการใช้แอปพลิเคชันด้วย

องค์ประกอบสำคัญอีกประการหนึ่งคือส่วนติดต่อผู้ใช้งานหรือ User Interface ซึ่งต้องได้รับการออกแบบให้เหมาะกับหน้าจอของอุปกรณ์พกพา เนื่องจากพื้นที่แสดงผลของสมาร์ตโฟนมีขนาด

จำกัดเมื่อเปรียบเทียบกับคอมพิวเตอร์ ผู้พัฒนาจึงต้องเลือกแสดงข้อมูลที่สำคัญ จัดลำดับเนื้อหาให้ชัดเจน และลดขั้นตอนที่ไม่จำเป็น เพื่อให้ผู้ใช้งานสามารถดำเนินการต่าง ๆ ได้ภายในระยะเวลาสั้น

นอกจากนี้ Mobile Application ยังสามารถนำความสามารถของอุปกรณ์มาใช้เพื่อเพิ่มประสิทธิภาพในการให้บริการได้ เช่น การระบุตำแหน่ง การเชื่อมต่อเครือข่าย และฟังก์ชันที่เกี่ยวข้องกับอุปกรณ์ ซึ่งช่วยให้สามารถพัฒนาระบบที่ตอบสนองต่อบริบทการใช้งานของผู้ใช้ได้มากขึ้น ตัวอย่างในเอกสารแสดงให้เห็นถึงการพัฒนาแอปพลิเคชันที่ทำงานร่วมกับอุปกรณ์และเทคโนโลยีอื่น เช่น แอปพลิเคชันด้านการติดตามการออกกำลังกายที่เชื่อมต่อกับอุปกรณ์ผ่านเทคโนโลยี Bluetooth Low Energy

ในปัจจุบัน Mobile Application ถูกนำไปประยุกต์ใช้ในอุตสาหกรรมที่หลากหลาย ไม่ว่าจะเป็นการเงิน สุขภาพ การศึกษา ความบันเทิง พาณิชย์อิเล็กทรอนิกส์ และการท่องเที่ยว โดยเอกสารแสดงให้เห็นว่าบริษัทพัฒนาซอฟต์แวร์จำนวนมากนำโมบายแอปพลิเคชันมาใช้เพื่อแก้ไขปัญหาและตอบสนองความต้องการเฉพาะของแต่ละอุตสาหกรรม ตัวอย่างเช่น การพัฒนาแอปพลิเคชันสำหรับระบบจอง การจัดการข้อมูล การให้บริการด้านสุขภาพ และการเชื่อมโยงผู้ใช้กับบริการต่าง ๆ ผ่านอุปกรณ์พกพา

สำหรับอุตสาหกรรมการท่องเที่ยว อุปกรณ์พกพามีความเหมาะสมกับลักษณะการใช้งานอย่างมาก เนื่องจากนักท่องเที่ยวสามารถพกพาสมาร์ตโฟนติดตัวไปตลอดการเดินทาง ผู้ใช้งานจึงแผนการเดินทาง แผนสามารถเปิดดูรายละเอียดสถานที่ ตารางการเดินทาง ข้อมูลการจอง หรือข้อมูลที่เกี่ยวข้องกับการเดินทางในขณะที่อยู่ภายนอกสถานที่พักอาศัยได้ เอกสารมีตัวอย่างบริษัทที่ให้บริการพัฒนา Web และ Mobile Application สำหรับอุตสาหกรรม Travel & Tourism รวมถึงระบบจองและบริการที่เกี่ยวข้องกับการเดินทาง

นอกจากนี้ เอกสารยังกล่าวถึง Technology Bench ซึ่งให้บริการด้าน Travel Tech และ Web and Mobile App Development โดยมีผลิตภัณฑ์ที่เกี่ยวข้องกับการเดินทาง เช่น Smartdestinations.ai แสดงให้เห็นว่าการนำเทคโนโลยีเว็บ โมบาย และระบบอัจฉริยะมาใช้ร่วมกันมีบทบาทในอุตสาหกรรมการท่องเที่ยวสมัยใหม่

ในด้านพฤติกรรมของผู้ใช้งาน เอกสาร Web and Mobile Applications ระบุว่าเศรษฐกิจของแอปพลิเคชันทั่วโลกมีการเติบโตอย่างต่อเนื่อง โดยค่าใช้จ่ายของผู้บริโภคใน Mobile Apps และ Games ผ่าน App Store และ Google Play มีมูลค่ารวม 127 พันล้านดอลลาร์สหรัฐในปี ค.ศ. 2024 เพิ่มขึ้นจากปีก่อนหน้า 15.7 เปอร์เซ็นต์ ข้อมูลดังกล่าวสะท้อนให้เห็นว่า Mobile Application กลายเป็นหนึ่งในช่องทางหลักสำหรับการเข้าถึงบริการและเนื้อหาดิจิทัลของผู้ใช้งานในปัจจุบัน

อีกประเด็นที่เห็นได้ชัดจากเอกสารคือการเปลี่ยนแปลงของพฤติกรรมผู้ใช้ในแต่ละภูมิภาค โดยบางภูมิภาคมีลักษณะการใช้งานแบบ Mobile-first ซึ่งหมายถึงการที่สมาร์ตโฟนเป็นอุปกรณ์หลัก

ในการเข้าถึงอินเทอร์เน็ตและบริการดิจิทัล เอกสารระบุว่าในบางพื้นที่ของแอฟริกาตอนใต้ของทะเลทรายซาฮารา การใช้งาน Mobile Web สูงกว่าการใช้งาน Desktop อย่างมาก แนวโน้มดังกล่าวทำให้ผู้พัฒนาต้องคำนึงถึงการออกแบบระบบให้เหมาะสมกับการใช้งานผ่านอุปกรณ์พกพามากยิ่งขึ้น

สำหรับระบบ TripAI แม้ว่าขอบเขตการใช้งานในปัจจุบันจะเน้น Web Application เป็นหลัก แต่รูปแบบของระบบได้รับการออกแบบให้สามารถใช้งานผ่านอุปกรณ์ที่มีขนาดหน้าจอแตกต่างกันได้ โดยเฉพาะสมาร์ตโฟน ซึ่งสอดคล้องกับบริบทของระบบด้านการท่องเที่ยว เพราะผู้ใช้งานอาจจำเป็นต้องเปิดดูแผนการเดินทางหรือข้อมูลสถานที่ในระหว่างเดินทาง นอกจากนี้ การเลือกใช้เทคโนโลยี Flutter ยังเป็นพื้นฐานที่สามารถนำระบบไปต่อยอดสำหรับอุปกรณ์พกพาในอนาคตได้ โดยรายละเอียดของการพัฒนาแบบข้ามแพลตฟอร์มจะกล่าวถึงในหัวข้อถัดไป

ความแตกต่างระหว่าง Web Application และ Mobile Application

Web Application และ Mobile Application มีวัตถุประสงค์ร่วมกันในการให้บริการฟังก์ชันต่าง ๆ แก่ผู้ใช้งานผ่านระบบดิจิทัล แต่มีความแตกต่างกันในหลายด้าน โดยเฉพาะวิธีการเข้าถึงระบบ การติดตั้ง การพัฒนา การปรับปรุงระบบ และการใช้งานบนอุปกรณ์

Web Application มีลักษณะเด่นคือสามารถเข้าถึงผ่านเว็บเบราว์เซอร์ ผู้ใช้งานจึงไม่จำเป็นต้องดาวน์โหลดและติดตั้งแอปพลิเคชันลงบนอุปกรณ์โดยตรง การแก้ไขและปรับปรุงระบบสามารถดำเนินการจากส่วนกลาง และเมื่อผู้ใช้งานเปิดระบบในครั้งถัดไปก็สามารถเข้าถึงระบบเวอร์ชันที่ได้รับการปรับปรุงแล้วได้ทันที

ในทางตรงกันข้าม Mobile Application มักถูกออกแบบสำหรับการใช้งานบนอุปกรณ์พกพาและระบบปฏิบัติการแจกจ่าย เช่น Application Store ข้อดีคือสามารถออกแบบระบบให้เหมาะสมกับอุปกรณ์พกพาและสามารถนำความสามารถเฉพาะของอุปกรณ์มาประยุกต์ใช้ได้

เอกสารแสดงให้เห็นว่าผู้ให้บริการด้านการพัฒนาซอฟต์แวร์ในปัจจุบันมักให้บริการทั้ง Web Application และ Mobile Application ควบคู่กัน โดยเว็บมักถูกออกแบบให้มีลักษณะ Responsive ในขณะที่ Mobile Application จะถูกพัฒนาให้เหมาะกับ Android และ iOS รวมถึงประสบการณ์การใช้งานบนอุปกรณ์พกพา

ตารางที่ 2.1 การเปรียบเทียบลักษณะของ Web Application และ Mobile Application

ประเด็นเปรียบเทียบ	Web Application	Mobile Application
รูปแบบการเข้าใช้งาน	เข้าใช้งานผ่าน Web Browser	เข้าใช้งานผ่านแอปพลิเคชันบนอุปกรณ์พกพา

ประเด็นเปรียบเทียบ	Web Application	Mobile Application
การติดตั้ง	โดยทั่วไปไม่จำเป็นต้องติดตั้งลงบนอุปกรณ์	โดยทั่วไปต้องติดตั้งแอปพลิเคชันก่อนใช้งาน
อุปกรณ์ที่รองรับ	สามารถใช้ได้บนอุปกรณ์ที่มี Browser ที่รองรับ	เน้นสมาร์ตโฟนและแท็บเล็ต
ระบบปฏิบัติการ	มีความยืดหยุ่น เนื่องจากทำงานผ่าน Browser	อาจต้องพิจารณา Android และ iOS
การปรับปรุงระบบ	ปรับปรุงระบบจากส่วนกลาง ผู้ใช้เข้าถึงเวอร์ชันใหม่ได้ทันที	อาจต้องมีการอัปเดตแอปพลิเคชันตามรูปแบบการแจกจ่าย
การออกแบบหน้าจอ	ต้องรองรับขนาดหน้าจอที่หลากหลายด้วย Responsive Design	ออกแบบโดยเน้นการใช้งานบนหน้าจออุปกรณ์พกพา
การใช้ความสามารถของอุปกรณ์	สามารถใช้ความสามารถของอุปกรณ์บางประเภทผ่าน Browser และเทคโนโลยีเว็บ	สามารถออกแบบให้ทำงานร่วมกับความสามารถของอุปกรณ์พกพาได้โดยตรงมากกว่า
การเชื่อมต่ออินเทอร์เน็ต	โดยทั่วไปต้องพึ่งพาการเชื่อมต่อเครือข่ายสำหรับการเข้าถึงบริการ	ขึ้นอยู่กับรูปแบบของแอปและบริการที่ใช้งาน
การดูแลรักษา	สามารถดูแลระบบจากส่วนกลางได้สะดวก	ต้องคำนึงถึงเวอร์ชันและแพลตฟอร์มที่รองรับ
ลักษณะการใช้งาน	เหมาะกับระบบที่ต้องการเข้าถึงง่ายจากหลายอุปกรณ์	เหมาะกับระบบที่เน้นการใช้งานผ่านสมาร์ตโฟนและการใช้งานระหว่างเคลื่อนที่

จากตารางที่ 2.1 จะเห็นได้ว่า Web Application และ Mobile Application มีข้อดีและลักษณะการใช้งานที่แตกต่างกัน การเลือกรูปแบบในการพัฒนาจึงควรพิจารณาจากความต้องการของระบบ กลุ่มผู้ใช้งาน อุปกรณ์ที่ต้องการรองรับ และข้อจำกัดด้านทรัพยากรในการพัฒนา

อย่างไรก็ตาม แนวทางการพัฒนาระบบในปัจจุบันเริ่มมีการเชื่อมโยงระหว่าง Web Application และ Mobile Application มากขึ้น เช่น การพัฒนา Responsive Web Application, Progressive Web Application และ Cross-Platform Application ซึ่งช่วยลดความแตกต่างด้านประสบการณ์ของผู้ใช้งานระหว่างแพลตฟอร์ม

จากเอกสารพบว่าความต้องการในการพัฒนาแบบ Cross-platform เพิ่มขึ้น เนื่องจากองค์กรต้องการวิธีการที่มีประสิทธิภาพและประหยัดต้นทุนในการนำแอปพลิเคชันไปให้บริการบนหลายแพลตฟอร์ม เช่น Android และ iOS โดยมี Framework ที่ได้รับการกล่าวถึง ได้แก่ Flutter, React Native และ Xamarin ทั้งนี้รายละเอียดเกี่ยวกับ Cross-Platform Application

Progressive Web Application (PWA)

Progressive Web Application หรือ PWA เป็นแนวทางในการพัฒนา Web Application ที่มีเป้าหมายในการสร้างประสบการณ์ใช้งานให้ใกล้เคียงกับแอปพลิเคชันบนอุปกรณ์พกพามากขึ้น โดยอาศัยเทคโนโลยีของเว็บในการนำเสนอระบบผ่านเว็บเบราว์เซอร์ แต่มีแนวคิดในการปรับปรุงความสะดวกและประสบการณ์ใช้งานให้คล้ายคลึงกับ Mobile Application

แนวคิดของ PWA เป็นตัวอย่างที่แสดงให้เห็นถึงการพัฒนาของเทคโนโลยีเว็บที่พยายามลดข้อจำกัดระหว่าง Web Application และ Mobile Application ผู้ใช้งานสามารถเข้าถึงระบบผ่านเว็บ ขณะเดียวกันระบบถูกออกแบบให้มีประสบการณ์ใช้งานในลักษณะ App-like มากขึ้น จึงเป็นทางเลือกหนึ่งสำหรับระบบที่ต้องการให้ผู้ใช้งานสามารถเข้าถึงได้ง่ายโดยไม่ต้องพัฒนาแอปพลิเคชันแยกสำหรับทุกแพลตฟอร์ม

จากเอกสาร *Web and Mobile Applications* ระบุว่า Progressive Web Apps กำลังได้รับความนิยมมากขึ้นเนื่องจากสามารถมอบประสบการณ์การใช้งานในลักษณะที่ใกล้เคียงกับแอปพลิเคชัน โดยมีต้นทุนในการพัฒนาที่ต่ำกว่าในบางกรณี นอกจากนี้ PWA ยังได้รับความสนใจในตลาดที่อุปกรณ์ของผู้ใช้งานอาจมีข้อจำกัดด้านประสิทธิภาพ

ความสำคัญของ PWA จึงไม่ได้อยู่เพียงเรื่องของการแสดงผลผ่านเว็บเท่านั้น แต่ยังสะท้อนให้เห็นแนวคิดในการพัฒนาระบบที่ต้องการลดอุปสรรคในการเข้าถึงบริการ ผู้ใช้งานสามารถเริ่มต้นใช้งานระบบผ่าน Browser ได้โดยตรง ขณะที่ผู้พัฒนาสามารถดูแลและปรับปรุงระบบจากส่วนกลางได้ ซึ่งเป็นคุณลักษณะที่สอดคล้องกับแนวโน้มของการให้บริการซอฟต์แวร์ผ่านอินเทอร์เน็ต

แนวโน้มดังกล่าวยังสัมพันธ์กับการเติบโตของ Software as a Service หรือ SaaS ซึ่งเป็นรูปแบบการให้บริการซอฟต์แวร์ผ่านเครือข่ายอินเทอร์เน็ต โดยเอกสารอธิบายว่า SaaS เป็นรูปแบบการให้บริการซอฟต์แวร์บน Cloud ที่ผู้ใช้งานเข้าถึง Application ผ่านอินเทอร์เน็ต แทนที่จะติดตั้งและบริหารจัดการซอฟต์แวร์ทั้งหมดไว้บนอุปกรณ์ของตนเอง ตัวอย่างบริการที่ใช้รูปแบบดังกล่าว ได้แก่ Gmail, Outlook, Salesforce, Microsoft 365, Asana และ Trello

การเติบโตของ Cloud-based Application, SaaS และ PWA แสดงให้เห็นถึงทิศทางของการพัฒนา Web Application ที่มุ่งเน้นความสะดวกในการเข้าถึง ความสามารถในการรองรับผู้ใช้งานบนอุปกรณ์ที่แตกต่างกัน และการลดข้อจำกัดในการติดตั้งซอฟต์แวร์ลงบนอุปกรณ์ของผู้ใช้

ตารางที่ 2.2 การเปรียบเทียบแนวคิด Web Application, Progressive Web Application และ Mobile

คุณลักษณะ	Web Application	Progressive Web Application (PWA)	Mobile Application
ช่องทางหลักในการใช้งาน	Web Browser	Web Browser ในลักษณะ App-like	แอปพลิเคชันบนอุปกรณ์
การติดตั้ง	ไม่จำเป็น	แนวคิดเน้นการเข้าถึงผ่านเว็บและประสบการณ์คล้ายแอป	โดยทั่วไปต้องติดตั้ง
การรองรับอุปกรณ์	รองรับอุปกรณ์ที่ใช้งาน Browser ได้	รองรับอุปกรณ์ผ่านเทคโนโลยีเว็บ	รองรับตามระบบปฏิบัติการหรือ Platform
ประสบการณ์ใช้งาน	รูปแบบเว็บไซต์หรือ Web Application	มุ่งสร้างประสบการณ์ใกล้เคียง Mobile Application	ออกแบบสำหรับอุปกรณ์พกพาโดยเฉพาะ
การปรับปรุงระบบ	ปรับปรุงจากส่วนกลาง	ปรับปรุงจากระบบเว็บส่วนกลาง	ขึ้นอยู่กับรูปแบบการแจกจ่ายและการอัปเดตแอป
ต้นทุนการพัฒนา	ขึ้นอยู่กับขอบเขตและความซับซ้อน	เอกสารระบุว่าสามารถลดต้นทุนได้เมื่อเทียบกับการสร้างประสบการณ์แบบแอปในบางกรณี	ขึ้นอยู่กับจำนวนแพลตฟอร์มและแนวทางในการพัฒนา

ตารางที่ 2.2 แสดงให้เห็นว่า PWA เป็นแนวคิดที่อยู่ระหว่างการพัฒนา Web Application แบบทั่วไปและประสบการณ์การใช้งานของ Mobile Application โดยยังคงข้อดีของเว็บในด้านความสะดวกในการเข้าถึง แต่พยายามเพิ่มลักษณะการใช้งานให้มีความใกล้เคียงกับแอปพลิเคชันมากขึ้น

การออกแบบ Responsive User Interface และ User Experience

การออกแบบส่วนติดต่อผู้ใช้งานถือเป็นองค์ประกอบสำคัญของการพัฒนา Web and Mobile Application เนื่องจากผู้ใช้งานเป็นผู้ที่มีปฏิสัมพันธ์กับระบบโดยตรงผ่านองค์ประกอบต่าง ๆ ที่ปรากฏบนหน้าจอ เช่น เมนู ปุ่ม ช่องกรอกข้อมูล รูปภาพ และส่วนแสดงผลข้อมูล ดังนั้นการออกแบบระบบจึงไม่ควรพิจารณาเพียงความถูกต้องของฟังก์ชันการทำงาน แต่ต้องให้ความสำคัญกับความสะดวกและประสบการณ์ของผู้ใช้งานควบคู่กันไป

User Interface หรือ UI หมายถึงองค์ประกอบที่ใช้เป็นช่องทางในการสื่อสารระหว่างผู้ใช้งานกับระบบ โดยในบริบทของ Web and Mobile Application การออกแบบ UI จำเป็นต้องคำนึงถึงการจัดวางองค์ประกอบ รูปแบบการแสดงข้อมูล และความเหมาะสมของหน้าจอต่ออุปกรณ์ที่แตกต่างกัน ขณะที่ User Experience หรือ UX เกี่ยวข้องกับประสบการณ์โดยรวมที่ผู้ใช้งานได้รับจากการใช้ระบบ เช่น ความเข้าใจง่าย ความต่อเนื่องของการทำงาน และความสามารถในการเข้าถึงฟังก์ชันที่ต้องการได้อย่างเหมาะสม

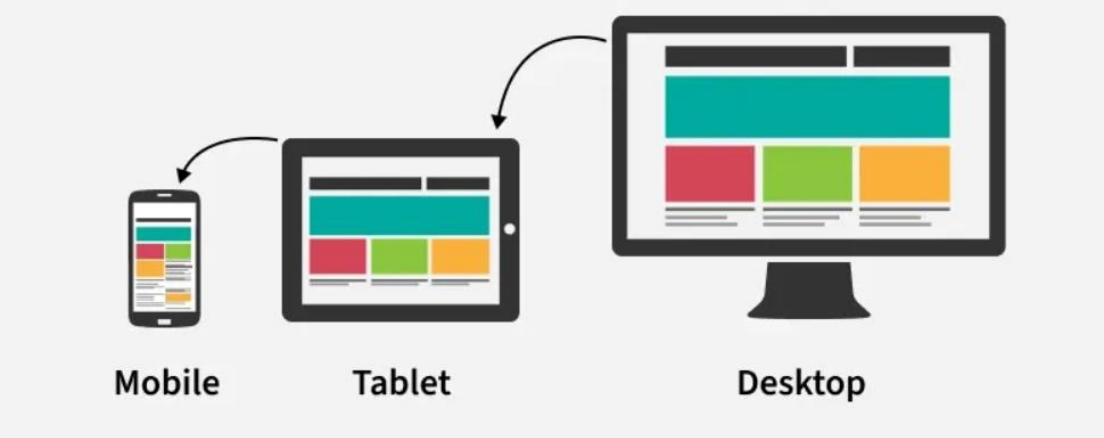
เอกสาร Web and Mobile Applications แสดงให้เห็นว่าการพัฒนา Web and Mobile Application ในปัจจุบันให้ความสำคัญกับ UI/UX ควบคู่กับคุณสมบัติด้านประสิทธิภาพและการรองรับการขยายตัวของระบบ ตัวอย่างเช่น Top Edge Technologies มีการนำเสนอการพัฒนาที่ครอบคลุมทั้ง Advanced UI/UX Design, Responsive Interfaces, Backend Systems, Performance รวมถึง Continuous Integration และ Deployment เพื่อสนับสนุนการทำงานของแอปพลิเคชันในระยะยาว

แนวคิด Responsive User Interface เป็นการออกแบบส่วนติดต่อผู้ใช้งานให้สามารถปรับรูปแบบการแสดงผลตามขนาดและลักษณะของอุปกรณ์ที่ใช้ โดยมีความสำคัญอย่างยิ่งต่อระบบที่ต้องการรองรับทั้งการใช้งานผ่านคอมพิวเตอร์และอุปกรณ์พกพา หากระบบถูกออกแบบโดยอ้างอิงขนาดหน้าจอเพียงรูปแบบเดียว อาจทำให้เกิดปัญหาในการแสดงผลเมื่อผู้ใช้เปลี่ยนไปใช้อุปกรณ์ที่มีพื้นที่หน้าจอแตกต่างกัน

เอกสารได้ยกตัวอย่างหลายกรณีที่ให้ความสำคัญกับ Responsive Design และ User Interface เช่น Izla Technologies ซึ่งมุ่งเน้นการพัฒนาเว็บไซต์และแอปพลิเคชันให้มีความสามารถในการปรับตัวต่อการใช้งาน พร้อมทั้งให้ความสำคัญกับ Usability และ Design เพื่อช่วยปรับปรุงประสบการณ์ของผู้ใช้งาน โดยบริการของบริษัทครอบคลุมทั้ง Mobile Development, Hybrid Application และ Frontend/Backend Development รวมถึงการปรับแต่ง User Interface ให้เหมาะกับความต้องการของระบบ

นอกจาก Responsive Design แล้ว ความง่ายต่อการใช้งานหรือ Usability ยังมีความสัมพันธ์กับคุณภาพของ Web and Mobile Application อย่างชัดเจน ระบบที่มีฟังก์ชันจำนวน

มากอาจไม่ได้สร้างประสบการณ์ที่ดีแก่ผู้ใช้งาน หากผู้ใช้ไม่สามารถเข้าใจขั้นตอนการทำงานหรือค้นหาฟังก์ชันที่ต้องการได้ ดังนั้นโครงสร้างหน้าจอ การจัดวางข้อมูล และลำดับการทำงานควรได้รับการออกแบบให้มีความชัดเจน



ภาพที่ 2.1 ตัวอย่าง Responsive Design

ในเอกสารมีการกล่าวถึง BytSolver ซึ่งพัฒนาตั้งแต่ Responsive Website ไปจนถึง Mobile Application และให้ความสำคัญกับการออกแบบส่วนติดต่อผู้ใช้ที่มีลักษณะ Intuitive เพื่อเพิ่ม User Engagement และ Usability ลักษณะดังกล่าวสะท้อนให้เห็นว่า UI/UX ไม่ใช่องค์ประกอบที่แยกออกจากกระบวนการพัฒนาระบบ แต่เป็นส่วนหนึ่งของการสร้างแอปพลิเคชันที่ตอบสนองต่อความต้องการของผู้ใช้งาน

สำหรับ Mobile Application การออกแบบส่วนติดต่อผู้ใช้งานมีความสำคัญเป็นพิเศษ เนื่องจากพื้นที่บนหน้าจอมีข้อจำกัดมากกว่าการใช้งานผ่านคอมพิวเตอร์ และผู้ใช้งานส่วนใหญ่โต้ตอบกับระบบผ่านหน้าจอสัมผัส ดังนั้นข้อมูลและฟังก์ชันสำคัญควรถูกจัดวางในตำแหน่งที่เหมาะสม และไม่ควรมีความซับซ้อนเกินความจำเป็น

ในขณะที่ Web Application มีข้อได้เปรียบในด้านพื้นที่หน้าจอเมื่อใช้งานผ่านคอมพิวเตอร์ แต่ระบบเดียวกันอาจถูกเปิดผ่านสมาร์ตโฟนหรือแท็บเล็ตได้ จึงจำเป็นต้องมีการออกแบบที่ยืดหยุ่น เอกสารมีตัวอย่างของ WebRange Solutions ที่ให้ความสำคัญกับ Responsive Design และ User Experience เพื่อพัฒนาเว็บไซต์ให้เหมาะสมกับการใช้งานและเพิ่มประสิทธิภาพในการเข้าถึงบริการของผู้ใช้

นอกเหนือจากรูปแบบการแสดงผลแล้ว ความสม่ำเสมอของส่วนติดต่อผู้ใช้งานยังเป็นองค์ประกอบที่ควรคำนึงถึง เช่น การใช้รูปแบบสี ปุ่ม ตัวอักษร และองค์ประกอบต่าง ๆ ในแนวทางเดียวกันตลอดทั้งระบบ ช่วยให้ผู้ใช้สามารถเรียนรู้รูปแบบการใช้งานและคาดการณ์การทำงานของระบบได้ง่ายขึ้น โดยเฉพาะระบบที่มีหลายหน้าหรือหลายฟังก์ชัน

การแสดงสถานะของระบบให้ผู้ใช้งานรับทราบเป็นอีกส่วนหนึ่งของ User Experience ตัวอย่างเช่น ในกรณีที่ระบบกำลังประมวลผลข้อมูลหรือเรียกใช้บริการจากภายนอก ควรมีองค์ประกอบที่ทำให้ผู้ใช้ทราบว่าระบบยังคงทำงานอยู่ การออกแบบลักษณะดังกล่าวช่วยลดความสับสนและทำให้ลำดับการใช้งานมีความชัดเจนยิ่งขึ้น

เอกสารยังสะท้อนให้เห็นถึงความสัมพันธ์ระหว่าง UI/UX และระบบที่สามารถรองรับการขยายตัวได้ ตัวอย่างเช่น Semexsoft ระบุถึง Mobile Solutions ที่มี User-friendly Interface, Cross-platform Support, Scalability และ Security ซึ่งแสดงให้เห็นว่าคุณภาพของแอปพลิเคชันต้องพิจารณาหลายด้านร่วมกัน ไม่ใช่เพียงความสวยงามของหน้าจอเท่านั้น

ตารางที่ 2.3 องค์ประกอบสำคัญในการออกแบบ User Interface และ User Experience สำหรับ Web and Mobile Application

องค์ประกอบ	**รายละเอียด**
Responsive Interface	การออกแบบหน้าจอให้สามารถปรับเปลี่ยนรูปแบบตามขนาดและประเภทของอุปกรณ์
Usability	การออกแบบระบบให้สามารถเข้าใจและใช้งานได้ง่าย
User-friendly Interface	การจัดองค์ประกอบของระบบให้เหมาะสมกับพฤติกรรมและความต้องการของผู้ใช้งาน
Consistency	การใช้รูปแบบสี ตัวอักษร ปุ่ม และองค์ประกอบของระบบให้เป็นไปในแนวทางเดียวกัน
Navigation	การจัดโครงสร้างการเข้าถึงหน้าหรือฟังก์ชันต่าง ๆ ให้มีความชัดเจน
User Feedback	การแสดงสถานะหรือผลตอบสนองของระบบเมื่อผู้ใช้ดำเนินการ
Performance	การออกแบบร่วมกับการพัฒนาระบบให้สามารถตอบสนองต่อผู้ใช้งานได้อย่างเหมาะสม
Scalability	การออกแบบระบบให้สามารถรองรับการเพิ่มขึ้นของผู้ใช้งานหรือฟังก์ชันในอนาคต
Security	การคำนึงถึงการปกป้องข้อมูลและการใช้งานระบบอย่างปลอดภัย

จากตารางที่ 2.3 จะเห็นได้ว่าการออกแบบ User Interface และ User Experience ของ Web and Mobile Application มีความสัมพันธ์กับหลายองค์ประกอบของระบบ ตั้งแต่การแสดงผล

จนถึงประสิทธิภาพและความปลอดภัย ดังนั้นกระบวนการพัฒนาแอปพลิเคชันจึงควรออกแบบประสบการณ์ของผู้ใช้งานไปพร้อมกับการวางโครงสร้างทางเทคนิค

สำหรับระบบ TripAI มีการนำแนวคิดดังกล่าวมาประยุกต์ใช้ในการออกแบบส่วนติดต่อผู้ใช้งาน โดยระบบต้องรองรับข้อมูลหลายประเภท เช่น แบบฟอร์มสำหรับกำหนดรายละเอียดการเดินทาง ผลลัพธ์แผนการเดินทาง รูปภาพสถานที่ และข้อมูลที่แสดงบนแผนที่ การจัดวางข้อมูลจึงต้องช่วยให้ผู้ใช้งานสามารถติดตามลำดับการใช้งานได้ง่าย

กระบวนการพัฒนา Web and Mobile Application

การพัฒนา Web and Mobile Application เป็นกระบวนการที่ประกอบด้วยหลายขั้นตอน ตั้งแต่การกำหนดความต้องการของระบบจนถึงการนำระบบไปใช้งานและการดูแลรักษาหลังการพัฒนาเสร็จสิ้น แต่ละขั้นตอนมีความสัมพันธ์กันและมีผลต่อคุณภาพของระบบที่พัฒนาขึ้น

เอกสาร *Web and Mobile Applications* แสดงให้เห็นว่าบริษัทพัฒนาซอฟต์แวร์หลายแห่งดำเนินงานในลักษณะ Full-cycle Development หรือ End-to-end Development ซึ่งครอบคลุมกระบวนการตั้งแต่การวิเคราะห์แนวคิด การวางแผน การพัฒนา ไปจนถึงการ Deploy ระบบ ตัวอย่างเช่น BariTechSol ระบุถึงการให้บริการพัฒนาซอฟต์แวร์ตั้งแต่แนวคิดจนถึงการนำระบบไปใช้งานจริง

ในขณะเดียวกัน Pioneer Logics ระบุถึงกระบวนการดำเนินโครงการที่ประกอบด้วย Requirements Gathering, Project Management, System Architecture, Software Development, Quality Assurance, Training และ Support ซึ่งแสดงให้เห็นว่าการพัฒนา Web and Mobile Application จำเป็นต้องครอบคลุมมากกว่าขั้นตอนการเขียนโปรแกรมเพียงอย่างเดียว

ขั้นตอนแรกของการพัฒนาคือ **การวิเคราะห์และรวบรวมความต้องการ (Requirement Analysis and Gathering)** ซึ่งเป็นกระบวนการศึกษาว่าระบบต้องสามารถทำอะไรได้บ้าง ผู้ใช้งานของระบบคือใคร และมีข้อกำหนดหรือข้อจำกัดใดที่ต้องนำมาพิจารณา การรวบรวมความต้องการที่ชัดเจนช่วยให้ทีมพัฒนาสามารถกำหนดขอบเขตของระบบและลดความคลาดเคลื่อนระหว่างความต้องการกับระบบที่พัฒนาขึ้น

หลังจากกำหนดความต้องการแล้วจึงเข้าสู่ **การออกแบบระบบ (System Design)** ซึ่งเป็นขั้นตอนในการกำหนดโครงสร้างและองค์ประกอบต่าง ๆ ของระบบ เช่น สถาปัตยกรรมของระบบ โครงสร้างส่วนติดต่อผู้ใช้งาน การไหลของข้อมูล และความสัมพันธ์ระหว่างองค์ประกอบต่าง ๆ ขั้นตอนนี้ช่วยให้ทีมพัฒนามีแนวทางที่ชัดเจนก่อนเข้าสู่การเขียนโปรแกรม

เอกสารกล่าวถึงการใช้ System Architecture เป็นหนึ่งในขั้นตอนสำคัญของวงจรการพัฒนา รวมทั้งการออกแบบ UI/UX และการสร้าง Prototype เพื่อใช้เก็บความคิดเห็นก่อนเริ่มพัฒนา

ระบบอย่างสมบูรณ์ การสร้างต้นแบบช่วยให้สามารถตรวจสอบรูปแบบการใช้งานและปรับปรุงแนวคิดก่อนนำไปพัฒนาเป็นระบบจริง

ขั้นตอนถัดมาคือ **การพัฒนาโปรแกรม (Implementation or Development)** ซึ่งเป็นขั้นตอนที่นำข้อกำหนดและการออกแบบมาแปลงเป็นซอฟต์แวร์ที่สามารถใช้งานได้จริง โดยผู้พัฒนาจะเลือกภาษา Framework และเทคโนโลยีให้เหมาะสมกับลักษณะของระบบ ในกรณีของ Web and Mobile Application อาจประกอบด้วยการพัฒนาส่วน Frontend, Backend และการเชื่อมต่อกับบริการหรือระบบอื่น

เอกสารแสดงให้เห็นว่าการพัฒนาสมัยใหม่มีการใช้เทคโนโลยีและ Framework หลากหลายตามวัตถุประสงค์ของระบบ ตัวอย่างเช่น Transviti มีการใช้ React Native, Flutter และ Ionic ในการพัฒนา Hybrid Mobile Application รวมถึงเทคโนโลยีสำหรับ Frontend และ Backend เช่น Angular.js, React.js, Vue.js และ Python การเลือกเทคโนโลยีจึงเป็นส่วนหนึ่งของการกำหนดแนวทางการพัฒนาให้เหมาะกับความต้องการของโครงการ

เมื่อพัฒนาระบบแล้ว ขั้นตอนที่มีความสำคัญอีกขั้นคือ **การทดสอบระบบ (Testing and Quality Assurance)** ซึ่งใช้ตรวจสอบว่าฟังก์ชันของระบบสามารถทำงานได้ตามข้อกำหนดหรือไม่ รวมถึงตรวจหาปัญหาที่อาจส่งผลต่อการใช้งาน เอกสารมีการกล่าวถึง Software Testing และ Quality Assurance ในฐานะส่วนหนึ่งของกระบวนการพัฒนาซอฟต์แวร์ เพื่อช่วยให้ผลิตภัณฑ์ได้รับการตรวจสอบคุณภาพก่อนนำไปใช้งานจริง

หลังจากการทดสอบและแก้ไขปัญหาแล้ว จะเข้าสู่ขั้นตอน **การนำระบบไปใช้งาน (Deployment)** ซึ่งเป็นการนำ Application ขึ้นสู่สภาพแวดล้อมที่ผู้ใช้งานสามารถเข้าถึงได้ เอกสารระบุถึงการนำแนวคิด Continuous Integration และ Deployment มาใช้ร่วมกับการพัฒนาระบบเพื่อช่วยสนับสนุนการนำซอฟต์แวร์เวอร์ชันใหม่ไปใช้งานและรักษาประสิทธิภาพของแอปพลิเคชัน

หลังจาก Deployment แล้ว ระบบยังจำเป็นต้องมี **การบำรุงรักษาและสนับสนุน (Maintenance and Support)** เพื่อแก้ไขปัญหาที่เกิดขึ้นหลังการใช้งาน ปรับปรุงประสิทธิภาพและเพิ่มเติมความสามารถเมื่อความต้องการของผู้ใช้งานเปลี่ยนแปลงไป เอกสารยกตัวอย่างบริการที่ครอบคลุมตั้งแต่การพัฒนาไปจนถึง Post-launch Support ซึ่งมีเป้าหมายเพื่อรักษาคุณภาพและความสามารถในการรองรับการขยายตัวของระบบ

Transviti ได้อธิบาย Product Development Lifecycle ในลักษณะที่ชัดเจน โดยครอบคลุมตั้งแต่ Analysis and Requirements Gathering, Design, Implementation, Testing, Deployment จนถึง Maintenance จึงสามารถสรุปกระบวนการพัฒนา Web and Mobile Application ได้ตามตารางที่ 2.4

ตารางที่ 2.4 ขั้นตอนการพัฒนา Web and Mobile Application

ขั้นตอน	รายละเอียด
การวิเคราะห์และรวบรวมความต้องการ	ศึกษาปัญหา ความต้องการของผู้ใช้งาน ขอบเขต และฟังก์ชันของระบบ
การออกแบบระบบ	กำหนดโครงสร้างระบบ สถาปัตยกรรม ส่วนติดต่อผู้ใช้งาน และการไหลของข้อมูล
การพัฒนาระบบ	นำแบบที่กำหนดไว้มาพัฒนาเป็นซอฟต์แวร์ด้วยภาษา Framework และเครื่องมือที่เหมาะสม
การทดสอบและประกันคุณภาพ	ตรวจสอบการทำงานของระบบ ค้นหาข้อผิดพลาด และตรวจสอบคุณภาพก่อนใช้งาน
การนำระบบไปใช้งาน	นำระบบที่ผ่านการทดสอบขึ้นสู่สภาพแวดล้อมจริงเพื่อให้ผู้ใช้งานเข้าถึงได้
การบำรุงรักษาและสนับสนุน	แก้ไขข้อผิดพลาด ปรับปรุงประสิทธิภาพ และพัฒนาระบบเพิ่มเติมหลังเปิดใช้งาน

จากตารางที่ 2.4 กระบวนการพัฒนา Web and Mobile Application มีลักษณะต่อเนื่องตั้งแต่การศึกษาความต้องการจนถึงการดูแลระบบหลังเปิดใช้งาน โดยแต่ละขั้นตอนมีความสำคัญต่อความสำเร็จของระบบ การข้ามหรือให้ความสำคัญกับขั้นตอนใดขั้นตอนหนึ่งน้อยเกินไปอาจส่งผลต่อคุณภาพและความเหมาะสมของระบบที่พัฒนาขึ้น

คุณลักษณะที่สำคัญของ Web and Mobile Application ที่มีประสิทธิภาพ

การพัฒนา Web and Mobile Application ให้สามารถตอบสนองต่อความต้องการของผู้ใช้งานได้อย่างมีประสิทธิภาพ จำเป็นต้องพิจารณาหลายองค์ประกอบร่วมกัน ทั้งด้านประสิทธิภาพ ความสามารถในการรองรับการขยายตัว ความปลอดภัย ความง่ายต่อการใช้งาน และความสามารถในการทำงานร่วมกับเทคโนโลยีอื่น โดยคุณลักษณะเหล่านี้ส่งผลโดยตรงต่อคุณภาพและความน่าเชื่อถือของระบบในระยะยาว

คุณลักษณะสำคัญประการแรกคือ **ความสามารถในการรองรับการขยายตัวของระบบ (Scalability)** ซึ่งหมายถึงความสามารถของระบบในการรองรับจำนวนผู้ใช้งาน ปริมาณข้อมูล หรือฟังก์ชันการทำงานที่เพิ่มขึ้นโดยไม่ส่งผลกระทบอย่างรุนแรงต่อประสิทธิภาพ เอกสาร *Web and Mobile Applications* แสดงให้เห็นว่าผู้ให้บริการด้านการพัฒนาซอฟต์แวร์จำนวนมากให้

ความสำคัญกับการพัฒนาโซลูชันที่สามารถขยายตัวได้ โดยเฉพาะระบบ Web, Mobile และ Cloud ที่ต้องรองรับความต้องการของผู้ใช้งานที่อาจเปลี่ยนแปลงในอนาคต

อีกองค์ประกอบหนึ่งคือ **ประสิทธิภาพของระบบ (Performance)** ระบบควรสามารถตอบสนองต่อคำสั่งของผู้ใช้งานได้อย่างเหมาะสมและลดระยะเวลารอคอยให้น้อยที่สุด โดยเฉพาะ Web Application และ Mobile Application ที่ให้บริการผ่านเครือข่ายอินเทอร์เน็ต ประสิทธิภาพจึงสัมพันธ์ทั้งกับการออกแบบระบบ การประมวลผล และโครงสร้างพื้นฐานที่รองรับการทำงาน

เอกสารยกตัวอย่าง Top Edge Technologies ซึ่งให้ความสำคัญกับ Backend Systems ที่มีความแข็งแรง ประสิทธิภาพของระบบ การออกแบบ Responsive Interface รวมถึงแนวทาง Continuous Integration และ Deployment เพื่อรักษาคุณภาพและความสามารถในการขยายตัวของแอปพลิเคชัน

ความปลอดภัย (Security) เป็นอีกหนึ่งองค์ประกอบสำคัญของ Web and Mobile Application เนื่องจากระบบจำนวนมากมีการจัดเก็บหรือประมวลผลข้อมูลของผู้ใช้งาน การออกแบบจึงต้องคำนึงถึงการป้องกันการเข้าถึงข้อมูลโดยไม่ได้รับอนุญาต รวมถึงการจัดการข้อมูลอย่างเหมาะสม เอกสารกล่าวถึงหลายบริษัทที่นำ Security มาเป็นส่วนหนึ่งของกระบวนการพัฒนา เช่น การใช้ Security Audit, Penetration Testing และการออกแบบระบบที่คำนึงถึงการปกป้องข้อมูล

ในบางกรณี ความปลอดภัยยังเชื่อมโยงกับมาตรฐานและกฎระเบียบด้านข้อมูล ตัวอย่างเช่น Evamp & Saanga ให้ความสำคัญกับ Information Security และแนวทางที่สอดคล้องกับ GDPR เพื่อช่วยปกป้องข้อมูลและปรับปรุงกระบวนการอย่างต่อเนื่อง การรักษาความปลอดภัยจึงไม่ได้เป็นเพียงประเด็นด้านเทคนิค แต่ยังเกี่ยวข้องกับการจัดการข้อมูลและข้อกำหนดที่เกี่ยวข้องกับความเป็นส่วนตัว

ความง่ายต่อการใช้งาน (Usability) และ **ประสบการณ์ของผู้ใช้งาน (User Experience)** เป็นองค์ประกอบที่มีผลโดยตรงต่อการยอมรับของระบบ หากระบบมีความสามารถสูง แต่ผู้ใช้งานไม่สามารถเข้าใจวิธีใช้งานหรือเข้าถึงฟังก์ชันได้อย่างสะดวก ประสิทธิผลของแอปพลิเคชันอาจลดลง เอกสารแสดงตัวอย่างบริษัทที่เน้น User-friendly Interface และการออกแบบที่ช่วยเพิ่ม Engagement และ Usability อย่างต่อเนื่อง

นอกจากนี้ Web and Mobile Application ที่มีประสิทธิภาพควรมี **ความยืดหยุ่นในการรองรับอุปกรณ์และแพลตฟอร์ม (Compatibility and Flexibility)** เนื่องจากผู้ใช้งานอาจเข้าถึงระบบจากอุปกรณ์ ระบบปฏิบัติการ หรือขนาดหน้าจอที่แตกต่างกัน การออกแบบ Responsive Interface และการเลือกเทคโนโลยีที่รองรับหลายแพลตฟอร์มจึงเป็นแนวทางที่ได้รับความสำคัญเพิ่มขึ้น

Izla Technologies เป็นตัวอย่างหนึ่งที่พัฒนา Mobile Application สำหรับ Android และ iOS รวมถึง Hybrid Application และ Frontend/Backend Development โดยให้ความสำคัญกับ Responsiveness, User Interface และความสามารถในการเชื่อมต่อกับ Cloud และ API-driven Architecture

ความน่าเชื่อถือ (Reliability) เป็นอีกปัจจัยหนึ่งที่ช่วยให้ผู้ใช้งานสามารถใช้ระบบได้อย่างต่อเนื่อง ระบบควรสามารถทำงานได้ตามที่กำหนดและลดโอกาสเกิดข้อผิดพลาดที่รบกวนการใช้งาน โดยการทดสอบซอฟต์แวร์ การประกันคุณภาพ และการบำรุงรักษาหลังนำระบบไปใช้งานเป็นกระบวนการที่ช่วยเพิ่มความน่าเชื่อถือของแอปพลิเคชัน เอกสารกล่าวถึงการนำ Software Testing, DevOps และ Quality Assurance มาใช้เพื่อให้ผลิตภัณฑ์ได้รับการตรวจสอบก่อนส่งมอบและสามารถรักษาคุณภาพของระบบได้อย่างต่อเนื่อง

อีกคุณลักษณะที่สำคัญคือ **ความสามารถในการบำรุงรักษา (Maintainability)** เนื่องจาก Web and Mobile Application จำเป็นต้องได้รับการแก้ไข ปรับปรุง หรือเพิ่มเติมความสามารถตามความต้องการที่เปลี่ยนแปลง การพัฒนาระบบที่มีโครงสร้างชัดเจนและมีแนวทางในการบริหารจัดการ Software Lifecycle จะช่วยให้การปรับปรุงระบบในอนาคตสามารถดำเนินการได้อย่างมีประสิทธิภาพ

เอกสารแสดงให้เห็นว่าแนวทางการพัฒนาสมัยใหม่ไม่ได้จบลงเมื่อระบบถูกนำไปใช้งาน แต่ยังครอบคลุมถึงการดูแลและสนับสนุนหลังการเปิดใช้งาน ตัวอย่างเช่น Transviti อธิบายวงจรการพัฒนาที่เริ่มจากการวิเคราะห์และเก็บความต้องการ การออกแบบ การพัฒนา การทดสอบ การนำไปใช้งาน และการบำรุงรักษา

ตารางที่ 2.5 คุณลักษณะสำคัญของ Web and Mobile Application ที่มีประสิทธิภาพ

คุณลักษณะ	รายละเอียด
Scalability	ความสามารถในการรองรับการเพิ่มขึ้นของผู้ใช้งาน ข้อมูล และฟังก์ชันของระบบ
Performance	ความสามารถในการประมวลผลและตอบสนองต่อผู้ใช้งานได้อย่างเหมาะสม
Security	การป้องกันข้อมูลและระบบจากการเข้าถึงหรือการใช้งานที่ไม่ได้รับอนุญาต
Usability	ความง่ายในการเรียนรู้ เข้าใจ และใช้งานระบบ
User Experience	ประสบการณ์โดยรวมของผู้ใช้งานในการมีปฏิสัมพันธ์กับระบบ

คุณลักษณะ	รายละเอียด
Compatibility	ความสามารถในการทำงานบนอุปกรณ์หรือแพลตฟอร์มที่แตกต่างกัน
Reliability	ความสามารถในการทำงานอย่างถูกต้องและต่อเนื่อง
Maintainability	ความสะดวกในการแก้ไข ปรับปรุง และเพิ่มเติมความสามารถของระบบ
Responsiveness	ความสามารถในการปรับส่วนติดต่อให้เหมาะสมกับขนาดหน้าจอและอุปกรณ์

จากตารางที่ 2.5 จะเห็นได้ว่าคุณภาพของ Web and Mobile Application ไม่ได้ขึ้นอยู่กับองค์ประกอบด้านใดด้านหนึ่ง แต่เป็นผลจากการออกแบบและพัฒนาหลายด้านร่วมกัน ตั้งแต่ประสิทธิภาพของระบบ ความปลอดภัย ไปจนถึงประสบการณ์ของผู้ใช้งาน ดังนั้นกระบวนการพัฒนาจึงจำเป็นต้องคำนึงถึงคุณลักษณะเหล่านี้ตั้งแต่ขั้นตอนการวิเคราะห์ ออกแบบ พัฒนา ทดสอบ และบำรุงรักษา

แนวโน้มของ Web and Mobile Application ในปัจจุบัน

อุตสาหกรรม Web and Mobile Application มีการเปลี่ยนแปลงอย่างต่อเนื่องตามพัฒนาการของเทคโนโลยี พฤติกรรมของผู้ใช้งาน และความต้องการขององค์กร เอกสาร *Web and Mobile Applications* ระบุว่าอุตสาหกรรมดังกล่าวกำลังเติบโตจากหลายปัจจัย ได้แก่ Digital Transformation การเพิ่มขึ้นของการใช้ Smartphone และความต้องการ Software Solution ที่สามารถรองรับการขยายตัวได้

แนวโน้มสำคัญประการหนึ่งคือ **การเพิ่มขึ้นของการใช้งานอุปกรณ์พกพา** สมาร์ตโฟนกลายเป็นอุปกรณ์หลักในการเข้าถึงข้อมูลและบริการออนไลน์ของผู้ใช้งานจำนวนมาก โดยเฉพาะในพื้นที่ที่มีพฤติกรรมการใช้งานแบบ Mobile-first การเปลี่ยนแปลงนี้ส่งผลให้การออกแบบระบบต้องคำนึงถึงหน้าจอขนาดเล็ก ความสะดวกในการใช้งาน และประสบการณ์บนอุปกรณ์พกพามากขึ้น

เอกสารระบุว่าในบางพื้นที่ของ Sub-Saharan Africa การใช้งาน Mobile Web มีสัดส่วนสูงกว่า Desktop อย่างชัดเจน ขณะที่ Mobile Application โดยเฉพาะด้าน Fintech มีส่วนช่วยเติมเต็มช่องว่างของโครงสร้างพื้นฐานด้านการเงิน แสดงให้เห็นว่าการใช้งาน Web และ Mobile อาจมีบทบาทแตกต่างกันตามบริบทของแต่ละภูมิภาค

แนวโน้มอีกประการคือ **การพัฒนาแบบ Cross-platform** ซึ่งเกิดจากความต้องการลดระยะเวลาและค่าใช้จ่ายในการพัฒนาแอปพลิเคชันสำหรับหลายแพลตฟอร์ม เอกสารระบุว่าความต้องการ Cross-platform Development เพิ่มขึ้นเนื่องจากองค์กรต้องการนำ Mobile Application

ไปใช้งานบน Android และ iOS ได้อย่างมีประสิทธิภาพ โดยมี Framework ที่สำคัญ ได้แก่ Flutter, React Native และ Xamarin

นอกจากนี้ **Progressive Web Application (PWA)** ยังเป็นแนวโน้มที่ได้รับความสนใจ เนื่องจากสามารถนำเสนอประสบการณ์ที่มีลักษณะใกล้เคียงกับ Mobile Application ผ่านเทคโนโลยีเว็บ เอกสารระบุว่า PWA ได้รับความนิยมเพิ่มขึ้นจากความสามารถในการสร้าง App-like Experience และมีข้อได้เปรียบด้านต้นทุนในบางบริบท

อีกแนวโน้มหนึ่งที่มีความสำคัญคือ **Cloud Computing และ Software as a Service (SaaS)** ซึ่งเปลี่ยนรูปแบบการให้บริการซอฟต์แวร์จากการติดตั้งภายในอุปกรณ์ไปสู่การเข้าถึงผ่านเครือข่ายอินเทอร์เน็ต เอกสารอธิบายว่า SaaS เป็นรูปแบบการให้บริการซอฟต์แวร์บน Cloud ที่ช่วยให้ผู้ใช้งานสามารถเข้าถึง Application โดยไม่จำเป็นต้องติดตั้งและบริหารจัดการซอฟต์แวร์ทั้งหมดด้วยตนเอง

การเติบโตของ SaaS ยังสัมพันธ์กับการเปลี่ยนแปลงทางดิจิทัลขององค์กร การทำงานระยะไกล และความสามารถในการขยายตัวของ Cloud-native Architecture โดยตลาด SaaS มีแนวโน้มขยายตัวอย่างรวดเร็วตามความต้องการใช้บริการซอฟต์แวร์ผ่านอินเทอร์เน็ต

Artificial Intelligence (AI) และ Machine Learning (ML) เป็นอีกหนึ่งเทคโนโลยีที่ถูกนำมาเชื่อมต่อกับ Web and Mobile Application มากขึ้น แอปพลิเคชันไม่ได้มีหน้าที่เพียงรับข้อมูลและแสดงผล แต่สามารถนำ AI มาใช้ในการวิเคราะห์ข้อมูล ระบบอัตโนมัติ การพยากรณ์ และสร้างบริการที่มีความซับซ้อนมากขึ้น

เอกสารกล่าวถึงการเติบโตของนวัตกรรมด้าน Artificial Intelligence ภายในตลาด Mobile Application และยกตัวอย่างแอปพลิเคชันที่นำ AI มาเป็นส่วนหนึ่งของผลิตภัณฑ์และบริการ นอกจากนี้บริษัทพัฒนาซอฟต์แวร์หลายแห่งในเอกสารยังให้บริการด้าน Generative AI, AI/ML Development และระบบ Automation ควบคู่กับการพัฒนา Web and Mobile Application

ตัวอย่างเช่น Cogent Labs ใช้เทคโนโลยี Generative AI ร่วมกับเทคโนโลยี Web และ Software Development เพื่อสร้างระบบอัจฉริยะและระบบอัตโนมัติ ขณะที่ TenX ให้ความสำคัญกับ Generative AI, Data Analytics, Cloud DevOps และ Robotic Process Automation เพื่อประยุกต์ใช้ในกระบวนการทางธุรกิจ

อีกแนวโน้มหนึ่งคือ **Low-code และ No-code Development** ซึ่งช่วยลดความซับซ้อนของกระบวนการพัฒนาแอปพลิเคชัน เอกสารระบุว่าแพลตฟอร์ม Low-code/No-code ช่วยให้การพัฒนา Web and Mobile Application สามารถทำได้รวดเร็วขึ้น ลดต้นทุน และเพิ่มความคล่องตัวในการพัฒนาระบบที่เหมาะกับความต้องการเฉพาะขององค์กร

แนวโน้มเหล่านี้แสดงให้เห็นว่า Web and Mobile Application ไม่ได้พัฒนาในลักษณะที่แยกออกจากเทคโนโลยีอื่น แต่มีการผสานเข้ากับ Cloud, Artificial Intelligence, Data Analytics, Automation และเทคโนโลยีการพัฒนาแบบใหม่อย่างต่อเนื่อง

ตารางที่ 2.6 แนวโน้มสำคัญของ Web and Mobile Application

แนวโน้ม	ลักษณะสำคัญ
Mobile-first	ให้ความสำคัญกับการใช้งานผ่านสมาร์ตโฟนและอุปกรณ์พกพา
Cross-platform Development	พัฒนาแอปพลิเคชันให้สามารถรองรับหลายแพลตฟอร์ม
Progressive Web Application	ใช้เทคโนโลยีเว็บเพื่อสร้างประสบการณ์ใกล้เคียง Mobile Application
Cloud Computing	ใช้โครงสร้างพื้นฐานและบริการผ่านระบบ Cloud
Software as a Service	ให้บริการ Application ผ่านอินเทอร์เน็ต
Artificial Intelligence	เพิ่มความสามารถด้านการวิเคราะห์ การสร้างเนื้อหา และระบบอัตโนมัติ
Low-code/No-code	ลดความซับซ้อนและระยะเวลาในการพัฒนา Application
Responsive Design	รองรับการแสดงผลบนอุปกรณ์และขนาดหน้าจอที่แตกต่างกัน

จากตารางที่ 2.6 จะเห็นได้ว่าแนวโน้มของ Web and Mobile Application มุ่งเน้นไปที่ความยืดหยุ่น ความสะดวกในการเข้าถึง การลดข้อจำกัดของแพลตฟอร์ม และการนำเทคโนโลยีอัจฉริยะเข้ามาเสริมความสามารถของระบบ แนวโน้มเหล่านี้มีแนวโน้มที่จะมีบทบาทเพิ่มขึ้นตามการเปลี่ยนแปลงของพฤติกรรมผู้ใช้งานและความต้องการขององค์กรในยุคดิจิทัล

การประยุกต์ใช้ Web and Mobile Application ในอุตสาหกรรมต่าง ๆ

Web and Mobile Application ถูกนำไปประยุกต์ใช้ในอุตสาหกรรมจำนวนมาก เนื่องจากสามารถออกแบบและปรับเปลี่ยนให้เหมาะสมกับลักษณะงานและความต้องการที่แตกต่างกัน เอกสาร *Web and Mobile Applications* แสดงตัวอย่างการนำเทคโนโลยีดังกล่าวไปใช้ในด้านการเงิน สุขภาพ การศึกษา พาณิชย์อิเล็กทรอนิกส์ การท่องเที่ยว การขนส่ง และระบบบริหารจัดการองค์กร

ในด้าน **การศึกษา** Web and Mobile Application สามารถใช้เป็นแพลตฟอร์มจัดการการเรียนรู้ ระบบจัดการข้อมูลนักเรียน และระบบการเรียนรู้ออนไลน์ เอกสารยกตัวอย่าง CECOS

Innovations ซึ่งพัฒนาระบบสำหรับบริหารจัดการการเข้าเรียนของนักศึกษาและแพลตฟอร์มที่เชื่อมโยงนักศึกษา นักลงทุน และภาคอุตสาหกรรม

ในด้าน **สุขภาพและการแพทย์** มีการนำ Web and Mobile Application มาใช้สำหรับการบริหารข้อมูลผู้ป่วย การให้บริการทางการแพทย์ และการเชื่อมต่อระหว่างผู้ใช้กับผู้เชี่ยวชาญ ตัวอย่างหนึ่งในเอกสารคือ Focusteck ซึ่งพัฒนา FixAFib ในรูปแบบ Mobile และ Web Platform เพื่อเชื่อมโยงผู้ป่วยกับผู้เชี่ยวชาญ

ด้าน **พาณิชย์อิเล็กทรอนิกส์** มีการนำ Web Application และ Mobile Application มาใช้ในการแสดงสินค้า การทำธุรกรรม การจัดการคำสั่งซื้อ และการเชื่อมต่อระบบชำระเงิน เอกสารกล่าวถึง Eweberinc ซึ่งพัฒนา E-commerce และ Mobile Commerce Solutions โดยใช้เทคโนโลยี เช่น WordPress, OpenCart และ Shopify รวมถึงการเชื่อมต่อระบบการชำระเงิน

ในด้าน **การเงินและ Fintech** แอปพลิเคชันถูกใช้เพื่อสนับสนุนการให้บริการด้านการเงิน การชำระเงิน และการเข้าถึงบริการธนาคารผ่านอุปกรณ์พกพา เอกสารกล่าวถึงการเติบโตของ Mobile Banking และ Fintech Application ซึ่งมีบทบาทในการเพิ่มการเข้าถึงบริการทางการเงินและสนับสนุนกิจกรรมทางเศรษฐกิจดิจิทัล

ด้าน **การท่องเที่ยวและการเดินทาง** Web and Mobile Application สามารถนำมาใช้ในการจัดการข้อมูลการเดินทาง การจอง การแสดงข้อมูลสถานที่ และการให้บริการนักท่องเที่ยว เอกสารยกตัวอย่าง Hayat Tech ซึ่งพัฒนาระบบ Bus Booking ที่สามารถตรวจสอบตารางเวลา จองตั๋ว และจัดการการสำรองที่นั่งผ่านระบบออนไลน์

นอกจากนี้ Technology Bench ยังให้บริการด้าน Travel Tech และ Web and Mobile App Development และมีผลิตภัณฑ์ที่เกี่ยวข้องกับบริการด้านการเดินทาง เช่น Smartdestinations.ai แสดงให้เห็นว่าเทคโนโลยี Web, Mobile และ Artificial Intelligence สามารถนำมาผสานกันเพื่อพัฒนาบริการในอุตสาหกรรมการท่องเที่ยวได้

ในด้าน **การขนส่งและโลจิสติกส์** Web and Mobile Application ถูกนำมาใช้เพื่อสนับสนุนการจัดการเส้นทาง การบริหารคลังสินค้า และการประสานงานระหว่างผู้เกี่ยวข้อง ตัวอย่างเช่น Vaival Technologies ให้บริการระบบสำหรับ Route Optimization และ Warehouse Automation รวมถึงการใช้ข้อมูลแบบ Real-time เพื่อเพิ่มประสิทธิภาพการดำเนินงาน

ในด้าน **การบริหารจัดการองค์กร** มีการพัฒนาระบบ Enterprise Resource Planning, Customer Relationship Management และ Management System ในรูปแบบ Web และ Mobile เพื่อช่วยจัดการข้อมูลและกระบวนการต่าง ๆ ภายในองค์กร เอกสารยกตัวอย่าง The Next Rex ซึ่งพัฒนาระบบ School Management, ERP, CRM, Hospital Management, Restaurant Management และ Inventory Management เป็นต้น

ตารางที่ 2.7 ตัวอย่างการประยุกต์ใช้ Web and Mobile Application ในอุตสาหกรรมต่าง ๆ

อุตสาหกรรม	ตัวอย่างการประยุกต์ใช้
การศึกษา	ระบบจัดการการเรียนรู้ ระบบบริหารข้อมูลนักเรียน และแพลตฟอร์มการศึกษา
สุขภาพ	ระบบข้อมูลผู้ป่วย แพลตฟอร์มเชื่อมต่อผู้ป่วยกับผู้เชี่ยวชาญ และ Telemedicine
พาณิชย์อิเล็กทรอนิกส์	ระบบร้านค้าออนไลน์ การจัดการสินค้า คำสั่งซื้อ และการชำระเงิน
การเงิน	Mobile Banking, Fintech และระบบการชำระเงินดิจิทัล
การท่องเที่ยว	ระบบจอง การจัดการข้อมูลการเดินทาง และบริการ Travel Tech
การขนส่งและโลจิสติกส์	การวางเส้นทาง การจัดการคลังสินค้า และการติดตามข้อมูลแบบ Real-time
การบริหารองค์กร	ERP, CRM, Management System และระบบบริหารกระบวนการภายในองค์กร

จากตารางที่ 2.7 จะเห็นได้ว่า Web and Mobile Application มีความสามารถในการประยุกต์ใช้ในขอบเขตที่กว้าง และสามารถออกแบบให้เหมาะสมกับวัตถุประสงค์เฉพาะของแต่ละอุตสาหกรรมได้ การพัฒนาเทคโนโลยี Cloud, Artificial Intelligence, Data Analytics และระบบ Cross-platform ยังช่วยเพิ่มขอบเขตความสามารถของ Web and Mobile Application ให้รองรับการใช้งานที่มีความซับซ้อนมากยิ่งขึ้น**(Pakistan Software Export Board [PSEB], 2025)**

2.1.2 Client-Server Architecture

ความหมายของ Client-Server Architecture

Client-Server Architecture หรือสถาปัตยกรรมแบบไคลเอนต์-เซิร์ฟเวอร์ เป็นรูปแบบสถาปัตยกรรมของระบบคอมพิวเตอร์ที่แบ่งหน้าที่การทำงานออกเป็นสองส่วนสำคัญ ได้แก่ Client ซึ่งเป็นส่วนที่ร้องขอบริการหรือทรัพยากร และ Server ซึ่งเป็นส่วนที่รับคำขอ ประมวลผล และส่งผลลัพธ์กลับไปยัง Client โดยการติดต่อสื่อสารระหว่างทั้งสองส่วนสามารถเกิดขึ้นผ่านระบบเครือข่ายหรืออินเทอร์เน็ต

Yadav และ Singh (2009) อธิบายว่า Client-Server เป็นแบบจำลองการประมวลผลที่แบ่งหน้าที่ระหว่างกระบวนการอิสระสองประเภท คือ Client และ Server โดย Client ทำหน้าที่ร้องขอบริการเฉพาะจาก Server ส่วน Server เป็นกระบวนการที่ให้บริการตามคำขอของ Client ทั้งสอง

กระบวนการอาจอยู่ภายในคอมพิวเตอร์เครื่องเดียวกันหรืออยู่บนคอมพิวเตอร์คนละเครื่องที่เชื่อมต่อผ่านเครือข่ายก็ได้

แนวคิดดังกล่าวแสดงให้เห็นว่า Client และ Server ไม่ได้หมายถึงฮาร์ดแวร์เพียงอย่างเดียว แต่สามารถหมายถึงกระบวนการหรือซอฟต์แวร์ที่มีบทบาทแตกต่างกันภายในระบบได้ คอมพิวเตอร์เครื่องหนึ่งจึงสามารถทำหน้าที่เป็น Client หรือ Server ได้ขึ้นอยู่กับซอฟต์แวร์และลักษณะการทำงานที่กำหนดไว้ Yadav และ Singh (2009) ยังกล่าวว่า Client-Server ไม่ควรถูกมองว่าเป็นเพียงฮาร์ดแวร์หรือซอฟต์แวร์ชนิดใดชนิดหนึ่ง แต่เป็นการทำงานร่วมกันของอุปกรณ์คอมพิวเตอร์โครงสร้างพื้นฐาน และโปรแกรมซอฟต์แวร์เพื่อดำเนินงานทางคอมพิวเตอร์ร่วมกัน

Nyabuto (2024) ให้ความหมายในลักษณะเดียวกันว่า Client-Server Architecture เป็น Software Model ที่ใช้สำหรับการให้บริการทรัพยากรและการตอบสนองคำขอผ่านเครือข่าย โดย Client จะส่งคำขอทรัพยากรผ่านเครือข่าย ขณะที่ Server รับคำขอนั้น ประมวลผล และตอบกลับไปยัง Client รูปแบบดังกล่าวยังสามารถรองรับผู้ใช้งานหลายรายในการเข้าถึงและใช้ทรัพยากรร่วมกันได้ในเวลาเดียวกัน

ในระบบ Client-Server สามารถมี Client ได้ตั้งแต่หนึ่งเครื่องขึ้นไป และสามารถเชื่อมต่อกับ Server หนึ่งเครื่องหรือหลายเครื่องเพื่อร่วมกันให้บริการตามคำขอ Server มักมีหน้าที่จัดเก็บข้อมูลหรือรันโปรแกรมที่ใช้ในการประมวลผลคำขอ ในขณะที่การสื่อสารระหว่าง Client และ Server อาศัยโพรโทคอลมาตรฐาน เช่น Hypertext Transfer Protocol (HTTP), File Transfer Protocol (FTP) และ Simple Mail Transfer Protocol (SMTP)

เอกสาร *Client-server Architecture* อธิบายเพิ่มเติมว่า Server เป็นส่วนที่โฮสต์ จัดส่ง และจัดการทรัพยากรหรือบริการส่วนใหญ่ที่ Client ต้องการใช้งาน โดย Client หนึ่งเครื่องหรือหลายเครื่องจะเชื่อมต่อกับ Server ส่วนกลางผ่านเครือข่ายหรืออินเทอร์เน็ต จึงเรียกสถาปัตยกรรมลักษณะนี้ว่า Network Computing Model หรือ Client-Server Network ได้เช่นกัน

จากแนวคิดของทั้งสามแหล่งจึงสามารถสรุปได้ว่า Client-Server Architecture เป็นการแบ่งความรับผิดชอบของระบบออกจากกันอย่างชัดเจน โดย Client มุ่งเน้นการติดต่อกับผู้ใช้และการร้องขอบริการ ส่วน Server มุ่งเน้นการประมวลผล การจัดการข้อมูล และการให้บริการ การแบ่งหน้าที่ดังกล่าวช่วยให้ทรัพยากรส่วนกลางสามารถถูกใช้งานร่วมกันโดย Client หลายราย และเป็นพื้นฐานสำคัญของระบบคอมพิวเตอร์ที่ทำงานผ่านเครือข่าย

องค์ประกอบของ Client-Server Architecture

Client-Server Architecture ประกอบด้วยองค์ประกอบหลายส่วนที่ทำงานร่วมกันเพื่อให้การร้องขอและการให้บริการสามารถดำเนินการได้อย่างถูกต้อง โดยองค์ประกอบพื้นฐานที่สำคัญ

ประกอบด้วย Client, Server และ Communication Middleware หรือระบบการสื่อสารระหว่าง Client และ Server

Yadav และ Singh (2009) แบ่งองค์ประกอบของ Client-Server Architecture ออกเป็น 3 ส่วนหลัก ได้แก่ Hardware ซึ่งประกอบด้วยอุปกรณ์ฝั่ง Client และ Server, Software ที่ทำให้ฮาร์ดแวร์สามารถทำงานตามหน้าที่ และ Communication Middleware ซึ่งทำหน้าที่เชื่อมโยง Hardware และ Software ผ่านระบบเครือข่าย

Client คือกระบวนการหรือโปรแกรมที่ร้องขอบริการจาก Server โดยทั่วไปถือเป็นส่วน Front-end เนื่องจากเป็นส่วนที่ผู้ใช้งานมีปฏิสัมพันธ์โดยตรง Client จะรับข้อมูลจากผู้ใช้งาน ตรวจสอบหรือประมวลผลข้อมูลในระดับหนึ่ง และสร้างคำขอเพื่อส่งไปยัง Server ตามบริการที่ต้องการ เอกสาร *Client-server Architecture* อธิบาย Client ว่าเป็นซอฟต์แวร์หรือ Application ที่รับข้อมูล Input และส่ง Request ไปยัง Server

Yadav และ Singh (2009) ระบุว่าหน้าที่ของ Client สามารถครอบคลุมการจัดการ User Interface การรับและตรวจสอบรูปแบบข้อมูลจากผู้ใช้งาน การประมวลผล Application Logic และการสร้างคำขอไปยังฐานข้อมูลหรือ Server Client จึงทำหน้าที่เป็นจุดเริ่มต้นของการติดต่อสื่อสารในระบบ และโดยทั่วไปจะเป็นฝ่ายเริ่มต้นการสนทนากับ Server ก่อนเสมอ

Server คือกระบวนการหรือโปรแกรมที่ทำหน้าที่ให้บริการแก่ Client โดยรับคำขอที่ส่งเข้ามา ตรวจสอบ ประมวลผล และส่งผลลัพธ์กลับไปยัง Client Server จึงมักเรียกว่า Back-end ของระบบ เนื่องจากทำหน้าที่อยู่เบื้องหลังและสามารถรองรับคำขอจาก Client หลายรายพร้อมกันได้

Server สามารถให้บริการได้หลายประเภทตามวัตถุประสงค์ของระบบ เช่น File Server สำหรับจัดเก็บและแบ่งปันไฟล์, Print Server สำหรับจัดการเครื่องพิมพ์, Application Server สำหรับประมวลผล Application, Mail Server สำหรับจัดการอีเมล, Web Server สำหรับให้บริการข้อมูลผ่านเว็บ และ Database Server สำหรับจัดเก็บและให้บริการข้อมูลจากฐานข้อมูล

โดยเฉพาะ Database Server มีหน้าที่รับคำขอข้อมูล ประมวลผลคำขอ และส่งเฉพาะข้อมูลที่ตรงกับความต้องการกลับไปยัง Client แนวทางดังกล่าวช่วยลดความจำเป็นในการส่งข้อมูลทั้งหมดผ่านเครือข่ายและช่วยให้การใช้ทรัพยากรมีประสิทธิภาพมากขึ้น

องค์ประกอบที่สามคือ **Communication Middleware** ซึ่งเป็นซอฟต์แวร์ที่ช่วยให้ Client และ Server ติดต่อสื่อสารกันในสภาพแวดล้อมแบบกระจาย Middleware ทำหน้าที่เป็นตัวกลางในการส่งข้อมูลและข้อมูลควบคุม รวมถึงช่วยซ่อนรายละเอียดของระบบเครือข่ายหรือฐานข้อมูลจาก Application ฝั่ง Client

Middleware มีบทบาทสำคัญในการทำให้ระบบสามารถทำงานร่วมกับ Network Protocol หรือ Database Server ที่แตกต่างกันได้ โดยช่วยลดความจำเป็นที่โปรแกรมฝั่ง Client ต้องเขียนโค้ด

เฉพาะสำหรับฐานข้อมูลหรือโพรโทคอลแต่ละประเภท Yadav และ Singh (2009) ระบุว่าการใช้ Database Middleware สามารถสร้าง Network Independence และ Database Server Independence ซึ่งช่วยให้ Front-end Application เข้าถึงข้อมูลจาก Server หลายประเภทได้โดยไม่ต้องผูกติดกับรายละเอียดเฉพาะของแต่ละระบบ

เอกสาร Client-server Architecture ยังกล่าวถึง **Load Balancer** ซึ่งเป็นองค์ประกอบที่ช่วยกระจาย Network Traffic หรือคำขอที่เข้ามาไปยัง Backend Server หลายเครื่อง เพื่อช่วยเพิ่มประสิทธิภาพในการใช้ทรัพยากรและลดภาระที่เกิดขึ้นกับ Server เครื่องใดเครื่องหนึ่ง

ตารางที่ 2.8 องค์ประกอบพื้นฐานของ Client-Server Architecture

องค์ประกอบ	หน้าที่
Client	รับข้อมูลจากผู้ใช้ เริ่มต้น Request และรับผลลัพธ์จาก Server
Server	รับและประมวลผล Request จัดการทรัพยากร และส่ง Response กลับไปยัง Client
Communication Middleware	เป็นตัวกลางในการติดต่อสื่อสารและรับส่งข้อมูลระหว่าง Client และ Server
Network Protocol	กำหนดรูปแบบและกฎในการรับส่งข้อมูล เช่น TCP/IP และ HTTP
Database Server	จัดเก็บ ประมวลผล และให้บริการข้อมูลตามคำขอ
Load Balancer	กระจาย Request ไปยัง Server หลายเครื่องเพื่อลดภาระและเพิ่มประสิทธิภาพ

จากตารางที่ 2.8 จะเห็นได้ว่า Client-Server Architecture ไม่ได้ประกอบด้วย Client และ Server เพียงสองส่วนเท่านั้น แต่ยังต้องอาศัยองค์ประกอบด้านเครือข่ายและซอฟต์แวร์ตัวกลางในการสนับสนุนการสื่อสาร โดยแต่ละองค์ประกอบมีหน้าที่แตกต่างกันแต่ทำงานร่วมกันเป็นระบบเดียว

รูปแบบการทำงานและการสื่อสารระหว่าง Client และ Server

กลไกพื้นฐานในการทำงานของ Client-Server Architecture มีลักษณะเป็น Request-Response กล่าวคือ Client เป็นผู้เริ่มต้นส่งคำขอไปยัง Server เมื่อ Server ได้รับคำขอจะดำเนินการตรวจสอบและประมวลผล หลังจากนั้นจึงส่งผลลัพธ์หรือ Response กลับไปยัง Client

Nyabuto (2024) ระบุว่าคำขอจะเริ่มต้นจาก Client และถูกส่งผ่านช่องทางการสื่อสาร เช่น Network ไปยัง Server โดย Client อาจอยู่ในรูปของคอมพิวเตอร์ที่ใช้ Web Browser, Mobile Application หรือ Application ประเภทอื่นที่สามารถร้องขอทรัพยากรหรือบริการได้ ในขณะที่ Server มีหน้าที่รับ ประมวลผล และตอบกลับคำขอดังกล่าว

เอกสาร *Client-server Architecture* แสดงลำดับการทำงานของระบบในรูปแบบที่มี Load Balancer และ Database โดยเริ่มจาก Client ส่งคำขอไปยัง Server จากนั้น Load Balancer จะเลือก Server ที่เหมาะสม Server ทำการประมวลผลและอาจส่งคำขอไปยัง Database เมื่อ Database ส่งข้อมูลกลับ Server จะประมวลผลข้อมูลดังกล่าวและส่งผลลัพธ์กลับไปยัง Client ก่อนที่จะเริ่มกระบวนการใหม่เมื่อมีคำขอถัดไป
ลำดับการทำงานดังกล่าวสามารถสรุปได้ดังนี้

ตารางที่ 2.9 ลำดับการทำงานของ Client-Server Architecture

ลำดับ	กระบวนการ
1	Client สร้างและส่ง Request
2	Request ถูกส่งผ่าน Network หรือ Middleware
3	Server รับและตรวจสอบ Request
4	Server ประมวลผลคำขอ และเรียกใช้ข้อมูลหรือบริการที่จำเป็น
5	หากจำเป็น Server ติดต่อ Database หรือ Server อื่น
6	Server สร้างผลลัพธ์หรือ Response
7	Response ถูกส่งกลับผ่านเครือข่ายไปยัง Client
8	Client รับและนำผลลัพธ์ไปแสดงหรือประมวลผลต่อ

การแยกกระบวนการดังกล่าวทำให้ Client ไม่จำเป็นต้องทราบรายละเอียดภายในเกี่ยวกับวิธีการที่ Server ใช้ในการดำเนินงาน ตัวอย่างเช่น Client อาจส่งคำขอเพื่อตรวจสอบข้อมูล แต่ไม่จำเป็นต้องทราบว่า Server ใช้ฐานข้อมูลชนิดใดหรือมีกระบวนการภายในอย่างไร เอกสาร *Client-server Architecture* ระบุว่าการแยก Functionality ระหว่าง Client และ Server ทำให้แต่ละส่วนสามารถทำหน้าที่ได้อย่างอิสระ และช่วยสร้าง Abstraction ระหว่างส่วนต่าง ๆ ของระบบ

ลักษณะดังกล่าวเป็นหนึ่งในหลักสำคัญของ Client-Server Architecture เนื่องจากทำให้สามารถปรับเปลี่ยนหรือพัฒนาส่วนหนึ่งของระบบได้โดยไม่จำเป็นต้องแก้ไขทุกส่วนพร้อมกัน และเป็นพื้นฐานสำคัญของการแบ่งระบบออกเป็นองค์ประกอบหรือ Layer ที่มีหน้าที่เฉพาะ

หลักการและคุณลักษณะของ Client-Server Architecture

Client-Server Architecture เป็นสถาปัตยกรรมที่อาศัยการแบ่งหน้าที่และกระจายกระบวนการทำงานระหว่าง Client และ Server อย่างชัดเจน โดยทั้งสองส่วนมีหน้าที่เป็นอิสระต่อกัน แต่ทำงานร่วมกันผ่านระบบเครือข่าย หลักการดังกล่าวช่วยให้สามารถแบ่งภาระการประมวลผลและจัดการทรัพยากรให้เหมาะสมกับแต่ละส่วนของระบบ

Yadav และ Singh (2009) ระบุว่าหลักการสำคัญของระบบ Client-Server ประกอบด้วย Hardware Independence, Software Independence, Open Access to Services, Process Distribution และ Standards ซึ่งเป็นองค์ประกอบพื้นฐานที่ช่วยให้ Client, Server และ Communication Middleware สามารถทำงานร่วมกันได้อย่างเหมาะสม

ความเป็นอิสระจากฮาร์ดแวร์ (Hardware Independence) หมายถึง Client, Server และส่วนของ Communication Middleware ควรสามารถทำงานบนฮาร์ดแวร์ที่แตกต่างกันได้โดยไม่ทำให้หน้าที่หลักของระบบเปลี่ยนแปลงไป แนวคิดดังกล่าวช่วยลดการผูกติดกับอุปกรณ์หรือผู้ผลิตรายใดรายหนึ่ง และทำให้สามารถปรับเปลี่ยนโครงสร้างพื้นฐานของระบบได้ตามความเหมาะสม

ความเป็นอิสระจากซอฟต์แวร์ (Software Independence) เป็นแนวคิดที่ทำให้ Client, Server และ Middleware สามารถรองรับระบบปฏิบัติการ Network Protocol และ Application ที่หลากหลายได้ ตัวอย่างของ Network Protocol ที่เอกสารกล่าวถึง ได้แก่ TCP/IP และ IPX ซึ่งแสดงให้เห็นว่าการทำงานแบบ Client-Server ไม่จำเป็นต้องจำกัดอยู่บนแพลตฟอร์มหรือซอฟต์แวร์รูปแบบเดียว

การเข้าถึงบริการแบบเปิด (Open Access to Services) หมายถึง Client ภายในระบบควรสามารถร้องขอบริการที่มีอยู่บนเครือข่ายได้โดยไม่ขึ้นอยู่กับตำแหน่งทางกายภาพของ Client หรือ Server โดยบริการควรถูกจัดเตรียมให้สามารถตอบสนองต่อคำขอของ Client เมื่อมีความต้องการใช้งาน

การกระจายการประมวลผล (Process Distribution) เป็นหนึ่งในคุณลักษณะหลักของ Client-Server Architecture โดยหน้าที่ในการประมวลผลข้อมูลถูกแบ่งระหว่าง Client และ Server ทั้งสองส่วนจะต้องมีขอบเขตของหน้าที่ที่ชัดเจนและสามารถทำงานอย่างเป็นอิสระ การแบ่งหน้าที่ดังกล่าวช่วยเพิ่มความเป็นโมดูล ความยืดหยุ่น และความสามารถในการใช้ทรัพยากรให้เหมาะสมกับงาน

นอกจากนี้ Server หนึ่งเครื่องสามารถให้บริการแก่ Client หลายรายได้ และระบบสามารถเพิ่มประสิทธิภาพโดยปรับปรุง Server หรือเพิ่มจำนวน Server เมื่อปริมาณงานเพิ่มขึ้น หลักการดังกล่าวมีส่วนสนับสนุน Scalability และ Flexibility ของระบบ รวมถึงช่วยให้ Client และ Server สามารถทำงานร่วมกันได้แม้มีความแตกต่างทางด้าน Hardware หรือ Software

Nyabuto (2024) อธิบายคุณลักษณะของ Client-Server Architecture ในแนวทางเดียวกัน โดยระบุว่า Client และ Server จำเป็นต้องใช้ Protocol ในการสื่อสาร Server หนึ่งเครื่องสามารถให้บริการคำขอหลายรายการได้พร้อมกัน และสถาปัตยกรรมสามารถรองรับได้ทั้ง Horizontal Scaling และ Vertical Scaling รวมทั้งสามารถทำงานบน Hardware และ Software ที่แตกต่างกันได้

Horizontal Scaling หมายถึงการเพิ่ม Server เข้าไปในระบบเพื่อช่วยรองรับปริมาณงานที่เพิ่มขึ้น ขณะที่ Vertical Scaling หมายถึงการเพิ่มความสามารถของ Server เดิม เช่น การเพิ่มหน่วยความจำหรือประสิทธิภาพของหน่วยประมวลผล แนวทางทั้งสองช่วยให้ระบบสามารถรองรับจำนวนผู้ใช้งานหรือคำขอที่เพิ่มขึ้นได้

ตารางที่ 2.10 หลักการสำคัญของ Client-Server Architecture

หลักการ	รายละเอียด
Hardware Independence	รองรับการทำงานบนฮาร์ดแวร์ที่แตกต่างกัน
Software Independence	รองรับระบบปฏิบัติการ Protocol และ Software ที่หลากหลาย
Open Access to Services	Client สามารถร้องขอบริการภายในเครือข่ายตามความต้องการ
Process Distribution	แบ่งกระบวนการประมวลผลระหว่าง Client และ Server
Scalability	รองรับการเพิ่มจำนวน Server หรือเพิ่มประสิทธิภาพของ Server
Interoperability	องค์ประกอบที่แตกต่างกันสามารถทำงานร่วมกันได้
Standards	ใช้มาตรฐานในการกำหนดรูปแบบการสื่อสารและการเข้าถึงบริการ

จากตารางที่ 2.10 จะเห็นได้ว่า Client-Server Architecture มุ่งเน้นการแบ่งหน้าที่ของระบบและการทำงานร่วมกันขององค์ประกอบที่มีความหลากหลาย โดยใช้มาตรฐานและ Protocol เป็นตัวกลางในการสื่อสาร ซึ่งช่วยสนับสนุนการขยายระบบและการปรับเปลี่ยนองค์ประกอบในอนาคต

รูปแบบของ Client-Server Architecture

Client-Server Architecture สามารถจำแนกตามจำนวนชั้นหรือ Tier ที่ใช้แบ่งหน้าที่ของระบบได้หลายรูปแบบ โดย Yadav และ Singh (2009) จำแนกรูปแบบหลักออกเป็น Two-Tier, Three-Tier และ N-Tier Client-Server Architecture ขณะที่ Nyabuto (2024) ใช้การจำแนกในลักษณะเดียวกัน โดยอธิบายว่าการเพิ่ม Tier มีวัตถุประสงค์เพื่อแบ่งความรับผิดชอบของระบบออกเป็นส่วนที่ชัดเจนขึ้น

Two-Tier Client-Server Architecture

Two-Tier Client-Server Architecture เป็นรูปแบบพื้นฐานที่ประกอบด้วยสองส่วน ได้แก่ Client และ Server โดย Client ทำหน้าที่เป็นส่วนติดต่อผู้ใช้งานและส่งคำขอไปยัง Server โดยตรง ส่วน Server ทำหน้าที่ประมวลผลหรือจัดการข้อมูลตามคำขอ

Yadav และ Singh (2009) ระบุว่า Client-Server Environment อย่างน้อยจะต้องประกอบด้วยสอง Tier โดย Client ถือเป็น Tier แรกและ Server เป็น Tier ที่สอง Client จะร้องขอบริการจาก Server โดยตรงโดยไม่มี Server หรือกระบวนการตัวกลางเข้ามาช่วยประสานงาน

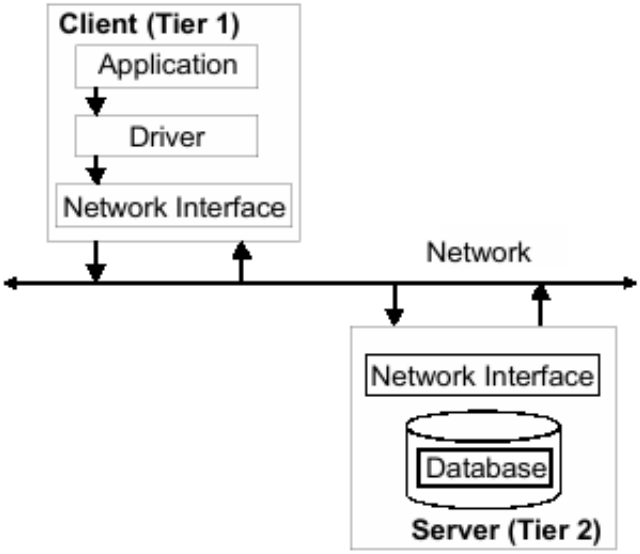
ในระบบ Two-Tier การประมวลผล Application Logic และ Business Rules อาจถูกวางไว้ที่ Client หรือ Server ก็ได้ โดยฐานข้อมูลมักอยู่ฝั่ง Server ขณะที่ Client ทำหน้าที่นำเสนอข้อมูล รับข้อมูลจากผู้ใช้ และส่งคำขอไปยังฐานข้อมูล

Nyabuto (2024) อธิบายว่า Two-Tier Architecture ประกอบด้วยอุปกรณ์ประมวลผลสองระดับ คือ Client และ Server โดย Client ทำหน้าที่แสดง User Interface และส่ง Request ขณะที่ Server เป็นส่วนที่ประมวลผลคำขอ รูปแบบนี้เหมาะสำหรับ Application ที่ไม่ซับซ้อนและไม่ต้องใช้กำลังประมวลผลสูงมาก

Two-Tier Architecture มีข้อดีด้านความเรียบง่ายและประสิทธิภาพ เนื่องจาก Client ติดต่อกับ Server โดยตรง ทำให้จำนวนขั้นตอนในการสื่อสารมีไม่มาก เอกสารระบุว่าระบบรูปแบบนี้สามารถนำไปพัฒนาได้รวดเร็วและมีความซับซ้อนในการติดตั้งน้อยเมื่อเปรียบเทียบกับสถาปัตยกรรมที่มีหลาย Tier

อย่างไรก็ตาม เมื่อจำนวน Client เพิ่มขึ้น ประสิทธิภาพของระบบอาจลดลง เนื่องจาก Client จำนวนมากต้องติดต่อกับ Server โดยตรงพร้อมกัน นอกจากนี้ หากมี Application Logic จำนวนมากอยู่บน Client การแก้ไขหรืออัปเดตระบบอาจจำเป็นต้องดำเนินการกับ Client แต่ละเครื่อง ซึ่งเพิ่มความซับซ้อนในการบำรุงรักษา

Nyabuto (2024) ยังระบุว่า Two-Tier Architecture เหมาะกับระบบที่มีจำนวนผู้ใช้งานพร้อมกันไม่มาก เนื่องจากเมื่อจำนวนผู้ใช้งานเพิ่มขึ้น Performance จะกลายเป็นปัญหา และรูปแบบนี้อาจมีข้อจำกัดด้าน Portability เนื่องจากองค์ประกอบของระบบมีความสัมพันธ์กันค่อนข้างแน่น



ภาพที่ 2.2 โครงสร้าง Two-Tier Client-Server Architecture

ที่มา: (Yadav และ Singh ,2009)

Three-Tier Client-Server Architecture

Three-Tier Client-Server Architecture เป็นการพัฒนาต่อยอดจาก Two-Tier Architecture โดยเพิ่ม Tier สำหรับประมวลผล Application Logic หรือ Business Logic ระหว่าง Client และ Database Server เพื่อแยกหน้าที่ของแต่ละส่วนออกจากกันอย่างชัดเจน

Yadav และ Singh (2009) อธิบายว่า การกระจาย Business Logic ไว้หลายส่วนทำให้การนำกลับมาใช้ใหม่และการบำรุงรักษาทำได้ยาก จึงมีการเพิ่ม Software Tier ที่สามเข้ามาระหว่าง Client และ Database โดย Business Logic ส่วนใหญ่จะถูกจัดไว้ใน Middle Tier ทำให้เมื่อกฎหรือกระบวนการทางธุรกิจเปลี่ยนแปลง สามารถแก้ไขที่ Middle Tier เป็นหลักได้

Three-Tier Architecture สามารถแบ่งหน้าที่ออกเป็นสามส่วน ได้แก่ **Presentation Tier**, **Application หรือ Business Logic Tier** และ **Data Tier** โดย Presentation Tier รับผิดชอบ User Interface และการนำเสนอข้อมูล Application Tier ทำหน้าที่ประมวลผลกฎและลำดับการทำงาน ส่วน Data Tier ทำหน้าที่จัดเก็บและจัดการข้อมูล

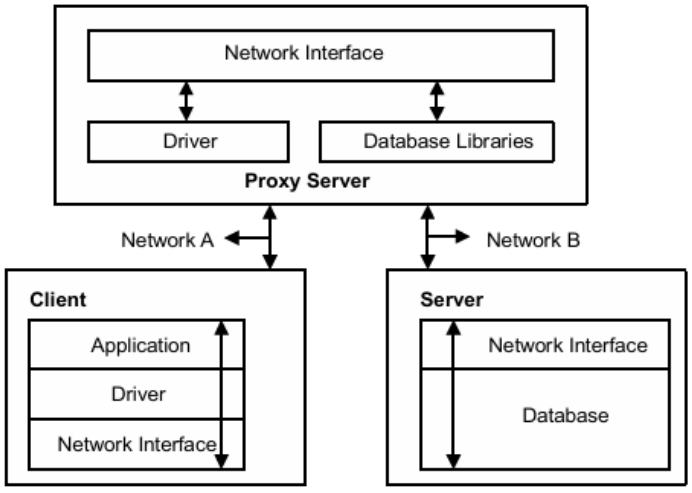
Nyabuto (2024) อธิบาย Three-Tier Architecture ในลักษณะเดียวกัน โดยแบ่งออกเป็น Client Tier ซึ่งทำหน้าที่แสดงข้อมูลและรับ Input, Logic/Application Tier ซึ่งประมวลผล

Request และเรียกข้อมูลที่จำเป็น และ Data Tier ซึ่งจัดเก็บและดึงข้อมูลผ่าน Database Management System

การเพิ่ม Application Tier ช่วยให้ Client ไม่จำเป็นต้องเข้าถึง Database โดยตรง ทุกคำขอสามารถถูกส่งผ่าน Application Tier ก่อน ซึ่งช่วยสร้างการแยกระหว่างส่วนแสดงผล กระบวนการประมวลผล และฐานข้อมูลอย่างชัดเจน อีกทั้งยังช่วยเพิ่มความปลอดภัยของ Database เนื่องจากผู้ใช้งานไม่สามารถเข้าถึงหรือแก้ไขฐานข้อมูลโดยตรงได้

Three-Tier Architecture มีข้อดีด้านการบำรุงรักษา เนื่องจาก Business Logic ถูกจัดเก็บไว้ใน Application Server ส่วนกลาง ทำให้ไม่จำเป็นต้องนำ Software Version ใหม่ไปติดตั้งบน Client ทุกเครื่อง นอกจากนี้ การแยก User Interface ออกจาก Application Logic ยังช่วยเพิ่มความสามารถในการนำ Components กลับมาใช้ใหม่และทำให้การเปลี่ยนแปลงระบบมีผลกระทบต่อส่วนอื่นน้อยลง

Nyabuto (2024) สรุปข้อดีของ Three-Tier Architecture ได้แก่ Maintainability ที่ดีขึ้น Scalability ที่เพิ่มขึ้น Security ที่ดีขึ้น และ Performance ที่สามารถบริหารจัดการแยกตามแต่ละ Tier ได้ อย่างไรก็ตาม รูปแบบดังกล่าวมีความซับซ้อนในการพัฒนามากกว่า Two-Tier เนื่องจากมีจุดติดต่อสื่อสารระหว่างระบบเพิ่มขึ้น



ภาพที่ 2.3 โครงสร้าง Three-Tier Client-Server Architecture

ที่มา: (Yadav และ Singh ,2009)

N-Tier Client-Server Architecture

N-Tier Architecture หรือ Multi-Tier Architecture เป็นการขยายแนวคิดจาก Three-Tier Architecture โดยแบ่งกระบวนการทำงานออกเป็น Tier เพิ่มเติมตามหน้าที่และความซับซ้อน

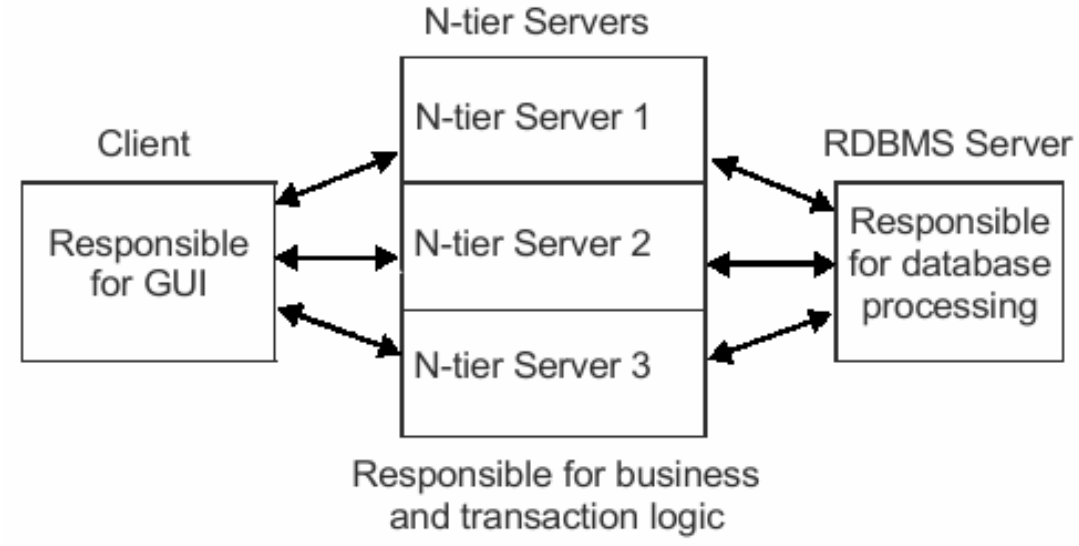
ของระบบ แต่ละ Tier สามารถรับผิดชอบ Function เฉพาะและสามารถแยกออกเป็นองค์ประกอบทาง Logical หรือ Physical ได้

Yadav และ Singh (2009) อธิบายว่า N-Tier Computing ช่วยให้ผู้พัฒนาสามารถกระจาย Functionality ของระบบไปยังหลาย Logical และ Physical Tiers ทำให้สามารถใช้ทรัพยากร Hardware และ Platform ได้มีประสิทธิภาพ และช่วยให้ Component ต่าง ๆ สามารถรองรับ Application ขนาดใหญ่ได้พร้อมกัน

การแยก Tier จำนวนมากยังช่วยให้ผู้ดูแลระบบสามารถระบุ Bottleneck ได้ง่ายขึ้น หากส่วนใดมีภาระการประมวลผลสูง สามารถเพิ่ม Hardware หรือ Server เฉพาะในส่วนดังกล่าวเพื่อรองรับ Load Balancing หรือ Failover ได้โดยไม่จำเป็นต้องปรับปรุงทั้งระบบ

Nyabuto (2024) ระบุว่า N-Tier Architecture มีการแยก Presentation, Processing และ Data Management Logic ออกจากกันทั้งทาง Logical และ Physical โดย Server หลายเครื่องสามารถทำงานร่วมกันเพื่อประมวลผลคำขอ รูปแบบดังกล่าวเหมาะสำหรับระบบขนาดใหญ่ที่ต้องการ Scalability, Data Integrity และ Security ในระดับสูง

ข้อดีสำคัญของ N-Tier Architecture คือสามารถเพิ่มประสิทธิภาพของระบบรวม จัด Business Logic ไว้ในส่วนกลาง และเพิ่มระดับความปลอดภัยของระบบได้ อย่างไรก็ตาม ยิ่งระบบถูกแบ่งออกเป็น Tier จำนวนมาก ความซับซ้อนในการพัฒนา Deployment และ Maintenance ก็เพิ่มสูงขึ้นตามไปด้วย



ภาพที่ 2.4โครงสร้าง N-Tier Client-Server Architecture

ที่มา: (Yadav และ Singh ,2009)

การเปรียบเทียบ Two-Tier, Three-Tier และ N-Tier Architecture

ตารางที่ 2.11 การเปรียบเทียบรูปแบบ Client-Server Architecture

ประเด็น	Two-Tier	Three-Tier	N-Tier
จำนวน Tier	2	3	ตั้งแต่ 3 Tier ขึ้นไป
โครงสร้างหลัก	Client และ Server	Presentation, Application, Data	แบ่งออกเป็นหลาย Layer ตามหน้าที่
การติดต่อ Database	Client สามารถติดต่อ Server โดยตรง	ผ่าน Application Tier	ผ่าน Service หรือ Tier ที่กำหนด
ความซับซ้อน	ต่ำ	ปานกลาง	สูง
Scalability	เหมาะกับระบบขนาดเล็ก	รองรับการขยายได้ดี	รองรับการขยายตัวสูง
Maintainability	ยากขึ้นเมื่อ Client มีจำนวนมาก	จัดการง่ายขึ้นเนื่องจากแยก Business Logic	สามารถจัดการแยกตามแต่ละ Tier
Security	มีข้อจำกัดมากกว่า	ดีขึ้นเพราะ Database ไม่ถูกเข้าถึงโดยตรง	สามารถแยก Security ตามแต่ละ Layer
การนำ Component กลับมาใช้	จำกัด	ทำได้ดีขึ้น	รองรับได้ดี
ลักษณะการใช้งาน	Application ขนาดเล็กและผู้ใช้ไม่มาก	Web/Application ทั่วไป	Application ขนาดใหญ่หรือระบบซับซ้อน

จากตารางที่ 2.11 จะเห็นได้ว่า การเพิ่มจำนวน Tier ช่วยให้สามารถแบ่งหน้าที่และความรับผิดชอบของระบบได้ชัดเจนมากขึ้น Two-Tier มีโครงสร้างเรียบง่ายและเหมาะกับระบบที่ไม่ซับซ้อน ขณะที่ Three-Tier เพิ่ม Application Layer เพื่อแยก Business Logic ออกจาก Client และ Database ส่วน N-Tier เป็นการขยายแนวคิดดังกล่าวให้สามารถรองรับระบบที่มีขนาดใหญ่และซับซ้อนมากขึ้น

ข้อดีของ Client-Server Architecture

Client-Server Architecture มีข้อดีหลายประการจากการแบ่งหน้าที่การทำงานระหว่าง Client และ Server อย่างชัดเจน โดยเฉพาะในด้านการจัดการทรัพยากร การแบ่งภาระการประมวลผล การเข้าถึง
ข้อมูล และความสามารถในการขยายระบบ

Yadav และ Singh (2009) อธิบายว่า Client-Server Computing ช่วยกระจายภาระการประมวลผลระหว่าง Client และ Server ทำให้การประมวลผลข้อมูลบางส่วนสามารถดำเนินการที่ Server ขณะที่ Client รับผิดชอบงานส่วนหน้า ส่งผลให้ลดภาระของ Workstation และลดปริมาณข้อมูลที่ต้องส่งผ่านเครือข่ายในบางกรณี เนื่องจากระบบสามารถส่งเฉพาะ Query และผลลัพธ์ที่เกี่ยวข้องแทนการถ่ายโอนข้อมูลทั้งหมด

ข้อดีอีกประการหนึ่งคือ **ความเป็นอิสระของ Workstation (Workstation Independence)** ซึ่งช่วยให้ Client ที่ทำงานบน Hardware หรือ Operating System ที่แตกต่างกันสามารถเข้าถึงบริการจาก Server ร่วมกันได้ โดย Client ไม่จำเป็นต้องใช้ Platform หรือ Software รูปแบบเดียวกันทั้งหมด

นอกจากนี้ Client-Server Architecture ยังมีคุณสมบัติด้าน **System Interoperability** กล่าวคือ องค์ประกอบของระบบ เช่น Client, Network และ Server สามารถทำงานร่วมกันได้แม้จะมีความแตกต่างด้านเทคโนโลยี และสามารถเปลี่ยนบาง Component โดยไม่จำเป็นต้องเปลี่ยนทั้งระบบ

Scalability เป็นข้อดีที่สำคัญอีกประการหนึ่ง เนื่องจากระบบสามารถปรับปรุงหรือเปลี่ยน Server ให้มีประสิทธิภาพสูงขึ้นโดยไม่ส่งผลกระทบต่อผู้ใช้งานโดยตรง และสามารถขยายองค์ประกอบของระบบตามปริมาณงานที่เพิ่มขึ้นได้

ในด้านข้อมูล Client-Server Architecture สนับสนุน **Data Integrity** โดย Server และ Database Management System สามารถจัดการกลไกต่าง ๆ เช่น การสำรองข้อมูล การทำ Transaction Processing และการรักษาความถูกต้องของข้อมูล ช่วยลดความเสี่ยงที่ข้อมูลจะสูญหายหรือไม่สอดคล้องกันเมื่อระบบเกิดความผิดพลาด

อีกข้อดีหนึ่งคือ **Data Accessibility** เนื่องจากข้อมูลส่วนใหญ่ถูกจัดเก็บไว้ในตำแหน่งส่วนกลาง ทำให้ผู้ใช้งานหลายรายสามารถเข้าถึงข้อมูลเดียวกันได้พร้อมกันภายใต้การควบคุมของระบบ แนวทางดังกล่าวช่วยสนับสนุนการแบ่งปันทรัพยากรและการใช้งานข้อมูลร่วมกันภายในระบบเครือข่าย

ในด้านการบริหารจัดการ ระบบ Client-Server สนับสนุน **Centralized Management** เนื่องจากข้อมูลและบริการหลักอยู่ที่ Server จึงสามารถจัดการด้าน Security, Data Integrity, Backup และ Recovery ได้จากส่วนกลาง

Nyabuto (2024) ให้ข้อสรุปในทิศทางเดียวกัน โดยระบุว่าข้อดีของ Client-Server Architecture ได้แก่ การจัดเก็บข้อมูลไว้ในตำแหน่งส่วนกลาง การเพิ่ม Scalability และ Efficiency การจัดการทรัพยากรจากส่วนกลาง การรองรับ Load Balancing และการแบ่งปันทรัพยากรระหว่าง Platform ที่แตกต่างกัน

ตารางที่ 2.12 ข้อดีของ Client-Server Architecture

ข้อดี	รายละเอียด
Distributed Processing	แบ่งภาระการประมวลผลระหว่าง Client และ Server
Scalability	สามารถเพิ่ม Server หรือเพิ่มประสิทธิภาพของ Server ได้
Centralized Management	จัดการข้อมูล Security และ Backup จากส่วนกลาง
Data Accessibility	ผู้ใช้งานหลายรายสามารถเข้าถึงข้อมูลร่วมกันได้
Data Integrity	มีระบบควบคุมความถูกต้องและความสอดคล้องของข้อมูล
Interoperability	รองรับการทำงานร่วมกันของ Hardware และ Software ที่แตกต่างกัน
Resource Sharing	สามารถใช้ทรัพยากรและบริการร่วมกันผ่านเครือข่าย
Load Balancing	กระจายคำขอไปยัง Server หลายเครื่องเพื่อลดภาระ

จากตารางที่ 2.12 จะเห็นได้ว่า Client-Server Architecture มีจุดเด่นทั้งด้านการบริหารจัดการข้อมูล การใช้ทรัพยากร และความสามารถในการขยายระบบ จึงเป็นรูปแบบสถาปัตยกรรมที่สามารถนำไปใช้กับระบบที่มีผู้ใช้งานหลายรายและต้องใช้ทรัพยากรร่วมกันผ่านเครือข่ายได้อย่างมีประสิทธิภาพ

ข้อจำกัดและข้อเสียของ Client-Server Architecture

แม้ Client-Server Architecture จะมีข้อดีหลายประการ แต่ก็มีข้อจำกัดที่ต้องพิจารณา โดยเฉพาะในด้านต้นทุน ความซับซ้อน การพึ่งพา Server ส่วนกลาง และความเสี่ยงด้านความปลอดภัย

Yadav และ Singh (2009) ระบุว่าข้อจำกัดประการหนึ่งคือ **Maintenance Cost** เนื่องจาก Server และ Database Server จำเป็นต้องได้รับการดูแลอย่างต่อเนื่อง เมื่อจำนวนผู้ใช้งานหรือขนาดของฐานข้อมูลเพิ่มขึ้น อาจจำเป็นต้องมีผู้ดูแลระบบหรือ Database Administrator โดยเฉพาะ ซึ่งทำให้ต้นทุนด้านบุคลากรเพิ่มสูงขึ้น

นอกจากนี้ยังมี **Hardware Cost** เนื่องจาก Server มักต้องมีความสามารถในการประมวลผล หน่วยความจำ และพื้นที่จัดเก็บข้อมูลสูงกว่าคอมพิวเตอร์ Client ทั่วไป เพื่อรองรับคำขอจากผู้ใช้งานหลายรายพร้อมกัน

ระบบ Client-Server ยังมีความซับซ้อนมากกว่าระบบที่มีโครงสร้างง่าย เนื่องจากประกอบด้วยหลาย Component เช่น Client, Server, Database, Middleware และ Network เมื่อองค์ประกอบของระบบมีจำนวนมากขึ้น การค้นหาสาเหตุของปัญหาและการบำรุงรักษาระบบก็อาจทำได้ยากขึ้น

เอกสาร *Client-server Architecture* ระบุถึงข้อจำกัดด้าน **Centralized Control** โดยหาก Server หลักหยุดทำงาน บริการทั้งหมดที่พึ่งพา Server ดังกล่าวอาจไม่สามารถใช้งานได้ ลักษณะดังกล่าวทำให้ Server กลายเป็นจุดสำคัญที่ต้องได้รับการออกแบบให้มีความน่าเชื่อถือและมีระบบสำรองที่เหมาะสม

นอกจากนี้ Server ยังอาจตกเป็นเป้าหมายของ **Denial of Service (DoS)** หรือ **Distributed Denial of Service (DDoS)** ซึ่งเป็นการส่งคำขอจำนวนมากเพื่อทำให้ทรัพยากรของระบบไม่สามารถรองรับคำขอจากผู้ใช้งานปกติได้

Nyabuto (2024) กล่าวถึงข้อเสียในลักษณะเดียวกัน โดยระบุว่า Security เป็นความท้าทายสำคัญของ Client-Server Architecture เนื่องจากข้อมูลจำนวนมากถูกเก็บไว้ที่ส่วนกลาง หาก Server หรือ Network ถูกโจมตี อาจทำให้บริการไม่สามารถใช้งานได้ รวมถึงมีความเสี่ยงจาก DoS, Man-in-the-Middle และ Packet Spoofing

ตารางที่ 2.13 ข้อจำกัดของ Client-Server Architecture

ข้อจำกัด	รายละเอียด
Centralized Failure	หาก Server ส่วนกลางล้มเหลว อาจส่งผลให้บริการทั้งหมดหยุดชะงัก
Security Risk	Server เป็นจุดสำคัญที่อาจตกเป็นเป้าหมายของการโจมตี
Maintenance Cost	ต้องใช้บุคลากรและทรัพยากรในการดูแล Server และ Database
Hardware Cost	Server ต้องมีประสิทธิภาพสูงเพื่อรองรับ Client หลายราย
Complexity	มีหลาย Component ทำให้การพัฒนาและแก้ไขปัญหาซับซ้อน

ข้อจำกัด	รายละเอียด
Network Dependency	การทำงานของระบบขึ้นอยู่กับคุณภาพและความพร้อมของเครือข่าย
Overload	Server อาจรับภาระมากเกินไปเมื่อมีคำขอจำนวนมากพร้อมกัน

ข้อจำกัดดังกล่าวสะท้อนให้เห็นว่า แม้ Client-Server Architecture จะช่วยให้การจัดการระบบเป็นศูนย์กลางและรองรับผู้ใช้งานจำนวนมากได้ แต่จำเป็นต้องมีการวางแผนด้าน Infrastructure, Security, Availability และ Resource Management อย่างเหมาะสม

ความปลอดภัยใน Client-Server Architecture

Security เป็นองค์ประกอบที่มีความสำคัญอย่างมากใน Client-Server Architecture เนื่องจากข้อมูลและบริการส่วนใหญ่มักถูกจัดการผ่าน Server หาก Server ถูกโจมตีหรือเกิดการเข้าถึงโดยไม่ได้รับอนุญาต อาจส่งผลกระทบต่อข้อมูลและบริการจำนวนมากในระบบ

Yadav และ Singh (2009) อธิบายว่า Security Threat หมายถึงสถานการณ์หรือเหตุการณ์ที่อาจก่อให้เกิดความเสียหายต่อข้อมูลหรือทรัพยากรของเครือข่าย เช่น การทำลายข้อมูล การเปิดเผยข้อมูล การแก้ไขข้อมูล การปฏิเสธการให้บริการ และการทุจริต โดย Client-Server Security เกี่ยวข้องกับกลไก Authorization และ Access Control เช่น Password, Smart Card, Biometrics และ Firewall

ภัยคุกคามต่อ Server สามารถเกิดขึ้นได้หลายรูปแบบ เช่น **Eavesdropping** ซึ่งเป็นการดักฟังข้อมูลที่ส่งผ่านเครือข่าย ทำให้ผู้โจมตีอาจได้รับข้อมูลสำคัญ เช่น Password หรือข้อมูลส่วนตัว โดยการเข้ารหัสข้อมูลสามารถช่วยลดความเสี่ยงจากการดักฟังได้

อีกหนึ่งภัยคุกคามคือ **Denial of Service** ซึ่งเกิดขึ้นเมื่อผู้โจมตีใช้ทรัพยากรของระบบในปริมาณมากจนผู้ใช้งานปกติไม่สามารถเข้าถึงบริการได้ ตัวอย่างรูปแบบการโจมตี ได้แก่ Service Overloading, Message Flooding และ Packet Replay

เอกสาร *Client-server Architecture* อธิบายเพิ่มเติมว่า DoS Attack มักเกิดจากการส่ง Request จำนวนมากไปยัง Server เพื่อให้ระบบ Overload จนไม่สามารถตอบสนองคำขอปกติได้ ส่วน DDoS มีความซับซ้อนมากกว่า เนื่องจาก Traffic ถูกส่งมาจาก Client หลายเครื่องพร้อมกัน ทำให้ไม่สามารถแก้ไขปัญหาได้เพียงการ Block Client เครื่องเดียว

การออกแบบ Security ใน Client-Server Architecture จึงควรให้ความสำคัญทั้งกับการตรวจสอบตัวตน การกำหนดสิทธิ์การเข้าถึง การเข้ารหัสข้อมูล และการป้องกันการโจมตีผ่านเครือข่าย เนื่องจาก Server ทำหน้าที่เป็นศูนย์กลางในการให้บริการและจัดการทรัพยากรของระบบ

Scalability ของ Client-Server Architecture

Scalability หมายถึงความสามารถของระบบในการรองรับปริมาณงาน ผู้ใช้งาน หรือ Request ที่เพิ่มขึ้นโดยยังคงสามารถให้บริการได้อย่างเหมาะสม Client-Server Architecture รองรับการขยายระบบได้ทั้งในแนวนอนและแนวตั้ง

เอกสาร *Client-server Architecture* แบ่ง Scalability ออกเป็นสองรูปแบบ ได้แก่ **Horizontal Scaling** และ **Vertical Scaling** โดย Horizontal Scaling คือการเพิ่มหรือลดจำนวน Server ใน Network ส่วน Vertical Scaling คือการเพิ่มประสิทธิภาพของ Server เช่น การใช้เครื่องที่มีขนาดใหญ่และเร็วขึ้น

Nyabuto (2024) ระบุในลักษณะเดียวกันว่า Client-Server Architecture สามารถรองรับ Horizontal และ Vertical Scalability โดยสามารถเพิ่ม Server เพื่อรองรับ Workload ที่เพิ่มขึ้น หรือเพิ่ม CPU และ RAM ให้ Server เดิมเพื่อเพิ่มความสามารถในการประมวลผล

การรองรับ Scalability มีความสำคัญอย่างยิ่งในระบบที่จำนวนผู้ใช้งานหรือปริมาณข้อมูลสามารถเพิ่มขึ้นตามเวลา หากระบบไม่มีความสามารถในการขยายตัว Server อาจเกิด Bottleneck และทำให้ Performance ลดลงอย่างมากเมื่อจำนวน Request เพิ่มขึ้น

Yadav และ Singh (2009) ระบุว่า Scalability เป็นหนึ่งในข้อดีของ Client-Server Architecture เนื่องจากความเป็น Modular ของระบบช่วยให้สามารถ Upgrade Server หรือเปลี่ยน Component เป็น Hardware ที่มีประสิทธิภาพสูงขึ้นได้โดยไม่ส่งผลต่อส่วนอื่นของระบบมากนัก

Availability และ Dependability

Availability หมายถึงความสามารถของระบบในการให้บริการได้อย่างต่อเนื่องและพร้อมใช้งานเมื่อต้องการ โดย Client-Server Architecture สามารถเพิ่ม Availability ผ่านการใช้ Server หลายเครื่องหรือระบบสำรอง

เอกสาร *Client-server Architecture* ระบุว่า Server สามารถทำงานต่อเนื่องได้เป็นเวลานาน และสามารถคง Uptime ระหว่างการบำรุงรักษาได้หากมีการทำ Server Duplication หรือมี Server สำรอง

ในด้าน Dependability การใช้ Server ที่ได้รับการออกแบบสำหรับงานเฉพาะช่วยให้การประมวลผลและการจัดสรรทรัพยากรสามารถดำเนินการได้อย่างมีประสิทธิภาพ โดย Server สามารถปรับแต่งให้เหมาะกับลักษณะงาน เช่น Database Processing หรือ Application Processing ได้

อย่างไรก็ตาม Availability อาจลดลงหากระบบพึ่งพา Server เพียงเครื่องเดียว เอกสารระบุว่าเมื่อมี Client จำนวนมากส่ง Request พร้อมกัน Server ส่วนกลางอาจ Overload และส่งผลให้ Performance ลดลงหรือระบบหยุดให้บริการได้

ดังนั้นการออกแบบ Client-Server Architecture ที่ต้องการ Availability สูงจึงควรหลีกเลี่ยงการพึ่งพา Server เพียงจุดเดียว และควรมีแนวทางด้าน Redundancy, Replication และ Load Distribution เพื่อช่วยลดความเสี่ยงจาก Single Point of Failure

Load Balancing

Load Balancing เป็นเทคนิคที่ใช้กระจาย Request หรือ Network Traffic ไปยัง Server หลายเครื่อง เพื่อให้แต่ละ Server รับภาระอย่างเหมาะสมและลดโอกาสเกิด Overload

เอกสาร *Client-server Architecture* ระบุว่า Load Balancer ทำหน้าที่กระจาย Incoming Network Traffic ไปยังกลุ่ม Backend Server เพื่อเพิ่มประสิทธิภาพในการใช้ Resource

ในการทำงานทั่วไป Client จะส่ง Request เข้าสู่ระบบ จากนั้น Load Balancer จะเลือก Server ที่เหมาะสมเพื่อรับคำขอ Server จะประมวลผลและอาจติดต่อ Database ก่อนส่งข้อมูลกลับไปยัง Client กระบวนการนี้ทำให้ Request ไม่ถูกส่งไปยัง Server เพียงเครื่องเดียวทั้งหมด

Nyabuto (2024) ระบุว่า Client-Server Architecture สามารถรองรับ Load Balancing ได้ด้วยการเพิ่ม Replica หรือ Redundant Server ซึ่งช่วยกระจาย Workload และยังช่วยเพิ่มทางเลือกในการ Recovery เมื่อ Server เครื่องหนึ่งเกิดความล้มเหลว

Yadav และ Singh (2009) ยังอธิบายว่าใน Three-Tier Architecture การแยก Business Logic ออกจาก Database Server ช่วยให้การทำ Load Balancing ทำได้ง่ายขึ้น และ Server Process สามารถถูกย้ายไปยัง Server เครื่องอื่นได้เมื่อเกิด Performance Bottleneck

ตารางที่ 2.14 แนวทางสนับสนุน Scalability และ Availability ใน Client-Server Architecture

แนวทาง	รายละเอียด
Horizontal Scaling	เพิ่มจำนวน Server เพื่อรองรับ Request ที่เพิ่มขึ้น
Vertical Scaling	เพิ่ม CPU, RAM หรือประสิทธิภาพให้ Server เดิม
Load Balancing	กระจาย Request ไปยัง Server หลายเครื่อง
Server Replication	ใช้ Server สำรองเพื่อเพิ่ม Availability
Redundancy	ลดความเสี่ยงจากการพึ่งพา Server เพียงเครื่องเดียว
Resource Distribution	กระจาย Process และทรัพยากรไปยัง Server ที่เหมาะสม

จากตารางที่ 2.14 จะเห็นได้ว่า Scalability และ Availability มีความสัมพันธ์กันโดยตรง การเพิ่ม Server ไม่เพียงช่วยรองรับ Workload ที่เพิ่มขึ้น แต่ยังสามารถช่วยเพิ่มความต่อเนื่องในการให้บริการได้ หากระบบมีการกระจาย Load และมี Server สำรองอย่างเหมาะสม

การประยุกต์ใช้ Client-Server Architecture

Client-Server Architecture ถูกนำไปใช้ในระบบคอมพิวเตอร์หลายประเภท เนื่องจากสามารถรองรับการแบ่งปันทรัพยากรและการให้บริการผ่านเครือข่ายได้

Nyabuto (2024) ระบุว่า Application หลายประเภททำงานบน Client-Server Model เช่น Email Exchange, Database System และ Internet Service ในบทสรุปของบทความยังระบุถึงตัวอย่างระบบ เช่น Mail System, Printing System และ ATM ซึ่งล้วนสามารถทำงานบนสถาปัตยกรรมแบบ Client-Server ได้

Yadav และ Singh (2009) แบ่งประเภทของ Server ตามบริการที่ให้ เช่น File Server, Print Server, Application Server, Mail Server, Web Server, Database Server และ Transaction Server ซึ่งสะท้อนให้เห็นว่าสถาปัตยกรรมนี้สามารถประยุกต์ใช้ได้กับงานหลายประเภท Web Application ถือเป็นหนึ่งในตัวอย่างสำคัญของระบบ Client-Server โดย Web Browser สามารถทำหน้าที่เป็น Thin Client และส่ง Request ไปยัง Web Server ผ่าน HTTP เพื่อร้องขอข้อมูลหรือบริการ

Nyabuto (2024) ยังกล่าวถึงการนำ Client-Server Architecture ไปใช้ร่วมกับเทคโนโลยีสมัยใหม่ เช่น Cloud Computing และ Microservices โดย Cloud Computing มีพื้นฐานของการให้บริการผ่านเครือข่าย ขณะที่ Microservices ใช้การแบ่งระบบออกเป็น Service ที่เป็นอิสระและสามารถสื่อสารระหว่างกันได้

สรุปแนวคิด Client-Server Architecture

Client-Server Architecture เป็นรูปแบบการออกแบบระบบที่แบ่งหน้าที่การทำงานระหว่าง Client ซึ่งเป็นผู้ร้องขอบริการ และ Server ซึ่งเป็นผู้ให้บริการ โดยทั้งสองส่วนสามารถสื่อสารผ่านเครือข่ายด้วย Protocol ที่กำหนด รูปแบบดังกล่าวช่วยให้สามารถแบ่งภาระการประมวลผล จัดการข้อมูลจากส่วนกลาง ใช้ทรัพยากรร่วมกัน และรองรับการขยายตัวของระบบได้

สถาปัตยกรรมแบบ Client-Server สามารถแบ่งเป็น Two-Tier, Three-Tier และ N-Tier ตามจำนวน Layer หรือ Tier ที่ใช้ในการแบ่งหน้าที่ โดยการเพิ่ม Tier ช่วยให้สามารถแยก Presentation, Application Logic และ Data Management ออกจากกันได้ชัดเจนมากขึ้น แม้จะทำให้โครงสร้างของระบบมีความซับซ้อนเพิ่มขึ้นก็ตาม จุดเด่นของ Client-Server Architecture คือ

Scalability, Centralized Management, Resource Sharing, Interoperability และความสามารถในการแบ่ง Process ระหว่าง Client และ Server ขณะที่ข้อจำกัดสำคัญ ได้แก่ ความซับซ้อน ต้นทุนในการดูแลรักษา ความเสี่ยงจาก Server Failure และภัยคุกคามด้าน Security ดังนั้นการออกแบบระบบในรูปแบบนี้จำเป็นต้องพิจารณาทั้งประสิทธิภาพ ความปลอดภัย การกระจาย Load และ Availability ควบคู่กันไป

2.1.3 API (Application Programming Interface)

ความหมายและความสำคัญของ Application Programming Interface

Application Programming Interface หรือ API เป็นกลไกที่ใช้สำหรับเชื่อมโยงการทำงานระหว่างระบบซอฟต์แวร์ เพื่อให้ระบบหนึ่งสามารถร้องขอ ส่ง หรือรับข้อมูลและความสามารถจากอีกระบบหนึ่งได้ผ่านรูปแบบการติดต่อที่กำหนดไว้ การใช้ API ช่วยให้ระบบซอฟต์แวร์ที่มีโครงสร้างหรือหน้าที่แตกต่างกันสามารถสื่อสารและทำงานร่วมกันโดยไม่จำเป็นต้องทราบรายละเอียดของกระบวนการภายในทั้งหมดของอีกระบบ

Office of Systems Engineering and Architecture (2026) ให้นิยาม API ว่าเป็นกลไกในระดับซอฟต์แวร์ที่มีโครงสร้างและรองรับการรับส่งข้อมูล ซึ่งช่วยให้นักพัฒนาระบบสามารถค้นหาความสามารถของระบบ และทำให้ระบบที่ถูกโปรแกรมไว้สามารถร้องขอ ส่ง หรือรับข้อมูลและฟังก์ชันการทำงานจากระบบเดียวกันหรือระบบอื่นได้ API ที่ได้รับการออกแบบอย่างเหมาะสมสามารถสนับสนุนคุณลักษณะสำคัญ ได้แก่ Interoperability, Security และ Scalability

จากนิยามดังกล่าว API จึงสามารถมองได้ว่าเป็นตัวกลางระหว่าง Software หรือ Application ที่ทำหน้าที่กำหนดวิธีการติดต่อระหว่างระบบ เมื่อ Application หนึ่งต้องการใช้ข้อมูลหรือความสามารถของอีก Application หนึ่ง สามารถดำเนินการผ่าน API ตามรูปแบบที่กำหนดไว้แทนที่จะเข้าไปเข้าถึงกระบวนการภายในของระบบโดยตรง

เอกสารระบุว่า Application Program เป็นระบบซอฟต์แวร์ที่ถูกพัฒนาขึ้นเพื่อตอบสนองต่อข้อกำหนดหรือความต้องการที่กำหนดไว้ ขณะที่ API ช่วยเชื่อมต่อ Application Program หลายระบบที่ถูกใช้ในการดำเนินงาน และช่วยทำให้การผสานรวมระหว่าง Application ต่าง ๆ ทำได้ง่ายขึ้น

API จึงมีบทบาทสำคัญต่อระบบซอฟต์แวร์สมัยใหม่ เนื่องจาก Application จำนวนมากไม่ได้ทำงานอย่างเป็นอิสระ แต่จำเป็นต้องแลกเปลี่ยนข้อมูลหรือเรียกใช้บริการจากระบบอื่น เอกสารระบุว่า API ช่วยสนับสนุนการแบ่งปันข้อมูล การทำงานร่วมกัน และการเชื่อมต่อระบบและความสามารถที่แตกต่างกันให้สามารถทำงานร่วมกันได้อย่างต่อเนื่อง

อีกหนึ่งความสำคัญของ API คือการสนับสนุน **Interoperability** หรือความสามารถในการทำงานร่วมกันของระบบ โดย API ทำหน้าที่เป็น Interface ที่กำหนดรูปแบบการแลกเปลี่ยนข้อมูลระหว่าง Software ซึ่งช่วยลดข้อจำกัดที่เกิดจากความแตกต่างของระบบและเทคโนโลยีที่ใช้ เอกสารระบุว่า API เป็นส่วนสำคัญของ Interoperability เนื่องจากช่วยให้ Software System ที่มีความหลากหลายสามารถสื่อสารและเชื่อมโยงกันได้

API ยังสามารถสนับสนุนแนวคิด **Composability** ซึ่งหมายถึงความสามารถในการนำบริการหรือความสามารถหลายส่วนมาประกอบเข้าด้วยกันเพื่อสร้างความสามารถใหม่ เอกสารอธิบายว่าคุณสมบัติดังกล่าวช่วยให้สามารถรวบรวมความสามารถจาก API หลายรายการเข้าด้วยกันอย่างรวดเร็วเพื่อสร้างความสามารถหรือบริการในรูปแบบใหม่

ในมุมมองด้านข้อมูล API สามารถทำหน้าที่เสมือนมาตรฐานการแลกเปลี่ยนข้อมูลอัตโนมัติระหว่างบริการสองส่วน โดยมาตรฐานข้อมูลครอบคลุมข้อตกลงเกี่ยวกับการแทนค่า รูปแบบความหมาย โครงสร้าง การส่ง การจัดการ และการใช้ข้อมูล API สามารถทำหน้าที่อยู่ในหลายระดับของระบบ ตั้งแต่การเชื่อมต่อบริการวิเคราะห์ข้อมูลกับระบบจัดเก็บข้อมูล ไปจนถึงการเชื่อมโยงบริการด้าน Data Integration และ Data Management

จากแนวคิดดังกล่าวจึงสามารถสรุปได้ว่า API ไม่ได้เป็นเพียงช่องทางสำหรับการส่งข้อมูลระหว่าง Software เท่านั้น แต่เป็นองค์ประกอบที่ช่วยสร้างการเชื่อมโยงระหว่างระบบ ทำให้ระบบสามารถนำข้อมูลและความสามารถจากแหล่งอื่นมาใช้ร่วมกัน รวมทั้งช่วยสนับสนุนการพัฒนาระบบที่มีความยืดหยุ่นและสามารถขยายตัวได้

องค์ประกอบและระบบนิเวศของ API

การทำงานของ API เกิดขึ้นภายในสภาพแวดล้อมที่ประกอบด้วยองค์ประกอบหลายส่วน ซึ่งมีหน้าที่แตกต่างกันในการร้องขอ ให้บริการ และส่งผ่านข้อมูล เอกสาร *API Technical Guidance* แบ่งระบบที่เกี่ยวข้องกับ API ออกเป็นส่วนของ **Provider System** และ **External User/Consumer System** โดย Provider System เป็นระบบที่มีข้อมูลหรือความสามารถที่ระบบอื่นต้องการ ขณะที่ Consumer System เป็นระบบที่ต้องการร้องขอและเข้าถึงข้อมูลหรือความสามารถดังกล่าว

API Consumer หมายถึงผู้ใช้หรือระบบที่ต้องการเรียกใช้ API เพื่อเข้าถึงข้อมูลหรือบริการ ในทางกลับกัน **API Provider** เป็นผู้ให้บริการ API และรับผิดชอบในการจัดเตรียมข้อมูลหรือ Functionality ให้ Consumer สามารถเข้าถึงได้ตาม Interface ที่กำหนด

เอกสารอธิบายว่า API Ecosystem สามารถประกอบด้วย Stakeholder หลายประเภทซึ่งเป็นตัวแทนของระบบ Component และ Function ที่แตกต่างกัน โดยระบบที่เกี่ยวข้องสามารถแบ่งออกได้เป็น External System, Provider System และ Communication System

External System คือระบบที่ส่งคำขอไปยัง API ของ Provider System เพื่อเข้าถึงข้อมูลหรือ Function ส่วน Provider System คือระบบที่มีข้อมูลที่ต้องการและจัดเตรียม API Endpoint สำหรับให้ระบบภายนอกเรียกใช้ ขณะที่ Communication System เป็นโครงสร้างพื้นฐานที่ทำหน้าที่เป็นช่องทางการสื่อสารระหว่างระบบที่ร้องขอข้อมูลและระบบที่ให้ข้อมูล

องค์ประกอบสำคัญอีกส่วนหนึ่งคือ **API Endpoint** ซึ่งเป็นจุดที่ Application หรือ Service ใช้สำหรับติดต่อกับ API ในเอกสารมีการอธิบาย API Ecosystem โดยแสดงความสัมพันธ์ระหว่าง API User, Application Program, Network, API Endpoint และ Provider System เพื่อแสดงให้เห็นว่าการใช้งาน API ไม่ได้เกิดขึ้นอย่างโดดเดี่ยว แต่เป็นส่วนหนึ่งของการติดต่อสื่อสารระหว่าง Application หลายระบบ

ตารางที่ 2.15 องค์ประกอบพื้นฐานของระบบนิเวศ API

องค์ประกอบ	หน้าที่
API Consumer	ระบบหรือผู้ใช้งานที่ร้องขอข้อมูลหรือบริการผ่าน API
API Provider	ระบบที่จัดเตรียมข้อมูลหรือความสามารถให้ระบบอื่นเรียกใช้
Application Program	โปรแกรมที่เรียกใช้หรือให้บริการความสามารถผ่าน API
API Endpoint	จุดสำหรับรับ Request จาก Consumer
Communication System	โครงสร้างพื้นฐานสำหรับรับส่งข้อมูลระหว่างระบบ
API Gateway	ตัวกลางสำหรับบริหารจัดการ การเข้าถึง และ Traffic ของ API
Data Store	แหล่งจัดเก็บข้อมูล เช่น Database หรือ Data Lake
API Documentation	ข้อมูลอธิบาย Endpoint, Parameter, Response และวิธีการใช้ API

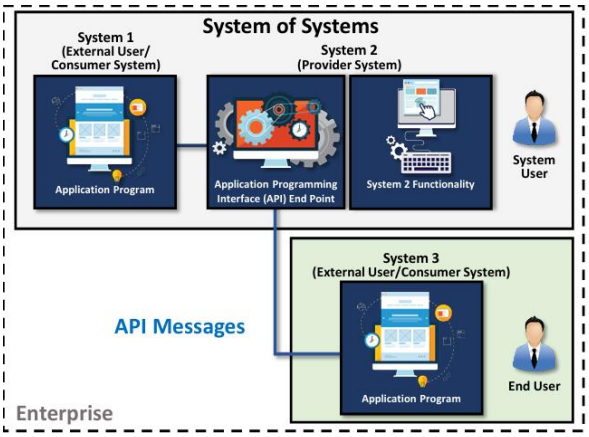
องค์ประกอบต่าง ๆ ในตารางที่ 2.15 มีความสัมพันธ์กัน โดย Consumer เริ่มต้นจากการร้องขอข้อมูลหรือ Function ผ่าน API Endpoint จากนั้น Provider System จะดำเนินการตามคำขอและส่งข้อมูลกลับผ่านช่องทางการสื่อสารที่กำหนดไว้

รูปแบบการสื่อสารและการแลกเปลี่ยนข้อมูลผ่าน API

API ทำหน้าที่เป็น Interface สำหรับการสื่อสารระหว่าง Application Program โดยระบบหนึ่งสามารถส่งข้อความหรือ Request ไปยังอีกระบบ และได้รับข้อมูลหรือ Response กลับมาตามข้อกำหนดของ API

เอกสารแสดงรูปแบบการทำงานของ API ในบริบทของ System of Systems โดยมี External User/Consumer System ติดต่อกับ Provider System ผ่าน API Messages ขณะที่ Provider System มี Application Programming Interface Endpoint สำหรับรับการติดต่อจากระบบภายนอก

ในมุมมองดังกล่าว API ทำหน้าที่เป็นขอบเขตหรือ Interface ที่แยกระหว่าง Application ภายใน Provider System กับระบบภายนอก โดย Consumer ไม่จำเป็นต้องทราบว่ากระบวนการภายใน Provider System ทำงานอย่างไร แต่จำเป็นต้องทราบ Interface รูปแบบข้อมูล และเงื่อนไขที่ API กำหนด



ภาพที่ 2.5 โครงสร้างการเชื่อมต่อ API ระหว่างระบบ

ที่มา: Office of Systems Engineering and Architecture (2026)

การสื่อสารผ่าน API สามารถอยู่ในรูปแบบต่าง ๆ โดยในภาพ API Ecosystem ของเอกสารมีการระบุรูปแบบ Message เช่น **Call/Response, Request/Response และ Publish/Subscribe** ซึ่งเป็นวิธีการที่ระบบสามารถใช้ในการแลกเปลี่ยนข้อมูลหรือเรียกใช้ Function ระหว่างกัน

รูปแบบ Request/Response เป็นการที่ Consumer ส่ง Request ไปยัง Provider ผ่าน API แล้ว Provider ส่ง Response กลับตามผลลัพธ์ของการดำเนินการ ขณะที่ Publish/Subscribe เป็นรูปแบบที่เกี่ยวข้องกับการเผยแพร่ข้อมูลจากฝั่งหนึ่งและให้ระบบที่สมัครรับข้อมูลสามารถรับข้อมูลตามกลไกที่กำหนดไว้

การแลกเปลี่ยนข้อมูลระหว่าง Application ยังต้องอาศัยรูปแบบข้อมูลและมาตรฐานที่ทั้งสองฝ่ายเข้าใจร่วมกัน เอกสารแนะนำให้ API ใช้มาตรฐานและ Protocol แบบเปิด เช่น OpenAPI รวมทั้งรูปแบบข้อมูล XML และ JSON เพื่อสนับสนุน Semantic และ Syntactic Interoperability ระหว่างระบบ

API และการทำงานร่วมกันระหว่างระบบ (Interoperability)

Interoperability เป็นหนึ่งในแนวคิดสำคัญของ API เนื่องจากระบบซอฟต์แวร์ในสภาพแวดล้อมจริงอาจถูกพัฒนาด้วย Technology, Platform หรือ Architecture ที่แตกต่างกัน การกำหนด Interface และ Standard กลางจึงช่วยให้ระบบเหล่านั้นสามารถแลกเปลี่ยนข้อมูลและใช้งานบริการร่วมกันได้

เอกสารระบุว่า Data Strategy, Architecture และ Framework เป็นองค์ประกอบสำคัญของ Interoperability และมักมีการใช้ API ร่วมด้วย API สามารถทำงานร่วมกับ Framework, Model, Ecosystem, Protocol และ Messaging Standard เพื่อสนับสนุนการเชื่อมต่อ Application และการแลกเปลี่ยนข้อมูล

API มีตำแหน่งอยู่ภายในแนวคิด Application Interoperability และมีความสัมพันธ์กับทั้ง Function Interoperability และ Data Interoperability แสดงให้เห็นว่า API ไม่ได้ทำหน้าที่เพียงเป็นช่องทางทางเทคนิค แต่ยังเกี่ยวข้องกับการทำให้ Function และข้อมูลของระบบต่าง ๆ สามารถใช้งานร่วมกันได้

การใช้มาตรฐานแบบเปิดมีบทบาทสำคัญในการสนับสนุน Interoperability เอกสารแนะนำให้ใช้ REST Principles และ OpenAPI รวมถึงรูปแบบข้อมูลมาตรฐาน เช่น JSON, XML และ EXI เพื่อช่วยให้ระบบสามารถแลกเปลี่ยนข้อมูลได้ด้วยรูปแบบที่มีความสอดคล้องกัน

นอกจากนี้ยังมีการกล่าวถึง HTTP, REST และ SOAP ในฐานะ Protocol หรือรูปแบบที่ใช้สนับสนุนการเชื่อมโยงระบบ พร้อมกับ JSON และ XML สำหรับการแทนข้อมูล การกำหนดมาตรฐานเหล่านี้ช่วยลดความแตกต่างทางเทคนิคระหว่างระบบและเพิ่มความสามารถในการเชื่อมต่อ Application จากแหล่งที่แตกต่างกัน

ตารางที่ 2.16 ตัวอย่างมาตรฐานและเทคโนโลยีที่เกี่ยวข้องกับ API

มาตรฐาน/เทคโนโลยี	บทบาทตามเอกสาร
HTTP	ใช้ในการสื่อสารและแลกเปลี่ยนข้อมูลระหว่างระบบ
REST	รูปแบบสถาปัตยกรรมสำหรับการออกแบบ Networked Application
SOAP	รูปแบบการสื่อสารที่สามารถใช้สำหรับเชื่อมต่อระบบ

มาตรฐาน/เทคโนโลยี	บทบาทตามเอกสาร
OpenAPI	ใช้อธิบายโครงสร้างและพฤติกรรมของ API
JSON	รูปแบบข้อมูลที่มีขนาดกะทัดรัดและมนุษย์สามารถอ่านได้
XML	รูปแบบมาตรฐานสำหรับการแทนและแลกเปลี่ยนข้อมูล
OAuth	Protocol สำหรับควบคุมการเข้าถึงทรัพยากร
TLS	ใช้สำหรับเข้ารหัสการสื่อสารระหว่าง Client และ Server

จากตารางที่ 2.16 จะเห็นได้ว่า API ทำงานร่วมกับมาตรฐานหลายประเภท ตั้งแต่มาตรฐานด้านการสื่อสาร รูปแบบข้อมูล ไปจนถึงกลไกด้านความปลอดภัย การเลือกใช้มาตรฐานแบบเปิดช่วยสนับสนุนให้ API สามารถเชื่อมโยงกับ Application และ Platform ที่หลากหลายได้มากขึ้น

แนวคิด API-First

API-First เป็นแนวคิดการออกแบบและพัฒนาระบบที่ให้ความสำคัญกับ API ตั้งแต่ช่วงเริ่มต้นของการออกแบบ โดย API จะถูกกำหนดโครงสร้าง หน้าที่ และรูปแบบการสื่อสารก่อนการพัฒนาระบบส่วนอื่น แนวคิดนี้ช่วยให้ API กลายเป็นองค์ประกอบพื้นฐานของระบบแทนที่จะถูกเพิ่มเติมภายหลัง

Office of Systems Engineering and Architecture (2026) ระบุว่า API-First Strategy ควรถูกให้ความสำคัญตั้งแต่เริ่มต้นการออกแบบระบบ โดย API ควรถูกมองเป็นองค์ประกอบพื้นฐานที่ได้รับการออกแบบก่อนการ Implementation เพื่อสนับสนุน Modularity, Scalability และ Reusability ของระบบ

แนวคิด API-First ยังช่วยแยกกระบวนการพัฒนา API ออกจากระบบส่วนอื่น ทำให้โครงสร้างของระบบมีความเป็น Modular มากขึ้น และช่วยให้ทีมพัฒนาส่วน Front-end และ Back-end สามารถทำงานคู่ขนานกันได้ตามขอบเขตความรับผิดชอบของแต่ละทีม

นอกจากนี้ การออกแบบ API ตั้งแต่ต้นยังช่วยให้สามารถกำหนดรูปแบบข้อมูล มาตรฐานการสื่อสาร และข้อกำหนดด้านความปลอดภัยได้อย่างชัดเจนก่อนเริ่มพัฒนา Application จริง ซึ่งช่วยลดความแตกต่างระหว่างระบบที่ต้องทำงานร่วมกันและลดปัญหาในขั้นตอน Integration

API-First ยังสัมพันธ์กับการกำหนด Common Data Model หรือ CDM ซึ่งเป็นโครงสร้างข้อมูลมาตรฐานที่ใช้ร่วมกันใน API Endpoint ต่าง ๆ เอกสารระบุว่า Common Data Model ช่วยสร้างความสอดคล้องของข้อมูล ลดความซับซ้อน และเพิ่ม Data Interoperability ระหว่าง Application และ Service

ดังนั้น แนวคิด API-First จึงไม่ได้มุ่งเน้นเพียงการสร้าง Interface สำหรับเชื่อมต่อระบบเท่านั้น แต่ยังช่วยกำหนดโครงสร้างการพัฒนาให้มีความเป็นระบบ รองรับการขยายตัว การนำกลับมาใช้ใหม่ และการทำงานร่วมกันของ Application ที่หลากหลาย

REST และมาตรฐานข้อมูลที่ใช้ร่วมกับ API

Representational State Transfer หรือ REST เป็นรูปแบบสถาปัตยกรรมที่ใช้ในการออกแบบ Networked Application และถูกนำมาใช้ร่วมกับ API อย่างแพร่หลาย เนื่องจากมีลักษณะ Lightweight และรองรับการขยายตัวของระบบได้

Office of Systems Engineering and Architecture (2026) ระบุว่า REST เป็น Architectural Style ที่มีน้ำหนักเบาและสามารถรองรับ Scalability สำหรับการพัฒนา Networked Applications ขณะที่ JSON เป็นรูปแบบข้อมูลที่มีโครงสร้างเรียบง่าย ขนาดกะทัดรัด และสามารถอ่านได้โดยมนุษย์

API ที่ใช้แนวทาง REST มักใช้ร่วมกับ HTTP เพื่อส่ง Request และ Response ระหว่าง Client และ Server โดยโครงสร้างของ API จะถูกกำหนดผ่าน Resource หรือ Endpoint ซึ่ง Consumer สามารถเรียกใช้งานตามเงื่อนไขที่กำหนดไว้

การใช้ REST ร่วมกับมาตรฐานแบบเปิดช่วยให้ API สามารถเชื่อมต่อกับระบบที่หลากหลายได้ง่ายขึ้น เนื่องจากลดการพึ่งพา Proprietary Protocol เอกสารแนะนำให้ใช้ Open Standards เช่น REST, OpenAPI, JSON และ XML เพื่อเพิ่ม Compatibility และ Interoperability ระหว่างระบบ

JSON (JavaScript Object Notation) เป็นรูปแบบการแทนข้อมูลที่มีโครงสร้างกะทัดรัดและมนุษย์สามารถอ่านได้ เหมาะสำหรับใช้แลกเปลี่ยนข้อมูลระหว่าง Application เนื่องจากสามารถนำไปใช้กับภาษาการเขียนโปรแกรมและ Platform ที่หลากหลายได้

XML (Extensible Markup Language) เป็นอีกรูปแบบหนึ่งที่ใช้สำหรับแทนและแลกเปลี่ยนข้อมูลระหว่างระบบ โดยเอกสารระบุว่า JSON และ XML เป็น Standardized Data Formats ที่ช่วยสนับสนุน Compatibility ของระบบ

นอกจากนี้ เอกสารยังกล่าวถึง SOAP ในฐานะหนึ่งในรูปแบบที่สามารถใช้ร่วมกับ HTTP, REST, JSON และ XML สำหรับการเชื่อมต่อและแลกเปลี่ยนข้อมูลระหว่าง Application

ตารางที่ 2.17 มาตรฐานและรูปแบบข้อมูลที่ใช้ในการพัฒนา API

รายการ	ลักษณะสำคัญ
REST	รูปแบบสถาปัตยกรรมสำหรับ Networked Application ที่มีลักษณะ Lightweight และรองรับ Scalability
HTTP	Protocol สำหรับการสื่อสารและรับส่ง Request/Response
OpenAPI	Specification สำหรับอธิบาย Endpoint โครงสร้าง และพฤติกรรมของ API
JSON	รูปแบบข้อมูลที่กะทัดรัด อ่านได้ง่าย และเหมาะสำหรับการแลกเปลี่ยนข้อมูล
XML	รูปแบบข้อมูลที่ใช้แทนและแลกเปลี่ยนข้อมูลระหว่างระบบ
SOAP	รูปแบบการสื่อสารระหว่างระบบที่สามารถใช้ร่วมกับ Web Service

จากตารางที่ 2.17 จะเห็นได้ว่า API ไม่ได้ทำงานโดยอาศัย Technology เพียงชนิดเดียว แต่ต้องทำงานร่วมกับมาตรฐานหลายประเภท ทั้งในด้านการสื่อสาร การอธิบาย Interface และรูปแบบข้อมูล ซึ่งช่วยให้ระบบสามารถแลกเปลี่ยนข้อมูลได้อย่างเป็นมาตรฐานและลดความซับซ้อนของการเชื่อมต่อระหว่าง Application

API Endpoint

API Endpoint หมายถึงจุดที่ Consumer ใช้สำหรับติดต่อกับ API เพื่อร้องขอข้อมูลหรือ Function จาก Provider System Endpoint จึงทำหน้าที่เป็นจุดเชื่อมต่อระหว่าง Consumer และ Provider โดยแต่ละ Endpoint สามารถถูกกำหนดให้รองรับ Function หรือ Resource ที่แตกต่างกัน

ในเอกสาร *API Technical Guidance* มีการอธิบาย Provider System ว่าเป็นระบบที่มีข้อมูลหรือ Function ที่ต้องการให้ระบบอื่นเข้าถึง และ Provider System จะจัดเตรียม API Endpoint ผ่าน Software หรือ Gateway เพื่อให้ External System สามารถส่ง Request เข้ามาได้

Endpoint สามารถมี Parameter เพื่อควบคุมลักษณะของข้อมูลที่ Consumer ต้องการ เช่น การกำหนดจำนวนรายการ การเรียงลำดับ หรือการกรองข้อมูล เอกสารกล่าวถึง Pagination, Sorting และ Filtering ในฐานะ Parameter สำคัญของ API ที่ช่วยเพิ่มประสิทธิภาพในการส่งข้อมูลและลดปริมาณข้อมูลที่ไม่จำเป็น

Pagination เป็นการแบ่งข้อมูลจำนวนมากออกเป็นหลายส่วนเพื่อไม่ให้ Client ต้องรับข้อมูลทั้งหมดในครั้งเดียว เอกสารระบุว่าการใช้ Parameter เช่น limit และ offset ช่วยควบคุมจำนวนข้อมูลในแต่ละหน้า ลด Payload Size และช่วยลดภาระของ Database เมื่อมีข้อมูลจำนวนมาก

Sorting ช่วยให้ Consumer สามารถกำหนดลำดับของข้อมูล เช่น การเรียงจากน้อยไปมากหรือมากไปน้อย ขณะที่ Filtering ช่วยเลือกเฉพาะข้อมูลที่ตรงตามเงื่อนไขที่กำหนด เช่น วันที่ หมวดหมู่ หรือ Location ซึ่งช่วยลดปริมาณข้อมูลที่ไม่เกี่ยวข้องและเพิ่มประสิทธิภาพของ Request และ Response

API Gateway

API Gateway เป็นองค์ประกอบที่ทำหน้าที่เป็นตัวกลางระหว่าง Client และ Service โดยใช้สำหรับบริหารจัดการการสื่อสารของ API จากจุดศูนย์กลาง

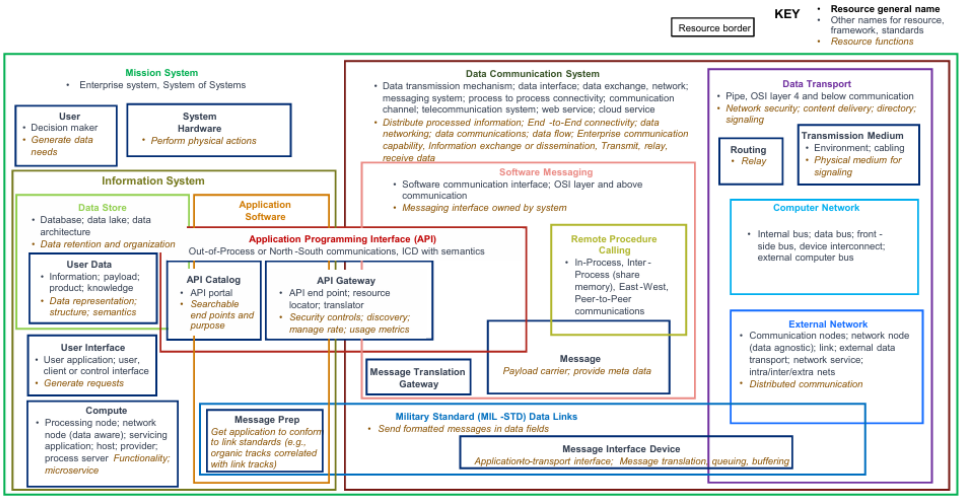
Office of Systems Engineering and Architecture (2026) อธิบายว่า API Gateway เป็น Component ของ Application Delivery Infrastructure ที่อยู่ระหว่าง Client และ Service และช่วยจัดการ API Communication แบบ Centralized

API Gateway สามารถทำหน้าที่หลายประการ เช่น การตรวจสอบ Authentication และ Authorization การกำหนดเส้นทางของ Request การตรวจสอบ Traffic การค้นหา Service และการจัดการ Rate ของการเรียกใช้ API

เอกสารระบุว่า API Gateway ที่มีประสิทธิภาพควรรองรับ Service Discovery, Authentication and Access Control, Load Balancing, Caching, Health Checking, Attack Detection, Security Logging, Monitoring และ Circuit Breakers

API Gateway จึงเป็นองค์ประกอบสำคัญในระบบที่มี Service หรือ API จำนวนมาก เนื่องจากสามารถรวมกระบวนการควบคุมการเข้าถึงและการตรวจสอบ Traffic ไว้ที่จุดเดียว ทำให้การจัดการและการรักษาความปลอดภัยสามารถทำได้อย่างเป็นระบบมากขึ้น

ในกรณีที่เกิด Denial-of-Service Attack หรือมี Traffic สูงเกินกว่าที่ระบบรองรับ API Gateway สามารถถูกกำหนดให้ลดหรือปฏิเสธ Traffic บางส่วนเพื่อป้องกัน Service ภายในจาก Overload ได้



ภาพที่ 2.6 ความสัมพันธ์ระหว่าง API และองค์ประกอบภายในระบบ

ที่มา: Office of Systems Engineering and Architecture (2026)

คุณลักษณะของ API ที่มีคุณภาพ

API ที่มีคุณภาพควรได้รับการออกแบบให้สามารถใช้งานได้ง่าย มีความน่าเชื่อถือ ปลอดภัย และรองรับการเปลี่ยนแปลงของระบบในอนาคต การออกแบบ API ที่ดีจึงไม่ได้พิจารณาเพียงความสามารถในการรับและส่งข้อมูลเท่านั้น แต่ยังต้องพิจารณาคุณลักษณะเชิงคุณภาพของ API ด้วย

Office of Systems Engineering and Architecture (2026) ระบุคุณลักษณะสำคัญของ High-Quality API หลายประการ เช่น Reusable, Reliable, Interoperable, Discoverable, Scalable, Consistent, Easy to Use, Clear, Secure, Compliant, Complete, Loose Coupling, High Cohesion และ Robustness

Reusable หมายถึงความสามารถในการนำ API หรือ Component ที่พัฒนาขึ้นกลับมาใช้ใน Application อื่น โดยช่วยลดความซ้ำซ้อนในการพัฒนา

Reliable หมายถึง API สามารถให้บริการได้อย่างสม่ำเสมอและทำงานตาม Documentation ที่กำหนด

Interoperable หมายถึง API สามารถถูกนำไปใช้ร่วมกับ Application หรือ System อื่นได้อย่างเหมาะสม

Discoverable หมายถึงนักพัฒนาสามารถค้นหา API และข้อมูลเกี่ยวกับ API ได้ง่าย ซึ่งช่วยเพิ่มโอกาสในการนำ API กลับมาใช้ใหม่

Scalable หมายถึง API สามารถรองรับจำนวนผู้ใช้หรือจำนวน Request ที่เพิ่มขึ้นได้

Consistent หมายถึง API มีรูปแบบการทำงานที่สอดคล้องกัน แม้ว่าจะถูกพัฒนาโดยผู้พัฒนาหลายคนหรือถูกนำไปใช้งานในหลายส่วนของระบบ

Easy to Use และ **Clear** หมายถึง API ควรมีโครงสร้างที่เข้าใจง่ายและมี Documentation ที่ชัดเจน เพื่อช่วยให้นักพัฒนาสามารถนำไปใช้งานได้อย่างถูกต้อง

Secure หมายถึง Security ถูกนำมาพิจารณาตั้งแต่การออกแบบและ Deployment ของ API ไม่ใช่ถูกเพิ่มเติมภายหลัง

Loose Coupling หมายถึงระบบที่ใช้งาน API ไม่จำเป็นต้องทราบรายละเอียดการทำงานภายในของ Service มากเกินความจำเป็น ซึ่งช่วยลดการพึ่งพากันระหว่างระบบ

High Cohesion หมายถึง API ควรรับและส่งเฉพาะข้อมูลที่จำเป็นสำหรับ Function ที่กำหนด เพื่อช่วยลด Complexity

Robustness หมายถึง API สามารถรับมือกับสถานการณ์ผิดปกติหรือปัญหาที่เกิดขึ้นได้อย่างเหมาะสม

ตารางที่ 2.18 คุณลักษณะสำคัญของ API ที่มีคุณภาพ

คุณลักษณะ	รายละเอียด
Reusable	สามารถนำ Component หรือ API กลับมาใช้ใหม่ได้
Reliable	สามารถให้บริการได้อย่างต่อเนื่องและทำงานตามที่กำหนด
Interoperable	สามารถเชื่อมต่อและทำงานร่วมกับระบบอื่นได้
Discoverable	สามารถค้นหา API และข้อมูลประกอบได้ง่าย
Scalable	รองรับจำนวนผู้ใช้งานและ Request ที่เพิ่มขึ้น
Consistent	มีรูปแบบและพฤติกรรมที่สม่ำเสมอ
Easy to Use	นักพัฒนาสามารถเข้าใจและใช้งานได้ง่าย
Clear	มีโครงสร้างและ Documentation ที่ชัดเจน
Secure	มี Security เป็นส่วนหนึ่งของการออกแบบ
Loose Coupling	ลดการพึ่งพารายละเอียดภายในระหว่างระบบ
High Cohesion	ทำหน้าที่เฉพาะและใช้ข้อมูลเท่าที่จำเป็น
Robustness	สามารถรับมือกับข้อผิดพลาดหรือสถานการณ์ผิดปกติได้

จากตารางที่ 2.18 จะเห็นได้ว่า API ที่มีคุณภาพต้องมีองค์ประกอบทั้งด้าน Functionality, Usability, Reliability, Scalability และ Security ซึ่งคุณลักษณะเหล่านี้มีผลต่อความสามารถในการนำ API ไปใช้ร่วมกับระบบอื่นในระยะยาว

ความปลอดภัยของ API

API Security เป็นองค์ประกอบสำคัญของการออกแบบและพัฒนาระบบ เนื่องจาก API เป็นจุดเชื่อมต่อที่เปิดให้ Application หรือ System ภายนอกสามารถร้องขอข้อมูลหรือ Function ได้ หากไม่มีมาตรการ Security ที่เหมาะสม อาจเกิดการเข้าถึงข้อมูลโดยไม่ได้รับอนุญาตหรือเกิด Vulnerability ภายในระบบ

เอกสารระบุว่า API สามารถเผชิญกับภัยคุกคามหลายประเภท เช่น Injection Attack, Authentication และ Authorization Problem, Data Breach และปัญหาด้าน Availability

แนวทางการรักษาความปลอดภัยที่สำคัญ ได้แก่ Authentication และ Authorization, Encryption, Input Validation, Output Encoding, Monitoring และ Logging

Authentication เป็นกระบวนการตรวจสอบว่า Entity หรือผู้ใช้ที่ร้องขอเข้าถึง API เป็นผู้ที่อ้างว่าเป็นจริงหรือไม่ ขณะที่ **Authorization** เป็นการตรวจสอบว่าสิ่งที่ผ่าน Authentication แล้วมีสิทธิ์เข้าถึง Resource หรือ Function ใดได้บ้าง

เอกสารแนะนำให้ใช้กลไก Authorization ที่มีความแข็งแรง รวมถึงการใช้ Digitally Signed Authentication Token การกำหนดอายุของ Token ให้เหมาะสม และหลีกเลี่ยงการฝัง API Key ที่มีข้อมูลสำคัญไว้ใน Request โดยตรง

การตรวจสอบ Input ยังมีความสำคัญต่อ API Security เนื่องจาก Input Validation ช่วยตรวจสอบและทำความสะอาดข้อมูลก่อนนำไปประมวลผล เพื่อลดความเสี่ยงจาก Injection Attack ขณะที่ Output Encoding ช่วยลดความเสี่ยงจากการนำข้อมูลที่ไม่น่าเชื่อถือไป Execute ในระบบปลายทาง

นอกจากนี้ การเข้ารหัสข้อมูลยังมีบทบาทในการรักษา Confidentiality และ Integrity ของข้อมูล โดยเอกสารระบุว่า SSL/TLS สามารถใช้สำหรับเข้ารหัสการสื่อสารระหว่าง Client และ Server เพื่อป้องกันการดักฟังข้อมูลระหว่างการส่งผ่านเครือข่าย

เอกสารยังระบุว่า OAuth และ JSON Web Token หรือ JWT สามารถนำมาใช้เพื่อสนับสนุน Authentication และ Authorization โดย OAuth ช่วยควบคุม Delegated Access ขณะที่ JWT ใช้สำหรับส่ง Claim ระหว่าง Parties ในรูปแบบที่เหมาะสมกับการตรวจสอบสิทธิ์

ตารางที่ 2.19 แนวทางสำคัญด้าน API Security

แนวทาง	วัตถุประสงค์
Authentication	ตรวจสอบตัวตนของผู้ใช้หรือ Application
Authorization	กำหนดสิทธิ์ในการเข้าถึง Resource
TLS/SSL	เข้ารหัสข้อมูลระหว่างการรับส่ง

แนวทาง	วัตถุประสงค์
OAuth	ควบคุม Delegated Access
JWT	ส่งข้อมูล Claim สำหรับ Authentication/Authorization
Input Validation	ตรวจสอบข้อมูลก่อนประมวลผล
Output Encoding	ลดความเสี่ยงจากข้อมูลที่อาจก่อให้เกิดการโจมตี
Monitoring and Logging	ตรวจสอบพฤติกรรมและค้นหากิจกรรมผิดปกติ
Security Testing	ตรวจหาช่องโหว่ก่อนและหลัง Deployment

การรักษาความปลอดภัยจึงควรดำเนินการตลอด API Life Cycle ตั้งแต่การออกแบบ การพัฒนา การทดสอบ การนำไปใช้ และการ Monitoring ภายหลัง Deployment ไม่ควรถูกมองว่าเป็นขั้นตอนที่ดำเนินการเฉพาะในช่วงท้ายของการพัฒนา

API Versioning และ Life Cycle Management

API มีการเปลี่ยนแปลงตามความต้องการของระบบ การเพิ่ม Function ใหม่ หรือการปรับปรุง Security ดังนั้นการจัดการ Version จึงเป็นองค์ประกอบสำคัญในการดูแล API

Office of Systems Engineering and Architecture (2026) แนะนำให้ใช้ Versioning Strategy ที่ชัดเจน เช่น Semantic Versioning เพื่อรักษา Backward Compatibility และกำหนด Upgrade Path ที่ชัดเจนสำหรับ Consumer

API Governance ควรครอบคลุม Versioning, Service Discovery, Stakeholder Engagement และ Life Cycle Management เพื่อให้ API มีความสอดคล้องและสามารถปรับตัวได้เมื่อระบบเปลี่ยนแปลง

เมื่อมี API Version ใหม่ Provider ควรมี Communication Plan เพื่อแจ้ง Consumer เกี่ยวกับ Feature ใหม่ การเปลี่ยนแปลง และแผนการยกเลิก Version เดิม เอกสารยังระบุว่าการรองรับ API หลาย Version พร้อมกันอาจเพิ่ม Infrastructure และ Labor Cost จึงควรกำหนดเงื่อนไขการ Deprecation อย่างชัดเจน

ดังนั้น API Life Cycle จึงครอบคลุมตั้งแต่ Design, Development, Deployment, Maintenance, Versioning จนถึง Deprecation หรือ Retirement ของ API

การทดสอบ API

API เช่นเดียวกับ Software ประเภทอื่นอาจเกิด Bug หรือ Vulnerability ได้ โดยเฉพาะ API ซึ่งเป็น System Access Point หากมีข้อผิดพลาดอาจเปิดโอกาสให้เกิดการเข้าถึงหรือพฤติกรรมที่ไม่คาดคิดภายในระบบ เอกสารจึงระบุว่าการออกแบบ Implementation และ Continuous Testing มีความสำคัญต่อการสร้างความมั่นใจว่า API ทำงานตามที่กำหนด

การทดสอบ API สามารถดำเนินการได้หลายระดับ เช่น Unit Testing, Integration Testing และ Application หรือ End-to-End Testing

Unit Testing เป็นการทดสอบ Component หรือ Function ขนาดเล็กเพื่อให้แน่ใจว่าสามารถทำงานได้อย่างถูกต้องในระดับอิสระ

Integration Testing เป็นการตรวจสอบว่า Component, Module หรือ Service ที่ถูกพัฒนาแยกจากกันสามารถทำงานร่วมกันได้อย่างถูกต้องเมื่อเชื่อมต่อเข้าด้วยกัน

Application Testing หรือ End-to-End Testing เป็นการตรวจสอบการทำงานโดยรวมของ Application รวมถึง Input, Output และ Service ที่เกี่ยวข้อง เพื่อประเมินว่าระบบทำงานได้ตามข้อกำหนดใน Environment ที่ใกล้เคียงกับการใช้งานจริงหรือไม่

นอกจากนี้ ยังมีการทดสอบด้าน Security เช่น Penetration Testing, Fuzz Testing, Static Application Security Testing และ Dynamic Application Security Testing เพื่อช่วยค้นหา Vulnerability ภายใน API

ตารางที่ 2.20 รูปแบบการทดสอบ API

การทดสอบ	วัตถุประสงค์
Unit Testing	ตรวจสอบ Function หรือ Component แต่ละส่วน
Integration Testing	ตรวจสอบการทำงานร่วมกันของ Module และ Service
End-to-End Testing	ตรวจสอบกระบวนการทำงานของระบบโดยรวม
Functional Testing	ตรวจสอบว่า API ทำงานตาม Function ที่กำหนด
Penetration Testing	จำลองการโจมตีเพื่อค้นหา Vulnerability
Fuzz Testing	ส่งข้อมูลผิดรูปแบบหรือไม่คาดคิดเพื่อตรวจหาข้อผิดพลาด
Security Code Review	ตรวจสอบ Source Code เพื่อค้นหาปัญหาด้าน Security

จากตารางที่ 2.20 การทดสอบ API จำเป็นต้องครอบคลุมทั้งด้าน Functionality, Integration และ Security เพื่อให้มั่นใจว่า API สามารถทำงานได้อย่างถูกต้องและมีความปลอดภัยก่อนนำไปใช้ใน Production Environment

API Monitoring และ Metrics

หลังจาก API ถูกนำไปใช้งานแล้ว การ Monitoring มีบทบาทสำคัญในการตรวจสอบ Performance, Availability, Security และรูปแบบการใช้งานของ Consumer

เอกสารระบุว่า API สามารถใช้เป็นแหล่งข้อมูลสำหรับ Monitoring ระบบ โดยข้อมูลจากการใช้งานสามารถช่วยตรวจพบ Pattern การใช้งาน ข้อผิดพลาด และพฤติกรรมที่ไม่ถูกต้อง รวมทั้งช่วยในการวางแผน Versioning และการปรับปรุงระบบ

API Metrics ที่สำคัญประกอบด้วย Response Time, Error Rate, Availability, Usage, Traffic, Growth, Latency, Rate Limiting, Authentication and Authorization Success Rate และ SLA Compliance

Response Time ใช้วัดระยะเวลาที่ API ใช้ในการตอบกลับ Request ซึ่งสามารถช่วยตรวจหาปัญหา Performance หรือ Bottleneck

Error Rate ใช้วัดสัดส่วน Request ที่เกิด Error เพื่อช่วยตรวจสอบ Stability และ Functionality ของ API

Availability ใช้วัด Uptime และความพร้อมในการให้บริการ API

Usage และ Traffic ใช้วัดจำนวน Request และปริมาณข้อมูลที่ถูกประมวลผล เพื่อใช้ประเมินรูปแบบการใช้งานและความต้องการด้าน Scalability

Latency ใช้วัดระยะเวลาที่ข้อมูลเดินทางจาก Client ไปยัง API Server และกลับมายัง Client

Rate Limiting ช่วยควบคุมจำนวน Request ต่อช่วงเวลาเพื่อลดความเสี่ยงจากการใช้งานเกินขีดจำกัดหรือการโจมตี

เอกสารยังกล่าวถึง SLA Compliance, API Retention และ Memory Usage เป็น Metrics สำหรับประเมินประสิทธิภาพและความน่าเชื่อถือของ API ในระยะยาว

ตารางที่ 2.21 ตัวชี้วัดสำคัญสำหรับ API

Metric	รายละเอียด
Response Time	เวลาที่ API ใช้ในการตอบกลับ Request
Error Rate	สัดส่วน Request ที่เกิด Error
Availability	ความพร้อมและ Uptime ของ API
Usage / Traffic	จำนวน Request และปริมาณข้อมูลที่ใช้งาน

Metric	รายละเอียด
Latency	ระยะเวลาในการเดินทางของข้อมูลระหว่าง Client และ Server
Rate Limiting	จำนวน Request ที่อนุญาตภายในช่วงเวลาที่กำหนด
Authentication / Authorization	ผลสำเร็จของกระบวนการตรวจสอบตัวตนและสิทธิ์
SLA Compliance	ระดับการปฏิบัติตาม Service-Level Agreement
Memory Usage	ปริมาณหน่วยความจำที่ API ใช้งาน

การติดตาม Metrics เหล่านี้ช่วยให้สามารถตรวจสอบประสิทธิภาพของ API ได้อย่างเป็นระบบ และช่วยให้ผู้ดูแลระบบสามารถตัดสินใจเกี่ยวกับการปรับปรุง Performance, Scalability และ Reliability โดยอาศัยข้อมูลจากการใช้งานจริง

สรุปแนวคิด Application Programming Interface

Application Programming Interface เป็นกลไกสำคัญที่ช่วยให้ Application และ System ที่แตกต่างกันสามารถติดต่อ แลกเปลี่ยนข้อมูล และเรียกใช้ Function ระหว่างกันได้โดยผ่าน Interface ที่กำหนด API ช่วยลดความจำเป็นที่ระบบหนึ่งต้องทราบรายละเอียดการทำงานภายในของอีกระบบ และสนับสนุนการพัฒนา Software ที่มีความเป็น Modular มากขึ้น

แนวทาง API-First ช่วยให้ API ถูกออกแบบเป็นองค์ประกอบพื้นฐานของระบบตั้งแต่เริ่มต้น โดยเน้น Modularity, Scalability และ Reusability ขณะเดียวกัน การใช้ Open Standards เช่น REST, OpenAPI, JSON และ XML ช่วยเพิ่มความสามารถด้าน Interoperability และลดการผูกติดกับเทคโนโลยีเฉพาะ

API ที่มีคุณภาพควรมีคุณลักษณะด้าน Reliability, Interoperability, Discoverability, Scalability, Security, Loose Coupling และ Robustness รวมถึงมี Documentation ที่ชัดเจนและง่ายต่อการใช้งาน

ในด้านการรักษาความปลอดภัย API จำเป็นต้องใช้กลไก Authentication, Authorization, Encryption, Input Validation, Monitoring และ Security Testing เพื่อป้องกันการเข้าถึงหรือการโจมตีที่ไม่ได้รับอนุญาต ขณะที่การจัดการ Version, Testing และ Monitoring ช่วยให้ API สามารถรักษาคุณภาพและรองรับการเปลี่ยนแปลงของระบบตลอด Software Life Cycle

ด้วยเหตุนี้ API จึงถือเป็นองค์ประกอบสำคัญของระบบ Software สมัยใหม่ที่ต้องการเชื่อมโยง Application, Data และ Service จากหลายแหล่งให้สามารถทำงานร่วมกันได้อย่างเป็นมาตรฐาน มีความยืดหยุ่น และรองรับการขยายตัวในอนาคต

2.1.4 Cross-Platform Application Development

ความหมายและแนวคิดของ Cross-Platform Application Development

การพัฒนาแอปพลิเคชันในปัจจุบันมีความจำเป็นต้องรองรับอุปกรณ์และระบบปฏิบัติการที่หลากหลาย โดยเฉพาะ Android และ iOS ซึ่งเป็นระบบปฏิบัติการหลักสำหรับอุปกรณ์พกพา ความแตกต่างระหว่างแต่ละแพลตฟอร์มส่งผลให้ผู้พัฒนาต้องพิจารณารูปแบบและเทคโนโลยีที่เหมาะสมสำหรับการพัฒนาแอปพลิเคชัน เพื่อให้สามารถรองรับผู้ใช้งานบนแพลตฟอร์มที่แตกต่างกันได้อย่างมีประสิทธิภาพ

Lahtinen (2024) ระบุว่า ความนิยมที่เพิ่มขึ้นของอุปกรณ์พกพาส่งผลให้ความต้องการ Mobile Application เพิ่มสูงขึ้น อย่างไรก็ตาม การพัฒนา Application ให้สามารถทำงานบน Platform ที่แตกต่างกันถือเป็นความท้าทาย เนื่องจากแต่ละ Platform มีข้อกำหนดและแนวทางในการพัฒนาที่แตกต่างกัน จึงเกิดแนวคิด Cross-Platform Mobile Development เพื่อช่วยให้ Application สามารถทำงานบนหลาย Platform ได้

Cross-Platform Application Development หมายถึงแนวทางการพัฒนา Application ที่สามารถนำ Source Code หรือ Codebase ส่วนหนึ่งมาใช้ร่วมกันเพื่อให้ Application ทำงานบนระบบปฏิบัติการหรือ Platform หลายประเภทได้ แนวทางดังกล่าวแตกต่างจาก Native Development ซึ่งต้องพัฒนา Application แยกสำหรับแต่ละ Platform

เอกสารอธิบายว่า ในปัจจุบันผู้พัฒนา Mobile Application มีแนวทางหลักสองรูปแบบ ได้แก่ Native Development และ Cross-Platform Development โดย Cross-Platform Development ช่วยให้ผู้พัฒนาสามารถสร้าง Application ที่ทำงานได้บนระบบปฏิบัติการหลายระบบ และสามารถแบ่งปัน Code ระหว่าง Platform ต่าง ๆ ได้

แนวคิดดังกล่าวมีจุดประสงค์สำคัญในการลดความซ้ำซ้อนของการพัฒนา เนื่องจากหากพัฒนา Application แบบ Native ผู้พัฒนาอาจจำเป็นต้องสร้าง Codebase แยกสำหรับ Android และ iOS ในขณะที่ Cross-Platform Development สามารถใช้ Code ส่วนใหญ่ร่วมกันได้ ทำให้การพัฒนาและการบำรุงรักษามีความเป็นศูนย์กลางมากขึ้น

Lahtinen (2024) ระบุว่า Cross-Platform Framework เช่น Flutter, React Native และ Kotlin Multiplatform Mobile ช่วยสนับสนุนการใช้ Code ร่วมกันระหว่าง Platform และสามารถเข้าถึงความสามารถทั่วไปของอุปกรณ์ได้

ดังนั้น Cross-Platform Application Development จึงสามารถมองได้ว่าเป็นแนวทางการพัฒนาซอฟต์แวร์ที่มีเป้าหมายในการเพิ่มประสิทธิภาพของกระบวนการพัฒนา ลดการสร้าง Code ที่ซ้ำซ้อน และช่วยให้ Application สามารถให้บริการแก่ผู้ใช้งานบน Platform ที่หลากหลายโดยใช้โครงสร้างการพัฒนาร่วมกัน

Native Application Development

Native Application Development เป็นแนวทางการพัฒนา Application สำหรับระบบปฏิบัติการใดระบบปฏิบัติการหนึ่งโดยเฉพาะ เช่น Android หรือ iOS โดยผู้พัฒนาจะใช้ภาษา เครื่องมือ และ Framework ที่ถูกออกแบบมาให้เหมาะสมกับ Platform นั้น

Lahtinen (2024) อธิบายว่า Native Development เป็นการพัฒนา Mobile Application สำหรับ Mobile Operating System เฉพาะ โดยผู้พัฒนาจะใช้ Platform-specific Programming Languages และ Development Tools เพื่อปรับ Application ให้ทำงานได้อย่างเหมาะสมกับ Platform เป้าหมาย

ข้อได้เปรียบสำคัญของ Native Development คือ Application สามารถได้รับการปรับแต่งให้เหมาะสมกับระบบปฏิบัติการโดยตรง ทำให้มี Performance และ Responsiveness ที่ดี อีกทั้งสามารถปฏิบัติตาม Platform-specific Design Guidelines เพื่อสร้างประสบการณ์ใช้งานที่มีความสอดคล้องกับ Platform นั้นได้

อีกคุณสมบัติหนึ่งของ Native Application คือความสามารถในการเข้าถึง Hardware Feature ของอุปกรณ์ได้โดยตรง เนื่องจาก Application ถูกพัฒนาด้วยเครื่องมือที่ออกแบบมาสำหรับ Platform นั้น จึงสามารถเรียกใช้ความสามารถต่าง ๆ ของอุปกรณ์ตาม API ที่ระบบปฏิบัติการจัดเตรียมไว้ได้

อย่างไรก็ตาม Native Development มีข้อจำกัดในด้านการพัฒนาและบำรุงรักษา เนื่องจากแต่ละ Platform ต้องมี Codebase แยกจากกัน หากต้องการรองรับทั้ง Android และ iOS อาจจำเป็นต้องมีทีมพัฒนาและเครื่องมือแยกสำหรับแต่ละ Platform ซึ่งทำให้ต้นทุนและความต้องการด้านการประสานงานเพิ่มขึ้น

การดูแลรักษา Codebase หลายชุดยังใช้เวลาและอาจเพิ่มโอกาสเกิดข้อผิดพลาด นอกจากนี้ Platform-specific Logic อาจส่งผลให้ Application บน Android และ iOS มีพฤติกรรมหรือความสามารถที่ไม่สอดคล้องกันหากแต่ละ Version ได้รับการพัฒนาและปรับปรุงในช่วงเวลาที่แตกต่างกัน

ดังนั้น Native Development จึงเหมาะกับกรณีที่ต้องการใช้ความสามารถเฉพาะของ Platform อย่างเต็มที่หรือให้ความสำคัญกับ Performance และ Native User Experience อย่าง

มาก แต่ต้องแลกกับภาระในการดูแล Codebase หลายชุดเมื่อ Application ต้องรองรับหลายระบบปฏิบัติการ

Cross-Platform Application Development

Cross-Platform Development เป็นแนวทางที่ช่วยให้ Application สามารถทำงานบนระบบปฏิบัติการหลายระบบจากการพัฒนาร่วมกัน โดยใช้ Framework หรือ Development Tool ที่ช่วยสร้าง Abstraction ระหว่าง Source Code กับ Platform ปลายทาง

Lahtinen (2024) ระบุว่า Cross-Platform Development เป็นการสร้าง Application ที่สามารถทำงานบน Operating System หลายระบบ และการใช้ Code ร่วมกันระหว่าง Platform สามารถช่วยลด Development Time และ Cost ได้

Cross-Platform Framework มีบทบาทสำคัญในการช่วยให้ผู้พัฒนาสามารถสร้าง Application โดยใช้แนวทางการพัฒนาเดียวกันแล้วนำไปใช้งานกับหลาย Platform ตัวอย่าง Framework ที่เอกสารกล่าวถึง ได้แก่ Flutter, React Native และ Kotlin Multiplatform Mobile

การใช้ Codebase ร่วมกันช่วยลดความจำเป็นในการสร้าง Function เดียวกันซ้ำหลายครั้งสำหรับแต่ละระบบปฏิบัติการ ตัวอย่างเช่น Business Logic, Data Model หรือ User Interface บางส่วนสามารถถูกพัฒนาร่วมกันแล้วนำไปใช้กับ Application บน Platform ที่แตกต่างกันได้

ข้อดีดังกล่าวช่วยให้ทีมพัฒนาสามารถจัดการ Source Code จากส่วนกลาง และเมื่อมีการปรับปรุง Function ที่สามารถใช้ร่วมกัน การเปลี่ยนแปลงสามารถถูกนำไปใช้กับหลาย Platform ได้โดยไม่ต้องแก้ไข Application แต่ละ Version แยกกันทั้งหมด

อย่างไรก็ตาม การใช้ Cross-Platform Framework ยังมีข้อจำกัด เอกสารระบุว่า Application แบบ Cross-Platform อาจมีปัญหาด้าน Performance เนื่องจากมี Abstraction Layer ของ Framework อยู่ระหว่าง Application และ Platform นอกจากนี้ การเข้าถึง Platform-specific API อาจต้องใช้ความพยายามเพิ่มเติมในการปรับแต่ง

อีกหนึ่งความท้าทายคือการรักษาความสอดคล้องของ User Interface ระหว่าง Platform เนื่องจาก Shared UI Component อาจไม่สามารถเลียนแบบ Native Design Pattern ของแต่ละระบบปฏิบัติการได้อย่างสมบูรณ์ การเลือกใช้ Cross-Platform Development จึงควรพิจารณาความต้องการของโครงการ กลุ่มผู้ใช้งานเป้าหมาย และทรัพยากรที่มีอยู่ร่วมกัน

การเปรียบเทียบ Native Development และ Cross-Platform Development

Native Development และ Cross-Platform Development มีเป้าหมายในการสร้าง Application ให้ผู้ใช้งานสามารถเข้าถึง Function ที่ต้องการเช่นเดียวกัน แต่แตกต่างกันในวิธีการพัฒนา การจัดการ Source Code การรองรับ Platform และทรัพยากรที่ใช้

ตารางที่ 2.22 การเปรียบเทียบ Native Development และ Cross-Platform Development

ประเด็น	Native Development	Cross-Platform Development
Codebase	ใช้ Codebase แยกสำหรับแต่ละ Platform	สามารถใช้ Code ร่วมกันระหว่างหลาย Platform
Platform	พัฒนาเฉพาะระบบปฏิบัติการเป้าหมาย	รองรับหลายระบบปฏิบัติการ
Performance	สามารถปรับแต่งให้เหมาะกับ Platform โดยตรง	อาจได้รับผลกระทบจาก Abstraction Layer
Hardware Access	เข้าถึง Hardware Feature โดยตรงได้ดี	รองรับ Feature ทั่วไป แต่ Feature เฉพาะอาจต้องปรับแต่ง
Development Time	ต้องพัฒนาแยกเมื่อรองรับหลาย Platform	ลดเวลาได้จากการใช้ Code ร่วมกัน
Development Cost	อาจสูงขึ้นเมื่อมีหลาย Codebase	สามารถลดต้นทุนจากการพัฒนาร่วมกัน
Maintenance	ต้องดูแลหลาย Codebase	สามารถดูแล Source Code ส่วนใหญ่จากฐานเดียว
User Interface	ปรับให้ตรงกับ Native Design ได้โดยตรง	อาจต้องปรับแต่งเพื่อให้เหมาะกับแต่ละ Platform
Team Coordination	อาจต้องมีทีมแยกตาม Platform	ทีมเดียวสามารถดูแลหลาย Platform ได้มากขึ้น

จากตารางที่ 2.22 จะเห็นได้ว่า Native Development มีข้อได้เปรียบด้าน Performance การเข้าถึง Platform-specific Feature และความสามารถในการออกแบบให้สอดคล้องกับ Native Platform โดยตรง ขณะที่ Cross-Platform Development มีข้อได้เปรียบด้านการใช้ Code ร่วมกัน เวลาในการพัฒนา และการลดภาระในการดูแล Codebase หลายชุด

การเลือกแนวทางใดจึงไม่สามารถพิจารณาจากปัจจัยเพียงด้านเดียว แต่ควรพิจารณาจาก Project Requirement, Target Audience และ Available Resources ซึ่งเป็นปัจจัยที่เอกสารระบุว่ามีความสำคัญในการตัดสินใจเลือกระหว่าง Native และ Cross-Platform Development

หลักการใช้ Codebase ร่วมกันหลายแพลตฟอร์ม

แนวคิดพื้นฐานของ Cross-Platform Development คือการแบ่งปัน Source Code ระหว่าง Platform เพื่อให้ Function ต่าง ๆ สามารถนำไปใช้ร่วมกันได้ โดยแทนที่จะสร้าง Application แยกอย่างสมบูรณ์สำหรับ Android และ iOS ผู้พัฒนาสามารถใช้ Codebase หลักร่วมกันและให้ Framework จัดการส่วนที่เกี่ยวข้องกับ Platform

เอกสารกล่าวถึงแนวคิดดังกล่าวตั้งแต่บทนำ โดยระบุว่า Cross-Platform Mobile Framework ช่วยให้ผู้พัฒนาสามารถ “เขียน Code ครั้งเดียวและ Deploy ไปยังหลาย Platform” ซึ่งช่วยตอบสนองต่อความต้องการ Application ที่สามารถทำงานบน Android และ iOS ได้

การใช้ Codebase ร่วมกันมีผลต่อกระบวนการพัฒนาในหลายด้าน ประการแรกคือการลด Code Duplication เพราะ Logic หรือ Component ที่ทำงานเหมือนกันบนแต่ละ Platform ไม่จำเป็นต้องถูกสร้างใหม่ทั้งหมด

ประการที่สองคือการรักษาความสอดคล้องของ Function หาก Function หลักถูกจัดเก็บอยู่ใน Codebase เดียวกัน การเปลี่ยนแปลงหรือแก้ไขสามารถถูกควบคุมจาก Source Code หลัก ซึ่งช่วยลดความแตกต่างของ Function ระหว่าง Platform

ประการที่สามคือการบำรุงรักษา เมื่อเกิดข้อผิดพลาดในส่วนที่ใช้ Code ร่วมกัน ผู้พัฒนาสามารถแก้ไขจาก Codebase กลาง แทนการแก้ปัญหาเดียวกันซ้ำในแต่ละ Native Application

อย่างไรก็ตาม แนวคิด Single หรือ Shared Codebase ไม่ได้หมายความว่า Source Code ทุกส่วนจะสามารถใช้ร่วมกันได้เสมอ ในกรณีที่ Application ต้องเรียกใช้ Function เฉพาะของ Platform อาจ จำเป็นต้องมี Platform-specific Modification เพิ่มเติม ซึ่งเป็นหนึ่งในข้อจำกัดที่ Lahtinen (2024) กล่าวถึงในการพัฒนาแบบ Cross-Platform

ข้อดีของ Cross-Platform Application Development

ข้อดีหลักของ Cross-Platform Development คือ **การลดระยะเวลาในการพัฒนา** เนื่องจาก Source Code สามารถแบ่งปันระหว่าง Platform ได้ ผู้พัฒนาจึงไม่จำเป็นต้องสร้าง Application ทั้งระบบใหม่สำหรับแต่ละ Operating System

Lahtinen (2024) ระบุว่าการแบ่งปัน Code ระหว่าง Platform สามารถช่วยลดทั้ง Development Time และ Development Cost ซึ่งเป็นหนึ่งในเหตุผลสำคัญที่ทำให้ Cross-Platform Framework ถูกนำมาใช้ในการพัฒนา Mobile Application

ข้อดีอีกด้านคือ **การลดความซ้ำซ้อนของ Source Code** เนื่องจาก Function ที่ทำงานเหมือนกันสามารถถูกพัฒนาใน Codebase หลัก ทำให้กระบวนการแก้ไขและเพิ่ม Function มีความเป็นศูนย์กลางมากขึ้น

ความสะดวกในการบำรุงรักษา

เป็นผลต่อเนื่องจากการใช้ Shared Codebase เพราะผู้พัฒนาไม่จำเป็นต้องติดตามการเปลี่ยนแปลงของ Application หลายชุดที่แยกจากกันทั้งหมด ซึ่งช่วยลดโอกาสเกิดความไม่สอดคล้องของ Function ระหว่าง Platform

นอกจากนี้ Cross-Platform Framework ยังช่วยให้ทีมพัฒนาสามารถรองรับกลุ่มผู้ใช้งานที่มีอุปกรณ์แตกต่างกันได้จากกระบวนการพัฒนาร่วมกัน ตัวอย่างเช่น Flutter ในเอกสารสามารถใช้สำหรับสร้าง Application ที่รองรับ iOS, Android, Web, macOS และ Windows ได้

ในส่วนของ Flutter โดยเฉพาะ เอกสารระบุว่าความสามารถในการรองรับ Cross-platform ช่วยลด Development Effort และ Development Time ขณะที่ Hot Reload ช่วยให้สามารถแก้ไข Code และเห็นผลของการเปลี่ยนแปลงได้อย่างรวดเร็ว

ตารางที่ 2.23 ข้อดีของ Cross-Platform Application Development

ข้อดี	รายละเอียด
Code Reusability	สามารถนำ Source Code ส่วนใหญ่กลับมาใช้บนหลาย Platform
Reduced Development Time	ลดเวลาในการสร้าง Application แยกสำหรับแต่ละ Platform
Reduced Development Cost	ลดภาระด้านทรัพยากรจากการพัฒนา Codebase หลายชุด
Easier Maintenance	สามารถแก้ไขและดูแล Source Code จากโครงสร้างร่วมกัน
Consistency	ช่วยให้ Function ของ Application บนแต่ละ Platform มีความสอดคล้องกัน

ข้อดี	รายละเอียด
Multi-Platform Support	สามารถนำ Application ไปใช้บนระบบปฏิบัติการหลายประเภท
Faster Development Cycle	Framework บางประเภทมีเครื่องมือช่วยให้แก้ไขและทดสอบได้รวดเร็วขึ้น

จากตารางที่ 2.23 ข้อได้เปรียบของ Cross-Platform Development ส่วนใหญ่เกี่ยวข้องกับการลดความซ้ำซ้อนของกระบวนการพัฒนาและการใช้ทรัพยากรร่วมกัน ซึ่งมีความสำคัญโดยเฉพาะเมื่อต้องพัฒนา Application สำหรับหลาย Platform ในเวลาเดียวกัน

ข้อจำกัดของ Cross-Platform Application Development

แม้ Cross-Platform Development จะช่วยลดระยะเวลาและต้นทุนในการพัฒนา แต่ไม่ได้หมายความว่าจะเหมาะสมกับ Application ทุกประเภท เนื่องจาก Framework ทำหน้าที่เป็นชั้นกลางระหว่าง Source Code และ Platform จริง ซึ่งอาจสร้างข้อจำกัดบางประการ

Lahtinen (2024) ระบุว่า Cross-Platform Application อาจเกิด Performance Issue จาก Abstraction Layer ที่ Framework เพิ่มเข้ามา เนื่องจาก Source Code ไม่ได้เชื่อมต่อกับ Platform โดยตรงในลักษณะเดียวกับ Native Application

ข้อจำกัดอีกประการคือ **Platform-specific API** หากต้องการใช้ความสามารถที่มีเฉพาะใน Android หรือ iOS อาจต้องเพิ่ม Source Code หรือ Plugin ที่ออกแบบเฉพาะ Platform ทำให้การใช้ Shared Codebase ไม่สามารถครอบคลุม Source Code ทั้งหมดได้เสมอไป

ด้าน User Interface ก็อาจเป็นข้อจำกัด เนื่องจากแต่ละ Platform มี Native Design Pattern ที่แตกต่างกัน การนำ UI Component เดียวกันไปใช้กับหลาย Platform อาจไม่สามารถสร้างประสบการณ์ที่เหมือน Native Application ได้ในทุกกรณี

นอกจากนี้ ความเหมาะสมของ Framework แต่ละชนิดยังแตกต่างกัน ทั้งในด้าน Programming Language, Performance, Development Tool, Platform Support, Community Support และ Learning Curve จึงจำเป็นต้องพิจารณา Framework ตามเงื่อนไขของโครงการ ไม่ควรเลือกจากความสามารถในการรองรับหลาย Platform เพียงอย่างเดียว

ตารางที่ 2.24 ข้อดีและข้อจำกัดของ Cross-Platform Application Development

ด้าน	ข้อดี	ข้อจำกัด
Source Code	ใช้ Code ร่วมกันได้	บาง Function ยังต้องใช้ Platform-specific Code
เวลา	ลด Development Time	การแก้ปัญหาเฉพาะ Platform อาจเพิ่มเวลา
ต้นทุน	ลดการสร้าง Codebase ซ้ำ	อาจมีต้นทุนเพิ่มเติมในการปรับแต่ง Platform
Performance	Framework บางประเภทให้ประสิทธิภาพใกล้ Native	Abstraction Layer อาจส่งผลต่อ Performance
User Interface	ใช้ Component ร่วมกันได้	อาจไม่ตรง Native Design Pattern ทุก Platform
Device Features	Framework รองรับ Feature ทั่วไป	Feature เฉพาะอาจต้องใช้ Plugin หรือ Native Code
Maintenance	ดูแลระบบจาก Codebase กลางได้	ต้องติดตามการเปลี่ยนแปลงของแต่ละ Platform และ Framework

จากการเปรียบเทียบดังกล่าวจะเห็นได้ว่า Cross-Platform Development เป็นการแลกเปลี่ยนระหว่างความสะดวกและประสิทธิภาพในการพัฒนา กับข้อจำกัดของการเข้าถึงความสามารถเฉพาะของ Platform ดังนั้นการพิจารณา Project Requirement, Target Platform, Performance Requirement และ Available Resources จึงเป็นส่วนสำคัญในการเลือกแนวทางการพัฒนา

Framework สำหรับ Cross-Platform Application Development

Framework สำหรับการพัฒนาแบบ Cross-Platform มีบทบาทสำคัญในการช่วยให้ผู้พัฒนาสามารถสร้าง Application ที่รองรับระบบปฏิบัติการหลายประเภทจากกระบวนการพัฒนาร่วมกัน โดย Framework แต่ละประเภทมีแนวทางในการจัดการ User Interface, Performance, Development Tool และ Platform Support ที่แตกต่างกัน จึงต้องพิจารณาให้เหมาะสมกับลักษณะของระบบและความต้องการของโครงการ

Lahtinen (2024) เปรียบเทียบ Framework สำหรับ Cross-Platform Development จำนวน 3 Framework ได้แก่ Flutter, React Native และ .NET MAUI โดยพิจารณาจาก

Development Language, User Interface, Performance, Development Tools, Platform Support, Community Support, Learning Curve และ Long-term Viability

Flutter ใช้ภาษา Dart ในการพัฒนาและมีลักษณะเด่นด้านการสร้าง User Interface ผ่าน Customizable Widgets และ Rendering Engine ของ Framework ขณะที่ React Native ใช้ JavaScript และ React และเน้นการสร้าง Interface ที่มีลักษณะใกล้เคียง Native ส่วน .NET MAUI ใช้ภาษา C# และทำงานร่วมกับ Ecosystem ของ .NET

ในด้าน Development Tools เอกสารระบุว่า Flutter มี Flutter DevTools และสามารถทำงานร่วมกับ Integrated Development Environment หลายประเภท React Native มี Community Support, Hot Reloading และ IDE Integration ขณะที่ .NET MAUI มีเครื่องมือจาก Ecosystem ของ Microsoft เช่น Visual Studio และ Visual Studio Code

ด้าน Platform Support ทั้ง Flutter และ React Native ในตารางรองรับ iOS, Android, Web, macOS และ Windows ขณะที่ .NET MAUI รองรับ iOS, Android, macOS และ Windows

ตารางที่ 2.25 การเปรียบเทียบ Framework สำหรับ Cross-Platform Application Development

ประเด็น	Flutter	React Native	.NET MAUI
Development Language	Dart	JavaScript, React	C#
User Interface	Customizable Widgets และ Rendering Engine	Customizable และเน้น Native Look and Feel	Native-like UI บนหลาย Platform
Performance	เอกสารระบุว่าให้ประสิทธิภาพสูงและรองรับ 60 FPS	รองรับ Native Modules เพื่อช่วยด้าน Performance	ใช้ Native UI และ .NET Libraries
Development Tools	Flutter DevTools และ IDE Support	Hot Reloading และ IDE Integration	Visual Studio และ VS Code
Platform Support	iOS, Android, Web, macOS, Windows	iOS, Android, Web, macOS, Windows	iOS, Android, macOS, Windows
Community Support	Community ที่กำลังเติบโตและได้รับการ	Community ขนาดใหญ่	ได้รับการสนับสนุนจาก Microsoft

ประเด็น	Flutter	React Native	.NET MAUI
	สนับสนุนจาก Google		
Learning Curve	ระดับปานกลาง	เข้าถึงได้ค่อนข้างง่าย	เหมาะกับผู้ที่มีพื้นฐาน C#
Long-term Viability	มีแนวโน้มเติบโต	เป็น Open-source ที่ใช้งานมาอย่างต่อเนื่อง	ได้รับการสนับสนุนจาก Microsoft

จากตารางที่ 2.25 จะเห็นได้ว่า Framework แต่ละประเภทมีลักษณะเด่นแตกต่างกัน จึงไม่มี Framework ใดที่เหมาะสมกับทุกสถานการณ์ การเลือก Framework ควรพิจารณาทั้งภาษาโปรแกรมที่ทีมพัฒนามีความเชี่ยวชาญ Platform ที่ต้องรองรับ Performance เครื่องมือในการพัฒนา Community Support และความต้องการของระบบโดยรวม ซึ่ง Lahtinen (2024) ระบุว่าการเลือก Framework ที่เหมาะสมถือเป็นส่วนสำคัญของกระบวนการ Cross-Platform Mobile Application Development

Flutter Framework

Flutter เป็น Framework แบบ Free และ Open-source ที่พัฒนาโดย Google สำหรับสร้าง Application แบบ Multi-platform จาก Codebase เดียว โดยสามารถพัฒนา Application สำหรับ Mobile, Web และ Desktop ได้ และใช้ภาษา Dart เป็นภาษาโปรแกรมหลัก

Lahtinen (2024) อธิบายว่า Flutter มีชุด Widget ที่ถูกเตรียมไว้สำหรับการสร้าง User Interface ซึ่งช่วยให้ผู้พัฒนาสามารถสร้าง Interface ที่มีลักษณะ Interactive และสามารถปรับแต่งได้ตามความต้องการ แนวทางดังกล่าวช่วยสนับสนุนการพัฒนา Application ที่มีรูปแบบการแสดงผลสอดคล้องกันบนหลาย Platform

Flutter สามารถรองรับ Application สำหรับ iOS, Android, Windows, macOS และ Platform อื่นที่ Framework รองรับ โดยเอกสารยกตัวอย่างองค์กรที่มีการใช้ Flutter เช่น Alibaba, BMW, eBay, Toyota และ Tencent

คุณลักษณะหนึ่งที่ทำให้ Flutter ได้รับความนิยมคือ **Hot Reload** ซึ่งช่วยให้นักพัฒนาสามารถเห็นผลจากการเปลี่ยนแปลง Source Code ได้อย่างรวดเร็วโดยไม่ต้องเริ่ม Application ใหม่ทุกครั้ง นอกจากนี้ Flutter ยังใช้ Dart ซึ่งรองรับคุณสมบัติอย่าง Strong Typing, Null Safety และ Asynchronous Programming

เอกสารยังระบุว่า Flutter สามารถให้ Performance ในลักษณะที่ใกล้เคียง Native ได้จากการใช้ภาษาแบบ Compiled และ High-performance Rendering Engine รวมทั้งสามารถเข้าถึง Native Feature ผ่าน Platform-specific Plugin ได้

ในด้านโครงสร้างโครงการ Flutter ส่งเสริมแนวคิด Modularity และ Separation of Concerns โดยสามารถแบ่ง Source Code และ Resource ออกเป็นส่วนต่าง ๆ ตามหน้าที่ เช่น Screen, Widget, Model และ Service ซึ่งช่วยให้สามารถจัดระเบียบ Source Code และนำ Component กลับมาใช้ใหม่ได้ง่ายขึ้น

สถาปัตยกรรมของ Flutter

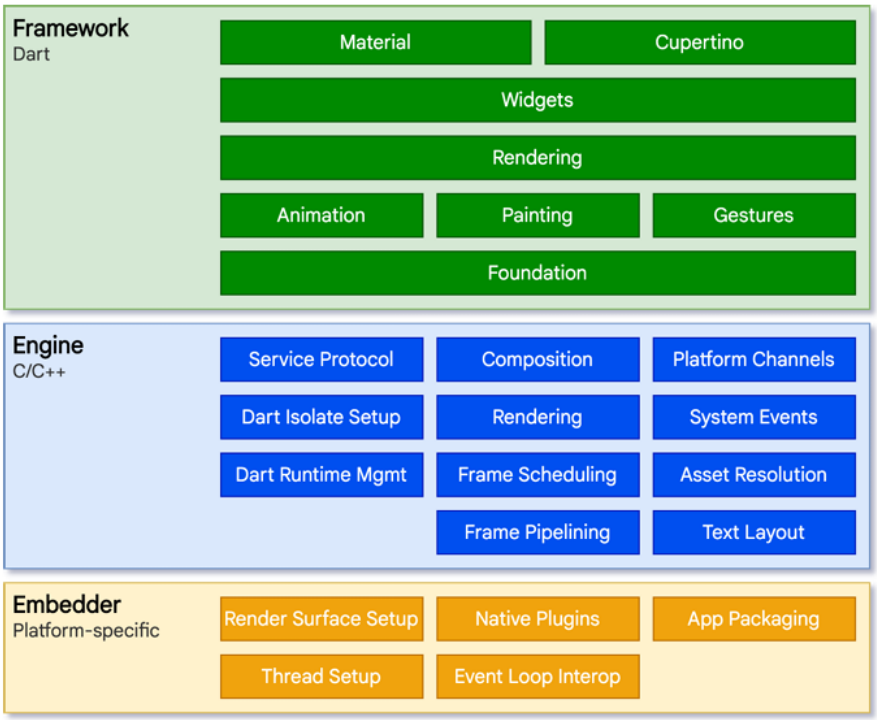
สถาปัตยกรรมของ Flutter สามารถแบ่งออกเป็นองค์ประกอบหลายระดับ โดยภาพ Flutter Architectural Overview ในเอกสารแสดงองค์ประกอบหลัก 3 ระดับ ได้แก่ **Framework, Engine และ Embedder**

ในระดับ Framework ประกอบด้วยองค์ประกอบต่าง ๆ เช่น Material, Cupertino, Widgets, Rendering, Animation, Painting, Gestures และ Foundation ซึ่งเป็นส่วนที่นักพัฒนาใช้ในการสร้าง User Interface และจัดการพฤติกรรมของ Application

ระดับ Engine เป็นส่วนที่อยู่ต่ำกว่า Framework และทำหน้าที่เกี่ยวกับการประมวลผลที่สำคัญ เช่น Rendering, Composition, Frame Scheduling, Text Layout, Asset Resolution และ Platform Channels จากภาพในเอกสารยังแสดงให้เห็นว่า Engine มีองค์ประกอบที่เกี่ยวข้องกับ Dart Runtime และ Service Protocol ด้วย

ส่วน Embedder เป็นชั้นที่เกี่ยวข้องกับ Platform-specific Operation เช่น Render Surface Setup, Native Plugin, Thread Setup, Event Loop Integration และ App Packaging ซึ่งทำให้ Flutter สามารถทำงานร่วมกับระบบปฏิบัติการแต่ละประเภทได้

แนวคิดการแบ่งสถาปัตยกรรมออกเป็นหลายระดับช่วยให้ Flutter แยกส่วนที่ใช้ร่วมกันของ Framework ออกจากรายละเอียดเฉพาะของ Platform ทำให้สามารถนำ Application Logic และ User Interface ส่วนใหญ่ไปใช้ร่วมกันระหว่าง Platform ได้ ขณะที่งานบางประเภทที่เกี่ยวข้องกับระบบปฏิบัติการสามารถดำเนินการผ่าน Platform-specific Component ได้



ภาพที่ 2.7 สถาปัตยกรรมของ Flutter Framework

ที่มา: สังเคราะห์จาก Lahtinen (2024)

โครงสร้างของโครงการ Flutter

โครงการ Flutter โดยทั่วไปถูกจัดโครงสร้างในรูปแบบ Modular และ Hierarchical เพื่อช่วยในการนำ Source Code กลับมาใช้ใหม่และแยกความรับผิดชอบของแต่ละส่วนออกจากกัน Lahtinen (2024) แบ่งองค์ประกอบพื้นฐานของ Flutter Project ออกเป็นส่วนสำคัญ เช่น main.dart, pubspec.yaml, lib, assets, test ตลอดจน Folder เฉพาะของ Android และ iOS

ไฟล์ main.dart เป็น Entry Point ของ Flutter Application โดยเป็นจุดเริ่มต้นของการทำงานของ Application และมีการเรียก runApp() เพื่อเริ่มต้น Root Widget

ไฟล์ pubspec.yaml ใช้สำหรับจัดการ Dependency และ Asset ของ Application เช่น Package, Image และ Resource อื่นที่ระบบจำเป็นต้องใช้งาน

Folder lib เป็นตำแหน่งหลักสำหรับจัดเก็บ Source Code ภาษา Dart และสามารถแบ่งออกเป็น Subdirectory ตามหน้าที่ เช่น screens หรือ views สำหรับ Code ของหน้าจอ, widgets สำหรับ UI Component ที่สามารถนำกลับมาใช้ใหม่, models สำหรับ Data Model และ services สำหรับ Business Logic และ Data Manipulation

Folder assets ใช้สำหรับจัดเก็บ Resource เช่น Image, Font หรือ Data File ส่วน test ใช้เก็บ Unit Test และ Folder android กับ ios มีไฟล์ที่จำเป็นต่อการ Build Application สำหรับ

แต่ละ Platform โดยเฉพาะ ซึ่งโดยทั่วไปจะถูกแก้ไขเฉพาะเมื่อต้องการปรับ Platform-specific Feature เช่น Application Icon หรือ Initial Loading Screen

ตารางที่ 2.26 องค์ประกอบพื้นฐานของโครงการ Flutter

องค์ประกอบ	หน้าที่
main.dart	Entry Point สำหรับเริ่มต้น Flutter Application
pubspec.yaml	จัดการ Dependency และ Asset
lib	จัดเก็บ Source Code ภาษา Dart
screens/views	จัดเก็บส่วนประกอบของหน้าจอ
widgets	จัดเก็บ UI Component ที่สามารถนำกลับมาใช้ใหม่
models	จัดเก็บ Data Model
services	จัดเก็บ Business Logic และการจัดการข้อมูล
assets	จัดเก็บรูปภาพ Font หรือ Resource อื่น
test	จัดเก็บ Unit Test
android / ios	จัดเก็บไฟล์เฉพาะของแต่ละ Platform

โครงสร้างดังกล่าวช่วยสนับสนุน Separation of Concerns โดยทำให้ Source Code แต่ละประเภทมีตำแหน่งและหน้าที่ที่ชัดเจน ซึ่งช่วยเพิ่มความสะดวกในการบำรุงรักษาและขยาย Application เมื่อระบบมีขนาดใหญ่ขึ้น

Widget-Based User Interface

Widget ถือเป็นองค์ประกอบพื้นฐานสำคัญของ Flutter โดย Lahtinen (2024) ระบุว่า องค์ประกอบต่าง ๆ ใน Flutter ถูกสร้างขึ้นในลักษณะ Widget ตั้งแต่องค์ประกอบโครงสร้าง เช่น Button และ Text Field ไปจนถึงองค์ประกอบด้าน Style เช่น Font และ Color

ผู้พัฒนาสามารถนำ Widget หลายประเภทมาประกอบกันเพื่อสร้าง User Interface ที่ซับซ้อน และแต่ละ Widget สามารถปรับแต่งให้เหมาะสมกับรูปแบบที่ต้องการได้ แนวทางดังกล่าวช่วยให้สามารถสร้าง Component ขนาดเล็กที่มีหน้าที่เฉพาะ แล้วนำ Component เหล่านั้นกลับมาใช้ซ้ำในส่วนอื่นของ Application

Widget-Based Approach จึงสนับสนุนแนวคิด Reusability และ Modularity เนื่องจาก User Interface สามารถแบ่งออกเป็น Component ย่อย ทำให้ Source Code มีโครงสร้างที่เข้าใจง่ายและลดความจำเป็นในการสร้าง UI Element เดิมซ้ำหลายครั้ง

เอกสารยังแสดงตัวอย่างการจัดโครงสร้าง Frontend โดยแยก Screen แต่ละหน้าออกเป็นไฟล์ และสร้างหน้าจอผ่าน Flutter Widget เช่น StatefulWidget, Scaffold, AppBar, Form, TextFormField และ ElevatedButton

Hot Reload และกระบวนการพัฒนา

Hot Reload เป็นคุณลักษณะของ Flutter ที่ช่วยเพิ่มความรวดเร็วในกระบวนการพัฒนา โดยช่วยให้นักพัฒนาสามารถเปลี่ยน Source Code และเห็นผลการเปลี่ยนแปลงได้อย่างรวดเร็วใน Application ที่กำลังทำงาน

Lahtinen (2024) ระบุว่า Hot Reload เป็นหนึ่งในคุณลักษณะที่ทำให้ Flutter ได้รับความนิยม เนื่องจากช่วยให้ผู้พัฒนาสามารถเห็นการเปลี่ยนแปลงของ Code ได้ทันทีและส่งผลให้ Development Cycle สั้นลง

ในส่วน Selection Criteria เอกสารยังย้ำว่า Hot Reload ช่วยให้สามารถเปลี่ยน Code และเห็น Visual Update ได้อย่างรวดเร็ว ซึ่งช่วยเพิ่มประสิทธิภาพในการพัฒนาและลดเวลาในกระบวนการทดสอบการเปลี่ยนแปลงของ User Interface

คุณสมบัติดังกล่าวมีประโยชน์โดยเฉพาะในขั้นตอนการพัฒนาและปรับแต่ง User Interface เนื่องจากผู้พัฒนาสามารถทดลองเปลี่ยน Layout, Widget หรือค่าอื่น ๆ และตรวจสอบผลที่เกิดขึ้นได้อย่างรวดเร็ว

ภาษา Dart สำหรับ Cross-Platform Development

Dart เป็น Programming Language ที่เป็นพื้นฐานของ Flutter และถูกออกแบบโดยเน้นการพัฒนา Application ฝั่ง Client บน Platform ที่หลากหลาย Lahtinen (2024) อธิบายว่า Dart เป็น Client-optimized Programming Language สำหรับพัฒนา Application ที่มีความรวดเร็วบนหลาย Platform และมี Runtime Environment ที่เหมาะกับ Application Framework

Dart สนับสนุนการพัฒนาแบบ Multi-platform และสามารถใช้กับ Compilation Target หลายประเภท ได้แก่ Web, Mobile และ Desktop นอกจากนี้ยังมีคุณสมบัติที่ช่วยสนับสนุน Development Workflow เช่น Stateful Hot Reload ซึ่งช่วยให้การเปลี่ยนแปลง Application ระหว่างการพัฒนาสามารถทำได้อย่างรวดเร็ว

Dart เป็นรากฐานสำคัญในการทำงานของ Flutter ทั้งในด้าน Language และ Runtime รวมถึงสนับสนุนงานด้าน Code Formatting, Analysis และ Testing

ในด้าน Type System ภาษา Dart ให้ความสำคัญกับ Type Safety ผ่าน Static Type Checking เพื่อให้ค่าของ Variable สอดคล้องกับประเภทข้อมูลที่กำหนด ขณะเดียวกันยังรองรับ Type Inference ซึ่งช่วยให้ Compiler สามารถระบุประเภทของข้อมูลโดยไม่จำเป็นต้องเขียน Type Annotation ทุกกรณี

อีกคุณสมบัติหนึ่งคือ **Sound Null Safety** ซึ่งกำหนดให้ Variable เป็น Non-nullable โดยค่าเริ่มต้น เว้นแต่จะระบุให้สามารถเป็น Null ได้อย่างชัดเจน คุณสมบัตินี้ช่วยลดข้อผิดพลาดที่เกิดจาก Null Value ผ่านการตรวจสอบ Source Code แบบ Static Analysis

ตารางที่ 2.27 คุณลักษณะของ Flutter และ Dart ที่สนับสนุน Cross-Platform Development

คุณลักษณะ	รายละเอียด
Single/Shared Codebase	ใช้ Source Code ร่วมกันสำหรับหลาย Platform
Widget-Based UI	สร้าง Interface จาก Widget ที่สามารถนำกลับมาใช้ใหม่
Hot Reload	แสดงผลการเปลี่ยนแปลง Code ได้อย่างรวดเร็ว
Multi-Platform Support	รองรับ Mobile, Web และ Desktop
Native-like Performance	ใช้ Compiled Language และ Rendering Engine
Platform-specific Plugins	รองรับการเข้าถึง Native Feature เมื่อจำเป็น
Dart Type Safety	ตรวจสอบประเภทข้อมูลผ่าน Static Type Checking
Sound Null Safety	ลดข้อผิดพลาดจาก Null Value
Modular Project Structure	ช่วยแบ่ง Source Code ตามหน้าที่และสนับสนุน Reusability

จากตารางที่ 2.27 จะเห็นได้ว่า Flutter และ Dart มีคุณลักษณะที่สนับสนุนการพัฒนาแบบ Cross-Platform ในหลายด้าน ทั้งการใช้ Code ร่วมกัน การสร้าง User Interface แบบ Component-based การเพิ่มความรวดเร็วของ Development Cycle และการรองรับ Platform ที่หลากหลาย

การรองรับหลายแพลตฟอร์มของ Cross-Platform Application

คุณลักษณะสำคัญของ Cross-Platform Application Development คือความสามารถในการพัฒนา Application ให้สามารถทำงานบน Platform ที่แตกต่างกันโดยใช้ Source Code

ร่วมกันในระดับหนึ่ง แนวทางดังกล่าวช่วยลดความจำเป็นในการสร้าง Application ใหม่ทั้งหมดสำหรับแต่ละระบบปฏิบัติการ และช่วยให้การบริหารจัดการ Source Code มีความเป็นศูนย์กลางมากขึ้น

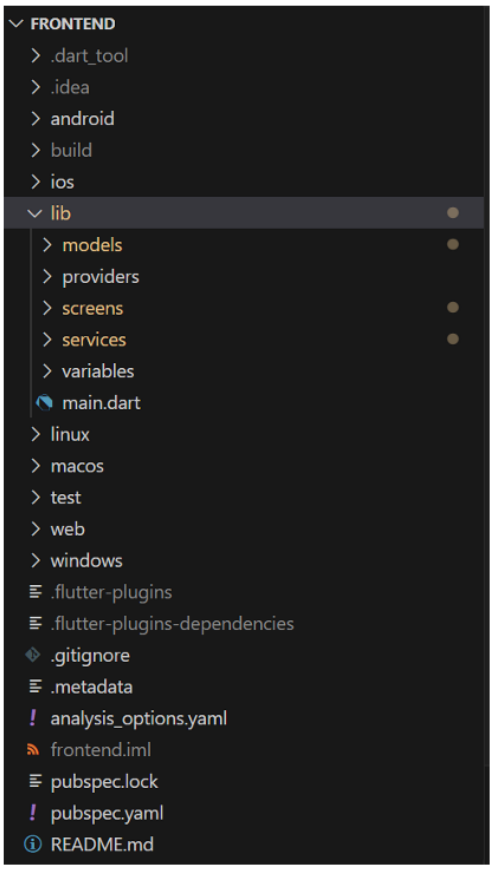
Lahtinen (2024) ระบุว่า Flutter เป็น Framework แบบ Free และ Open-source ที่พัฒนาโดย Google ซึ่งได้รับการออกแบบสำหรับการสร้าง Application แบบ Multi-platform จาก Codebase เดียว โดยสามารถนำไปใช้ในการพัฒนา Application สำหรับ Mobile, Web และ Desktop ได้

จากการเปรียบเทียบ Framework สำหรับ Cross-Platform Development ในเอกสาร Flutter สามารถรองรับ Platform ได้แก่ iOS, Android, Web, macOS และ Windows ซึ่งแสดงให้เห็นถึงความสามารถของ Framework ในการนำ Source Code ไปใช้งานบน Environment ที่หลากหลาย

ความสามารถดังกล่าวทำให้ผู้พัฒนาสามารถจัดการ Application Logic และ User Interface ส่วนใหญ่จาก Codebase เดียวกัน ขณะที่รายละเอียดบางส่วนที่เกี่ยวข้องกับระบบปฏิบัติการโดยเฉพาะยังสามารถแยกจัดการผ่าน Folder หรือ Component สำหรับ Platform นั้นได้

โครงสร้างของ Flutter Project สะท้อนแนวคิดดังกล่าวได้อย่างชัดเจน โดยภายในโครงการจะมี Source Code หลักอยู่ใน Folder lib ขณะที่มี Folder เฉพาะสำหรับระบบปฏิบัติการ เช่น android และ ios เพื่อจัดเก็บไฟล์ที่จำเป็นต่อการ Build และปรับแต่ง Application สำหรับแต่ละ Platform

แนวทางนี้ช่วยสร้างสมดุลระหว่างการใช้ Source Code ร่วมกันกับความสามารถในการปรับแต่ง Application ให้เหมาะสมกับ Platform เฉพาะ กล่าวคือ ส่วนที่มีลักษณะร่วมกันสามารถพัฒนาและบำรุงรักษาจาก Codebase กลาง ขณะที่ส่วนที่จำเป็นต้องใช้ความสามารถเฉพาะของอุปกรณ์หรือระบบปฏิบัติการสามารถแยกดำเนินการได้ตามความจำเป็น



ภาพที่ 2.8 โครงสร้างของโครงการ Flutter สำหรับการพัฒนาแบบ Cross-Platform

การเชื่อมต่อ Frontend และ Backend ใน Cross-Platform Application

การพัฒนา Application แบบ Cross-Platform ไม่ได้จำกัดอยู่เพียงการสร้าง User Interface ที่สามารถทำงานบนหลาย Platform เท่านั้น แต่ Application จำนวนมากยังต้องมีการเชื่อมต่อกับ Backend เพื่อจัดเก็บ ประมวลผล และเรียกใช้ข้อมูลจากส่วนกลาง

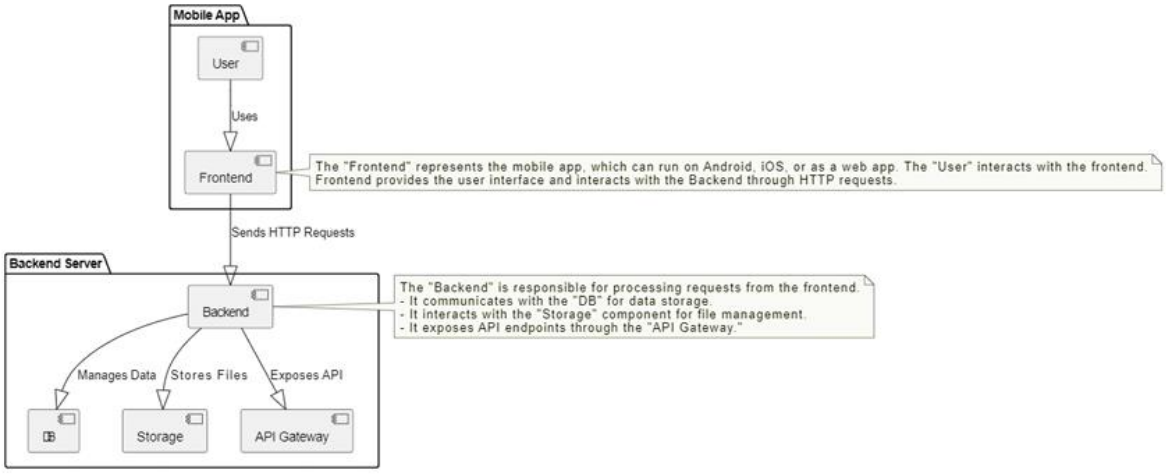
Lahtinen (2024) แสดงตัวอย่างสถาปัตยกรรมของ Cross-Platform Application ซึ่งแบ่งระบบออกเป็น Frontend และ Backend โดย Frontend เป็น Application ที่ผู้ใช้งานมีปฏิสัมพันธ์ด้วย และสามารถทำงานบน Android, iOS หรือในรูปแบบ Web Application ได้ ส่วน Backend ทำหน้าที่รับและประมวลผล Request ที่ถูกส่งมาจาก Frontend

การติดต่อระหว่าง Frontend และ Backend ในตัวอย่างดังกล่าวดำเนินการผ่าน HTTP Request ซึ่งช่วยแยกส่วน User Interface ออกจากกระบวนการจัดการข้อมูลและ Business Logic ที่อยู่ทางฝั่ง Backend

Backend สามารถแบ่งออกเป็นองค์ประกอบต่าง ๆ ได้แก่ Database สำหรับจัดเก็บข้อมูล Storage สำหรับจัดเก็บ File และ API Gateway สำหรับเปิด API Endpoint ให้ Frontend สามารถติดต่อกับ Backend ได้

การแยก Frontend และ Backend ในลักษณะดังกล่าวมีความเหมาะสมกับ Cross-Platform Development เนื่องจาก Application บนหลาย Platform สามารถใช้ Backend และ Data Source ร่วมกันได้ โดยไม่จำเป็นต้องสร้างระบบจัดการข้อมูลแยกสำหรับแต่ละ Platform

Frontend จึงรับผิดชอบในส่วนที่เกี่ยวข้องกับ User Interface และการติดต่อกับผู้ใช้งาน ขณะที่ Backend ทำหน้าที่ประมวลผล Request จัดการข้อมูล และให้บริการ Function ที่ Application ต้องการ ซึ่งช่วยให้แต่ละส่วนสามารถพัฒนาและบำรุงรักษาได้อย่างเป็นสัดส่วน



ภาพที่ 2.9 สถาปัตยกรรมการเชื่อมต่อ Frontend และ Backend ของ Cross-Platform Application

ที่มา: สังเคราะห์จาก Lahtinen (2024)

การจัดโครงสร้าง Source Code สำหรับ Cross-Platform Application

การจัดโครงสร้าง Source Code เป็นอีกหนึ่งองค์ประกอบที่มีความสำคัญต่อการพัฒนา Cross-Platform Application เนื่องจากการใช้ Codebase ร่วมกันอาจทำให้ Source Code มีขนาดใหญ่ขึ้นเมื่อ Application มี Function จำนวนมาก ดังนั้นการแบ่ง Code ตามหน้าที่จึงช่วยให้สามารถบำรุงรักษาและพัฒนาระบบเพิ่มเติมได้ง่ายขึ้น

Lahtinen (2024) อธิบายว่า Flutter Project มีโครงสร้างแบบ Modular และ Hierarchical ซึ่งสนับสนุน Code Reusability และ Separation of Concerns โดย Source Code ภายใน lib สามารถแบ่งออกเป็นส่วนต่าง ๆ เช่น Screens หรือ Views, Widgets, Models และ Services

Screens หรือ Views ใช้สำหรับจัดเก็บ User Interface ของแต่ละหน้าจอ ขณะที่ Widgets ใช้สำหรับจัดเก็บ UI Component ขนาดเล็กที่สามารถนำกลับมาใช้ซ้ำ Models ใช้กำหนดโครงสร้างข้อมูล และ Services ใช้จัดการ Business Logic หรือกระบวนการเกี่ยวกับข้อมูล

แนวทางดังกล่าวช่วยลดการรวม Source Code ทุกประเภทไว้ภายใน File หรือ Module เดียว และทำให้แต่ละส่วนมีหน้าที่รับผิดชอบที่ชัดเจน ส่งผลให้สามารถค้นหา แก้ไข และทดสอบ Source Code ได้ง่ายขึ้น

ในส่วนของ Frontend เอกสารยังแสดงตัวอย่างการแบ่ง Folder ออกเป็น Models, Providers, Screens, Services และ Variables โดยแต่ละส่วนมีหน้าที่เฉพาะ เช่น Models ใช้แทน Data Entity, Providers ใช้จัดการ State และ Data Flow, Screens ใช้จัดเก็บหน้าจอ และ Services ใช้จัดการการติดต่อ API

การจัดโครงสร้างในลักษณะดังกล่าวสนับสนุน Modularity และ Reusability ซึ่งเป็นแนวคิดสำคัญของ Cross-Platform Application Development เนื่องจาก Component ที่ถูกออกแบบให้มีความเป็นอิสระสามารถนำกลับมาใช้ได้หลายส่วน และสามารถปรับปรุงโดยลดผลกระทบต่อส่วนอื่นของระบบ

เอกสารยังกล่าวถึงการจัดเก็บ Configuration และค่าที่ใช้ร่วมกันไว้ในตำแหน่งส่วนกลาง ซึ่งช่วยสร้าง Single Source of Truth และลดโอกาสเกิดข้อผิดพลาดจากค่าที่ไม่สอดคล้องกัน นอกจากนี้ยังช่วยสนับสนุน Maintainability และ Scalability เมื่อ Application มี Function เพิ่มขึ้น

การสื่อสารข้อมูลระหว่าง Platform และ Backend

การสื่อสารข้อมูลเป็นส่วนหนึ่งที่ช่วยให้ Cross-Platform Application สามารถใช้ข้อมูลและบริการจาก Backend ร่วมกันได้ แม้ Application จะทำงานบน Platform ที่แตกต่างกัน

ในตัวอย่างของ Lahtinen (2024) Flutter Frontend มี Service สำหรับจัดการ API Request และใช้ HTTP Library ติดต่อกับ Backend ผ่าน RESTful API Endpoint โดยข้อมูลถูกส่งและรับในรูปแบบ JSON

การใช้รูปแบบข้อมูลร่วมกันช่วยให้ Frontend สามารถแปลงข้อมูลที่ได้รับจาก API ไปเป็น Object ภายใน Application ได้ ตัวอย่างในเอกสารมีการใช้ fromJson() สำหรับแปลง JSON ที่ได้รับจาก API Response ให้เป็น Object และใช้ toJson() สำหรับแปลง Object กลับเป็น JSON เพื่อส่งข้อมูลไปยัง Backend

แนวทางดังกล่าวช่วยสร้างการแยกระหว่างโครงสร้างภายใน Frontend กับ Backend โดยทั้งสองส่วนไม่จำเป็นต้องใช้ Programming Language หรือ Framework เดียวกัน เพียงแต่ต้องตกลงรูปแบบการสื่อสารและข้อมูลที่ใช้แลกเปลี่ยนร่วมกัน

ในตัวอย่างจากเอกสาร Frontend พัฒนาด้วย Flutter และ Dart ขณะที่ Backend ใช้ Node.js และ Express แต่ยังสามารถทำงานร่วมกันผ่าน HTTP และ REST API ได้

ลักษณะนี้แสดงให้เห็นถึงความยืดหยุ่นของ Cross-Platform Application Development ซึ่ง Frontend สามารถมุ่งเน้นการรองรับหลาย Platform ขณะที่ Backend สามารถใช้ Technology Stack ที่เหมาะกับการประมวลผลและการจัดการข้อมูลได้อย่างอิสระ

หลักเกณฑ์ในการเลือกเทคโนโลยีสำหรับ Cross-Platform Development

การเลือก Framework และเครื่องมือสำหรับ Cross-Platform Development จำเป็นต้องพิจารณาปัจจัยหลายด้านร่วมกัน เนื่องจาก Framework แต่ละประเภทมีจุดเด่นและข้อจำกัดที่แตกต่างกัน

Lahtinen (2024) ระบุเกณฑ์สำคัญในการพิจารณาเทคโนโลยี ได้แก่ **Community Support, Documentation, Performance, Ecosystem และ Compatibility with Project Requirements**

Community Support มีความสำคัญเนื่องจาก Framework ที่มี Community ขนาดใหญ่สามารถช่วยให้ผู้พัฒนาเข้าถึง Package, Tutorial, Forum และแนวทางแก้ไขปัญหาได้มากขึ้น ในกรณีของ Flutter เอกสารระบุว่ามี Community ที่มีการใช้งานอย่างต่อเนื่องและช่วยสนับสนุนการพัฒนา Package และแหล่งความรู้ต่าง ๆ

Documentation เป็นอีกปัจจัยหนึ่งที่ช่วยให้ผู้พัฒนาสามารถเรียนรู้ Function และ Feature ของ Framework ได้อย่างถูกต้อง เอกสารกล่าวว่า Flutter มี Documentation ที่ครอบคลุมและได้รับการปรับปรุงอย่างต่อเนื่อง ซึ่งช่วยสนับสนุนกระบวนการเรียนรู้และการนำ Feature ไปใช้งาน

Performance ควรถูกพิจารณาตามลักษณะของ Application หาก Application มีความต้องการด้าน Graphic หรือ Interaction สูง Framework จำเป็นต้องสามารถตอบสนองได้อย่างเหมาะสม ขณะเดียวกัน Ecosystem ของ Framework มีผลต่อความพร้อมของ Library, Plugin และเครื่องมือที่สามารถนำมาใช้เพิ่มเติม

Platform Support ก็เป็นปัจจัยสำคัญ เนื่องจาก Framework แต่ละประเภทอาจรองรับระบบปฏิบัติการแตกต่างกัน การกำหนด Platform เป้าหมายก่อนเลือก Framework จึงช่วยลดปัญหาที่อาจเกิดขึ้นในขั้นตอนการพัฒนา

นอกจากนี้ต้องพิจารณาความรู้และประสบการณ์ของทีมพัฒนา Programming Language ที่ใช้ ตลอดจนทรัพยากรและระยะเวลาของโครงการ เพราะแม้ Framework จะมีความสามารถสูง แต่หากไม่สอดคล้องกับความต้องการหรือความพร้อมของผู้พัฒนา อาจทำให้ประสิทธิภาพของกระบวนการพัฒนาลดลง

ดังนั้น การเลือก Cross-Platform Framework ไม่ควรพิจารณาเฉพาะจำนวน Platform ที่สามารถรองรับ แต่ควรประเมินร่วมกันทั้งด้าน Performance, Development Tool, Ecosystem, Documentation, Community และ Requirement ของระบบ

สรุปแนวคิด Cross-Platform Application Development

Cross-Platform Application Development เป็นแนวทางการพัฒนา Application ที่ช่วยให้ Source Code สามารถนำไปใช้ร่วมกันบนหลาย Platform โดยมีจุดประสงค์เพื่อลดความซ้ำซ้อนในการพัฒนา ลดระยะเวลาและต้นทุน ตลอดจนช่วยให้การดูแลรักษา Application หลาย Platform สามารถดำเนินการจาก Codebase ร่วมกันได้

เมื่อเปรียบเทียบกับ Native Development การพัฒนาแบบ Cross-Platform มีข้อได้เปรียบด้าน Code Reusability และ Development Efficiency ขณะที่ Native Development มีข้อได้เปรียบด้านการเข้าถึง Platform-specific Feature และความสามารถในการปรับแต่ง Application ให้เหมาะสมกับระบบปฏิบัติการโดยตรง อย่างไรก็ตาม Cross-Platform Framework ยังคงสามารถเชื่อมต่อกับ Native Feature ผ่าน Plugin หรือ Platform-specific Code ได้เมื่อมีความจำเป็น

Framework สำหรับ Cross-Platform Development มีหลายประเภท เช่น Flutter, React Native และ .NET MAUI ซึ่งแตกต่างกันในด้าน Programming Language, User Interface, Performance, Development Tools และ Platform Support ดังนั้นการเลือก Framework จึงจำเป็นต้องพิจารณาตามความต้องการและข้อจำกัดของระบบ

Flutter เป็นหนึ่งใน Framework ที่รองรับแนวคิด Cross-Platform โดยใช้ภาษา Dart และสามารถสร้าง Application สำหรับ Mobile, Web และ Desktop จาก Codebase เดียวกัน มีระบบ Widget สำหรับสร้าง User Interface และมี Hot Reload ซึ่งช่วยเพิ่มความรวดเร็วของ Development Cycle

นอกจากนี้ โครงสร้างของ Flutter ยังสนับสนุน Modularity, Separation of Concerns และ Code Reusability ผ่านการแบ่ง Source Code เป็น Screens, Widgets, Models และ Services ตลอดจนสามารถแยกส่วนที่เกี่ยวข้องกับ Android และ iOS ไว้ภายใน Project เดียวกัน

โดยสรุป Cross-Platform Application Development เป็นแนวทางที่ช่วยให้การพัฒนา Application สำหรับหลาย Platform สามารถดำเนินการได้อย่างมีประสิทธิภาพมากขึ้น อย่างไรก็ตาม การเลือกใช้แนวทางดังกล่าวควรคำนึงถึง Performance, Platform-specific Requirements, User Interface, Ecosystem, Community Support และทรัพยากรที่มีอยู่ เพื่อให้ Framework ที่เลือกมีความเหมาะสมกับลักษณะและข้อกำหนดของระบบ

2.1.5 Map Integration and Geolocation

ความหมายของ Map Integration and Geolocation

การประยุกต์ใช้ข้อมูลเชิงพื้นที่ร่วมกับระบบสารสนเทศและแอปพลิเคชันมีบทบาทสำคัญต่อการนำเสนอข้อมูลที่เกี่ยวข้องกับตำแหน่งทางภูมิศาสตร์ โดยเฉพาะระบบที่ต้องการแสดงตำแหน่งพื้นที่ ขอบเขต หรือข้อมูลที่สัมพันธ์กับสถานที่ การนำข้อมูลดังกล่าวมาแสดงบนแผนที่ช่วยให้ผู้ใช้งานสามารถเข้าใจความสัมพันธ์ระหว่างข้อมูลกับพื้นที่จริงได้ชัดเจนมากกว่าการนำเสนอในรูปแบบข้อความเพียงอย่างเดียว

Map Integration หมายถึงการนำข้อมูลหรือข้อมูลเชิงพื้นที่มาผสานกับแผนที่ เพื่อให้ข้อมูลสามารถถูกแสดงและวิเคราะห์ตามตำแหน่งทางภูมิศาสตร์ได้ การผสานข้อมูลกับแผนที่อาจประกอบด้วยการกำหนดพิกัดของตำแหน่ง การแสดง Point, Polygon หรือพื้นที่ต่าง ๆ ตลอดจนการเพิ่มข้อมูลเชิงคุณลักษณะลงบนแผนที่

Kelly (2026) อธิบายว่าข้อมูลเชิงพื้นที่สามารถถูกแทนด้วย Geometry ซึ่งประกอบด้วยคู่ของ Coordinate โดยตัวอย่างข้อมูล Polygon จะประกอบด้วยชุด Coordinate ที่อ้างอิงตำแหน่ง Longitude และ Latitude

ในขณะที่ **Geolocation** หมายถึงกระบวนการที่ใช้ในการระบุตำแหน่งทางภูมิศาสตร์ของวัตถุ อุปกรณ์ หรือสถานที่ โดยข้อมูลตำแหน่งมักถูกแสดงในรูปแบบ Latitude และ Longitude

Universal Service Administrative Company อธิบายว่า Geolocation คือกระบวนการในการรวบรวม Latitude และ Longitude Coordinate ที่ใช้แทนตำแหน่งเฉพาะ โดยสามารถรวบรวมข้อมูลตำแหน่งได้หลายวิธี เช่น GPS, Web-based Map และ Automated Address Geocoding

แนวคิด Map Integration และ Geolocation จึงมีความสัมพันธ์กันโดยตรง กล่าวคือ Geolocation ทำหน้าที่ระบุตำแหน่งหรือสร้างข้อมูล Coordinate ส่วน Map Integration ทำหน้าที่นำ Coordinate และข้อมูลเชิงพื้นที่เหล่านั้นมาแสดงหรือผสานเข้ากับแผนที่

การทำงานร่วมกันของทั้งสองแนวคิดช่วยให้ระบบสามารถนำข้อมูลที่เป็นนามธรรม เช่น Address, IP Address หรือชื่อของสถานที่ มาเชื่อมโยงกับตำแหน่งทางภูมิศาสตร์ และแสดงตำแหน่งดังกล่าวผ่านแผนที่ได้

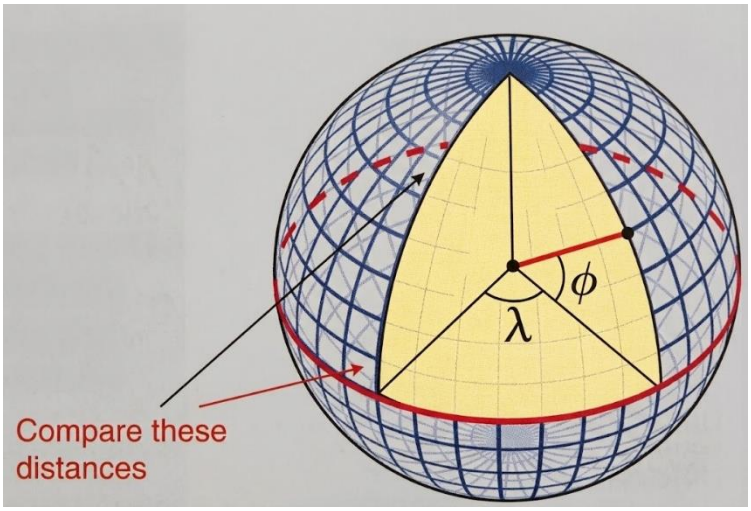
ระบบพิกัดภูมิศาสตร์ (Geographic Coordinate System)

Geographic Coordinate System หรือ GCS เป็นระบบที่ใช้สำหรับระบุตำแหน่งบนพื้นผิวโลกผ่านค่าพิกัด โดยทั่วไปใช้ Latitude และ Longitude เป็นองค์ประกอบหลัก

Kelly (2026) อธิบาย Geographic Coordinate System ว่าเป็นระบบพิกัดแบบทรงกลมหรือทรงรีที่ใช้สำหรับวัดและสื่อสารตำแหน่งบนพื้นโลกโดยตรงผ่าน Latitude และ Longitude โดยค่าที่วัดในระบบดังกล่าวมีลักษณะเป็นมุม ไม่ได้เป็นระยะทางโดยตรงเหมือน Cartesian Coordinate System

Latitude ใช้สำหรับระบุตำแหน่งทางเหนือหรือใต้ของเส้นศูนย์สูตร ส่วน Longitude ใช้สำหรับระบุตำแหน่งทางตะวันออกหรือตะวันตกเมื่อเทียบกับเส้นเมริเดียนหลักที่ผ่าน Greenwich ประเทศอังกฤษ

การระบุตำแหน่งด้วย Latitude และ Longitude สามารถนำไปใช้กับข้อมูลหลายประเภท เช่น ตำแหน่งของอาคาร ถนน สถานที่สำคัญ เขตการปกครอง หรืออุปกรณ์ที่สามารถรายงานตำแหน่งของตนเองได้ เมื่อค่าพิกัดถูกผสานกับระบบแผนที่ ระบบสามารถใช้ค่าดังกล่าวกำหนดจุดที่ต้องการแสดงได้



ภาพที่ 2.10 ระบบพิกัดภูมิศาสตร์ (Geographic Coordinate System)

ที่มา: Kelly (2026)

Latitude และ Longitude

Latitude และ Longitude เป็นค่าพื้นฐานสำหรับการระบุตำแหน่งทางภูมิศาสตร์ Latitude เป็นเส้นแนวนอนที่ใช้วัดตำแหน่งเหนือหรือใต้จากเส้นศูนย์สูตร ขณะที่ Longitude หรือ Meridian เป็นเส้นที่ลากจากขั้วโลกหนึ่งไปยังอีกขั้วโลกหนึ่งและใช้วัดตำแหน่งตะวันออกหรือตะวันตกจาก Greenwich

ค่าพิกัดสามารถแสดงได้หลายรูปแบบ เช่น Degrees, Minutes and Seconds หรือ Decimal Degrees ตัวอย่าง Decimal Degrees ในเอกสารคือ 40.934797, -73.296459

Decimal Degrees เป็นรูปแบบที่สะดวกต่อการใช้งานกับระบบคอมพิวเตอร์และฐานข้อมูล เนื่องจาก Latitude และ Longitude สามารถจัดเก็บเป็นค่าตัวเลขและนำไปใช้ในการคำนวณหรือแสดงบนแผนที่ได้โดยตรง

Universal Service Administrative Company แนะนำว่าข้อมูล Latitude และ Longitude สำหรับการจัดเก็บตำแหน่งควรรายงานในรูปแบบ Decimal Degree และในบริบทของเอกสารกำหนดให้มีความละเอียดอย่างน้อยหกตำแหน่งทศนิยม เพื่อให้สามารถอ้างอิงตำแหน่งได้อย่างละเอียด

อย่างไรก็ตาม การนำ Latitude และ Longitude ไปใช้งานจำเป็นต้องคำนึงถึงลำดับของ Coordinate ด้วย เนื่องจากระบบหรือ Data Format บางประเภทกำหนดลำดับเป็น Latitude/Longitude ขณะที่บางระบบใช้ Longitude/Latitude การสลับลำดับของค่าอาจทำให้ตำแหน่งที่แสดงบนแผนที่ผิดจากตำแหน่งที่ต้องการได้

Kelly (2026) แสดงตัวอย่างของข้อมูล GeoJSON และ Google Maps ซึ่งมีความแตกต่างในเรื่องลำดับของ Latitude และ Longitude จึงจำเป็นต้องตรวจสอบรูปแบบ Coordinate ของระบบที่นำข้อมูลไปใช้

Coordinate Reference System

Coordinate Reference System หรือ CRS เป็นระบบอ้างอิงที่กำหนดวิธีการเชื่อมโยงค่าพิกัดกับตำแหน่งจริงบนพื้นผิวโลก ข้อมูลแผนที่แต่ละชุดอาจใช้ CRS ที่แตกต่างกัน ดังนั้นการผสานข้อมูลหลายชุดจึงจำเป็นต้องตรวจสอบว่าข้อมูลเหล่านั้นใช้ระบบอ้างอิงที่สอดคล้องกัน

Kelly (2026) ยกตัวอย่าง **WGS84** ซึ่งใช้ Latitude และ Longitude และมีรหัส **EPSG:4326** รวมถึง UTM Zone สำหรับพื้นที่ในซีกโลกเหนือและใต้

EPSG:4326 เป็นระบบพิกัดที่ใช้ Latitude และ Longitude และถูกนำมาใช้กับข้อมูลแผนที่อย่างแพร่หลาย ตัวอย่างในเอกสารแสดงข้อมูล GeoJSON ที่มี CRS เป็น EPSG:4326

การใช้ CRS ที่แตกต่างกันสามารถทำให้รูปทรง ตำแหน่ง หรือระยะทางที่ปรากฏบนแผนที่มีความแตกต่างกันได้ เนื่องจากการฉายข้อมูลจากพื้นผิวโลกที่มีลักษณะโค้งไปยังพื้นที่สองมิติสามารถทำได้หลายรูปแบบ

Kelly (2026) ระบุว่า Map Projection ที่แตกต่างกันสามารถทำให้รูปร่างของพื้นที่ปรากฏแตกต่างกัน และเมื่อมีการรวมข้อมูลแผนที่หลายชุด CRS ของข้อมูลแต่ละชุดควรเป็นระบบเดียวกัน

ดังนั้น ก่อนนำข้อมูลเชิงพื้นที่หลายแหล่งมาแสดงร่วมกัน จำเป็นต้องตรวจสอบ CRS และหากระบบอ้างอิงแตกต่างกันอาจต้องแปลงข้อมูลให้อยู่ใน CRS เดียวกันก่อน

เอกสารแสดงตัวอย่างการแปลงข้อมูลจาก EPSG:26918 ไปยัง EPSG:4326 ซึ่งหลังจากแปลงแล้วค่าบนแกนของแผนที่เปลี่ยนจากค่าพิกัดของระบบเดิมเป็น Latitude และ Longitude

ข้อมูลเชิงพื้นที่และ Geometry

การทำ Map Integration จำเป็นต้องมีรูปแบบสำหรับแทนข้อมูลเชิงพื้นที่ โดยข้อมูลดังกล่าวสามารถแทนได้ด้วย Geometry ประเภทต่าง ๆ ตามลักษณะของวัตถุหรือพื้นที่ที่ต้องการแสดง

Kelly (2026) อธิบายตัวอย่าง Geometry ที่สำคัญ ได้แก่ **Point, Polygon และ MultiPolygon** โดย Point ใช้แทนตำแหน่งเดียว Polygon ใช้แทนพื้นที่ที่มีเส้นขอบล้อมรอบ และ MultiPolygon เป็นกลุ่มของ Polygon ตั้งแต่หนึ่งรูปขึ้นไป

Point เหมาะสำหรับการแสดงข้อมูลสถานที่ที่สามารถแทนด้วยตำแหน่งเดียว เช่น อาคาร จุดบริการ หรือ Landmark ขณะที่ Polygon เหมาะกับข้อมูลที่มีขอบเขตพื้นที่ เช่น เขตการปกครอง จังหวัด หรือพื้นที่ให้บริการ

การเลือก Geometry ที่เหมาะสมจึงขึ้นอยู่กับลักษณะของข้อมูล หากต้องการแสดงเพียงตำแหน่งสามารถใช้ Point แต่หากต้องการแสดงพื้นที่หรือขอบเขตจะต้องใช้ Polygon หรือ MultiPolygon

นอกจากนี้ Geometry จะต้องถูกเชื่อมโยงกับ Coordinate Reference System ที่เหมาะสม เพื่อให้ค่าพิกัดสามารถถูกตีความเป็นตำแหน่งจริงบนพื้นผิวโลกได้อย่างถูกต้อง

การผสานข้อมูลกับแผนที่ (Map Integration)

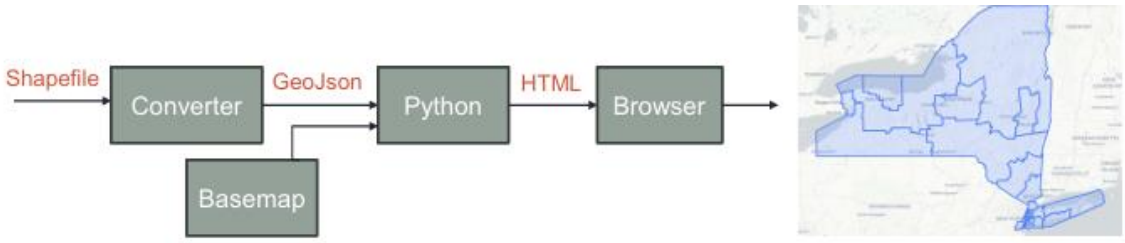
Map Integration เป็นกระบวนการนำข้อมูลทางภูมิศาสตร์หรือข้อมูลทั่วไปมาเชื่อมโยงและแสดงผลบนแผนที่ โดยข้อมูลอาจอยู่ในรูปของ Latitude/Longitude, Point, Polygon หรือข้อมูล Attribute ที่มีความสัมพันธ์กับพื้นที่

Kelly (2026) แสดงตัวอย่างการนำข้อมูลขอบเขต Congressional District มาเชื่อมเข้ากับ Basemap โดยเริ่มจากการอ่าน Shapefile แปลงข้อมูลเป็น GeoJSON จากนั้นนำ GeoJSON ไปแสดงบนแผนที่

แนวทางนี้แสดงให้เห็นว่าการ Map Integration อาจประกอบด้วยหลายขั้นตอน ได้แก่ การเตรียมข้อมูล การแปลง Format การตรวจสอบ Coordinate System การสร้าง Basemap และการเพิ่มข้อมูลเชิงพื้นที่ลงบนแผนที่

GeoJSON เป็นหนึ่งใน Format ที่สามารถใช้สำหรับจัดเก็บและแลกเปลี่ยนข้อมูลเชิงพื้นที่ โดยเอกสารระบุว่า Application จำนวนมากต้องใช้ GeoJSON และสามารถแปลงข้อมูลจาก Shapefile ไปเป็น GeoJSON ก่อนนำไปใช้กับระบบแผนที่ได้

การแยก Basemap ออกจากข้อมูลที่ต้องการนำเสนอช่วยให้ระบบสามารถใช้แผนที่เป็นพื้นหลังและเพิ่มข้อมูลเฉพาะด้านเป็น Layer เพิ่มเติม ตัวอย่างเช่น ขอบเขตพื้นที่สามารถถูกวางทับบน Basemap เพื่อช่วยให้ผู้ใช้งานสามารถเข้าใจตำแหน่งและความสัมพันธ์ของพื้นที่ได้ง่ายขึ้น



ภาพที่ 2.11 กระบวนการผสานข้อมูลเชิงพื้นที่เข้ากับแผนที่

ที่มา: Kelly (2026)

Map Tiles และ Basemap

ระบบ Web Mapping จำนวนมากใช้แนวคิด **Map Tiles** ในการแสดงแผนที่ โดยแผนที่จะถูกแบ่งเป็นภาพขนาดเล็กหลาย Tile เมื่อผู้ใช้งานเลื่อนหรือ Zoom ระบบจะโหลด Tile ที่เหมาะสมกับพื้นที่และระดับการ Zoom มาแสดง

Kelly (2026) ระบุว่าระบบ Web Mapping เช่น Google Maps และ Leaflet ใช้แนวคิดการจัดเก็บแผนที่ในรูป Map Tiles และเมื่อผู้ใช้ Pan หรือ Zoom Browser จะโหลด Tile ที่แตกต่างกันตามบริเวณและระดับรายละเอียดที่ต้องการ

แนวทางดังกล่าวช่วยให้ระบบไม่จำเป็นต้องโหลดข้อมูลแผนที่ทั้งหมดพร้อมกัน แต่สามารถโหลดเฉพาะส่วนที่อยู่ในบริเวณที่ผู้ใช้งานกำลังดู ทำให้การแสดงผลแผนที่ผ่าน Web Application มีประสิทธิภาพมากขึ้น

Basemap คือแผนที่พื้นฐานที่ใช้เป็น Layer หลักสำหรับแสดงข้อมูลทางภูมิศาสตร์ เช่น ถนน เมือง หรือขอบเขตพื้นที่ จากนั้น Application สามารถเพิ่ม Layer ข้อมูลประเภทอื่นทับลงไปได้

เอกสารยกตัวอย่าง Folium ซึ่งใช้ **OpenStreetMap เป็น Default Tile** และยังสามารถเลือก Tile Set อื่น เช่น CartoDB Positron, CartoDB Voyager และ NASAGIBS Blue Marble ได้

ดังนั้นการเลือก Basemap จึงควรพิจารณาตามประเภทของข้อมูลที่ต้องการนำเสนอและระดับรายละเอียดที่เหมาะสม เพราะ Basemap ทำหน้าที่เป็นบริบททางภูมิศาสตร์สำหรับข้อมูลอื่นที่แสดงทับอยู่บนแผนที่

Geolocation

Geolocation เป็นกระบวนการที่ใช้ในการระบุตำแหน่งของสถานที่หรืออุปกรณ์บนพื้นผิวโลก โดยผลลัพธ์โดยทั่วไปอยู่ในรูป Latitude และ Longitude

Universal Service Administrative Company อธิบายว่า Geolocation คือกระบวนการสำหรับรวบรวม Latitude และ Longitude Coordinate ซึ่งใช้แทนตำแหน่งเฉพาะ และแบ่งวิธีการรวบรวมออกเป็นสามแนวทาง ได้แก่ **GPS ในพื้นที่จริง, Desktop Geolocation โดยใช้ Web-based Maps and Imagery และ Automated Address Geocoding**

แต่ละวิธีมีความแตกต่างกันด้านความแม่นยำ ความสะดวก และทรัพยากรที่ต้องใช้ โดยการเลือกวิธีที่เหมาะสมขึ้นอยู่กับลักษณะของข้อมูล จำนวนตำแหน่งที่ต้องการรวบรวม และระดับความแม่นยำที่ต้องการ

การระบุตำแหน่งด้วย GPS

การใช้ GPS เป็นวิธีหนึ่งที่สามารถรวบรวม Latitude และ Longitude จากตำแหน่งจริง โดยใช้อุปกรณ์ที่รองรับ GPS เช่น Smartphone, Tablet หรืออุปกรณ์พกพาอื่น

Universal Service Administrative Company ระบุว่าการเดินทางไปยังตำแหน่งจริงและเก็บ Coordinate ผ่านอุปกรณ์ GPS เป็นวิธีที่ให้ผลลัพธ์แม่นยำและมีความน่าเชื่อถือสูงที่สุดในบรรดาวิธีที่เอกสารเปรียบเทียบ เนื่องจากตำแหน่งถูกเก็บจากสถานที่จริงโดยตรง

ข้อดีของแนวทางนี้คือสามารถเพิ่มความมั่นใจว่าพิกัดที่ได้เป็นตำแหน่งจริงที่ต้องการ อย่างไรก็ตาม การเดินทางไปเก็บข้อมูลในสถานที่แต่ละแห่งอาจใช้เวลาและต้นทุนสูง โดยเฉพาะเมื่อมีสถานที่จำนวนมาก

Desktop Geolocation ผ่าน Web-based Maps

Desktop Geolocation เป็นการใช้ Web-based Map หรือภาพถ่ายเพื่อค้นหาตำแหน่งและระบุ Latitude/Longitude โดยไม่ต้องเดินทางไปยังตำแหน่งจริง

กระบวนการทั่วไปประกอบด้วยการเปิดแผนที่ ค้นหา Address หรือเลื่อนแผนที่ไปยังพื้นที่ที่ต้องการ เปิดภาพหรือรายละเอียดที่ช่วยระบุตำแหน่ง จากนั้นเลือกตำแหน่งและอ่านค่า Latitude/Longitude จากระบบ

วิธีนี้สามารถให้ผลลัพธ์ที่มีความน่าเชื่อถือ หากแผนที่และภาพที่ใช้อ้างอิงมีคุณภาพและตำแหน่งสามารถระบุได้ชัดเจน อย่างไรก็ตาม ข้อมูลภาพอาจล้าสมัย โดยเฉพาะพื้นที่ชนบทหรือบริเวณที่มีสิ่งปลูกสร้างใหม่ ซึ่งอาจส่งผลต่อความแม่นยำ

Geocoding

Geocoding เป็นกระบวนการแปลงข้อมูล Address ให้กลายเป็น Coordinate หรือ Latitude/Longitude โดยใช้ Geocoder และข้อมูลอ้างอิงทางภูมิศาสตร์

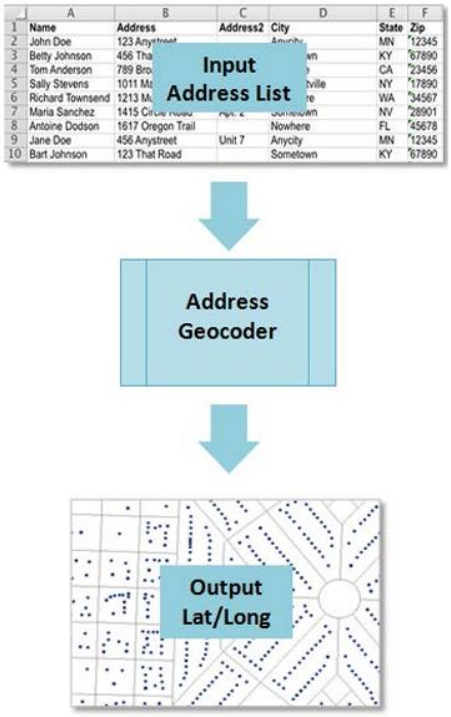
Universal Service Administrative Company ระบุว่า Automated Address Geocoding เป็นวิธีที่ใช้ Geocoder Program เพื่อแปลง Address Information ให้เป็น Latitude และ Longitude โดยสามารถประมวลผล Address จำนวนมากแบบ Batch ได้ด้วยต้นทุนที่ค่อนข้างต่ำ

Geocoder จะวิเคราะห์องค์ประกอบของ Address และค้นหาข้อมูลอ้างอิงที่ตรงกันเพื่อระบุ Coordinate หากข้อมูลอ้างอิงมี Address ที่ตรงกันอย่างชัดเจน ผลลัพธ์ที่ได้จะมีความน่าเชื่อถือมากขึ้น

อย่างไรก็ตาม หาก Address ไม่สามารถจับคู่กับข้อมูลอ้างอิงได้โดยตรง ระบบอาจใช้วิธี **Interpolation** หรือประมาณตำแหน่งจากข้อมูลใกล้เคียง ส่งผลให้ Coordinate ที่ได้มีความคลาดเคลื่อนจากตำแหน่งจริง

ตัวอย่างเช่น หากข้อมูลอ้างอิงมีช่วงหมายเลข Address ตั้งแต่ 100 ถึง 200 และต้องค้นหา Address หมายเลข 150 Geocoder อาจประมาณตำแหน่งให้อยู่บริเวณกึ่งกลางของช่วงถนน แม้ตำแหน่งจริงของอาคารอาจไม่ได้อยู่บริเวณดังกล่าว

ความแม่นยำของ Geocoding จึงขึ้นอยู่กับทั้งคุณภาพของ Input Address และคุณภาพของ Reference Data Address ที่ไม่สมบูรณ์ มีรูปแบบไม่สอดคล้อง หรือมีความกำกวมสามารถทำให้ผลลัพธ์ผิดพลาดได้



ภาพที่ 2.12 กระบวนการแปลงข้อมูลที่อยู่เป็นค่าพิกัดด้วย Geocoding
ที่มา: Universal Service Administrative Company (ม.ป.ป.)

การเปรียบเทียบวิธีการระบุตำแหน่ง

จากเอกสาร *Geolocation Methods* ทั้งสามวิธีมีความแตกต่างกันด้านความแม่นยำและทรัพยากรที่ต้องใช้ โดย GPS Field Collection ให้ความน่าเชื่อถือสูงที่สุดแต่ต้องใช้ความพยายามสูง Desktop Geolocation ให้ผลลัพธ์ในระดับดีมากและใช้ทรัพยากรระดับปานกลาง ขณะที่ Automated Geocoding สามารถจัดการ Address จำนวนมากได้อย่างรวดเร็ว แต่ผลลัพธ์ขึ้นอยู่กับ Input และ Reference Data อย่างมาก

ตารางที่ 2.28 การเปรียบเทียบวิธีการระบุตำแหน่งทางภูมิศาสตร์

วิธี	ความน่าเชื่อถือ	ทรัพยากรที่ใช้	ลักษณะเด่น
GPS Field Collection	สูงมาก	สูง	เก็บ Coordinate จากตำแหน่งจริง
Desktop Geolocation	สูง	ปานกลาง	ใช้ Web Map และ Imagery ค้นหาตำแหน่ง

วิธี	ความน่าเชื่อถือ	ทรัพยากรที่ใช้	ลักษณะเด่น
Automated Geocoding	ปานกลางถึงสูง ขึ้นอยู่กับข้อมูล	ต่ำ	ประมวลผล Address จำนวนมากได้รวดเร็ว

IP Geolocation

นอกจาก GPS และ Address Geocoding แล้ว ยังมีการระบุตำแหน่งจาก IP Address หรือ IP Geolocation ซึ่งเป็นการเชื่อมโยงข้อมูลของ IP Address กับตำแหน่งทางภูมิศาสตร์

Zhu et al. (2020) อธิบายว่า IP Geolocation เป็นเทคโนโลยีที่ใช้ระบุตำแหน่งทางภูมิศาสตร์ของอุปกรณ์เป้าหมายจาก IP Address โดย IP Address ทำหน้าที่เป็น Identifier ของ Host ในระบบอินเทอร์เน็ต และสามารถนำมาเชื่อมโยงกับข้อมูลตำแหน่งของอุปกรณ์ได้

IP Geolocation สามารถประยุกต์ใช้ในหลายด้าน เช่น Targeted Content, Network Performance Optimization, Social Network, Internet of Things และ Network Security

ตัวอย่างเช่น บริการออนไลน์สามารถนำตำแหน่งที่ได้จาก IP มาใช้เลือก Content ที่เกี่ยวข้องกับพื้นที่ของผู้ใช้งาน หรือระบบ Network สามารถใช้ข้อมูลตำแหน่งเพื่อเลือก Node ที่อยู่ใกล้กันเพื่อลดต้นทุนในการสื่อสารระยะไกล

ในด้าน Security ข้อมูลตำแหน่งของ IP สามารถนำมาใช้ช่วยตรวจสอบตัวตน ตรวจจับ IP ที่มีลักษณะผิดปกติ หรือเปรียบเทียบตำแหน่งที่คาดหมายกับตำแหน่งการใช้งานจริงได้

อย่างไรก็ตาม IP Geolocation แตกต่างจาก GPS เนื่องจากตำแหน่งที่ได้ไม่ได้มาจาก Sensor ที่อุปกรณ์วัดโดยตรง แต่เป็นการประเมินหรือจับคู่ IP Address กับข้อมูลตำแหน่ง ดังนั้นจึงมีความคลาดเคลื่อนและระดับ Precision ที่แตกต่างกัน

ความแม่นยำของ Geolocation

ความแม่นยำเป็นองค์ประกอบสำคัญในการประเมินระบบ Geolocation โดยสามารถพิจารณาจากระยะห่างระหว่างตำแหน่งที่ระบบระบุและตำแหน่งจริง

Zhu et al. (2020) ระบุว่าในบริบทของ IP Geolocation สามารถใช้ **Error Distance** ระหว่างผลลัพธ์ของ IP Geolocation และตำแหน่งจริงเป็นเกณฑ์สำหรับประเมิน Accuracy

ในงานวิจัยดังกล่าว ผู้วิจัยใช้ GPS Location ที่ได้รับจากผู้ใช้โดยได้รับอนุญาตเป็นข้อมูลอ้างอิงสำหรับทดสอบความแม่นยำของ IP Geolocation และแสดงทั้งตำแหน่งของผู้ใช้และผล Geolocation บนแผนที่เพื่อช่วยในการเปรียบเทียบ

ผลการทดลองรายงานว่า IP Geolocation System ที่ศึกษาให้ City-level Consistency เท่ากับ 98.88% และสำหรับ Street-level IP มี Median Error Distance 0.13 กิโลเมตร

อย่างไรก็ตาม ตัวเลขดังกล่าวเป็น **ผลของระบบและชุดข้อมูลเฉพาะในงานวิจัยนี้** ไม่ควรตีความว่า IP Geolocation ทุกระบบจะมีความแม่นยำในระดับเดียวกัน เนื่องจาก Accuracy ขึ้นอยู่กับ Algorithm, Reference Data, Network Type และบริบทการใช้งาน



ภาพที่ 2.13 ตัวอย่างการแสดงผลข้อมูล Geolocation บนแผนที่

ที่มา: Zhu et al. (2020)

การเชื่อมโยง Geolocation กับการแสดงผลบนแผนที่

ข้อมูล Geolocation จะมีประโยชน์มากขึ้นเมื่อถูกนำไปเชื่อมโยงกับระบบแผนที่ เพราะค่าพิกัด Latitude และ Longitude สามารถใช้เป็นตำแหน่งอ้างอิงสำหรับการแสดงผลข้อมูลบนพื้นที่จริงได้ การเชื่อมโยงดังกล่าวช่วยให้ผู้ใช้งานสามารถมองเห็นตำแหน่ง ความสัมพันธ์ของพื้นที่ และบริบททางภูมิศาสตร์ได้อย่างชัดเจนมากกว่าการแสดงค่าพิกัดในรูปตัวเลขเพียงอย่างเดียว

Zhu et al. (2020) แสดงตัวอย่างการทำงานของระบบที่ผสานข้อมูล Geolocation เข้ากับ Map Component โดยส่วน Front-end สามารถแสดงผลแผนที่และข้อมูลตำแหน่งจาก IP Geolocation ขณะเดียวกันระบบสามารถรับตำแหน่ง GPS ของผู้ใช้งานเมื่อได้รับอนุญาต และนำข้อมูลดังกล่าวมาใช้เปรียบเทียบกับผลลัพธ์จาก IP Geolocation

การทำงานในลักษณะดังกล่าวสะท้อนให้เห็นถึงความสัมพันธ์ระหว่าง Geolocation และ Map Integration กล่าวคือ กระบวนการ Geolocation ทำหน้าที่สร้างหรือค้นหาข้อมูลตำแหน่ง ส่วน Map Integration ทำหน้าที่นำข้อมูลตำแหน่งนั้นไปแสดงร่วมกับข้อมูลเชิงพื้นที่ประเภทอื่นบนแผนที่

ในงานของ Zhu et al. ระบบสามารถรับ Public Network IP Address จากอุปกรณ์ จากนั้นค้นหาข้อมูลทางภูมิศาสตร์ที่สัมพันธ์กับ IP และนำผลลัพธ์ไปแสดงบนแผนที่ โดยข้อมูลที่ได้สามารถครอบคลุมรายละเอียด เช่น Continent, Country, Province, ZIP Code และ Time Zone

เมื่อผู้ใช้งานอนุญาตให้เข้าถึง GPS ระบบสามารถนำตำแหน่ง GPS มาแสดงร่วมกับผลจาก IP Geolocation ซึ่งช่วยให้สามารถเปรียบเทียบตำแหน่งที่ได้จากสองวิธีและประเมินความแตกต่างของผลลัพธ์ได้

การนำข้อมูลหลายแหล่งมาแสดงบนแผนที่จึงไม่ได้มีวัตถุประสงค์เพียงเพื่อสร้างภาพประกอบ แต่ยังสามารถใช้เป็นเครื่องมือสำหรับการตรวจสอบ วิเคราะห์ และเปรียบเทียบข้อมูลเชิงพื้นที่ได้ด้วย

ความคลาดเคลื่อนและข้อจำกัดของ Geocoding

แม้ Geocoding จะช่วยให้สามารถแปลง Address จำนวนมากเป็น Latitude และ Longitude ได้อย่างรวดเร็ว แต่ผลลัพธ์ที่ได้ไม่ได้มีความแม่นยำเท่ากันในทุกกรณี ความถูกต้องของพิกัดขึ้นอยู่กับทั้งคุณภาพของข้อมูล Address ที่นำเข้าและคุณภาพของ Reference Data ที่ Geocoder ใช้ในการค้นหาตำแหน่ง

Universal Service Administrative Company ระบุว่า Geocoder สามารถให้ผลลัพธ์ที่น่าเชื่อถือได้หากมีข้อมูลอ้างอิงที่ตรงกับ Address อย่างชัดเจน แต่ในหลายกรณี โดยเฉพาะพื้นที่ชนบท ระบบอาจไม่สามารถค้นหา Address ที่ตรงกันและอาจต้องใช้วิธีประมาณตำแหน่งหรือ Interpolation แทน

ข้อจำกัดประการหนึ่งเกิดจาก **ข้อมูล Address ที่ไม่สมบูรณ์หรือไม่สอดคล้องกัน** ตัวอย่างเช่น การขาด ZIP Code การเขียนชื่อถนนต่างรูปแบบ หรือ Address ที่มีความกำกวม สามารถทำให้ Geocoder ระบุตำแหน่งผิดพลาดได้

อีกปัญหาหนึ่งคือ **Reference Data ที่ไม่ครบถ้วนหรือไม่ทันสมัย** โดยเฉพาะในพื้นที่ชนบท Geocoder อาจไม่มีข้อมูลตำแหน่งของอาคารหรือ Address ที่ต้องการ จึงจำเป็นต้องประมาณตำแหน่งจากช่วงหมายเลขของถนน ซึ่งอาจส่งผลให้พิกัดอยู่ห่างจากตำแหน่งจริง

Geocoder บางระบบยังอาจประมาณตำแหน่งให้อยู่ผิดด้านของถนน เนื่องจากสมมติให้บ้านเลขคู่และเลขคี่อยู่คนละฝั่ง ทั้งที่ในสถานการณ์จริงอาจไม่เป็นไปตามรูปแบบดังกล่าว

หากระบบไม่สามารถจับคู่กับ Street Address ได้ อาจลดระดับความละเอียดของข้อมูลลงไปยังพื้นที่ที่กว้างขึ้น เช่น ZIP Code และใช้จุดกึ่งกลางของพื้นที่นั้นเป็นตำแหน่งแทน ซึ่งอาจทำให้ผลลัพธ์มีความคลาดเคลื่อนจากตำแหน่งจริงอย่างมาก

ดังนั้นผลลัพธ์จาก Automated Geocoding ควรได้รับการตรวจสอบเมื่อความถูกต้องของตำแหน่งมีความสำคัญ โดยเฉพาะข้อมูลที่มาจาก Address ซึ่งมีความกำกวม หรือพื้นที่ซึ่ง Reference Data มีคุณภาพต่ำ

เอกสาร *Geolocation Methods* ยังเปรียบเทียบผลจากหลายวิธีและแสดงให้เห็นว่า GPS Field Collection, Desktop Geolocation และ Geocoding สามารถให้พิกัดที่แตกต่างกันสำหรับสถานที่เดียวกันได้ โดยตัวอย่างในหน้า 8 แสดงตำแหน่งจากหลาย Geocoder ซึ่งบางระบบให้ตำแหน่งใกล้อาคารจริง ขณะที่บางระบบมีความคลาดเคลื่อนหลายร้อยฟุตหรือหลายไมล์



ภาพที่ 2.14 ตัวอย่างความแตกต่างของตำแหน่งที่ได้จากวิธี Geolocation หลายรูปแบบ

ที่มา: Universal Service Administrative Company (ม.ป.ป.)

การอนุญาตให้เข้าถึงข้อมูลตำแหน่ง

ข้อมูลตำแหน่งมีความสัมพันธ์กับอุปกรณ์และผู้ใช้งานโดยตรง การเข้าถึงข้อมูลบางประเภทจึงจำเป็นต้องมีการจัดการสิทธิ์ในการเข้าถึงอย่างเหมาะสม โดยเฉพาะตำแหน่ง GPS ซึ่งสามารถให้ข้อมูลตำแหน่งของอุปกรณ์ได้ในระดับที่มีความละเอียดสูง

Zhu et al. (2020) อธิบายระบบที่ใช้ **GPS Authorization** ร่วมกับ Login Authorization โดย Front-end จะสามารถรวบรวม GPS Position ของผู้ใช้งานได้เมื่อได้รับอนุญาตจากผู้ใช้ จากนั้นจึงนำข้อมูล GPS ไปใช้สำหรับตรวจสอบผล IP Geolocation

ในด้าน Back-end ระบบยังมีการใช้ Token Authentication ร่วมกับ Server เพื่อจำกัดการเข้าถึงตามผู้ใช้งาน และบันทึก Geolocation Log ไว้ใน Database

ลักษณะดังกล่าวแสดงให้เห็นว่า การใช้งาน Geolocation ไม่ได้มีเพียงขั้นตอนทางเทคนิคในการค้นหาพิกัด แต่ยังเกี่ยวข้องกับการควบคุมการเข้าถึงข้อมูลตำแหน่งและการกำหนดสิทธิ์ของผู้ใช้งาน

สำหรับระบบที่ใช้ GPS โดยตรง การขออนุญาตจากผู้ใช้งานจึงเป็นส่วนหนึ่งของขั้นตอนการรวบรวมข้อมูลตำแหน่ง และช่วยให้การเข้าถึงข้อมูลเป็นไปตามขอบเขตที่ระบบกำหนดไว้

การใช้ Layer และข้อมูลขอบเขตบนแผนที่

Map Integration สามารถประกอบด้วยข้อมูลหลายชั้นหรือหลายประเภท โดย Basemap ทำหน้าที่เป็นพื้นหลัง ขณะที่ข้อมูลเชิงพื้นที่อื่นสามารถถูกเพิ่มเข้าไปเป็น Layer เพื่อแสดงข้อมูลเพิ่มเติม

Kelly (2026) ยกตัวอย่างการนำข้อมูลขอบเขต Congressional District มาเพิ่มบน Basemap โดยอ่านข้อมูลจาก Shapefile แปลงข้อมูลเป็น GeoJSON แล้วเพิ่มข้อมูลดังกล่าวลงในแผนที่

แนวทางดังกล่าวช่วยให้ข้อมูล Boundary หรือพื้นที่สามารถถูกแสดงทับบนแผนที่พื้นฐาน โดยระบบสามารถรักษาความสัมพันธ์ระหว่าง Geometry ของพื้นที่กับข้อมูล Attribute ที่เกี่ยวข้องได้

นอกจากการแสดง Boundary แล้ว Map Integration ยังสามารถนำข้อมูลเชิงปริมาณมาเชื่อมโยงกับพื้นที่ได้ ตัวอย่างหนึ่งคือ **Choropleth Map** ซึ่งเป็นแผนที่ที่ใช้สีแสดงปริมาณหรือค่าที่สัมพันธ์กับพื้นที่คงที่ เช่น County, Congressional District หรือ State

การสร้าง Choropleth Map จำเป็นต้องมีข้อมูล Geometry สำหรับกำหนดขอบเขต และ Data Value ที่สามารถเชื่อมโยงกับแต่ละพื้นที่ได้ ในตัวอย่างของ Kelly (2026) มีการใช้ GeoJSON สำหรับขอบเขตพื้นที่และ DataFrame สำหรับค่าข้อมูล จากนั้นเชื่อมสองส่วนเข้าด้วยกันผ่าน Key ที่ตรงกัน

แนวคิดดังกล่าวแสดงให้เห็นว่า Map Integration สามารถใช้ได้มากกว่าการแสดงจุดตำแหน่ง โดยสามารถรวมข้อมูลเชิงพื้นที่กับข้อมูลเชิงคุณลักษณะเพื่อสร้างการนำเสนอในรูปแบบที่ช่วยให้เห็น Pattern และความแตกต่างของแต่ละพื้นที่

การเลือกระบบพิกัดให้สอดคล้องกับข้อมูลแผนที่

การนำข้อมูลหลายแหล่งมารวมกันบนแผนที่จำเป็นต้องตรวจสอบ Coordinate Reference System ของแต่ละชุดข้อมูล หากข้อมูลใช้ CRS แตกต่างกัน ตำแหน่งหรือรูปร่างของ Geometry อาจไม่ตรงกันเมื่อนำมาแสดงบนแผนที่เดียวกัน

Kelly (2026) ระบุว่าการ Projection พิกัดจากพื้นผิวทรงกลมไปสู่พื้นที่สองมิติสามารถทำให้รูปร่างของพื้นที่แตกต่างกัน และเมื่อรวม Map Data หลายชุด CRS ของแต่ละชุดควรเป็นระบบเดียวกัน

เอกสารแสดงตัวอย่างการแปลงข้อมูลจาก EPSG:26918 ไปเป็น EPSG:4326 ก่อนใช้ Latitude/Longitude ในการตรวจสอบตำแหน่งกับ Geometry ของ Congressional District

ขั้นตอนดังกล่าวเป็นประเด็นสำคัญในการทำ Map Integration เนื่องจากแม้ข้อมูลแต่ละชุดจะมีค่าพิกัดที่ถูกต้องใน CRS ของตนเอง แต่หากนำข้อมูลที่อ้างอิงคนละระบบมาเปรียบเทียบโดยไม่แปลงก่อน อาจส่งผลให้ตำแหน่งที่แสดงไม่ตรงกัน

ดังนั้นการตรวจสอบ CRS และการ Transform Coordinate ให้อยู่ในระบบเดียวกันจึงควรดำเนินการก่อนการรวมข้อมูลเชิงพื้นที่หลายแหล่ง

การนำเสนอข้อมูลผ่านแผนที่แบบโต้ตอบ

ระบบแผนที่บน Web สามารถรองรับการโต้ตอบ เช่น Pan และ Zoom ทำให้ผู้ใช้งานสามารถเปลี่ยนพื้นที่หรือระดับรายละเอียดที่ต้องการดูได้

Kelly (2026) อธิบายว่า Web Mapping System เช่น Google Maps และ Leaflet ใช้ Map Tiles เป็นพื้นฐาน เมื่อผู้ใช้งาน Pan หรือ Zoom Browser จะโหลด Tile ใหม่ที่ตรงกับพื้นที่และระดับรายละเอียดที่กำลังแสดง

ในตัวอย่าง Folium สามารถสร้าง Map Object โดยกำหนด Location เริ่มต้นและ Zoom Level จากนั้นบันทึกเป็น HTML เพื่อแสดงผ่าน Browser ได้

เอกสารยังระบุว่า Folium ทำงานโดยแปลงการเรียกใช้ผ่าน Python API ไปเป็นการเรียกใช้ Leaflet API และสามารถนำข้อมูล GeoJSON รวมเข้ากับ Basemap ได้

การใช้ Map Tile และ Layer จึงช่วยให้ระบบแผนที่สามารถนำเสนอข้อมูลเชิงพื้นที่จำนวนมากโดยไม่จำเป็นต้องโหลดข้อมูลทั้งหมดในครั้งเดียว ขณะเดียวกันยังสามารถเพิ่มข้อมูลเฉพาะ เช่น Boundary หรือข้อมูลเชิงคุณลักษณะลงบน Basemap ได้

ประโยชน์ของ Map Integration and Geolocation

Map Integration และ Geolocation สามารถสนับสนุนการทำงานของระบบที่เกี่ยวข้องกับตำแหน่งในหลายด้าน โดยประโยชน์พื้นฐานคือการทำให้ข้อมูลที่อยู่ในรูป Address, IP Address หรือ Coordinate สามารถเชื่อมโยงกับตำแหน่งทางภูมิศาสตร์ได้

Geolocation ช่วยสร้างข้อมูลตำแหน่งจากหลายแหล่ง เช่น GPS, Web-based Map, Address Geocoding และ IP Address แต่ละวิธีมีระดับความแม่นยำและข้อจำกัดที่แตกต่างกัน การเลือกวิธีจึงควรพิจารณาตามวัตถุประสงค์และระดับ Precision ที่ต้องการ

ในขณะที่ Map Integration ช่วยนำข้อมูลตำแหน่งดังกล่าวมาแสดงร่วมกับ Basemap, Boundary และข้อมูลเชิงพื้นที่ประเภทอื่น ช่วยให้สามารถสังเกตตำแหน่งและความสัมพันธ์ของข้อมูลภายในพื้นที่ได้

Zhu et al. (2020) ระบุว่า IP Geolocation สามารถนำไปใช้ในหลายบริบท ได้แก่ Targeted Content, Network Performance Optimization, Social Network, Internet of Things และ Network Security ซึ่งสะท้อนให้เห็นว่าข้อมูลตำแหน่งสามารถนำไปใช้เป็นองค์ประกอบสนับสนุนการทำงานของระบบได้หลายรูปแบบ

นอกจากนี้ การเชื่อมข้อมูลเชิงปริมาณเข้ากับ Geometry ยังสามารถใช้สร้าง Choropleth Map เพื่อช่วยให้มองเห็นความแตกต่างของข้อมูลในแต่ละพื้นที่ได้ โดยใช้สีแทนค่าที่สัมพันธ์กับ Geographic Region

ด้วยเหตุนี้ Map Integration และ Geolocation จึงเป็นแนวคิดที่เกี่ยวข้องทั้งกับการค้นหาตำแหน่ง การจัดการ Coordinate การผสานข้อมูล และการนำเสนอข้อมูลในรูปแบบเชิงพื้นที่

สรุปแนวคิด Map Integration and Geolocation

Map Integration เป็นกระบวนการนำข้อมูลทางภูมิศาสตร์และข้อมูลประเภทอื่นมาเชื่อมโยงกับแผนที่ โดยอาศัยข้อมูล Coordinate, Geometry, Coordinate Reference System, Basemap และ Layer เพื่อให้ข้อมูลสามารถถูกนำเสนอในบริบททางภูมิศาสตร์

Geolocation เป็นกระบวนการระบุตำแหน่งโดยใช้ Latitude และ Longitude เป็นข้อมูลพื้นฐาน ซึ่งสามารถรวบรวมได้หลายวิธี ได้แก่ GPS Field Collection, Desktop Geolocation ผ่าน Web-based Map, Automated Address Geocoding และในอีกบริบทหนึ่งสามารถประมาณตำแหน่งจาก IP Address ได้

แต่ละวิธีมีระดับความแม่นยำแตกต่างกัน GPS ที่เก็บจากพื้นที่จริงสามารถให้ความน่าเชื่อถือสูง ขณะที่ Automated Geocoding มีข้อได้เปรียบในการประมวลผล Address จำนวนมาก แต่ Accuracy ขึ้นอยู่กับคุณภาพของ Input และ Reference Data

การใช้ข้อมูลตำแหน่งร่วมกับแผนที่ยังต้องคำนึงถึง Coordinate Reference System เนื่องจากข้อมูลที่ใช้ CRS แตกต่างกันอาจไม่สามารถนำมาวางซ้อนกันได้อย่างถูกต้อง จึงจำเป็นต้องตรวจสอบและ Transform ข้อมูลให้อยู่ใน Coordinate System ที่เหมาะสมก่อนการ Integration

นอกจากนี้ Map Integration สามารถรองรับข้อมูลหลายรูปแบบ ตั้งแต่ Point สำหรับตำแหน่งเดี่ยว Polygon สำหรับขอบเขตพื้นที่ ไปจนถึงการแสดงข้อมูลเชิงปริมาณผ่าน Choropleth Map การใช้ Basemap และ Map Tiles ยังช่วยให้ Web Mapping System สามารถรองรับการ Pan และ Zoom พร้อมโหลดรายละเอียดของพื้นที่ตามระดับที่ผู้ใช้งานต้องการ

ดังนั้น Map Integration and Geolocation จึงเป็นองค์ประกอบสำคัญของระบบที่ต้องจัดการข้อมูลเชิงพื้นที่ ตั้งแต่การค้นหาหรือรวบรวมพิกัด การตรวจสอบความถูกต้อง การแปลงระบบพิกัด ไปจนถึงการนำข้อมูลไปแสดงและวิเคราะห์บนแผนที่อย่างเป็นระบบ

2.1.6 NoSQL and Cloud Database

แนวคิดของฐานข้อมูลและฐานข้อมูล NoSQL

ฐานข้อมูล (Database) เป็นองค์ประกอบสำคัญของระบบสารสนเทศที่ใช้สำหรับจัดเก็บ จัดระเบียบ และเรียกใช้ข้อมูลให้สามารถนำไปประมวลผลได้อย่างมีประสิทธิภาพ รูปแบบของฐานข้อมูลสามารถแตกต่างกันตามลักษณะข้อมูล โครงสร้างที่ใช้จัดเก็บ และความต้องการของระบบ โดยฐานข้อมูลสามารถจำแนกออกเป็นฐานข้อมูลเชิงสัมพันธ์ (Relational Database) และฐานข้อมูลแบบไม่เชิงสัมพันธ์ (Non-Relational Database) หรือ NoSQL

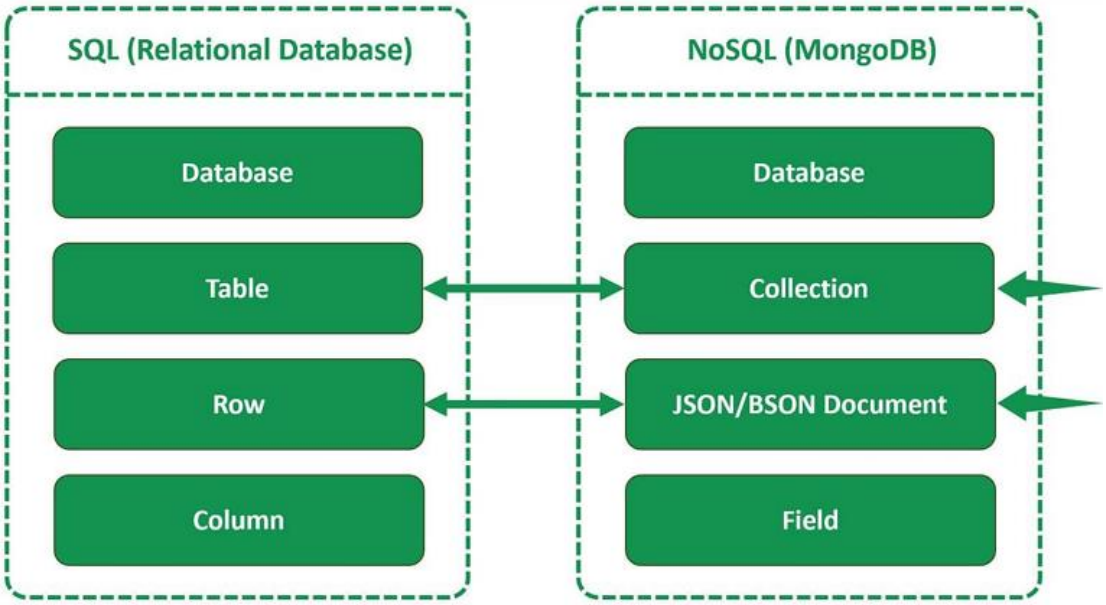
เอกสาร *Introduction to NoSQL and MongoDB* อธิบายว่า Relational Database จัดเก็บข้อมูลในรูปแบบตาราง ซึ่งประกอบด้วย Row และ Column มีโครงสร้างที่ชัดเจน และใช้ Primary Key กับ Foreign Key ในการกำหนดความสัมพันธ์ระหว่างข้อมูล ส่วน Non-Relational Database หรือ NoSQL ถูกออกแบบให้มีความยืดหยุ่นในการจัดเก็บข้อมูลมากขึ้น ไม่มีรูปแบบโครงสร้างที่ตายตัว และสามารถปรับขนาดหรือเปลี่ยนแปลงโครงสร้างได้ง่ายกว่า

คำว่า NoSQL โดยทั่วไปหมายถึง **Not Only SQL** ซึ่งไม่ได้หมายความว่าระบบดังกล่าวปฏิเสธการใช้ SQL โดยสิ้นเชิง แต่สะท้อนถึงแนวทางการจัดเก็บข้อมูลที่แตกต่างจากฐานข้อมูลเชิงสัมพันธ์แบบดั้งเดิม โดยมุ่งรองรับข้อมูลและรูปแบบการประมวลผลที่มีความยืดหยุ่นมากขึ้น

เอกสารระบุว่า NoSQL เหมาะสำหรับงานที่เกี่ยวข้องกับ Big Data และ Real-time Web Application เนื่องจากสามารถรองรับโครงสร้างข้อมูลที่เปลี่ยนแปลงได้ง่ายและมีความสามารถในการปรับขนาดของระบบ

ในฐานข้อมูลเชิงสัมพันธ์ ข้อมูลจะถูกแบ่งออกเป็น Table, Row และ Column ขณะที่ฐานข้อมูล NoSQL บางประเภท เช่น Document Database จะใช้โครงสร้าง Database,

Collection, Document และ Field แทน ซึ่งช่วยให้ข้อมูลภายใน Document แต่ละรายการสามารถมีโครงสร้างที่ยืดหยุ่นมากขึ้น



ภาพที่ 2.15 การเปรียบเทียบโครงสร้างข้อมูลระหว่าง SQL และ NoSQL

ที่มา: ชโลธร ชูทอง และคณะ (ม.ป.ป.)

ความแตกต่างระหว่าง Relational Database และ NoSQL Database

Relational Database หรือฐานข้อมูลเชิงสัมพันธ์มีแนวคิดในการจัดเก็บข้อมูลให้อยู่ในรูปแบบ Table โดยแต่ละ Table ประกอบด้วย Row และ Column และสามารถสร้างความสัมพันธ์ระหว่าง Table ผ่าน Key ต่าง ๆ เช่น Primary Key และ Foreign Key โครงสร้างดังกล่าวเหมาะสำหรับข้อมูลที่สามารถกำหนด Schema ได้อย่างชัดเจนและต้องการรักษาความสัมพันธ์ระหว่างข้อมูลอย่างเป็นระบบ

ในทางตรงกันข้าม NoSQL Database ไม่จำเป็นต้องจัดเก็บข้อมูลเป็น Table ในลักษณะเดียวกับ Relational Database โดยสามารถใช้รูปแบบข้อมูลที่หลากหลาย เช่น Key-Value, Document, Graph หรือ Wide Column ขึ้นอยู่กับประเภทของฐานข้อมูล

ความแตกต่างสำคัญอีกประการหนึ่งคือ Schema โดย Relational Database มักมี Schema ที่ชัดเจน การเปลี่ยนแปลงโครงสร้างข้อมูลจึงอาจต้องมีการปรับโครงสร้าง Table ขณะที่ NoSQL มีความยืดหยุ่นมากกว่าในด้านโครงสร้างข้อมูล เอกสารเปรียบเทียบว่า Relational Database มักขยายระบบแบบ Vertical Scaling หรือเพิ่มประสิทธิภาพของ Server ขณะที่ NoSQL สามารถรองรับ Horizontal Scaling หรือเพิ่มจำนวน Server ได้ง่ายกว่าในหลายรูปแบบ

อย่างไรก็ตาม ความแตกต่างดังกล่าวไม่ได้หมายความว่า NoSQL สามารถแทนที่ Relational Database ได้ในทุกกรณี แต่ระบบฐานข้อมูลแต่ละประเภทได้รับการออกแบบเพื่อตอบสนองต่อรูปแบบข้อมูลและข้อกำหนดที่แตกต่างกัน การเลือกใช้จึงควรพิจารณาจากลักษณะของข้อมูล ปริมาณข้อมูล รูปแบบ Query ความสัมพันธ์ของข้อมูล และความต้องการด้าน Scalability

ตารางที่ 2.29 การเปรียบเทียบ Relational Database และ NoSQL Database

ประเด็น	Relational Database	NoSQL Database
รูปแบบข้อมูล	Table, Row, Column	แตกต่างตามประเภท เช่น Document, Key-Value, Graph
ความสัมพันธ์	ใช้ Primary Key และ Foreign Key	ไม่จำเป็นต้องใช้ความสัมพันธ์ในลักษณะเดียวกับ RDBMS
Schema	โครงสร้างค่อนข้างชัดเจน	มีความยืดหยุ่นสูงกว่า
การขยายระบบ	มักใช้ Vertical Scaling	รองรับ Horizontal Scaling ได้ดีในหลายระบบ
รูปแบบข้อมูล	เหมาะกับ Structured Data	รองรับ Structured, Semi-structured และข้อมูลที่ยืดหยุ่น
ตัวอย่าง	MySQL, Oracle, SQL Server, PostgreSQL	MongoDB, Redis, Cassandra, Neo4j

ประเภทของข้อมูลที่สัมพันธ์กับฐานข้อมูล NoSQL

ลักษณะของข้อมูลเป็นปัจจัยสำคัญในการเลือกประเภทของฐานข้อมูล เนื่องจากข้อมูลในระบบสารสนเทศไม่ได้มีโครงสร้างเหมือนกันทั้งหมด เอกสาร *Introduction to NoSQL and MongoDB* แบ่งข้อมูลออกเป็น 3 ประเภทหลัก ได้แก่ Structured Data, Unstructured Data และ Semi-structured Data

Structured Data หรือข้อมูลแบบมีโครงสร้าง หมายถึงข้อมูลที่มีรูปแบบการจัดเรียงอย่างเป็นระบบ มีโครงสร้างชัดเจน และสามารถกำหนด Field หรือประเภทข้อมูลได้ ตัวอย่างเช่นข้อมูลที่จัดเก็บในตาราง Excel หรือ CSV รวมถึงข้อมูลเชิงตัวเลขต่าง ๆ

Unstructured Data หรือข้อมูลแบบไม่มีโครงสร้าง หมายถึงข้อมูลที่ไม่ได้ถูกกำหนดรูปแบบการจัดเก็บที่ชัดเจน เช่น ข้อความ รูปภาพ เสียง และวิดีโอ เนื้อหาประเภทนี้ไม่สามารถจัดลงใน Row และ Column ได้อย่างตรงไปตรงมาเหมือนข้อมูลเชิงสัมพันธ์

ส่วน **Semi-structured Data** หรือข้อมูลกึ่งโครงสร้าง เป็นข้อมูลที่มีโครงสร้างอยู่ในระดับหนึ่ง แต่อาจไม่ได้อยู่ในรูปแบบตาราง เช่น XML, JSON และ Web Page โดยข้อมูลสามารถมีโครงสร้างแบบลำดับชั้นหรือประกอบด้วย Field ที่แตกต่างกันได้

ข้อมูลแบบ Semi-structured มีความสัมพันธ์กับ NoSQL อย่างมาก เนื่องจาก Document Database จำนวนมากสามารถจัดเก็บข้อมูลในรูปแบบ JSON หรือโครงสร้างที่ใกล้เคียงกัน ทำให้ไม่จำเป็นต้องกำหนด Column แบบตายตัวสำหรับข้อมูลทุก Record

การรองรับข้อมูลที่มีโครงสร้างยืดหยุ่นจึงเป็นหนึ่งในเหตุผลที่ NoSQL ถูกนำมาใช้กับ Application ที่โครงสร้างข้อมูลสามารถเปลี่ยนแปลงได้ตามเวลา หรือระบบที่ต้องจัดเก็บข้อมูลหลายรูปแบบภายในฐานข้อมูลเดียวกัน

ประเภทของฐานข้อมูล NoSQL

NoSQL Database ไม่ได้หมายถึงฐานข้อมูลประเภทเดียว แต่เป็นกลุ่มของฐานข้อมูลหลายรูปแบบที่ได้รับการออกแบบให้เหมาะสมกับลักษณะข้อมูลและวิธีการเข้าถึงข้อมูลที่แตกต่างกัน

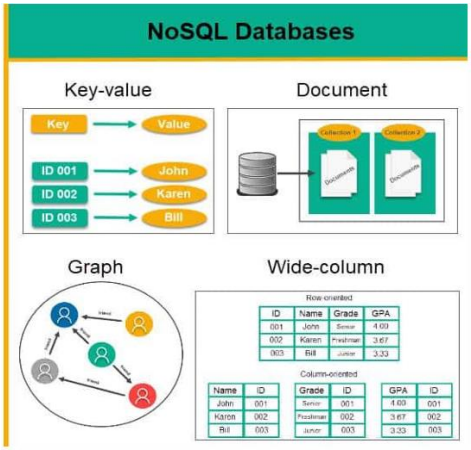
เอกสารแบ่ง NoSQL Database ออกเป็น 4 ประเภทหลัก ได้แก่ **Key-Value Store, Document Database, Graph Database และ Wide Column Store**

Key-Value Store เป็นฐานข้อมูลที่จัดเก็บข้อมูลในลักษณะคู่ของ Key และ Value โดยใช้ Key เป็นตัวระบุข้อมูลและใช้ Value เป็นค่าที่สัมพันธ์กับ Key นั้น โครงสร้างมีความเรียบง่ายและเหมาะกับระบบที่ต้องการเข้าถึงข้อมูลอย่างรวดเร็ว

Document Database จัดเก็บข้อมูลเป็น Document โดย Document สามารถประกอบด้วย Key-Value Pair และมักใช้โครงสร้าง JSON หรือรูปแบบที่คล้าย JSON ทำให้ข้อมูลแต่ละ Document ไม่จำเป็นต้องมี Field เหมือนกันทั้งหมด

Graph Database มุ่งเน้นการจัดเก็บความสัมพันธ์ระหว่างข้อมูล จึงเหมาะกับข้อมูลที่มีการเชื่อมโยงระหว่าง Object จำนวนมาก

ส่วน **Wide Column Store** จัดเก็บข้อมูลในลักษณะ Column Family ซึ่งช่วยรองรับข้อมูลขนาดใหญ่และโครงสร้างที่สามารถเปลี่ยนแปลงได้ตามความต้องการ



ภาพที่ 2.16 ประเภทของฐานข้อมูล NoSQL

ที่มา: ชโลธร ชูทอง และคณะ (ม.ป.ป.)

Document Database

Document Database เป็นฐานข้อมูล NoSQL ประเภทหนึ่งที่จัดเก็บข้อมูลเป็น Document แทนการจัดเก็บข้อมูลในรูปแบบ Row และ Column โดย Document สามารถมีโครงสร้างข้อมูลที่ยืดหยุ่นและประกอบด้วย Field หลายประเภท

เอกสารระบุว่า Document Database สามารถจัดเก็บข้อมูลในรูปแบบ JSON Structure หรือ Document และไม่จำเป็นต้องกำหนดรูปแบบหรือประเภทของข้อมูลทั้งหมดไว้ล่วงหน้าเหมือนฐานข้อมูลเชิงสัมพันธ์

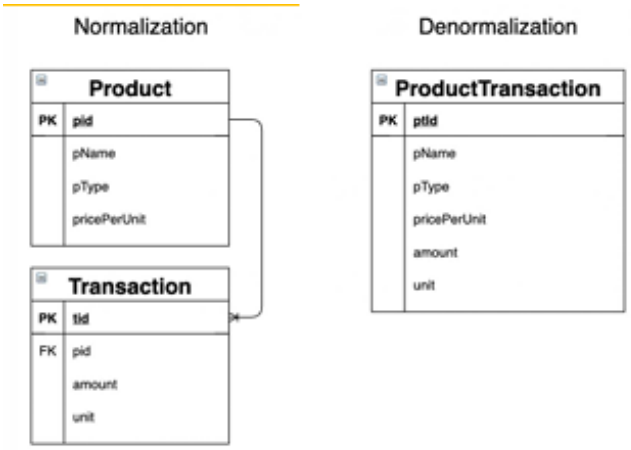
ภายใน Database จะประกอบด้วย Collection และภายใน Collection จะประกอบด้วย Document หลายรายการ โดยแต่ละ Document สามารถจัดเก็บข้อมูลแบบ Key-Value Pair และสามารถมี Nested Data ได้

Collection สามารถเปรียบเทียบในระดับแนวคิดกับ Table ของฐานข้อมูลเชิงสัมพันธ์ ขณะที่ Document มีลักษณะใกล้เคียงกับ Record แต่สามารถมีโครงสร้างภายในที่ยืดหยุ่นกว่า และแต่ละ Document ไม่จำเป็นต้องมี Field เหมือนกันทั้งหมด

Document Database ยังมีความสัมพันธ์กับแนวคิด **Denormalization** ซึ่งแตกต่างจาก Relational Database ที่มักใช้ Normalization เพื่อแบ่งข้อมูลออกเป็นหลาย Table ลดข้อมูลซ้ำ และเชื่อมความสัมพันธ์ผ่าน Key ในทางกลับกัน Document Database สามารถรวมข้อมูลที่เกี่ยวข้องไว้ภายใน Document เดียวเพื่อให้สามารถดึงข้อมูลได้โดยไม่จำเป็นต้อง Join หลาย Collection

แนวทาง Denormalization สามารถลดขั้นตอนในการเรียกข้อมูลบางประเภทและทำให้ Data Model มีความใกล้เคียงกับข้อมูลที่ Application ใช้งานจริง อย่างไรก็ตาม การออกแบบ

Document ที่รวมข้อมูลหลายส่วนไว้ด้วยกันจำเป็นต้องพิจารณาความซ้ำซ้อนของข้อมูลและรูปแบบการ Update ด้วย



ภาพที่ 2.17 การเปรียบเทียบ Normalization และ Denormalization

ที่มา: ชโลธร ชูทอง และคณะ (ม.ป.ป.)

JSON และ BSON

JSON หรือ JavaScript Object Notation เป็นรูปแบบข้อมูลแบบข้อความที่ใช้โครงสร้าง Key และ Value และสามารถใช้สำหรับการแลกเปลี่ยนข้อมูลระหว่างระบบได้ เอกสารอธิบายว่า JSON จัดเก็บข้อมูลโดยกำหนดชื่อ Field และ Value ภายในวงเล็บปีกกา และเป็นรูปแบบข้อมูลที่มีขนาดค่อนข้างเล็กสำหรับใช้แลกเปลี่ยนข้อมูล

ข้อมูล JSON สามารถประกอบด้วย Object ซ้อนกันหลายระดับ ทำให้สามารถแทนข้อมูลที่มีโครงสร้างแบบ Hierarchy ได้ และจัดเป็น Semi-structured Data เนื่องจากมีโครงสร้าง Field ที่ชัดเจนแต่ไม่จำเป็นต้องใช้ Schema แบบตายตัว

BSON หรือ Binary JSON เป็นรูปแบบที่ใช้แทนข้อมูลในรูป Binary และมีความสัมพันธ์กับระบบฐานข้อมูล Document บางประเภท เอกสารระบุว่า BSON เป็นการแทนค่าข้อมูลในรูปแบบ Binary ซึ่งแตกต่างจาก JSON ที่อยู่ในรูป Text

การใช้ JSON หรือรูปแบบ Document ทำให้โครงสร้างข้อมูลในฐานข้อมูลสามารถมีความใกล้เคียงกับ Object ที่ Application ใช้งาน ช่วยลดขั้นตอนบางส่วนในการแปลงข้อมูลระหว่าง Application กับ Database

Cloud Database

Cloud Database เป็นแนวทางการจัดเก็บและบริหารจัดการฐานข้อมูลโดยใช้โครงสร้างพื้นฐานของ Cloud Computing แทนการติดตั้งและดูแลระบบทั้งหมดบน Server ภายในองค์กร

โดยผู้ให้บริการ Cloud สามารถจัดเตรียม Computing Resource, Storage, Network และ Database Service สำหรับให้ผู้ใช้งานเลือกตามความต้องการ

Neu et al. (2019) อธิบายว่าผู้ให้บริการ Cloud รายใหญ่มี Database Service หลายประเภท ทั้งฐานข้อมูลเชิงสัมพันธ์และฐานข้อมูลแบบ Nonrelational เช่น Key/Value และ Document Store และองค์กรสามารถย้ายข้อมูลจากระบบ On-premises ไปใช้งานฐานข้อมูลที่ผู้ให้บริการ Cloud จัดเตรียมไว้ได้

Cloud Database ไม่ได้จำกัดอยู่ที่ฐานข้อมูลประเภทใดประเภทหนึ่ง ผู้ให้บริการสามารถนำเสนอทั้ง Relational Database, Data Warehouse และ Nonrelational Database ตามรูปแบบการใช้งานที่แตกต่างกัน Neu et al. (2019) ระบุว่าผู้ให้บริการ Cloud มีทั้ง Relational Database เช่น MySQL และ PostgreSQL ตลอดจนฐานข้อมูลแบบ Key/Value, Document, Graph, Search, Time Series และ Ledger Database

การนำฐานข้อมูลไปทำงานบน Cloud ทำให้รูปแบบการบริหารจัดการทรัพยากรแตกต่างจาก On-premises Environment โดยองค์กรไม่จำเป็นต้องซื้อ Hardware ทั้งหมดล่วงหน้า แต่สามารถเลือก CPU, Memory และ Storage ตามขนาดของ Database Instance ที่ต้องการใช้งาน

นอกจากนี้ ผู้ให้บริการ Cloud สามารถมี Region และ Availability Zone ในหลายพื้นที่ทางภูมิศาสตร์ ซึ่งช่วยให้สามารถเลือกตำแหน่งของ Server ให้ใกล้ผู้ใช้งาน กระจายข้อมูลเพื่อเพิ่ม Redundancy และรองรับ Load Balancing หรือ Disaster Recovery ได้

อีกหนึ่งลักษณะเด่นคือ **Elastic Scaling** โดยเมื่อ Workload เพิ่มขึ้น ระบบสามารถเพิ่ม Instance เพื่อช่วยแบ่งภาระ และลดจำนวน Instance ลงเมื่อความต้องการลดลง ทำให้สามารถปรับ Resource ให้สอดคล้องกับการใช้งานได้มากกว่าการจัดเตรียม Physical Server แบบคงที่

Self-Managed Database และ Managed Database

การใช้งานฐานข้อมูลบน Cloud สามารถแบ่งเป็นสองรูปแบบหลัก ได้แก่ Self-Managed Database และ Managed Database

Neu et al. (2019) อธิบายว่า **Self-Managed Database** เป็นรูปแบบที่ผู้ให้บริการ Cloud จัดเตรียม Hardware, Hypervisor และเครื่องมือสำหรับจัดการ Deployment ขณะที่ผู้ใช้งานยังมีหน้าที่สร้าง Virtual Machine ติดตั้ง Operating System ติดตั้ง Database และจัดการ Network รวมถึงงานดูแลระบบส่วนใหญ่ด้วยตนเอง

ในทางตรงกันข้าม **Managed Database** เป็นบริการที่ผู้ให้บริการจัดเตรียมทั้ง Hardware และ Server Software และช่วยดำเนินงานด้านการติดตั้ง การ Update การ Configuration และ Backup ของฐานข้อมูลหลายส่วนให้โดยอัตโนมัติ

Managed Database ช่วยลดภาระงานทางด้าน Infrastructure และ Routine Maintenance แต่ผู้ใช้งานยังคงต้องรับผิดชอบการตัดสินใจบางส่วน เช่น การกำหนด User Account, Access Permission และการควบคุมสิทธิ์ในการเข้าถึงข้อมูล

การเลือกระหว่าง Self-Managed และ Managed จึงสัมพันธ์กับระดับการควบคุมที่ต้องการ หากองค์กรต้องการควบคุม Version หรือ Database Feature อย่างละเอียด อาจเลือก Self-Managed แต่หากต้องการลดภาระด้านการติดตั้งและ Maintenance บริการ Managed Database สามารถช่วยลดงานดังกล่าวได้

Cloud-Native Database

Cloud-Native Database เป็นฐานข้อมูลที่ได้รับการออกแบบให้สอดคล้องกับลักษณะของ Cloud Environment ตั้งแต่ต้น และสามารถใช้ความสามารถของ Cloud เช่น Distributed Processing, Elastic Scaling และ High Availability ได้อย่างเหมาะสม

Neu et al. (2019) แบ่ง Managed Database ออกเป็น Traditional Database และ Cloud-Native Database โดยฐานข้อมูลแบบ Traditional เช่น Oracle, SQL Server, MySQL และ PostgreSQL สามารถถูกนำมาให้บริการในรูป Managed Database บน Cloud ได้ ขณะที่ Cloud Provider ยังพัฒนาฐานข้อมูลของตนเองที่ถูกออกแบบให้ทำงานบน Cloud โดยเฉพาะ

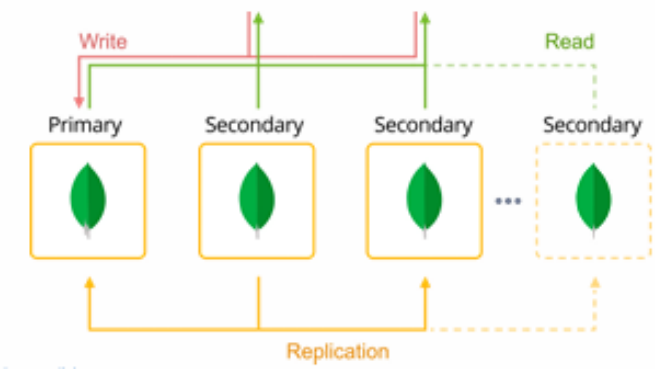
Cloud-Native Database มีเป้าหมายในการรองรับ Scalability และการบริหาร Resource ที่มีความยืดหยุ่น โดยเอกสารระบุว่าฐานข้อมูลประเภทนี้ได้รับการออกแบบให้สามารถขยายระบบได้ในระดับสูง ซึ่งเป็นงานที่ทำได้ยากกว่าสำหรับ Relational Database แบบดั้งเดิมบางประเภท

แนวคิดดังกล่าวทำให้ Cloud Database และ NoSQL มีความสัมพันธ์กัน เนื่องจากผู้ให้บริการ Cloud สามารถให้บริการฐานข้อมูลแบบ Nonrelational หลายประเภท เช่น Key/Value, Document และ Graph Database เพื่อรองรับ Workload ที่มีลักษณะแตกต่างกัน

NoSQL Database บนระบบ Cloud

ฐานข้อมูล NoSQL สามารถถูกติดตั้งบน Server ภายในองค์กรหรือให้บริการผ่านระบบ Cloud ได้ ตัวอย่างในเอกสารอบรมคือ MongoDB ซึ่งสามารถใช้งานได้ทั้งการติดตั้งบน Server และผ่านบริการ Cloud

ในระบบ Cloud สามารถสร้างฐานข้อมูลในลักษณะ Cluster และเลือก Cloud Provider รวมถึง Region ที่ต้องการ Deploy ได้ เพื่อสนับสนุน High Availability ของระบบ



ภาพที่ 2.18 โครงสร้าง Cluster และ Replication ของฐานข้อมูล NoSQL บน Cloud
ที่มา: ชโลธร ชูทอง และคณะ (ม.ป.ป.)

Scalability และ Elastic Scaling

Scalability เป็นความสามารถของระบบฐานข้อมูลในการรองรับปริมาณข้อมูล จำนวนผู้ใช้งาน หรือภาระการประมวลผลที่เพิ่มขึ้น โดยยังคงสามารถให้บริการได้อย่างเหมาะสม คุณลักษณะนี้มีความสำคัญต่อทั้ง NoSQL Database และ Cloud Database เนื่องจากระบบสมัยใหม่อาจมีปริมาณข้อมูลและจำนวน Request เปลี่ยนแปลงตลอดเวลา

ในเอกสาร *Introduction to NoSQL and MongoDB* มีการเปรียบเทียบว่า Relational Database มักขยายระบบแบบ **Vertical Scaling** ซึ่งเป็นการเพิ่มความสามารถของ Server เดิม ขณะที่ NoSQL Database สามารถรองรับ **Horizontal Scaling** โดยการเพิ่มจำนวน Server เพื่อกระจายภาระของระบบ

แนวคิด Horizontal Scaling มีความสัมพันธ์กับลักษณะของระบบแบบ Distributed เนื่องจากข้อมูลหรือการประมวลผลสามารถถูกกระจายไปยังหลาย Node ได้ แทนที่จะพึ่งพา Server เครื่องเดียวที่มีประสิทธิภาพสูงขึ้นเรื่อย ๆ

ในบริบทของ Cloud Database, Neu et al. (2019) อธิบายว่าผู้ให้บริการ Cloud มีความสามารถในการเพิ่มหรือลด Resource ให้สอดคล้องกับ Workload เมื่อปริมาณงานเพิ่มขึ้น ระบบสามารถเริ่ม Instance ใหม่เพื่อช่วยแบ่งภาระ และเมื่อความต้องการลดลง Instance ที่ไม่จำเป็นสามารถหยุดการทำงานได้ แนวทางนี้เรียกว่า **Elastic Scaling**

Elastic Scaling แตกต่างจากการเตรียม Hardware แบบ On-premises ซึ่งต้องซื้อและติดตั้งทรัพยากรล่วงหน้า การใช้ Cloud ช่วยให้การเพิ่ม Resource สามารถทำได้ตามความต้องการและลดปัญหา Overprovisioning ในบางกรณี

Neu et al. (2019) ยังระบุว่า Autoscaling สามารถทำงานได้ทั้งในระดับ Virtual Machine และระดับ Database โดยระบบสามารถเพิ่ม Instance เมื่อ Load สูงขึ้น และลด Instance เมื่อ Load ลดลง เพื่อไม่ให้เกิดการใช้ Resource เกินความจำเป็น

ความสามารถในการ Scale จึงเป็นหนึ่งในจุดเชื่อมสำคัญระหว่าง NoSQL และ Cloud Database เพราะ NoSQL หลายประเภทถูกออกแบบให้รองรับการกระจายข้อมูลหรือการประมวลผล ขณะที่ Cloud Infrastructure ช่วยให้สามารถเพิ่มหรือลด Resource ได้อย่างยืดหยุ่น

High Availability และ Replication

High Availability หรือ HA หมายถึงความสามารถของระบบในการให้บริการได้อย่างต่อเนื่อง แม้บางส่วนของ Infrastructure จะเกิดความผิดพลาด การออกแบบให้มี Server หรือข้อมูลสำรองหลายชุดจึงเป็นแนวทางสำคัญในการลดความเสี่ยงจาก Single Point of Failure

Replication หมายถึงการสร้างและรักษาสำเนาของข้อมูลไว้ในหลายตำแหน่ง เมื่อระบบหลักมีปัญหา สำเนาที่อยู่บน Node อื่นสามารถช่วยสนับสนุนการให้บริการหรือกระบวนการ Recovery ได้

ในระดับ Cloud Infrastructure, Neu et al. (2019) อธิบายว่าผู้ให้บริการ Cloud มีเครื่องมือสำหรับ Scalability, Fault Tolerance และ Load Balancing หาก Database Instance ล้มเหลวและระบบถูกกำหนดให้รองรับ High Availability สามารถมี Instance อื่นเข้ามาทดแทนได้

สำหรับระบบฐานข้อมูลที่ต้องการ Availability สูง การใช้หลาย Node หรือหลาย Instance ช่วยลดผลกระทบจากความเสียหายของอุปกรณ์หนึ่งส่วน และสามารถใช้ร่วมกับ Load Balancing เพื่อช่วยกระจาย Workload ระหว่าง Resource ที่มีอยู่

Region และ Availability Zone

Cloud Database มีคุณลักษณะที่แตกต่างจากฐานข้อมูลบน Server ภายในองค์กรตรงที่สามารถเลือกตำแหน่งทางภูมิศาสตร์ของ Infrastructure ได้ ผู้ให้บริการ Cloud มักจัด Resource ออกเป็น Region และ Availability Zone เพื่อช่วยในการกระจายระบบ

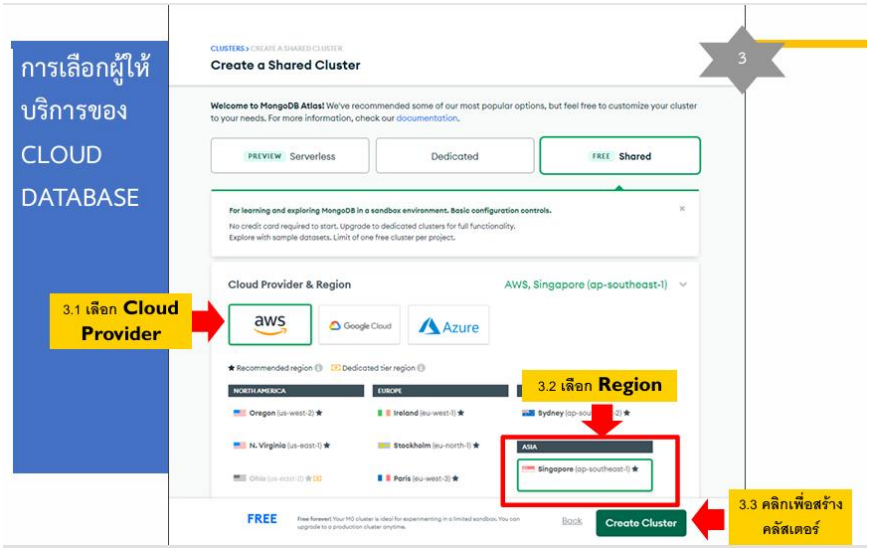
Neu et al. (2019) ระบุว่าผู้ให้บริการ Cloud รายใหญ่มีหลาย Location ซึ่งสามารถใช้เพื่อวัตถุประสงค์ต่าง ๆ เช่น วาง Server ให้ใกล้กับผู้ใช้งาน เพิ่ม Redundancy รองรับ Disaster Recovery และ Load Balancing

Region หมายถึงพื้นที่ทางภูมิศาสตร์ขนาดใหญ่ที่ผู้ให้บริการมี Infrastructure ขณะที่ **Availability Zone (AZ)** เป็นพื้นที่ย่อยภายใน Region ที่สามารถใช้สำหรับกระจาย Workload หรือสำเนาข้อมูล

การกระจาย Database ไปยังหลาย Availability Zone ช่วยเพิ่ม Resilience หาก Zone หนึ่งเกิดความเสียหาย ระบบสามารถใช้ Resource จาก Zone อื่นเพื่อให้บริการต่อได้ Neu et al. (2019) ระบุว่าระบบ Cloud สามารถกระจาย Workload ไปยังหลาย AZ พร้อมกับ Replication และ Load Balancing เพื่อช่วยเพิ่มความต่อเนื่องในการให้บริการ

ในด้าน Performance การเลือก Region ที่อยู่ใกล้กับกลุ่มผู้ใช้งานยังสามารถช่วยลด Response Time ได้ โดยเอกสารระบุว่าการเลือก Region ใกล้กับ Database User เป็นหนึ่งในปัจจัยสำคัญของการ Provision Resource บน Cloud

เอกสาร NoSQL ยังแสดงตัวอย่างการสร้าง Cluster บน Cloud โดยสามารถเลือก **Cloud Provider และ Region** ก่อนสร้าง Cluster แสดงให้เห็นว่าตำแหน่งของ Database Infrastructure สามารถกำหนดได้ในขั้นตอน Deployment



ภาพที่ 2.19 ตัวอย่างการเลือก Cloud Provider และ Region สำหรับ Cloud Database

ที่มา: ชโลธร ชูทอง และคณะ (ม.ป.ป.)

Backup และ Snapshot ใน Cloud Database

การสำรองข้อมูลเป็นกระบวนการสำคัญของระบบฐานข้อมูล เนื่องจากช่วยลดความเสี่ยงจากการสูญหายของข้อมูลและสนับสนุนการ Recovery เมื่อเกิดความผิดพลาด

Neu et al. (2019) อธิบายว่า Cloud Database มีวิธีสำรองข้อมูลพื้นฐานสองรูปแบบ ได้แก่ **Automated Backup** และ **Snapshot** โดย Automated Backup มักสามารถใช้สำหรับ Point-in-Time Recovery ขณะที่ Snapshot เป็นสำเนาที่ผู้ดูแลระบบเริ่มสร้างขึ้นตามช่วงเวลาที่ต้องการ

Snapshot สามารถใช้สำหรับเก็บข้อมูลระยะยาว Replication ไปยัง Region อื่น หรือรองรับข้อกำหนดด้าน Compliance ได้ นอกจากนี้ Managed Cloud Database ยังช่วยลดความซับซ้อนของการสร้างและคืนค่า Snapshot เมื่อเปรียบเทียบกับการจัดการ Infrastructure ด้วยตนเอง

การสำรองข้อมูลและ Replication มีวัตถุประสงค์ที่สัมพันธ์กันแต่ไม่เหมือนกัน Replication เน้นการสร้างสำเนาข้อมูลสำหรับการทำงานและ Availability ขณะที่ Backup และ Snapshot มีบทบาทสำคัญต่อ Recovery และการเก็บสถานะของข้อมูลในช่วงเวลาหนึ่ง

ในกรณีของ Managed Database ผู้ให้บริการสามารถดำเนินงานด้าน Backup บางส่วนให้โดยอัตโนมัติ ซึ่งช่วยลดภาระของ Database Administrator แต่ผู้ใช้งานยังจำเป็นต้องกำหนดนโยบายที่เหมาะสม เช่น ระยะเวลาการเก็บข้อมูลและรูปแบบการ Recovery

ความปลอดภัยของ Cloud Database

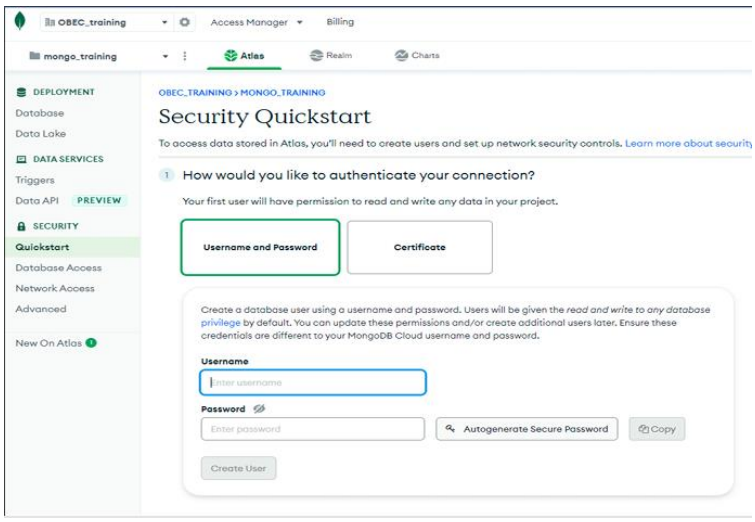
Security เป็นหนึ่งในประเด็นสำคัญของ Cloud Database เนื่องจากข้อมูลถูกจัดเก็บและเข้าถึงผ่าน Infrastructure ที่เชื่อมต่อกับเครือข่าย การรักษาความปลอดภัยจึงต้องครอบคลุมทั้ง Infrastructure, Network, User Account และข้อมูล

Neu et al. (2019) อธิบายแนวคิด **Shared Security Model** ซึ่งแบ่งความรับผิดชอบด้าน Security ระหว่าง Cloud Provider กับผู้ใช้บริการ โดย Provider รับผิดชอบ Infrastructure Security ขณะที่ผู้ใช้ยังมีหน้าที่รับผิดชอบ Security ของข้อมูลและ User Information ของตน

หน้าที่ของ Cloud Provider สามารถครอบคลุมการรักษาความปลอดภัยทางกายภาพของ Datacenter, Replication, Backup, Software Update, Network Firewall, การป้องกัน DoS และ Identity Management Service ขณะที่ผู้ใช้ต้องกำหนด User, Role, Access Right, Firewall Rule, Alert, Encryption และ Authentication ให้เหมาะสม

หลักการ Shared Responsibility จึงหมายความว่าการใช้ Managed Database ไม่ได้ทำให้ผู้ใช้งานไม่ต้องรับผิดชอบ Security แต่เป็นการแบ่งงานส่วนหนึ่งให้ Provider ดูแล โดยงานที่เกี่ยวข้องกับข้อมูลและการกำหนดสิทธิ์ยังคงอยู่ในความรับผิดชอบของผู้ใช้

ในเอกสาร NoSQL มีตัวอย่างหน้าจอ **Security Quickstart** สำหรับสร้าง Database User ด้วย Username และ Password ซึ่งสะท้อนแนวคิดการตรวจสอบตัวตนก่อนอนุญาตให้เข้าถึงฐานข้อมูล



ภาพที่ 2.20 ตัวอย่างการกำหนด Authentication สำหรับ Cloud Database

ที่มา: ชโลธร ชูทอง และคณะ (ม.ป.ป.)

Access Control และ Identity and Access Management

Access Control เป็นกระบวนการควบคุมว่าผู้ใช้หรือระบบใดสามารถเข้าถึง Resource ประเภทใดและสามารถดำเนินการอะไรได้บ้าง การควบคุมสิทธิ์มีความสำคัญต่อ Cloud Database เนื่องจาก Database อาจถูกใช้งานโดยผู้ใช้หรือ Application หลายประเภท

Neu et al. (2019) อธิบายว่า Cloud Provider ใช้ **Identity and Access Management (IAM)** สำหรับจัดการ User, Credential, Permission และ Authorization Policy เพื่อควบคุม Resource และ Service ที่แต่ละ User สามารถเข้าถึงได้

IAM สามารถใช้กำหนดสิทธิ์ที่แตกต่างกันสำหรับ User แต่ละประเภท และสามารถใช้ร่วมกับ Database-specific Access Rule เพื่อสร้างการควบคุมสิทธิ์ที่ละเอียดมากขึ้น เช่น จำกัด Operation หรือข้อมูลที่ผู้ใช้สามารถเข้าถึงได้

ใน Managed Database ผู้ดูแลระบบยังคงมีหน้าที่กำหนด Account, Group และ Role รวมถึงดูแลการเข้าถึง Database แม้ว่าการติดตั้งและ Maintenance หลายส่วนจะถูกดำเนินการโดย Cloud Provider แล้วก็ตาม

เอกสาร NoSQL แสดงตัวอย่างการเพิ่มสมาชิกเข้า Organization และ Project พร้อมกับกำหนด Permission ที่แตกต่างกันให้แต่ละ User ซึ่งเป็นตัวอย่างของการจัดการสิทธิ์ในระบบ Cloud Database

Network Security และการควบคุม IP Address

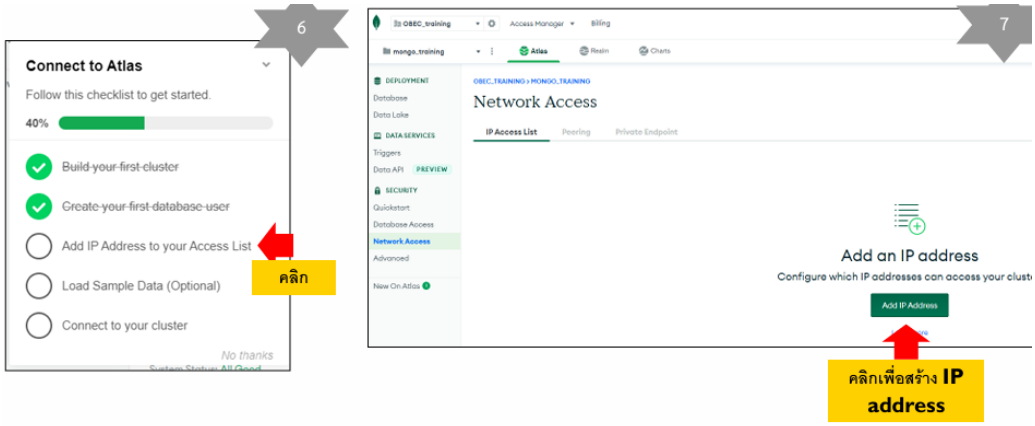
การรักษาความปลอดภัยของ Cloud Database ไม่ได้จำกัดอยู่ที่ Username และ Password เท่านั้น แต่ยังสามารถกำหนดเงื่อนไขด้านเครือข่ายเพื่อควบคุมว่าเครื่องหรือ Network ใดสามารถเชื่อมต่อกับ Database ได้

Neu et al. (2019) ระบุว่า Cloud Environment สามารถใช้ Virtual Private Cloud (VPC), Firewall และ Network Access Control List เพื่อสร้าง Network Isolation โดยสามารถกำหนด IP Range ที่ได้รับอนุญาตให้เข้าถึง Database Instance ได้

Security Group และ Firewall Rule สามารถใช้ควบคุม Network Traffic ที่เข้าสู่หรือออกจาก Database ขณะที่ VPN หรือ Direct Connection สามารถใช้เชื่อม Infrastructure ภายนอกเข้ากับ Cloud Environment ผ่านช่องทางที่มีการควบคุม

ในเอกสาร NoSQL มีตัวอย่างการกำหนด **Connection IP Address** เพื่อสร้าง Access List สำหรับควบคุมการเชื่อมต่อกับ Cluster ผ่านเครือข่าย

การกำหนด Network Access จึงเป็นอีก Layer หนึ่งของ Security นอกเหนือจาก User Authentication กล่าวคือ แม้ผู้ใช้จะมี Credential ที่ถูกต้อง แต่หาก Request มาจาก Network หรือ IP Address ที่ไม่ได้รับอนุญาต ระบบก็สามารถปฏิเสธการเชื่อมต่อได้



ภาพที่ 2.21 การกำหนดสิทธิ์การเข้าถึง Cloud Database ผ่าน IP Address

ที่มา: ชโลธร ชูทอง และคณะ (ม.ป.ป.)

การเข้ารหัสข้อมูล (Data Encryption)

Encryption เป็นกลไกสำคัญในการปกป้องข้อมูลจากการเข้าถึงโดยไม่ได้รับอนุญาต โดย Cloud Database จำเป็นต้องพิจารณาการเข้ารหัสทั้งข้อมูลที่ถูกจัดเก็บและข้อมูลที่อยู่ระหว่างการรับส่ง

Neu et al. (2019) อธิบายว่าข้อมูลบน Cloud สามารถถูกเข้ารหัสในขณะจัดเก็บหรือ **Data at Rest** และการ Encryption สามารถครอบคลุมไปถึง Backup ของฐานข้อมูลด้วย

นอกจากนี้ ข้อมูลที่ส่งระหว่างระบบหรือ **Data in Transit** ควรได้รับการเข้ารหัสเพื่อป้องกันการดักฟัง โดยเอกสารกล่าวถึงการใช้ SSL/TLS สำหรับสร้าง End-to-End Encryption ในการสื่อสารระหว่างระบบ

การเข้ารหัสมีความสำคัญโดยเฉพาะสำหรับระบบที่ต้องปฏิบัติตามข้อกำหนดด้าน Compliance เช่น ระบบที่จัดเก็บข้อมูลสุขภาพ การเงิน หรือข้อมูลการชำระเงิน ซึ่งอาจมีกฎเกณฑ์เกี่ยวกับการรักษาความลับและการปกป้องข้อมูล

อย่างไรก็ตาม Encryption จำเป็นต้องทำงานร่วมกับการจัดการ Encryption Key ที่เหมาะสม เนื่องจากความปลอดภัยของข้อมูลยังขึ้นอยู่กับการปกป้อง Key ที่ใช้สำหรับเข้ารหัสและถอดรหัสข้อมูล

Monitoring และ Performance ของ Cloud Database

การ Monitoring ช่วยให้สามารถตรวจสอบสถานะและประสิทธิภาพของ Database อย่างต่อเนื่อง และสามารถตรวจพบปัญหาก่อนที่จะส่งผลกระทบรุนแรงต่อ Application

Neu et al. (2019) ระบุว่า Cloud Provider สามารถให้ข้อมูล Metric หลายประเภท เช่น CPU Utilization, Query Latency, Query Throughput, I/O และ Memory Usage รวมถึงสามารถตั้ง Alert เมื่อค่าต่าง ๆ ออกนอกช่วงที่คาดการณ์ไว้

Cloud Platform ยังสามารถรายงาน Event เช่น Failover, Backup หรือการเปลี่ยน Configuration ทำให้ผู้ดูแลระบบสามารถติดตามสถานะของ Database และสร้าง Automated Action เมื่อเกิด Event ที่กำหนด

Performance Optimization ยังคงเป็นหน้าที่สำคัญ แม้ Database จะทำงานบน Cloud เพราะการย้ายระบบไป Cloud ไม่ได้ทำให้ Query หรือ Data Model ที่ไม่มีประสิทธิภาพดีขึ้นโดยอัตโนมัติ แต่ Cloud ช่วยเพิ่มทางเลือกในการ Scale Resource และมีเครื่องมือ Monitoring ที่ช่วยวิเคราะห์ Performance ได้ง่ายขึ้น

เมื่อระบบมี Workload เพิ่มขึ้น สามารถ Scale Up หรือ Scale Out ตามความเหมาะสม แต่ยังจำเป็นต้องตรวจสอบ Query, Index, Network และรูปแบบการใช้ข้อมูลเพื่อรักษาประสิทธิภาพโดยรวม

ข้อดีของ NoSQL และ Cloud Database

NoSQL และ Cloud Database มีลักษณะที่สนับสนุน Application สมัยใหม่หลายด้าน โดยเฉพาะระบบที่มีข้อมูลเปลี่ยนแปลงได้รวดเร็ว มีปริมาณข้อมูลสูง หรือต้องรองรับผู้ใช้งานจำนวนมาก

NoSQL มีข้อดีด้าน **Schema Flexibility** เนื่องจากไม่จำเป็นต้องกำหนดโครงสร้างข้อมูลแบบตายตัวเหมือน Relational Database และสามารถรองรับ Semi-structured Data เช่น JSON ได้โดยตรง

NoSQL ยังมีความสามารถในการรองรับ Horizontal Scaling และเหมาะกับ Big Data หรือ Real-time Web Application ตามที่เอกสารระบุ

Cloud Database มีข้อดีในด้านการลดภาระการจัดการ Infrastructure เนื่องจาก Cloud Provider สามารถดูแล Hardware, Backup, Update และงาน Routine Administration หลายประเภทได้ โดยเฉพาะในรูปแบบ Managed Database

นอกจากนี้ Cloud ยังสนับสนุน Elastic Scaling, High Availability, Multiple Region, Monitoring และ Automation ซึ่งช่วยให้ระบบสามารถปรับตัวตาม Workload และลดภาระในการจัดเตรียม Physical Infrastructure ล่วงหน้า

ข้อจำกัดของ NoSQL และ Cloud Database

แม้ NoSQL จะมีความยืดหยุ่นสูง แต่การไม่มี Schema ที่เข้มงวดในลักษณะเดียวกับ Relational Database ทำให้การออกแบบ Data Model จำเป็นต้องพิจารณารูปแบบการใช้งานข้อมูลอย่างรอบคอบ โดยเฉพาะเมื่อใช้ Denormalization ซึ่งอาจทำให้มีข้อมูลซ้ำใน Document หลายตำแหน่ง

ฐานข้อมูล NoSQL แต่ละประเภทได้รับการออกแบบสำหรับ Use Case ที่แตกต่างกัน เช่น Key-Value, Document, Graph หรือ Wide Column ดังนั้นการเลือกฐานข้อมูลที่ไม่เหมาะกับลักษณะของข้อมูลอาจทำให้การ Query หรือการบริหารข้อมูลมีความซับซ้อนขึ้น

ในด้าน Cloud Database, Managed Service มีข้อจำกัดเรื่องระดับการควบคุม เนื่องจากผู้ใช้งานไม่สามารถเลือก Database Engine หรือ Version ที่ Provider ไม่รองรับ และ Configuration บางประเภทอาจถูกจำกัดโดยบริการ

นอกจากนี้ Managed Database บางประเภทอาจมีข้อจำกัดด้าน Database Size หรือ IOPS หาก Workload มีขนาดใหญ่เกินข้อจำกัดของบริการ อาจจำเป็นต้องใช้ Cloud-Native Database หรือ Self-Managed Infrastructure แทน

การใช้ Cloud ยังต้องคำนึงถึง Cost, Region, Network, Security และ Compliance รวมถึงความเหมาะสมของ Service ที่ Provider จัดเตรียมไว้ จึงไม่ควรพิจารณาเพียงความสะดวกในการ Deployment อย่างเดียว

ตารางที่ 2.30 ข้อดีและข้อจำกัดของ NoSQL และ Cloud Database

ด้าน	ข้อดี	ข้อจำกัด
Data Structure	NoSQL รองรับโครงสร้างที่ยืดหยุ่น	ต้องออกแบบ Data Model ให้เหมาะกับรูปแบบการใช้งาน
Scalability	รองรับ Horizontal และ Elastic Scaling	การ Scale ต้องมีการวาง Architecture ที่เหมาะสม
Infrastructure	Cloud ลดภาระการจัดการ Hardware	ขึ้นอยู่กับขอบเขตบริการของ Provider
Availability	รองรับ Replication และ Multi-zone	ต้องกำหนด Configuration สำหรับ HA ให้เหมาะสม
Maintenance	Managed Service ช่วยดูแล Backup/Update	การควบคุมบางส่วนลดลง
Security	มี IAM, Network Control และ Encryption	ผู้ใช้ยังต้องรับผิดชอบ Data และ Access Security
Cost	จ่ายตาม Resource ที่ใช้งานได้	Resource ที่ Scale มากเกินไปสามารถเพิ่มค่าใช้จ่าย

ที่มา: สังเคราะห์จาก ชโลธร ชูทอง และคณะ (ม.ป.ป.) และ Neu et al. (2019)

สรุปแนวคิด NoSQL and Cloud Database

NoSQL Database เป็นแนวทางการจัดเก็บข้อมูลแบบ Non-Relational ที่มีความยืดหยุ่นด้าน Data Structure และสามารถรองรับข้อมูลได้หลายรูปแบบ ฐานข้อมูล NoSQL สามารถแบ่งออกเป็น Key-Value Store, Document Database, Graph Database และ Wide Column Store โดยแต่ละประเภทเหมาะกับลักษณะข้อมูลและรูปแบบการใช้งานที่แตกต่างกัน

Document Database เป็นรูปแบบหนึ่งที่จัดเก็บข้อมูลเป็น Document ซึ่งสามารถมีโครงสร้าง JSON หรือ Key-Value Pair และถูกจัดกลุ่มภายใน Collection ทำให้สามารถรองรับ

Semi-structured Data และปรับเปลี่ยน Data Model ได้ยืดหยุ่นกว่าการกำหนด Table และ Column แบบตายตัว

Cloud Database เป็นการนำระบบฐานข้อมูลไปทำงานบน Cloud Infrastructure ซึ่งสามารถรองรับได้ทั้ง Relational และ Nonrelational Database และสามารถให้บริการในรูปแบบ Self-Managed หรือ Managed Database

การทำงานบน Cloud ช่วยเพิ่มความยืดหยุ่นด้าน Resource ผ่าน Elastic Scaling และช่วยรองรับ High Availability ด้วย Region, Availability Zone, Replication และ Failover ขณะที่ Managed Database สามารถลดงานด้าน Routine Maintenance เช่น Installation, Update และ Backup ให้ผู้ให้บริการเป็นผู้ดำเนินการบางส่วน

ด้าน Security ของ Cloud Database ใช้แนวคิด Shared Security Model ซึ่งแบ่งความรับผิดชอบระหว่าง Cloud Provider และผู้ใช้บริการ โดย Provider ดูแล Infrastructure Security ขณะที่ผู้ใช้งานยังต้องจัดการ User, Permission, Authentication, Network Access และการปกป้องข้อมูลของตนเอง

การใช้ NoSQL ร่วมกับ Cloud จึงเป็นแนวทางที่สามารถรองรับระบบที่มีข้อมูลยืดหยุ่นและต้องการความสามารถด้าน Scalability และ Availability อย่างไรก็ตาม การออกแบบจำเป็นต้องพิจารณาทั้ง Data Model, Performance, Security, Cost และข้อจำกัดของ Cloud Service เพื่อให้ฐานข้อมูลมีความเหมาะสมกับลักษณะการใช้งาน

2.2 เทคโนโลยีและเครื่องมือที่เกี่ยวข้อง

ในการพัฒนาระบบออกแบบทริปการท่องเที่ยวด้วย AI จำเป็นต้องใช้เทคโนโลยีและเครื่องมือหลายประเภท เพื่อสนับสนุนการทำงานของระบบให้สามารถประมวลผลข้อมูลและให้บริการแก่ผู้ใช้งานได้อย่างมีประสิทธิภาพ โดยเทคโนโลยีที่นำมาใช้ในการพัฒนาระบบประกอบด้วย เทคโนโลยีการพัฒนาโมบายแอปพลิเคชัน เครื่องมือสำหรับพัฒนาและเชื่อมต่อระบบฐานข้อมูล รวมถึงบริการแผนที่ดิจิทัลและเทคโนโลยีด้านปัญญาประดิษฐ์ที่ช่วยในการวิเคราะห์ข้อมูลและแนะนำสถานที่ท่องเที่ยวให้เหมาะสมกับผู้ใช้งานแต่ละบุคคล ซึ่งเทคโนโลยีเหล่านี้มีบทบาทสำคัญในการสนับสนุนการทำงานของระบบให้สามารถวางแผนการเดินทางและนำเสนอข้อมูลการท่องเที่ยวได้อย่างสะดวกและมีประสิทธิภาพ

2.2.1 Flutter and Dart

Flutter เป็นเครื่องมือสำหรับพัฒนา Application ที่พัฒนาโดย Google โดยมีจุดเด่นด้านการสร้างส่วนติดต่อผู้ใช้งานหรือ User Interface (UI) และสนับสนุนการพัฒนา Application แบบ

Cross-Platform ทำให้สามารถพัฒนา Application จาก Source Code ร่วมกันและนำไปใช้งานบน Platform ที่แตกต่างกันได้ เอกสารประกอบการสอนระบุว่า Flutter สามารถใช้พัฒนา Mobile Application สำหรับทั้ง iOS และ Android และเป็น Open Source ที่สามารถนำมาใช้งานได้โดยไม่มีค่าใช้จ่าย

การพัฒนาแบบ Cross-Platform ช่วยลดความจำเป็นในการสร้าง Source Code แยกสำหรับแต่ละระบบปฏิบัติการ โดยเอกสารอธิบายว่า Cross-Platform Application สามารถทำงานบนระบบปฏิบัติการของคอมพิวเตอร์หลายประเภท เช่น Windows, macOS และ Linux รวมถึงระบบปฏิบัติการบน Smartphone เช่น iOS และ Android และสามารถนำแนวทางดังกล่าวไปใช้กับ Web Browser ได้ด้วย

Flutter ใช้ภาษา **Dart** เป็น Programming Language หลักในการพัฒนา Application โดย Dart ทำหน้าที่สำหรับกำหนด Logic การทำงาน การจัดการข้อมูล และการสร้าง User Interface ผ่าน Widget ต่าง ๆ การทำงานร่วมกันระหว่าง Flutter และ Dart ทำให้นักพัฒนาสามารถสร้างทั้งโครงสร้างหน้าจอและกระบวนการทำงานของ Application ภายใน Development Environment เดียวกัน

หนังสือ *สูตรลัด Flutter* ของอนุชิต ชโลธร ระบุว่าเนื้อหาการพัฒนา Flutter สามารถเรียนรู้ผ่านตัวอย่าง Source Code โดยครอบคลุมตั้งแต่การออกแบบหน้าจอ การจัด Layout และการตกแต่ง Widget ไปจนถึงการเชื่อมต่อ REST API การใช้งานแผนที่ การอ่านตำแหน่ง GPS การสร้าง Responsive Screen และการเชื่อมต่อกับ Gemini นอกจากนี้ผู้เขียนยังอธิบายว่าตัวอย่าง Code สามารถใช้เพื่อเรียนรู้แนวคิดการออกแบบหน้าจอและนำไปประยุกต์กับ Application รูปแบบอื่นได้

Widget ใน Flutter

แนวคิดสำคัญของ Flutter คือการสร้าง User Interface จาก **Widget** โดยส่วนประกอบที่แสดงบนหน้าจอสามารถนำ Widget หลายประเภทมาประกอบกันเพื่อสร้าง Interface ตามที่ต้องการ ตัวอย่างเช่น Scaffold, AppBar, Text, Container, Column, Row, NavigationBar และ FloatingActionButton

Scaffold เป็น Widget ที่ใช้สร้างโครงสร้างพื้นฐานของหน้าจอตามแนวทาง Material Design และสามารถกำหนดองค์ประกอบอื่น เช่น AppBar, Drawer, Navigation Bar และ Floating Action Button ได้

ส่วน AppBar ใช้สำหรับแสดงส่วนหัวของหน้าจอ เช่น ชื่อหน้าและปุ่ม Action ต่าง ๆ โดยทำงานอยู่ภายใต้ Scaffold

ตัวอย่างการพัฒนา Application ด้วย Flutter และ Dart

ตัวอย่างต่อไปนี้แสดงการใช้ภาษา Dart ร่วมกับ Flutter เพื่อสร้างหน้าจออย่างง่าย ประกอบด้วย AppBar, ข้อความ และปุ่มสำหรับเปลี่ยนข้อความเมื่อผู้ใช้งานกดปุ่ม

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: HomePage(),
    );
  }
}

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  String message = 'Welcome to Flutter';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Flutter and Dart'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              message,
              style: const TextStyle(fontSize: 24),
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
```

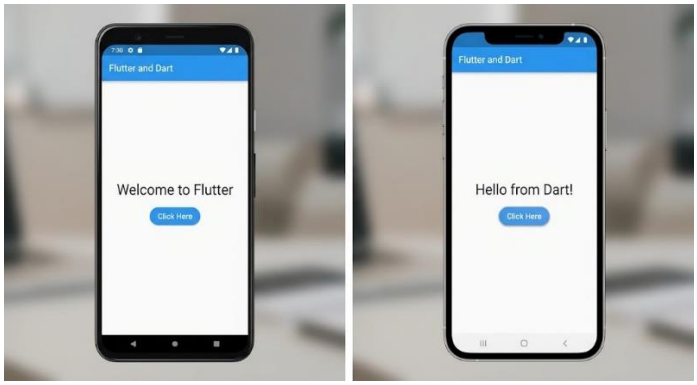
```
            setState(() {
              message = 'Hello from Dart!';
            });
          },
          child: const Text('Click Here'),
        ),
      ],
    ),
  ),
);
  }
}
```

คำอธิบายตัวอย่างโค้ด

ในตัวอย่างดังกล่าว MaterialApp ทำหน้าที่เป็นส่วนหลักของ Application ขณะที่ Scaffold ใช้กำหนดโครงสร้างของหน้าจอ และ AppBar ใช้แสดงชื่อหน้าจอ ส่วน Column ใช้จัดเรียง Widget ในแนวตั้ง ซึ่งสอดคล้องกับเอกสารที่อธิบายว่า Widget ภายใน Column จะถูกจัดเรียงตามแนวตั้งและสามารถกำหนดตำแหน่งด้วย MainAxisAlignment ได้

ElevatedButton เป็นปุ่มที่รับ Event ผ่าน onPressed เมื่อผู้ใช้งานกดปุ่มจะเรียก setState() เพื่อเปลี่ยนค่าของตัวแปร message จาก "Welcome to Flutter" เป็น "Hello from Dart!" และ Flutter จะ Update User Interface เพื่อแสดงข้อความใหม่

ผลลัพธ์



ภาพที่ 2.22 ผลลัพธ์การพัฒนา User Interface ด้วย Flutter และ Dart

การจัดวาง User Interface

Flutter มี Widget สำหรับการจัด Layout หลายประเภท เช่น Center, Align, Row, Column และ Stack ซึ่งแต่ละ Widget มีหน้าที่แตกต่างกัน ตัวอย่างเช่น Center ใช้จัด Widget ให้อยู่กึ่งกลาง ส่วน Align สามารถกำหนดตำแหน่งได้หลายรูปแบบ เช่น กึ่งกลาง ซ้ายบน ขวาบน ซ้ายล่าง และขวาล่าง

Stack สามารถใช้จัด Widget ในลักษณะซ้อนกันเป็นชั้น และสามารถใช้ร่วมกับ Align เพื่อกำหนดตำแหน่งของ Widget แต่ละ Layer ได้อย่างอิสระ

สำหรับการจัดเรียงองค์ประกอบในแนวตั้งสามารถใช้ Column และหากต้องการจัดเรียงในแนวนอนสามารถใช้ Row แนวคิดดังกล่าวทำให้สามารถประกอบ Widget ขนาดเล็กขึ้นมาเป็นหน้าจอที่มีความซับซ้อนได้

การนำ Flutter และ Dart มาใช้ในการพัฒนาระบบ

Flutter และ Dart สามารถใช้เป็นเทคโนโลยีหลักสำหรับพัฒนาส่วน Frontend ของ Application ตั้งแต่การสร้างหน้าจอ การรับข้อมูลจากผู้ใช้งาน การจัดการ Navigation การแสดงข้อมูล ไปจนถึงการติดต่อกับ External Service ผ่าน API

นอกจากนี้ หนังสือ *สูตรลัด Flutter* ยังแสดงขอบเขตการใช้งาน Flutter ที่ครอบคลุมการเชื่อมต่อ REST API, Permission, Database, Map, GPS, Responsive Interface และ Gemini ซึ่งสะท้อนว่า Flutter สามารถนำ Package และ Service ภายนอกมาทำงานร่วมกับ Application ได้หลายประเภท

ด้วยคุณลักษณะของ Cross-Platform Development และระบบ Widget ทำให้ Flutter สามารถใช้สร้าง User Interface ที่มีโครงสร้างเป็น Component และนำ Component กลับมาใช้ซ้ำได้ ขณะที่ภาษา Dart ทำหน้าที่ควบคุม Logic และพฤติกรรมของ Application จึงสามารถใช้งานทั้งสองเทคโนโลยีร่วมกันในกระบวนการพัฒนา Application ได้อย่างเป็นระบบ

การสร้าง Navigation Bar ด้วย Flutter

การพัฒนา Application ที่มีหลายหน้าหรือหลายส่วนจำเป็นต้องมีองค์ประกอบสำหรับช่วยให้ผู้ใช้งานสามารถเปลี่ยนไปยังส่วนต่าง ๆ ของระบบได้ Flutter มี Widget สำหรับสร้างเมนูบริเวณด้านล่างของหน้าจอ เช่น BottomNavigationBar และ NavigationBar

อนุชิต ชโลธร อธิบายว่า Flutter สามารถสร้างเมนูบริเวณด้านล่างหน้าจอด้วย BottomNavigationBar หรือ NavigationBar โดยหากต้องการรูปแบบที่สอดคล้องกับ Material Design 3 สามารถใช้ NavigationBar ได้

ตัวอย่างต่อไปนี้แสดงการสร้าง Navigation Bar จำนวน 3 เมนู ได้แก่ Home, Trips และ Profile โดยใช้ภาษา Dart

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MainPage(),
    );
  }
}

class MainPage extends StatefulWidget {
  const MainPage({super.key});

  @override
  State<MainPage> createState() => _MainPageState();
}

class _MainPageState extends State<MainPage> {
  int currentPage = 0;

  final List<Widget> pages = const [
    Center(
      child: Text(
        'Home Page',
        style: TextStyle(fontSize: 28),
      ),
    ),
    Center(
      child: Text(
        'My Trips',
        style: TextStyle(fontSize: 28),
      ),
    ),
    Center(
      child: Text(
        'Profile',
        style: TextStyle(fontSize: 28),
      ),
    ),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Flutter Navigation'),
      ),
```

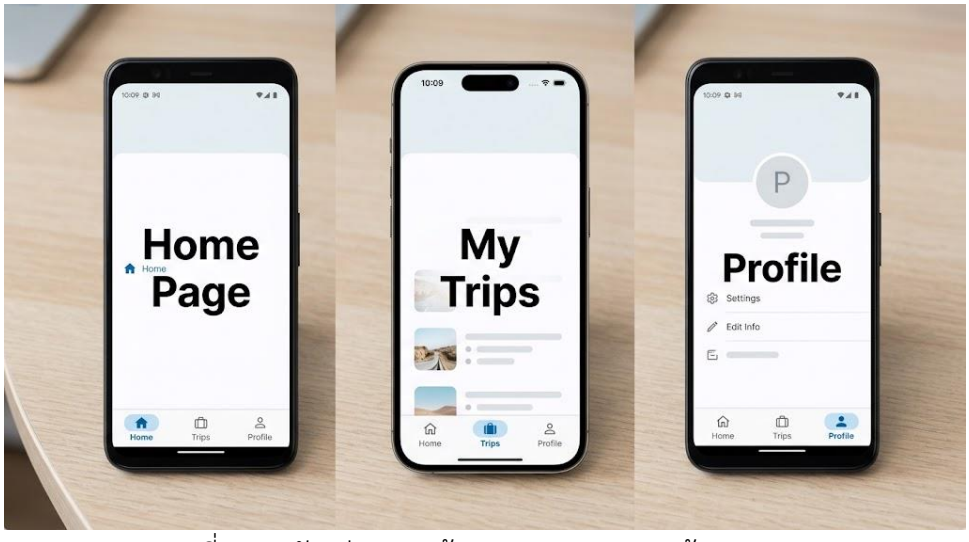
```
      body: pages[currentPage],

      bottomNavigationBar: NavigationBar(
        selectedIndex: currentPage,
        onDestinationSelected: (index) {
          setState(() {
            currentPage = index;
          });
        },
        destinations: const [
          NavigationDestination(
            icon: Icon(Icons.home),
            label: 'Home',
          ),
          NavigationDestination(
            icon: Icon(Icons.map),
            label: 'Trips',
          ),
          NavigationDestination(
            icon: Icon(Icons.person),
            label: 'Profile',
          ),
        ],
      ),
    );
  }
}
```

ในตัวอย่างดังกล่าว ตัวแปร currentPage ใช้สำหรับเก็บหมายเลขหน้าที่กำลังถูกเลือก เมื่อผู้ใช้งานกดเมนู NavigationDestination ค่า index จะถูกส่งเข้าสู่ onDestinationSelected() จากนั้น setState() จะเปลี่ยนค่าของ currentPage และทำให้ Flutter แสดง Widget ใน List pages ที่ตรงกับเมนูที่ผู้ใช้งานเลือก

เอกสารตัวอย่าง Flutter แสดงการกำหนด NavigationBar ผ่าน Property bottomNavigationBar ของ Scaffold และใช้ NavigationDestination สำหรับกำหนด Icon และ Label ของแต่ละเมนู

ผลลัพธ์



ภาพที่ 2.23 ตัวอย่างการสร้าง Navigation Bar ด้วย Flutter

การสร้าง Drawer Menu

นอกจาก Navigation Bar แล้ว Flutter ยังมี Drawer สำหรับสร้างเมนูที่เลื่อนออกมาจากด้านข้างของหน้าจอ ซึ่งสามารถนำมาใช้สำหรับแสดงเมนูที่มีจำนวนมากกว่าพื้นที่ของ Bottom Navigation

เอกสาร *สูตรลัด Flutter* อธิบายว่า Scaffold มี Property สำหรับกำหนด Drawer และสามารถสร้างรายการเมนูภายในผ่าน Drawer Widget โดย Flutter สามารถเพิ่มปุ่มสำหรับเปิด Drawer ให้โดยอัตโนมัติเมื่อกำหนด Drawer ไว้ใน Scaffold

ตัวอย่างการสร้าง Drawer สามารถเขียนได้ดังนี้

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: HomePage(),
    );
  }
```

```
}

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Menu Example'),
      ),

      drawer: Drawer(
        child: ListView(
          padding: EdgeInsets.zero,
          children: [
            const DrawerHeader(
              child: Center(
                child: Text(
                  'Application Menu',
                  style: TextStyle(fontSize: 22),
                ),
              ),
            ),

            ListTile(
              leading: const Icon(Icons.home),
              title: const Text('Home'),
              onTap: () {
                Navigator.pop(context);
              },
            ),

            ListTile(
              leading: const Icon(Icons.map),
              title: const Text('My Trips'),
              onTap: () {
                Navigator.pop(context);
              },
            ),

            ListTile(
              leading: const Icon(Icons.person),
              title: const Text('Profile'),
              onTap: () {
                Navigator.pop(context);
              },
```

```
          ),

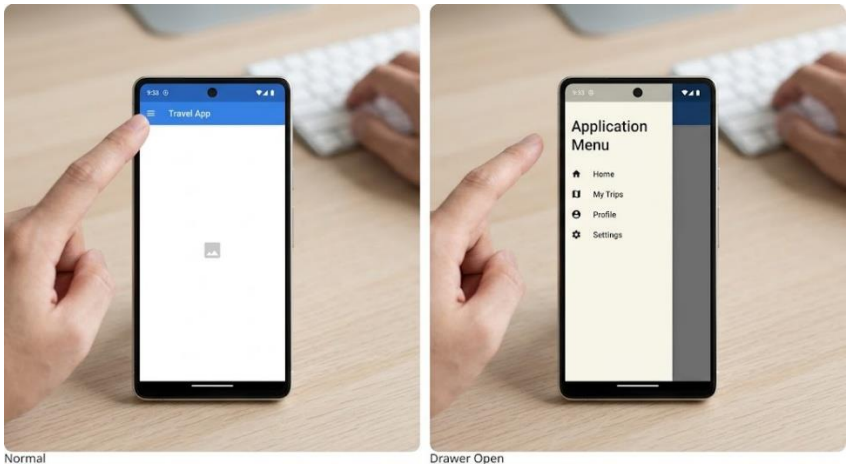
          ListTile(
            leading: const Icon(Icons.settings),
            title: const Text('Settings'),
            onTap: () {
              Navigator.pop(context);
            },
          ),
        ],
      ),
    ),

    body: const Center(
      child: Text(
        'Home Page',
        style: TextStyle(fontSize: 28),
      ),
    ),
  );
 }
}
```

DrawerHeader ใช้สร้างส่วนหัวของเมนู ขณะที่ ListTile ใช้สร้างรายการเมนูแต่ละรายการ โดยสามารถกำหนด leading สำหรับ Icon และ title สำหรับข้อความของเมนูได้

ตัวอย่างในเอกสารใช้โครงสร้าง Drawer ร่วมกับ ListView และ ListTile สำหรับเมนู Home, Notification, Profile และ Settings ซึ่งแสดงให้เห็นว่า Widget หลายชนิดสามารถนำมาประกอบกันเพื่อสร้าง Navigation Menu ได้

ผลลัพธ์



ภาพที่ 2.24 ตัวอย่างการสร้าง Drawer Menu ด้วย Flutter

การจัดวาง User Interface ด้วย Column และ Row

Flutter ใช้ Widget สำหรับควบคุม Layout โดย Column ใช้จัด Widget ในแนวตั้ง ส่วน Row ใช้จัด Widget ในแนวนอน การนำ Widget ทั้งสองชนิดมาทำงานร่วมกันทำให้สามารถสร้าง Interface ที่มีองค์ประกอบหลายระดับได้

เอกสารอธิบายว่า Widget ที่อยู่ภายใน Column จะถูกเรียงตัวในแนวตั้ง และสามารถควบคุมตำแหน่งหรือการกระจาย Widget ผ่านการกำหนดค่าต่าง ๆ ของ Column ได้

ตัวอย่างต่อไปเป็นการสร้าง Card อย่างง่ายสำหรับแสดงข้อมูลสถานที่

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: PlacePage(),
    );
  }
}

class PlacePage extends StatelessWidget {
  const PlacePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Travel Place'),
      ),

      body: Center(
        child: Container(
          width: 350,
          padding: const EdgeInsets.all(20),
          margin: const EdgeInsets.all(20),

          decoration: BoxDecoration(
            border: Border.all(),
            borderRadius: BorderRadius.circular(15),
```

```
        ),

        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text(
              'Wat Arun',
              style: TextStyle(
                fontSize: 26,
                fontWeight: FontWeight.bold,
              ),
            ),

            const SizedBox(height: 10),

            const Text(
              'Bangkok, Thailand',
              style: TextStyle(fontSize: 18),
            ),

            const SizedBox(height: 20),

            Row(
              children: const [
                Icon(Icons.location_on),
                SizedBox(width: 8),
                Text('View location'),
              ],
            ),

            const SizedBox(height: 15),

            Row(
              children: const [
                Icon(Icons.schedule),
                SizedBox(width: 8),
                Text('Recommended visit: 09:00'),
              ],
            ),
          ],
        ),
      ),
    ),
  );
 }
}
```

ในตัวอย่างนี้ Column ทำหน้าที่จัดข้อมูลชื่อสถานที่ ที่ตั้ง และรายละเอียดอื่นในแนวตั้ง ขณะที่ Row ใช้จัด Icon และข้อความให้อยู่ในแนวนอน

Property mainAxisSize: MainAxisSize.min ทำให้ Column ใช้พื้นที่ตามขนาดของ Widget ที่อยู่ภายใน ซึ่งสอดคล้องกับตัวอย่างในเอกสารที่ใช้ MainAxisSize.min เพื่อกำหนดให้ขนาดของ Column สัมพันธ์กับ Content ภายใน

ผลลัพธ์



ภาพที่ 2.25 ตัวอย่างการจัดวางข้อมูลด้วย Column และ Row ใน Flutter

การจัดวาง Widget แบบซ้อนด้วย Stack

ในกรณีที่ต้องการให้ Widget หลายรายการซ้อนทับกัน สามารถใช้ Stack ซึ่งเหมาะกับ Interface เช่น การวางข้อความบนรูปภาพ การวาง Badge หรือ Icon บน Card และการวางปุ่มในตำแหน่งเฉพาะ

อนุชิต ชโลธรอธิบายว่า Stack ใช้แสดง Widget แบบซ้อนกันเป็นชั้นหรือ Layer และสามารถใช้ร่วมกับ Align เพื่อกำหนดตำแหน่งของ Widget แต่ละชั้นได้อย่างอิสระ

ตัวอย่าง:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(
    debugShowCheckedModeBanner: false,
    home: StackExample(),
  ));
}

class StackExample extends StatelessWidget {
  const StackExample({super.key});
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Stack Example'),
    ),

    body: Center(
      child: Stack(
        children: [
          Container(
            width: 350,
            height: 220,
            color: Colors.grey,
          ),

          const Positioned(
            left: 20,
            bottom: 50,
            child: Text(
              'Bangkok',
              style: TextStyle(
                fontSize: 30,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),

          const Positioned(
            left: 20,
            bottom: 20,
            child: Text(
              'Explore your next destination',
              style: TextStyle(fontSize: 16),
            ),
          ),

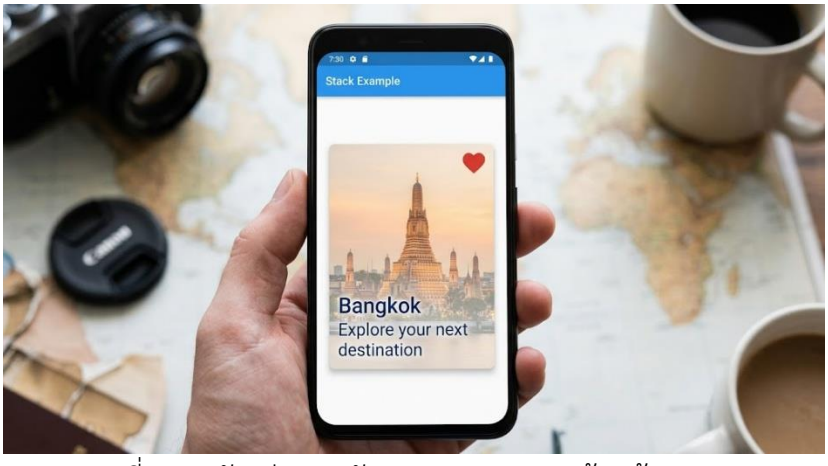
          const Positioned(
            right: 15,
            top: 15,
            child: Icon(
              Icons.favorite_border,
              size: 32,
            ),
          ),
        ],
```

```
        ),
      ),
    );
  }
}
```

ในตัวอย่าง Stack เป็น Container หลักสำหรับวาง Widget แบบซ้อนกัน ขณะที่ Positioned ใช้กำหนดตำแหน่งของ Text และ Icon ภายในพื้นที่ Stack

เอกสารแสดงตัวอย่างการใช้ Stack ร่วมกับ Align เพื่อวาง Container ในตำแหน่งซ้ายบน กึ่งกลาง และขวาล่าง ซึ่งเป็นหลักการเดียวกับการสร้าง Interface แบบ Layer

ผลลัพธ์



ภาพที่ 2.26 ตัวอย่างการจัดวาง Widget แบบซ้อนด้วย Stack

สรุปการใช้ Flutter และ Dart

จากตัวอย่างข้างต้น Flutter สามารถใช้ Widget หลายประเภทประกอบกันเพื่อสร้าง User Interface ตั้งแต่โครงสร้างพื้นฐานของหน้าจอด้วย Scaffold การสร้างส่วนหัวด้วย AppBar การจัดเมนูด้วย NavigationBar และ Drawer ไปจนถึงการจัด Layout ด้วย Column, Row, Align และ Stack

ภาษา Dart ทำหน้าที่กำหนดโครงสร้างและ Logic ของ Application เช่น การกำหนด Variable การเรียก Function การตอบสนองต่อ Event และการเปลี่ยน State ของ Widget ขณะที่ Flutter ทำหน้าที่นำโครงสร้างเหล่านั้นมาแสดงเป็น User Interface (Watchara Kokram,2021) (อนุชิต ชโลธร, ม.ป.ป.)

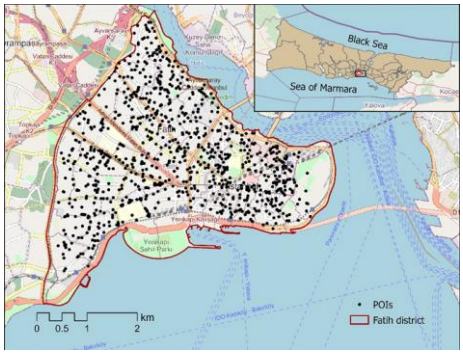
2.2.2 OpenStreetMap and Geocoding

OpenStreetMap หรือ OSM เป็นแพลตฟอร์มข้อมูลภูมิศาสตร์แบบเปิดที่เปิดโอกาสให้ผู้ใช้งานมีส่วนร่วมในการสร้างและปรับปรุงข้อมูลเชิงพื้นที่ เช่น ถนน อาคาร สถานที่สำคัญ และจุดที่น่าสนใจต่าง ๆ ข้อมูลของ OpenStreetMap จึงเกิดจากการมีส่วนร่วมของผู้ใช้งานจำนวนมากและสามารถนำไปประยุกต์ใช้ในระบบสารสนเทศภูมิศาสตร์และแอปพลิเคชันที่เกี่ยวข้องกับตำแหน่งได้

Kilic, Hacar และ Gülgen (2023) อธิบายว่า OpenStreetMap เป็นแพลตฟอร์ม Volunteered Geographic Information หรือ VGI ที่ให้ผู้ใช้งานสามารถสร้าง จัดโครงสร้าง และแบ่งปันข้อมูลทางภูมิศาสตร์ร่วมกัน และแตกต่างจากแพลตฟอร์มเชิงพาณิชย์ตรงที่ทั้งอาสาสมัครและผู้ใช้ทั่วไปสามารถมีส่วนร่วมในการปรับปรุงข้อมูลแผนที่ได้

โครงสร้างข้อมูลของ OpenStreetMap ประกอบด้วยองค์ประกอบสำคัญ ได้แก่ **Node, Way และ Relation** โดย Node เป็นจุดที่มีค่า Latitude และ Longitude, Way ใช้สำหรับสร้างเส้นหรือพื้นที่จากชุดของ Node และ Relation ใช้สำหรับเชื่อมโยง Node และ Way หลายรายการเพื่อแทนวัตถุทางภูมิศาสตร์ที่มีความซับซ้อน นอกจากนี้แต่ละองค์ประกอบยังสามารถกำหนด Tag ในรูปแบบ Key-Value เพื่อระบุชื่อ ประเภท ที่อยู่ หรือรายละเอียดอื่นของสถานที่ได้ (Clemens, 2015)

แนวคิดดังกล่าวทำให้ OpenStreetMap สามารถนำมาใช้เป็นข้อมูลพื้นฐานสำหรับระบบแผนที่ โดย Application สามารถแสดงข้อมูลเชิงตำแหน่ง เช่น ถนน สถานที่ และ Point of Interest หรือ POI ผ่านพิกัดทางภูมิศาสตร์ได้



ภาพที่ 2.27 ตัวอย่างการแสดงข้อมูล Point of Interest บน OpenStreetMap

ที่มา: Kilic, Hacar และ Gülgen (2023)

Geocoding

Geocoding เป็นกระบวนการที่ใช้แปลงข้อมูลในรูปข้อความ เช่น ชื่อสถานที่หรือที่อยู่ ให้กลายเป็นค่าพิกัดทางภูมิศาสตร์ ได้แก่ Latitude และ Longitude

Kilic et al. (2023) ให้ความหมายว่า Geocoding เป็นกระบวนการกำหนดค่าพิกัดบนพื้นผิวโลกให้กับข้อมูลที่มีการอ้างอิงสถานที่ เช่น Postal Address หรือ Place Name ส่วนกระบวนการในทิศทางกลับกันเรียกว่า **Reverse Geocoding** ซึ่งเป็นการนำค่าพิกัดไปค้นหาข้อมูลสถานที่หรือที่อยู่

สามารถอธิบายความสัมพันธ์ได้ดังนี้

Geocoding
ชื่อสถานที่ / ที่อยู่
↓
Geocoding Service
↓
Latitude + Longitude

ส่วน Reverse Geocoding จะเป็น

Reverse Geocoding
Latitude + Longitude
↓
Reverse Geocoding Service
↓
ชื่อสถานที่ / ที่อยู่

Geocoding จึงเหมาะสำหรับระบบที่มีข้อมูลสถานที่เป็นข้อความ แต่ต้องการนำสถานที่ดังกล่าวไปแสดงบนแผนที่ เนื่องจากระบบแผนที่จำเป็นต้องใช้ Coordinate เพื่อกำหนดตำแหน่งที่ต้องการแสดง

Nominatim

Nominatim เป็น Open-source Search and Geocoding Engine ที่ทำงานร่วมกับข้อมูลจาก OpenStreetMap โดยสามารถค้นหาสถานที่จากชื่อหรือที่อยู่และคืนค่า Latitude และ Longitude กลับมาได้

Clemens (2015) ระบุว่า Nominatim ทำงานบนข้อมูล OpenStreetMap เพื่อให้บริการ Geocoding โดยสามารถแปลง Named Location เช่น Full Address หรือ Named Area ให้เป็นค่าพิกัด Latitude และ Longitude และภายในระบบมีขั้นตอนสำหรับ Parse, Assemble และ Index ข้อมูล OSM ก่อนนำไปใช้ในการค้นหา

นอกจากนี้ Nominatim ยังรองรับ Reverse Geocoding โดย Kilic et al. (2023) อธิบายว่าบริการสามารถเข้าถึงผ่าน Web API และสามารถแปลงคู่ Coordinate ให้เป็นข้อมูลข้อความที่เกี่ยวข้องกับตำแหน่ง เช่น Postal Address, Place Name และข้อมูลอื่นที่เกี่ยวข้อง โดยสามารถส่งผลลัพธ์ออกมาได้หลายรูปแบบ เช่น JSON และ XML

Kilic et al. (2023) ยังระบุว่า OpenStreetMap มี Nominatim เป็น Search Engine แบบ Free และ Open-source โดย Geocoding สามารถค้นหาข้อมูลจากคำอธิบาย เช่น ชื่อหรือที่อยู่ ขณะที่ Reverse Geocoding สามารถใช้ Coordinate เพื่อค้นหาข้อมูล Label หรือ Address จากข้อมูล OSM ได้

ดังนั้น Nominatim จึงทำหน้าที่เป็นตัวกลางระหว่างข้อมูลสถานที่ในรูปข้อความและข้อมูลพิกัดที่ระบบแผนที่สามารถนำไปใช้งานได้

การทำงานร่วมกันระหว่าง OpenStreetMap และ Nominatim

OpenStreetMap และ Nominatim มีหน้าที่แตกต่างกันแต่ทำงานร่วมกัน โดย OpenStreetMap เป็นแหล่งข้อมูลทางภูมิศาสตร์ ส่วน Nominatim ทำหน้าที่ค้นหาและประมวลผลข้อมูลดังกล่าวเพื่อคืนข้อมูลตำแหน่ง

สามารถอธิบายกระบวนการได้ดังนี้

ชื่อสถานที่
↓
Nominatim API
↓
ค้นหาข้อมูลจาก OpenStreetMap
↓
Latitude / Longitude
↓
นำพิกัดไปแสดงบนแผนที่

ตัวอย่างเช่น เมื่อ Application มีชื่อสถานที่ "Wat Arun Bangkok" ระบบสามารถส่งคำดังกล่าวไปยัง Nominatim จากนั้น Nominatim จะค้นหาข้อมูลใน OpenStreetMap และคืนผลลัพธ์ที่ประกอบด้วยข้อมูล เช่น Latitude, Longitude, Display Name และรายละเอียดประเภทของสถานที่

แนวทางดังกล่าวช่วยให้ Application ไม่จำเป็นต้องเก็บค่าพิกัดของสถานที่ทุกแห่งไว้ภายในระบบล่วงหน้า แต่สามารถค้นหา Coordinate ตามข้อมูลสถานที่เมื่อมีความต้องการใช้งาน

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<void> searchLocation(String placeName) async {
  final Uri url = Uri.parse(
    'https://nominatim.openstreetmap.org/search'
    '?q=${Uri.encodeComponent(placeName)}'
    '&format=json'
    '&limit=1',
  );

  final response = await http.get(
    url,
    headers: {
      'User-Agent': 'FlutterGeocodingExample/1.0',
    },
  );

  if (response.statusCode == 200) {
    final List<dynamic> data = jsonDecode(response.body);

    if (data.isNotEmpty) {
      final location = data[0];

      print('ชื่อสถานที่: ${location['display_name']}');
      print('Latitude: ${location['lat']}');
      print('Longitude: ${location['lon']}');
    } else {
      print('ไม่พบสถานที่');
    }
  } else {
    print('เกิดข้อผิดพลาดในการเชื่อมต่อ');
  }
}

void main() {
  searchLocation('Wat Arun Bangkok');
}
```

ในตัวอย่างนี้ http.get() ใช้ส่ง Request ไปยัง Nominatim Search Service โดย Parameter q คือคำค้นหาสถานที่ ส่วน format=json กำหนดให้ข้อมูลที่ตอบกลับมาอยู่ในรูป JSON ซึ่งสอดคล้องกับรูปแบบ Output ที่ Nominatim รองรับตามที่ Kilic et al. (2023) อธิบายไว้

เมื่อได้รับข้อมูลแล้ว jsonDecode() จะใช้แปลง JSON Response ให้อยู่ในรูป Object ที่ Dart สามารถประมวลผลได้ จากนั้นจึงอ่านค่า display_name, lat และ lon

ตัวอย่างผลลัพธ์

เมื่อค้นหาสถานที่ ตัว Console จะมีลักษณะประมาณ

```
ชื่อสถานที่: Wat Arun, Bangkok, Thailand
Latitude: <ค่าละติจูดที่ Nominatim ส่งกลับ>
Longitude: <ค่าลองจิจูดที่ Nominatim ส่งกลับ>
```

ตัวอย่างการนำพิกัดไปใช้งานกับ Flutter

หลังจากได้รับ Latitude และ Longitude แล้ว สามารถนำค่าไปใช้กับ Widget แผนที่ภายใน Flutter ได้

ตัวอย่างเชิงแนวคิด เช่น

```dart
double latitude = 13.0;
double longitude = 100.0;

print('ตำแหน่งบนแผนที่');
print('Latitude: $latitude');
print('Longitude: $longitude');
```

ใน Application จริง ขั้นตอนการทำงานสามารถเป็น

```
ค้นหาสถานที่
      ↓
Nominatim API
      ↓
รับ Latitude / Longitude
      ↓
สร้างตำแหน่งบนแผนที่
      ↓
แสดง Marker
```

หากใช้ Package เช่น flutter_map พิกัดที่ได้สามารถนำไปสร้างตำแหน่งสำหรับ Marker บนแผนที่ที่ใช้ข้อมูลจาก OpenStreetMap ได้ ทำให้ผู้ใช้งานสามารถมองเห็นตำแหน่งของสถานที่จากข้อมูลที่ค้นหาได้โดยตรง

ความแม่นยำและข้อจำกัดของ Geocoding

แม้ Geocoding จะช่วยให้สามารถค้นหาตำแหน่งจากชื่อหรือที่อยู่ได้ แต่ผลลัพธ์อาจไม่ถูกต้องในทุกกรณี เนื่องจากความแม่นยำขึ้นอยู่กับคุณภาพของข้อมูลต้นทางและรูปแบบของข้อความที่ใช้ค้นหา

Clemens (2015) พบจากการทดลองว่า Nominatim มีข้อจำกัดเมื่อข้อมูล Address ไม่สมบูรณ์หรือองค์ประกอบของ Address อยู่ในลำดับที่แตกต่างจากรูปแบบที่ระบบคาดหมาย และผลการทดลองชี้ว่าประสิทธิภาพขึ้นอยู่กับรูปแบบของ Query ที่นำเข้า

Kilic et al. (2023) ระบุเช่นกันว่า Nominatim และ Reverse Geocoding อาจให้ผลลัพธ์ที่คลาดเคลื่อนได้ในกรณีที่สถานที่ที่ต้องการไม่มีอยู่ในฐานข้อมูล OSM หรือในพื้นที่เมืองที่มี POI อยู่ใกล้กันจำนวนมาก ระบบอาจคืนสถานที่ที่อยู่ใกล้ที่สุดแทนสถานที่ที่ต้องการจริง

งานวิจัยของ Kilic et al. ยังพบว่าแม้ตำแหน่งที่คืนมาจะอยู่ใกล้พิกัดจริง แต่ความใกล้ทางระยะทางไม่ได้หมายความว่าจะเป็นสถานที่ที่ถูกต้องเสมอไป โดยเฉพาะพื้นที่ที่มีความหนาแน่นของสถานที่สูง

ดังนั้นระบบที่ใช้ Geocoding ควรมีการตรวจสอบผลลัพธ์ เช่น ชื่อสถานที่ เมือง ประเทศ หรือประเภทของสถานที่ ก่อนนำพิกัดไปแสดงต่อผู้ใช้งาน

โครงสร้างข้อมูลของ OpenStreetMap

OpenStreetMap จัดเก็บข้อมูลทางภูมิศาสตร์โดยอาศัยองค์ประกอบพื้นฐานหลายประเภท ได้แก่ Node, Way และ Relation ซึ่งแต่ละองค์ประกอบมีบทบาทแตกต่างกันในการแทนข้อมูลบนพื้นผิวโลก Clemens (2015) อธิบายว่า Node เป็นองค์ประกอบที่มีค่าพิกัด Longitude และ Latitude จึงสามารถใช้แทนตำแหน่งแบบจุดบนพื้นผิวโลกได้ ส่วน Way ประกอบด้วย Node หลายรายการเรียงตามลำดับ และสามารถใช้แทนถนน เส้นทาง หรือขอบเขตของพื้นที่ได้ ขณะที่ Relation สามารถเชื่อมโยง Node, Way หรือ Relation อื่นเข้าด้วยกันเพื่อแทนข้อมูลเชิงพื้นที่ที่มีความซับซ้อนมากขึ้น

องค์ประกอบเหล่านี้ยังสามารถมี **Tag** ในรูปแบบ Key-Value Pair สำหรับใช้ระบุรายละเอียดเพิ่มเติม เช่น ชื่อของสถานที่ ประเภทของสถานที่ ข้อมูล Address ช่วงหมายเลขบ้าน แหล่งที่มาของข้อมูล หรือคุณลักษณะอื่นของวัตถุในโลกจริง

ตัวอย่างเชิงแนวคิดของข้อมูลสามารถแสดงได้ดังนี้

```
Node
 ├──── Latitude
 ├──── Longitude
 └──── Tags
     ├──── name = Place Name
     ├──── amenity = restaurant
     └──── tourism = attraction
```

โครงสร้างในลักษณะดังกล่าวทำให้ OpenStreetMap สามารถแทนข้อมูลได้มากกว่าการแสดงตำแหน่งบนแผนที่เพียงอย่างเดียว เนื่องจากแต่ละตำแหน่งสามารถมีข้อมูลเชิงความหมายหรือ Semantic Information ประกอบอยู่ด้วย

Kilic et al. (2023) ระบุว่า OpenStreetMap ประกอบด้วยทั้ง **Geometric Information** เช่น Node, Way และ Relation รวมถึง **Semantic Information** ที่จัดเก็บอยู่ในรูป Tag ทำให้ข้อมูล OSM สามารถใช้ได้ทั้งในด้านตำแหน่งและการจำแนกประเภทของวัตถุทางภูมิศาสตร์

การมีข้อมูลเชิงความหมายร่วมกับข้อมูลพิกัดมีประโยชน์ต่อกระบวนการ Geocoding เนื่องจากระบบไม่ได้พิจารณาเฉพาะ Coordinate เท่านั้น แต่ยังสามารถใช้ชื่อ Address หรือข้อมูลประเภทของสถานที่เพื่อช่วยระบุว่าผลลัพธ์ที่ค้นพบสัมพันธ์กับคำค้นหาหรือไม่

Point of Interest ใน OpenStreetMap

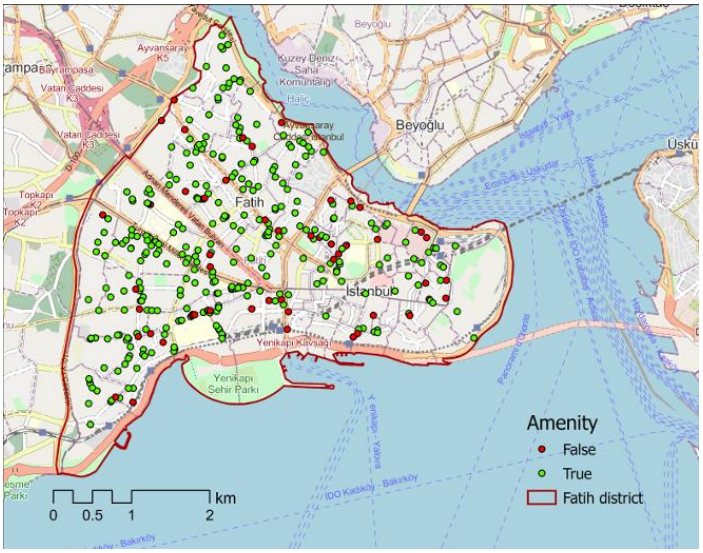
Point of Interest หรือ POI หมายถึงตำแหน่งที่มีความสำคัญหรือมีความสนใจสำหรับผู้ใช้งาน เช่น ร้านค้า โรงพยาบาล สถานศึกษา สถานที่ราชการ สถานที่ทางศาสนา หรือสถานที่ท่องเที่ยว

ในงานของ Kilic et al. (2023) มีการใช้ข้อมูล POI หลายประเภท เช่น Sales, Government Organization, Tourism, Infrastructure, Health, Religious Facility, Cultural, Transportation และ Educational Institution เพื่อเปรียบเทียบกับข้อมูล OpenStreetMap

ข้อมูล POI มีความสำคัญต่อ Application ที่เกี่ยวข้องกับแผนที่ เนื่องจากผู้ใช้งานส่วนใหญ่ไม่ได้ค้นหาด้วย Coordinate โดยตรง แต่ใช้ชื่อสถานที่หรือประเภทของสถานที่ เช่น ร้านอาหาร โรงแรม พิพิธภัณฑ์ หรือสถานที่ท่องเที่ยว แล้วให้ระบบค้นหาตำแหน่งที่สัมพันธ์กับข้อความดังกล่าว

OpenStreetMap ใช้ Tag ในการจำแนกข้อมูลประเภทต่าง ๆ ตัวอย่างจากงานวิจัยพบ Tag เช่น amenity, building, historic, leisure, office, shop และ tourism ซึ่งแสดงถึงประเภทของวัตถุที่ถูกบันทึกไว้ในระบบ

ลักษณะดังกล่าวทำให้ผลจาก Geocoding ไม่ควรถูกประเมินจากระยะทางเพียงอย่างเดียว แต่ควรพิจารณาความหมายหรือประเภทของสถานที่ร่วมด้วย เช่น หากผู้ใช้ค้นหาสถานที่ท่องเที่ยว แต่ระบบคืนผลลัพธ์เป็นอาคารหรือร้านค้าที่มีชื่อคล้ายกัน แม้พิกัดจะอยู่ใกล้กัน ผลลัพธ์ดังกล่าวอาจไม่ตรงกับความต้องการของผู้ใช้



ภาพที่ 2.28 ตัวอย่างการกระจายของข้อมูล Point of Interest บน OpenStreetMap
ที่มา: Kilic, Hacar และ Gülgen (2023)

การประมวลผลข้อมูลของ Nominatim

Nominatim ทำหน้าที่เป็นระบบค้นหาและ Geocoding ที่ทำงานบนข้อมูล OpenStreetMap โดยกระบวนการสำคัญประกอบด้วยการประมวลผลข้อมูล OSM การประกอบข้อมูล Address และการสร้าง Index เพื่อให้สามารถค้นหาสถานที่จากข้อความได้

Clemens (2015) ระบุว่า Address Information ใน OpenStreetMap อาจไม่ได้อยู่ภายใน Entity เดียวทั้งหมด ตัวอย่างเช่น Node อาจมีเพียงหมายเลขบ้าน ในขณะที่ Way ที่อ้างถึง Node อาจมีข้อมูลชื่อถนน และ Relation อีกส่วนหนึ่งอาจระบุ Postal Code ดังนั้นก่อนที่จะนำข้อมูลไปให้บริการ Geocoding ระบบจำเป็นต้องประกอบข้อมูล Address จากองค์ประกอบต่าง ๆ เข้าด้วยกัน

Nominatim จึงมี Pipeline สำหรับ Parse, Assemble และ Index ข้อมูลจาก OpenStreetMap เพื่อจัดเตรียมข้อมูลให้พร้อมสำหรับกระบวนการค้นหา

สามารถสรุปกระบวนการในเชิงแนวคิดได้ว่า

OpenStreetMap Data

↓

Node / Way / Relation

↓

Address + Tags

↓

Nominatim Processing

↓

Index

↓

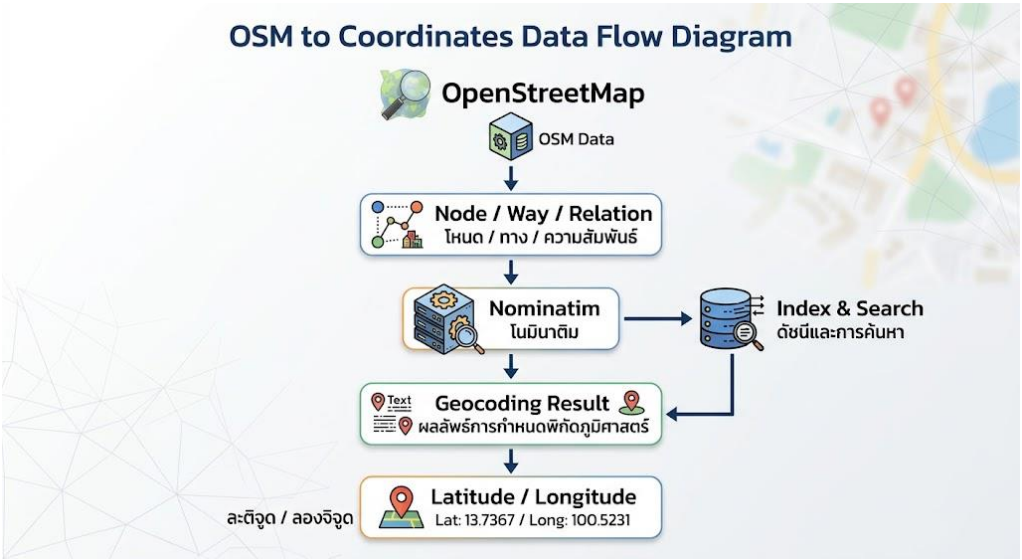
Search / Geocoding

↓

Latitude + Longitude

การสร้าง Index มีความสำคัญต่อ Search Engine เนื่องจากทำให้สามารถค้นหาข้อมูลจากข้อความได้โดยไม่ต้องตรวจสอบข้อมูล OSM ทั้งหมดทุกครั้งที่มี Request เข้ามา

การทำงานดังกล่าวยังสะท้อนว่ากระบวนการ Geocoding ไม่ใช่เพียงการนำข้อความหนึ่งข้อความไปจับคู่กับ Coordinate โดยตรง แต่ต้องผ่านการวิเคราะห์องค์ประกอบของ Address การค้นหาข้อมูลที่เกี่ยวข้อง และการจัดอันดับ Candidate Result ก่อนเลือกผลลัพธ์ที่เหมาะสม



ภาพที่ 2.29 กระบวนการประมวลผลข้อมูล OpenStreetMap ด้วย Nominatim

รูปแบบข้อมูลผลลัพธ์จาก Nominatim

Nominatim สามารถให้ผลลัพธ์ของกระบวนการ Geocoding หรือ Reverse Geocoding ในรูปแบบที่ Application สามารถนำไปประมวลผลต่อได้

Kilic et al. (2023) ระบุว่า Nominatim รองรับ Output หลาย Format เช่น **JavaScript Object Notation (JSON)** และ **Extensible Markup Language (XML)**

ในงานวิจัยดังกล่าว Nominatim ถูกใช้เพื่อดึงข้อมูลหลายประเภท เช่น Postal Address, Coordinate Pair, OSM ID, OSM Type, Place ID และ OSM Category รวมทั้งข้อมูลในรูปแบบ GeoJSON ก่อนนำข้อมูลไปใช้ใน GIS Database

การรองรับ JSON มีประโยชน์ต่อ Web และ Mobile Application เนื่องจากข้อมูลสามารถถูกส่งผ่าน HTTP และนำไปแปลงเป็น Data Object สำหรับใช้งานภายใน Application ได้

ตัวอย่างเชิงโครงสร้างสามารถแสดงได้ว่า

ผลลัพธ์จาก Nominatim
- Place Information
- Address
- Latitude
- Longitude
- OSM ID
- OSM Type
- Category

ข้อมูลดังกล่าวสามารถนำไปใช้ได้หลายรูปแบบ เช่น ใช้ Latitude และ Longitude เพื่อกำหนด Marker ใช้ชื่อสถานที่แสดงบน User Interface หรือใช้ Category เพื่อช่วยตรวจสอบประเภทของสถานที่ก่อนนำไปใช้

การตรวจสอบผลลัพธ์ของ Geocoding

กระบวนการ Geocoding อาจคืน Candidate Result มากกว่าหนึ่งรายการ โดยเฉพาะชื่อสถานที่ที่ซ้ำกันหรือสถานที่ที่มีชื่อคล้ายกัน ดังนั้นการเลือกผลลัพธ์ควรพิจารณาข้อมูลประกอบมากกว่าการเลือกจากชื่อเพียงอย่างเดียว

ข้อมูลที่สามารถใช้ประกอบการพิจารณา ได้แก่ ชื่อสถานที่ Address, Coordinate และประเภทของสถานที่ เนื่องจากองค์ประกอบเหล่านี้ช่วยลดโอกาสในการเลือกสถานที่ที่มีชื่อเหมือนกันแต่ตั้งอยู่คนละพื้นที่

จากงานของ Kilic et al. (2023) พบว่าการประเมินคุณภาพของผลลัพธ์สามารถพิจารณาได้ทั้งด้าน **Positional Accuracy** และ Semantic Information เช่น Address และ Tag

Positional Accuracy สามารถประเมินจากระยะทางระหว่างตำแหน่ง Reference กับตำแหน่งที่ได้จาก Geocoding โดยระยะทางที่น้อยแสดงว่าตำแหน่งมีความใกล้เคียงกันมากขึ้น อย่างไรก็ตาม ผู้วิจัยพบว่าตำแหน่งที่อยู่ใกล้กันไม่ได้หมายความว่าผลลัพธ์จะเป็นสถานที่เดียวกันเสมอไป

ในพื้นที่เมืองที่มี Point of Interest จำนวนมากอยู่ใกล้กัน Candidate ที่อยู่ใกล้พิกัดมากที่สุดอาจเป็นร้านค้า อาคาร หรือสถานที่อีกแห่งหนึ่งที่ไม่ตรงกับตำแหน่งที่ต้องการ

ดังนั้นการตรวจสอบผล Geocoding จึงควรพิจารณาทั้ง **Geometric Information** และ **Semantic Information** ร่วมกัน

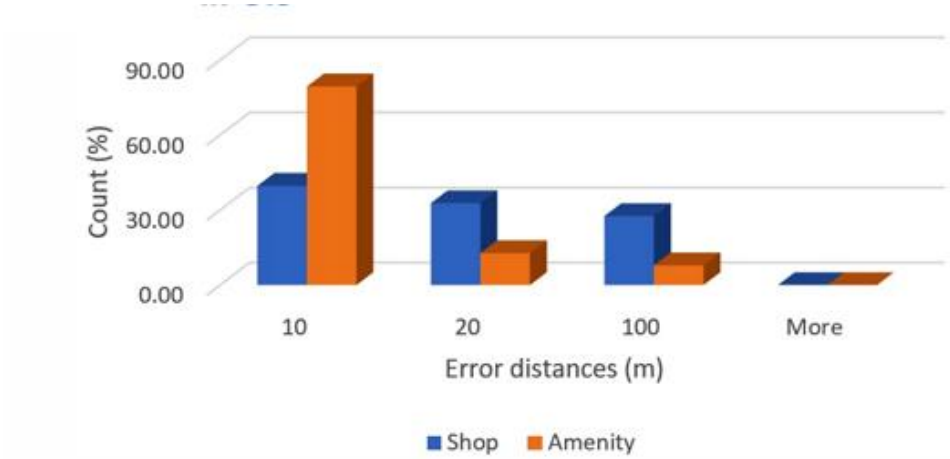
ความสัมพันธ์ระหว่างระยะทางกับความถูกต้องของผลลัพธ์

ระยะทางเป็นหนึ่งในเกณฑ์พื้นฐานที่สามารถใช้ตรวจสอบความถูกต้องของตำแหน่งได้ แต่ไม่ควรใช้เป็นเกณฑ์เพียงอย่างเดียว

งานของ Kilic et al. (2023) ใช้ Euclidean Distance ระหว่าง Reference Address กับ Address ที่ได้จากบริการ OSM เพื่อประเมิน Positional Accuracy โดยระยะทางดังกล่าวแสดงถึงความใกล้เคียงระหว่าง Geocoded Object และตำแหน่งจริง

ผลการทดลองพบว่าวัตถุประเภท amenity ประมาณ 80% มี Positional Error ต่ำกว่า 10 เมตร ขณะที่ผลการจับคู่ทั้งหมดในกลุ่มที่ศึกษาอยู่ภายในระยะ 100 เมตร อย่างไรก็ตามผู้วิจัยพบว่ามีบางกรณีที่ผล Matching ไม่ถูกต้องแม้ระยะทางจะต่ำกว่า 100 เมตร เนื่องจากพื้นที่ศึกษามีความหนาแน่นของ POI สูง

ผลดังกล่าวแสดงให้เห็นว่า **Distance ต่ำไม่ได้รับประกัน Semantic Correctness** กล่าวคือ จุดที่อยู่ใกล้กันทางภูมิศาสตร์อาจเป็นคนละสถานที่กัน



ภาพที่ 2.30 ความคลาดเคลื่อนด้านตำแหน่งของผล Reverse Geocoding จาก OpenStreetMap
ที่มา: Kilic, Hacar และ Gülgen (2023)

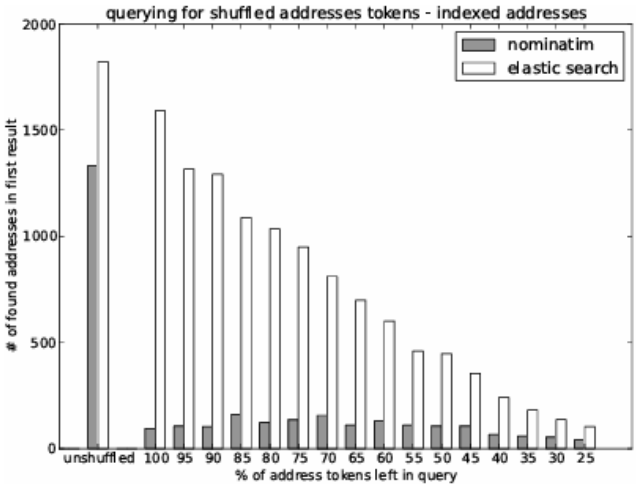
ผลกระทบของความสมบูรณ์ของข้อมูลต่อ Geocoding

คุณภาพของข้อมูล Input มีผลต่อประสิทธิภาพของ Geocoding โดยตรง หาก Address หรือชื่อสถานที่มีข้อมูลไม่ครบถ้วน การค้นหา Candidate ที่ถูกต้องจะทำได้ยากขึ้น

Clemens (2015) ทดลองกับ Address ที่มีข้อมูลครบและ Address ที่ถูกตัดองค์ประกอบบางส่วนออก โดยผลการทดลองแสดงให้เห็นว่า Nominatim มีความไวต่อรูปแบบ Address และมีประสิทธิภาพลดลงอย่างชัดเจนเมื่อ Token ของ Address ถูกสลับหรือลดจำนวนลง

ในการทดลองกับ Address ของร้านขายยาในประเทศเยอรมนีจำนวน 1,000 รายการ Nominatim สามารถค้นหาผลลัพธ์ Full Address ภายในระยะ 100 เมตรได้ 946 รายการ และไม่พบผลลัพธ์จำนวน 54 รายการ เมื่อเหลือเพียง 75% ของ Address Token จำนวนผลที่อยู่ภายใน 100 เมตรลดเหลือ 399 รายการ

ผลการทดลองดังกล่าวแสดงให้เห็นว่าความสมบูรณ์และรูปแบบของข้อความ Input เป็นปัจจัยสำคัญต่อความสำเร็จของ Geocoding ดังนั้นชื่อสถานที่หรือ Address ที่นำไปค้นหาควรมีข้อมูลประกอบที่เพียงพอ เช่น เมือง จังหวัด หรือประเทศ หากข้อมูลเหล่านั้นสามารถระบุได้



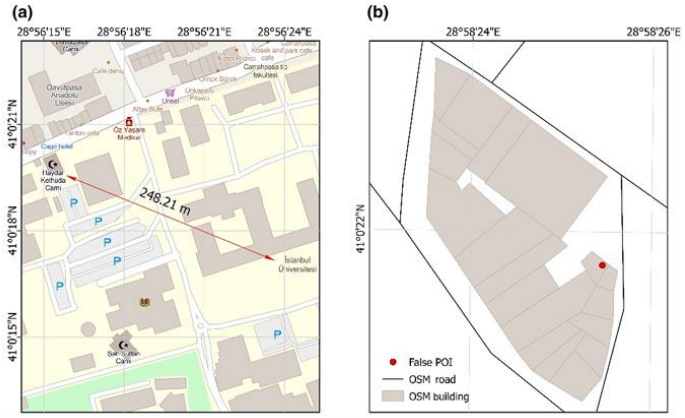
ภาพที่ 2.31 ผลกระทบของความสมบูรณ์ของ Address ต่อผลการ Geocoding

ที่มา: Clemens (2015)

ตัวอย่างความผิดพลาดของผลการระบุตำแหน่ง

งานวิจัยของ Kilic et al. (2023) แสดงตัวอย่างความผิดพลาดของ Reverse Geocoding ในพื้นที่เมือง โดยกรณีหนึ่งระบบคืน Istanbul University ซึ่งอยู่ห่างประมาณ 250 เมตรแทนสถานที่ที่ต้องการ แม้ว่าข้อมูล Reference ของสถานที่นั้นจะมีอยู่ก็ตาม

ตัวอย่างดังกล่าวแสดงให้เห็นว่าการเลือกวัตถุที่อยู่ใกล้ที่สุดไม่ได้หมายความว่าจะให้ผลลัพธ์ที่ถูกต้องเสมอ โดยเฉพาะบริเวณที่มีอาคารหรือ POI หลายแห่งอยู่ใกล้กัน



ภาพที่ 2.32 ตัวอย่างความผิดพลาดของการระบุตำแหน่งจากข้อมูล OpenStreetMap

ที่มา: Kilic, Hacar และ Gülgen (2023)

จากภาพที่ 2.32 แสดงให้เห็นว่าความผิดพลาดสามารถเกิดขึ้นได้ทั้งจากการเลือก Object ที่ไม่ตรงกับสถานที่เป้าหมายและจากข้อจำกัดของข้อมูลเชิงพื้นที่ ดังนั้นการนำผล Geocoding หรือ Reverse Geocoding ไปใช้งานควรคำนึงถึงคุณภาพของข้อมูลและบริบทของตำแหน่งร่วมด้วย

แนวทางตรวจสอบข้อมูลก่อนนำไปแสดงบนแผนที่

จากข้อจำกัดดังกล่าว การนำผล Geocoding ไปใช้ภายใน Application ควรมีกระบวนการตรวจสอบผลลัพธ์ก่อนเลือก Coordinate โดยสามารถพิจารณาข้อมูลหลายองค์ประกอบร่วมกัน เช่น ชื่อสถานที่ Address ประเภทของ POI เมือง ประเทศ และ Coordinate

กระบวนการสามารถอธิบายในเชิงแนวคิดได้ดังนี้

รับชื่อสถานที่
↓
ส่งคำค้นไปยัง Geocoding Service
↓
รับ Candidate Result
↓
ตรวจสอบชื่อ / Address / ประเภท
↓
ผลลัพธ์ตรงกับข้อมูลที่ต้องการหรือไม่?
↓
ใช่ ไม่ใช่
↓ ↓
เลือกพิกัด ตรวจสอบผลลัพธ์อื่น
↓
แสดงบนแผนที่

แนวทางดังกล่าวช่วยลดความเสี่ยงจากกรณีที่ชื่อสถานที่ซ้ำกันหรือมีสถานที่หลายแห่งอยู่ในพื้นที่ใกล้กัน ซึ่งเป็นปัญหาที่งานของ Kilic et al. (2023) พบว่าส่งผลต่อคุณภาพของ Geocoding และ Reverse Geocoding โดยเฉพาะพื้นที่เมืองที่มี POI หนาแน่น

สรุป OpenStreetMap and Geocoding

OpenStreetMap เป็นแพลตฟอร์มข้อมูลทางภูมิศาสตร์แบบเปิดที่จัดเก็บข้อมูลในรูปของ Node, Way และ Relation รวมถึง Tag ซึ่งใช้ระบุรายละเอียดเชิงความหมายของวัตถุ เช่น ชื่อ Address และประเภทของสถานที่

ข้อมูลดังกล่าวสามารถนำมาใช้งานร่วมกับ Nominatim ซึ่งเป็น Search and Geocoding Engine สำหรับ OpenStreetMap โดยสามารถแปลง Named Location หรือ Address ให้เป็น Latitude และ Longitude รวมถึงรองรับ Reverse Geocoding สำหรับแปลง Coordinate กลับเป็นข้อมูลสถานที่

Nominatim สามารถให้ข้อมูล Output ในรูปแบบที่ Application สามารถนำไปประมวลผล เช่น JSON และ XML รวมถึงข้อมูล Address, Coordinate, OSM Identifier และ Category

อย่างไรก็ตาม ความถูกต้องของผลลัพธ์ขึ้นอยู่กับทั้งคุณภาพของข้อมูล OpenStreetMap ความสมบูรณ์ของคำค้น รูปแบบ Address และความหนาแน่นของสถานที่ในบริเวณที่ค้นหา โดยงานวิจัยแสดงให้เห็นว่าระยะทางที่ใกล้ไม่ได้รับประกันว่าผลลัพธ์จะเป็นสถานที่ที่ถูกต้องเสมอไป

ดังนั้นการใช้งาน OpenStreetMap และ Geocoding ใน Application ควรประกอบด้วยทั้งกระบวนการค้นหาพิกัด การตรวจสอบข้อมูลผลลัพธ์ และการนำ Coordinate ที่ผ่านการพิจารณาแล้วไปแสดงบนระบบแผนที่ เพื่อให้การนำเสนอข้อมูลตำแหน่งมีความถูกต้องและสอดคล้องกับวัตถุประสงค์ของระบบ(Kilic et al., 2023) และ (Clemens, 2015)

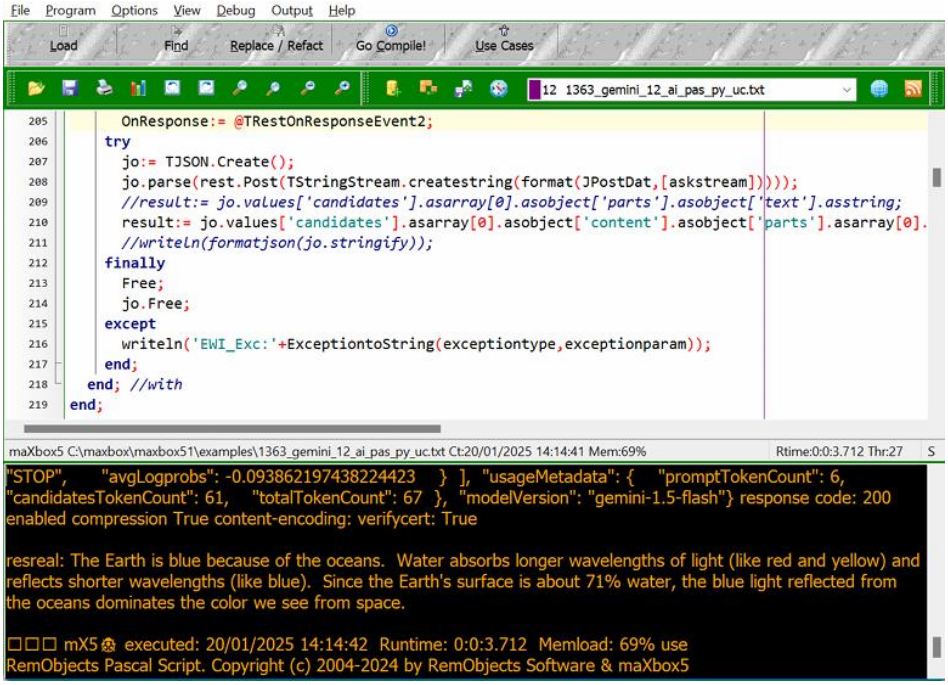
2.2.3 Google Gemini API

Google Gemini API เป็นชุดเครื่องมือและส่วนเชื่อมต่อสำหรับให้นักพัฒนาสามารถนำความสามารถของโมเดลปัญญาประดิษฐ์ตระกูล Gemini ไปใช้งานร่วมกับ Application หรือ Service อื่นได้ โดย Application สามารถส่งข้อมูลหรือคำสั่งไปยังโมเดลและรับผลลัพธ์กลับมาเพื่อนำไปประมวลผลหรือแสดงต่อผู้ใช้งาน

เอกสาร gemini.R อธิบายว่าเป็น Interface สำหรับ Google Gemini API ซึ่งช่วยให้สามารถเข้าถึงความสามารถของ Large Language Model หรือ LLM ของ Gemini และรองรับงานด้านการประมวลผลภาษา การสร้างข้อความ และความสามารถด้าน AI ประเภทอื่น

ในทำนองเดียวกัน maxbox4 (2025) อธิบายว่า Gemini API เป็นชุดเครื่องมือและ Interface ที่เปิดให้นักพัฒนานำโมเดลภายในตระกูล Gemini ไปเชื่อมต่อกับ Application และ Service ของตนได้ โดย API ทำหน้าที่เป็นช่องทางสำหรับส่ง Request ไปยังโมเดลและรับ Response กลับมา

Gemini เป็นกลุ่มโมเดล AI ที่มีความสามารถแบบ **Multimodal** กล่าวคือสามารถประมวลผลข้อมูลได้มากกว่าข้อความเพียงอย่างเดียว โดยเอกสารกล่าวถึงความสามารถในการทำความเข้าใจและสร้างข้อมูลประเภท Text, Code, Image, Audio และ Video ซึ่งทำให้สามารถนำไปประยุกต์ใช้กับ Application ได้หลายรูปแบบ



ภาพที่ 2.33 ตัวอย่างการใช้งาน Google Gemini API

ที่มา: maxbox4 (2025)

จากภาพที่ 2.32 แสดงตัวอย่างการนำ Gemini API มาเชื่อมต่อกับโปรแกรม โดยโปรแกรมส่ง Prompt ไปยังโมเดลและรับข้อความตอบกลับจากระบบ ซึ่งแสดงให้เห็นว่า Gemini API สามารถทำหน้าที่เป็นตัวกลางในการนำความสามารถของ Generative AI ไปใช้ภายใน Application ได้

ความสามารถของ Google Gemini API

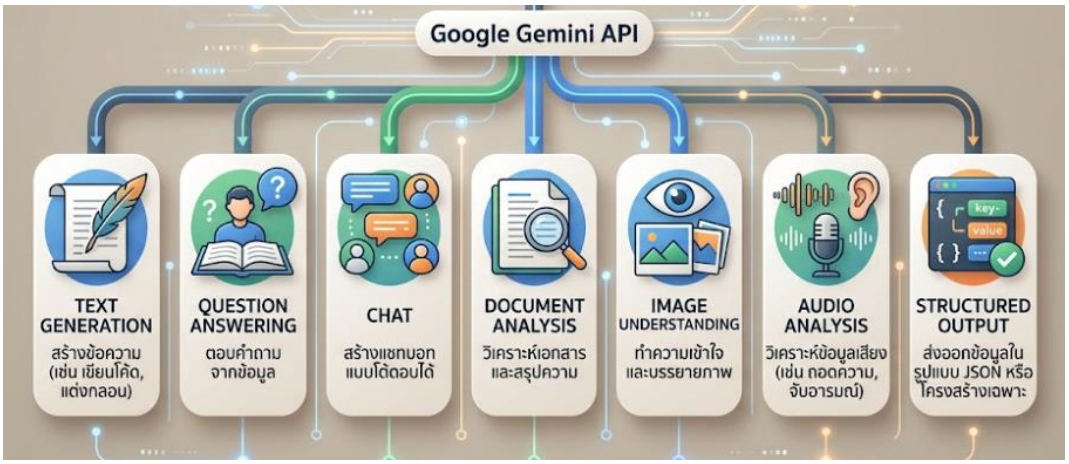
Google Gemini API สามารถนำไปใช้กับงานด้าน Natural Language Processing และ Generative AI ได้หลายประเภท เช่น การสร้างข้อความ การสรุปเนื้อหา การแปลภาษา การตอบคำถาม การวิเคราะห์ข้อความ และการสร้างระบบสนทนา

maxbox4 (2025) ยกตัวอย่างความสามารถด้านข้อความ ได้แก่ **Text Generation, Text Summarization, Text Translation, Sentiment Analysis, Question Answering และ Chatbot/Conversational AI** รวมถึงความสามารถแบบ Multimodal เช่น Image Understanding, Video Analysis และ Audio Analysis

นอกจากนี้ Gemini ยังสามารถนำมาใช้กับงานด้าน Code เช่น Code Completion, Code Explanation และ Code Translation ซึ่งช่วยให้โมเดลไม่ได้จำกัดอยู่เพียงการประมวลผลภาษาธรรมชาติ แต่สามารถประมวลผลข้อมูลที่มีโครงสร้างและ Programming Language ได้ด้วย

เอกสาร gemini.R แสดง Function สำหรับการใช้งาน Gemini หลายลักษณะ เช่น gemini() สำหรับสร้างข้อความจากข้อความ, gemini_chat() สำหรับการสนทนาหลายรอบ, gemini_docs() สำหรับสรุปหรือวิเคราะห์เอกสาร, gemini_image() สำหรับวิเคราะห์ข้อมูลที่ประกอบด้วยข้อความและรูปภาพ และ gemini_structured() สำหรับสร้างผลลัพธ์ที่มีโครงสร้าง

ดังนั้นความสามารถของ Gemini API สามารถแบ่งเชิงแนวคิดได้เป็น



ภาพที่ 2.34 ความสามารถที่สามารถเรียกใช้งานผ่าน Google Gemini API

การทำงานของ Gemini API

การนำ Gemini API ไปใช้ใน Application มีหลักการพื้นฐานในรูปแบบ Client–Server โดย Application ทำหน้าที่สร้าง Request และส่งข้อมูลไปยัง API Endpoint จากนั้น Gemini Model จะประมวลผล Prompt และส่ง Response กลับมายัง Application

สามารถแสดงกระบวนการได้ดังนี้

ผู้ใช้งาน
↓
Application
↓
สร้าง Prompt
↓
HTTP Request
↓
Gemini API

```
  ↓
Gemini Model
  ↓
Generated Response
  ↓
Application
  ↓
แสดงผลแก่ผู้ใช้งาน
```

API จึงทำหน้าที่เป็นตัวกลางระหว่าง Application และ AI Model โดย Application ไม่จำเป็นต้องติดตั้งหรือประมวลผลโมเดล AI ภายในอุปกรณ์โดยตรง แต่สามารถส่งข้อมูลไปให้ Service ประมวลผลแล้วนำผลลัพธ์ที่ได้กลับมาใช้งาน

maxbox4 (2025) อธิบายว่า Gemini API เป็น Standardized Interface สำหรับส่ง Request ไปยัง Gemini Model และรับ Response กลับมา ทำให้สามารถนำความสามารถด้าน AI ไปเพิ่มเป็น Function ภายใน Application ได้

Prompt

Prompt คือข้อความหรือคำสั่งที่ Application ส่งไปยังโมเดลเพื่อระบุสิ่งที่ต้องการให้โมเดลประมวลผลหรือสร้างผลลัพธ์ เอกสาร gemini.R ระบุว่า Function gemini() รับค่า prompt เป็นข้อมูลสำหรับให้โมเดลสร้างข้อความตอบกลับ

ตัวอย่างพื้นฐานจากเอกสารคือ

```r
library(gemini.R)

setAPI("YOUR_API_KEY")

gemini("Explain dplyr's mutate function")
```

โดย setAPI() ใช้กำหนด API Key และข้อความภายใน gemini() คือ Prompt ที่ส่งไปยังโมเดล

Prompt สามารถมีตั้งแต่คำถามสั้น ๆ ไปจนถึงคำสั่งที่กำหนดรายละเอียดหลายเงื่อนไข การกำหนด Prompt ที่ชัดเจนมีผลต่อรูปแบบของ Response เนื่องจากโมเดลจะใช้ข้อมูลและเงื่อนไขที่ได้รับเป็นบริบทสำหรับสร้างคำตอบ

ตัวอย่างเชิงแนวคิด เช่น

```
Prompt:
"สร้างแผนการเดินทางในกรุงเทพฯ 3 วัน
โดยมีงบประมาณ 5,000 บาท
เน้นสถานที่ทางวัฒนธรรมและอาหาร"
```

AI Model สามารถนำข้อมูลแต่ละเงื่อนไขภายใน Prompt มาประมวลผลร่วมกัน แล้วสร้างข้อความตอบกลับตามรูปแบบที่กำหนด

Request และ Response

การเรียก Gemini API มีการส่งข้อมูล Request ในรูปแบบที่มีโครงสร้าง โดยตัวอย่างในเอกสาร maxbox4 (2025) แสดงโครงสร้าง JSON ที่มี contents และ parts สำหรับบรรจุ Prompt ที่ต้องการส่งไปยังโมเดล

ตัวอย่าง JSON:

```
{
  "contents": [
    {
      "parts": [
        {
          "text": "Your text prompt goes here."
        }
      ]
    }
  ]
}
```

เมื่อ API ได้รับข้อมูลดังกล่าว Gemini Model จะประมวลผลข้อความภายใน text และสร้าง Response กลับมา

หากต้องการกำหนดพฤติกรรมของการสร้างผลลัพธ์ สามารถเพิ่ม Generation Configuration เช่น

```
{
  "contents": [
    {
      "parts": [
        {
          "text": "Your text prompt goes here."
        }
      ]
    }
  ],
```

```
  "generation_config": {
    "temperature": 0.7,
    "max_output_tokens": 100
  }
}
```

เอกสารยกตัวอย่างโครงสร้างดังกล่าวพร้อม temperature และ max_output_tokens สำหรับควบคุมการสร้างผลลัพธ์



```
{$I .\GAPIKEY.INC}

function TRestClient3_AskChatGPTGemini(askstream: string;
                                       aResponseHeader:TRestResponseHandler):string;
var JPostdat: string;
    jo: TJSON; rest: TRestResource;
begin
 (* JPostDat:= '{'+
    '"model": "text-davinci-003",'+
    '"prompt": "%s",'+
    '"max_tokens": 2048,'+
    '"temperature": 0.15}';   *)
   JPostDat:= '{'+
    '"contents": [{'+
    '"parts": [{"text": "%s"}]'+
    '}]}';

  with TRestClient.create(self) do begin
      rest:= Resource('https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateCont
                                              +GEMINI_APIKEY);
      println('@addr:'+objtostr(rest))
      rest.ContentType('application/json');
      //rest.Authorization('Bearer '+CHATGPT_APIKEY2);
      ConnectionType:= hctWinInet;
      OnResponse:= @TRestOnResponseEvent2;
    try
      jo:= TJSON.Create();
      jo.parse(rest.Post(TStringStream.createstring(format(JPostDat,[askstream]))));
      //result:= jo.values['candidates'].asarray[0].asobject['parts'].asobject['text'].asstring;
      result:= jo.values['candidates'].asarray[0].asobject['content'].asobject['parts'].asarray[0].asobject[
      //writeln(formatjson(jo.stringify));
    finally
      Free;
      jo.Free;
    except
      writeln('EWI_Exc:'+ExceptiontoString(exceptiontype,exceptionparam));
    end;
  end; //with
end;

Call:
writeln('resreal: '+TRestClient3_AskChatGPTGemini('Why is the earth blue?', Nil));
```

ภาพที่ 2.35 ตัวอย่างโครงสร้าง Request สำหรับ Google Gemini API (Source Code)

ที่มา: maxbox4 (2025)

```
{
  "contents": [
    {
      "parts": [
        {
          "text": "Your text prompt goes here."
        }
      ]
    }
  ],
  "generation_config": {
      "temperature": 0.7,
      "max_output_tokens": 100
    }
}
```

```
JPostDat:= '{'+
    '"contents": [{'+
    '"parts": [{"text": "%s"}]'+
    '            }],              '+
    '"generation_config": {       '+
    '  "temperature": 0.7,        '+
    '  "max_output_tokens": 100   '+
    '   }                         '+
    '}';
```

ภาพที่ 2.36 ตัวอย่างโครงสร้าง Request สำหรับ Google Gemini API (JSON Structure)

ที่มา: maxbox4 (2025)

Parameter สำหรับควบคุมการสร้างผลลัพธ์

การสร้างข้อความด้วย Generative AI สามารถกำหนด Parameter เพื่อควบคุมลักษณะของ Response ได้ เอกสาร gemini.R แสดง Parameter สำคัญ เช่น temperature, maxOutputTokens, topK, topP และ seed

Temperature เป็นค่าที่ใช้ควบคุมระดับความแปรผันของผลลัพธ์ เอกสารกำหนดช่วงค่าระหว่าง 0 ถึง 2 โดยการปรับค่าดังกล่าวมีผลต่อกระบวนการเลือก Token ของโมเดล

maxOutputTokens ใช้กำหนดจำนวน Token สูงสุดของ Response เพื่อควบคุมความยาวของข้อความที่โมเดลสามารถสร้างได้

topK และ **topP** เป็น Parameter ที่เกี่ยวข้องกับกระบวนการ Sampling ของโมเดล และใช้กำหนดขอบเขตของ Candidate Token ที่โมเดลสามารถเลือกในแต่ละขั้นตอน

ส่วน **seed** เป็นค่าที่สามารถกำหนดเพื่อใช้ควบคุมลักษณะบางส่วนของกระบวนการสร้างผลลัพธ์

ตรงนี้ **ไม่ต้องทำตารางครับ** เขียนเป็นย่อหน้าและใส่ Code จะดูดีกว่า

```
final generationConfig = {
 'temperature': 0.7,
```

```
  'maxOutputTokens': 2048,
  'topP': 0.95,
  'topK': 40,
};
```

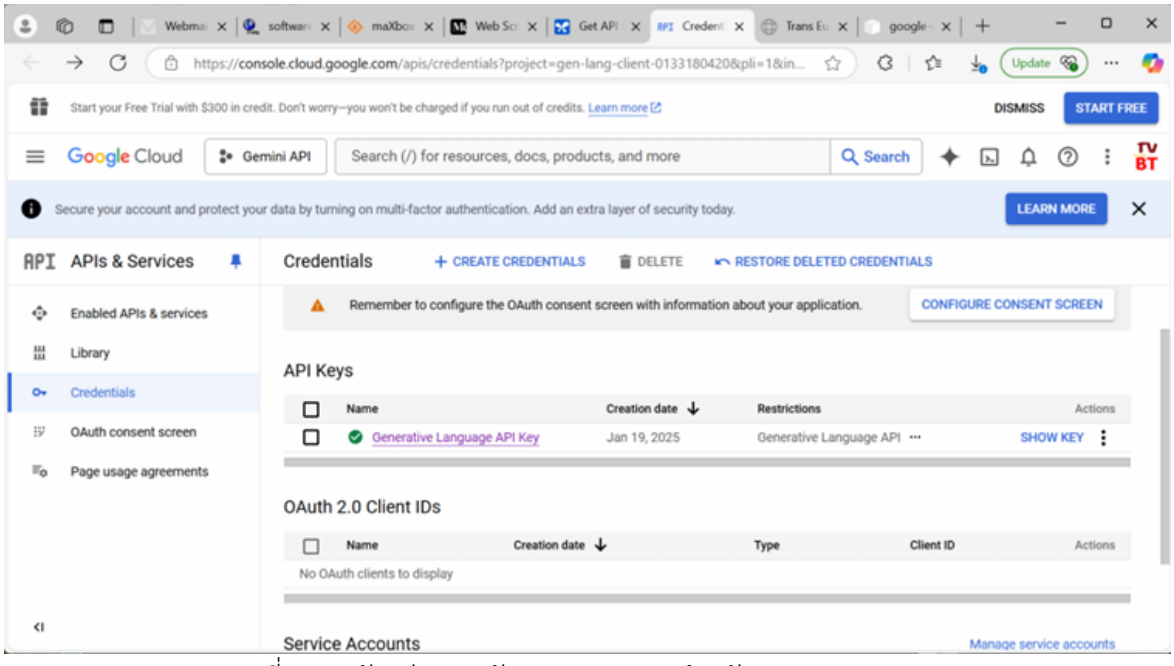
การใช้งาน API Key

การเรียกใช้ Gemini API จำเป็นต้องมี Credential สำหรับยืนยันสิทธิ์ในการใช้งาน Service หนึ่งในวิธีที่ใช้คือ API Key

เอกสาร gemini.R มี Function setAPI() สำหรับกำหนด Gemini API Key เป็น Environment Variable และระบุชัดเจนว่า API Key เป็นข้อมูลที่มีความละเอียดอ่อนและควรได้รับการปกป้อง

ใน Function อื่นของเอกสาร เช่น gemini_audio() และ gemini_docs() ระบุว่า API Key ถูกส่งผ่าน HTTP Header x-goog-api-key

ดังนั้น Application ไม่ควรแสดง API Key ให้ผู้ใช้งานเห็นโดยตรง หรือเผยแพร่ Key ภายใน Repository สาธารณะ เนื่องจากบุคคลอื่นอาจนำ Credential ดังกล่าวไปใช้เรียก API ได้



ภาพที่ 2.37 ตัวอย่างการจัดการ API Key สำหรับ Gemini API

ที่มา: maxbox4 (2025)

ตัวอย่างการเรียก Google Gemini API ด้วย Dart

สำหรับรายงานของคุณ ผมแนะนำให้มี Code ภาษา **Dart** เพราะเป็นภาษาที่ใช้ร่วมกับ Flutter และจะดีกว่าการยก R หรือ Python มาเป็น Code หลัก แม้แหล่งข้อมูลจะใช้ภาษาเหล่านั้นเป็นตัวอย่างก็ตาม

Code ต่อไปนี้เป็น **ตัวอย่างที่ผู้จัดทำสามารถทดลองเอง** โดยยึดแนวคิด Request แบบ JSON จากเอกสาร ไม่ใช่ Code ที่คัดลอกจากแหล่งอ้างอิง

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<String> askGemini(String prompt) async {
  const String apiKey = 'YOUR_API_KEY';

  final Uri url = Uri.parse(
    'https://generativelanguage.googleapis.com/'
    'v1beta/models/gemini-2.0-flash:generateContent',
  );

  final Map<String, dynamic> requestBody = {
    'contents': [
      {
        'parts': [
          {
            'text': prompt,
          }
        ]
      }
    ],
    'generationConfig': {
      'temperature': 0.7,
      'maxOutputTokens': 1024,
    }
  };

  final response = await http.post(
    url,
    headers: {
      'Content-Type': 'application/json',
      'x-goog-api-key': apiKey,
    },
    body: jsonEncode(requestBody),
  );

  if (response.statusCode == 200) {
    final Map<String, dynamic> data =
        jsonDecode(response.body);
```

```
    return data['candidates'][0]
        ['content']['parts'][0]['text'];
  } else {
    return 'Error: ${response.statusCode}';
  }
}

void main() async {
  final result = await askGemini(
    'แนะนำสถานที่ท่องเที่ยวในกรุงเทพ 3 แห่ง',
  );

  print(result);
}
```

โครงสร้าง contents ⟶ parts ⟶ text ในตัวอย่างสอดคล้องกับรูปแบบ Request ที่เอกสารแสดงไว้ ขณะที่แนวคิดการส่ง API Key ผ่าน Header สอดคล้องกับเอกสาร gemini.R

ผลลัพธ์

เมื่อ Run Code ผลลัพธ์ที่ Console อาจมีลักษณะประมาณ

```
1. วัดอรุณราชวราราม
   วัดสำคัญริมแม่น้ำเจ้าพระยา

2. พระบรมมหาราชวัง
   สถานที่สำคัญทางประวัติศาสตร์
3. ตลาดนัดจตุจักร
   แหล่งช้อปปิ้งและอาหารที่มีชื่อเสียง
```

ผลลัพธ์จริงอาจแตกต่างกันในแต่ละครั้ง เนื่องจากเป็นข้อความที่โมเดลสร้างขึ้น ดังนั้นสำหรับรายงานควร Run Code จริงแล้ว Capture Screenshot ของผลลัพธ์ของคุณเอง

Structured Output

อีกความสามารถหนึ่งที่มีประโยชน์สำหรับ Application คือการกำหนดให้ AI สร้างข้อมูลในรูปแบบ Structured Data แทนการคืนข้อความธรรมดา

เอกสาร gemini.R มี Function gemini_structured() ซึ่งสามารถกำหนด JSON Schema สำหรับ Response และผลลัพธ์ที่ได้จะถูกคืนเป็น Structured JSON

ตัวอย่างเชิงแนวคิด เช่น Application อาจต้องการข้อมูลในรูปแบบ

```
[
  {
    "place": "Place A",
    "time": "09:00",
    "description": "Description A"
  },
  {
    "place": "Place B",
    "time": "13:00",
    "description": "Description B"
  }
]
```

การได้รับ Response แบบ Structured มีข้อได้เปรียบต่อ Application เนื่องจากสามารถอ่าน Field แต่ละรายการแล้วนำไปจัดวางใน User Interface หรือบันทึกลง Database ได้ง่ายกว่าการ Parsing ข้อความที่ไม่มีโครงสร้าง

ตัวอย่างจากเอกสารกำหนด JSON Schema ที่ประกอบด้วย Array ของ Object และกำหนด Property ของแต่ละ Object ก่อนส่งไปยัง Gemini

ส่วนนี้ผมแนะนำให้ใส่ครับ เพราะ **เชื่อมกับลักษณะ Application ของคุณได้ดีมาก** โดยไม่จำเป็นต้องกล่าวชื่อโปรเจกต์ในเนื้อหาทฤษฎี

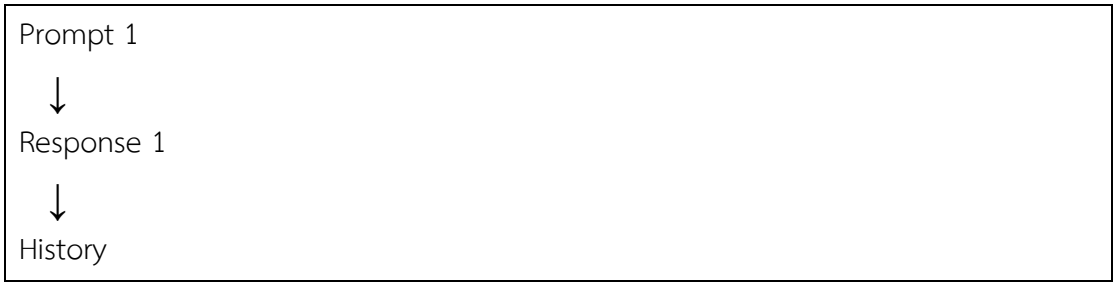
การสนทนาแบบ Multi-turn

Gemini API ยังรองรับการสนทนาที่มีหลายข้อความต่อเนื่อง โดยระบบสามารถเก็บ History ของการสนทนาและใช้ข้อมูลก่อนหน้าเป็น Context สำหรับข้อความถัดไป

เอกสาร gemini.R มี Function gemini_chat() ซึ่งรับ prompt และ history โดย History ทำหน้าที่เก็บลำดับของการสนทนา

ตัวอย่างในเอกสารแสดงการถามคำถามแรก จากนั้นนำ History ที่ได้จากคำตอบก่อนหน้าไปใช้กับคำถามถัดไป ทำให้โมเดลสามารถรักษาบริบทของ Conversation ได้

สามารถเขียน Flow เป็น





ระบบลักษณะนี้เหมาะกับ Application ที่ต้องการให้ผู้ใช้งานสนทนาหรือปรับแก้ผลลัพธ์จาก AI ต่อเนื่องจากข้อมูลเดิม

Token และความยาวของข้อมูล

การใช้งาน Large Language Model มีแนวคิด Token ซึ่งใช้เป็นหน่วยสำหรับประมวลผลข้อมูล Input และ Output

เอกสาร gemini.R มี Function countTokens() ซึ่งใช้คำนวณจำนวน Token ของ Content ทั้งข้อความ รูปภาพ หรือข้อมูลหลายส่วนร่วมกัน

การควบคุมจำนวน Token มีความสำคัญเนื่องจาก Model มีข้อจำกัดด้านปริมาณ Input และ Output ที่สามารถประมวลผลได้ และ Parameter maxOutputTokens สามารถใช้กำหนดจำนวน Token สูงสุดของผลลัพธ์

เมื่อสร้าง Prompt ที่มีรายละเอียดจำนวนมาก จึงควรเลือกข้อมูลที่มีความเกี่ยวข้องและกำหนดขนาด Output ให้เหมาะสมกับลักษณะการใช้งาน เพื่อไม่ให้ Response ยาวเกินความจำเป็น

การประมวลผลข้อมูลแบบ Multimodal

ความสามารถอีกด้านหนึ่งของ Gemini คือการรับข้อมูลหลายรูปแบบ เอกสาร gemini.R มี Function gemini_image() สำหรับส่ง Image พร้อม Prompt ไปยัง Gemini และได้รับข้อความวิเคราะห์กลับมา

นอกจากนี้ยังมี gemini_audio() ซึ่งส่ง Audio ไปยัง API และรับ Text Description กลับมา และ gemini_docs() สำหรับสรุป เปรียบเทียบ หรือวิเคราะห์ Document เช่น PDF, TXT, HTML, CSV และ XML

ดังนั้น Gemini API จึงไม่ได้จำกัดเพียง Text-to-Text แต่สามารถใช้เป็น Interface สำหรับการพัฒนา Application ที่ต้องประมวลผลข้อมูลหลายชนิดได้

ข้อควรพิจารณาในการใช้งาน Gemini API

การใช้ Gemini API จำเป็นต้องพิจารณาทั้ง Authentication, Response Format, API Error, Network Connection และความถูกต้องของผลลัพธ์ที่ AI สร้างขึ้น

API Key เป็นข้อมูลที่มีความละเอียดอ่อนและไม่ควรถูกเปิดเผย เอกสาร gemini.R ระบุอย่างชัดเจนว่า API Key เป็น Sensitive Information

Application ควรตรวจสอบ HTTP Status Code ก่อนประมวลผล Response รวมถึงรองรับกรณี API ไม่ตอบสนอง Network Error หรือ Response ไม่ตรงกับ Structure ที่คาดไว้

อีกประเด็นหนึ่งคือข้อความที่สร้างโดย Generative AI ไม่ควรถูกมองว่าเป็นข้อมูลที่ถูกต้องเสมอโดยอัตโนมัติ ดังนั้น Application ที่นำ Response ไปใช้กับข้อมูลสำคัญควรมีขั้นตอนตรวจสอบหรือ Validation ที่เหมาะสมก่อนนำข้อมูลไปใช้ต่อ

สรุป Google Gemini API

Google Gemini API เป็น Interface ที่ช่วยให้นักพัฒนาสามารถนำความสามารถของ Gemini Large Language Model ไปเชื่อมต่อกับ Application ผ่าน API ได้ โดย Application สามารถส่ง Prompt ไปให้โมเดลและรับ Generated Response กลับมาเพื่อนำไปใช้ภายในระบบ

Gemini API รองรับความสามารถหลายประเภท ได้แก่ Text Generation, Chat, Document Analysis, Image Understanding, Audio Processing และ Structured Output จึงสามารถนำไปใช้ได้ทั้งงานด้าน Natural Language Processing และ Multimodal Application

การเรียก API สามารถกำหนด Parameter เช่น Temperature, Maximum Output Token, Top-K และ Top-P เพื่อควบคุมกระบวนการสร้างผลลัพธ์ และสามารถใช้ API Key สำหรับ Authentication

นอกจากนี้ Structured Output ช่วยให้ Application สามารถรับข้อมูลในรูป JSON Structure ที่กำหนดไว้ล่วงหน้า ซึ่งเหมาะกับการนำ Response ไปแสดงใน User Interface หรือประมวลผลต่อในระบบ

ดังนั้น Google Gemini API จึงเป็นเครื่องมือที่ช่วยเชื่อมความสามารถของ Generative AI เข้ากับ Application โดยไม่จำเป็นต้องพัฒนาและติดตั้ง Large Language Model ขึ้นเองภายในระบบ และช่วยให้ Application สามารถเพิ่มความสามารถด้านการสร้าง วิเคราะห์ และประมวลผลข้อมูลผ่าน API ได้

2.2.4 Firebase Platform

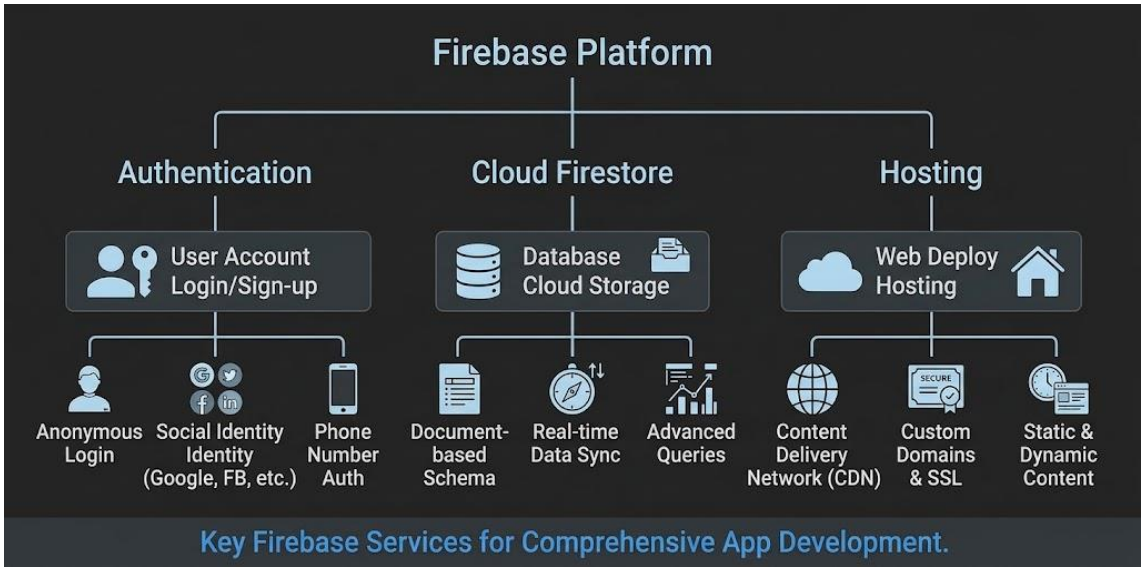
Firebase เป็นแพลตฟอร์มสำหรับสนับสนุนการพัฒนา Mobile Application และ Web Application โดยรวบรวมบริการหลายประเภทที่สามารถนำมาใช้ร่วมกันในการพัฒนาและบริหารจัดการระบบ เช่น ระบบจัดการผู้ใช้งาน ฐานข้อมูล การ Hosting และบริการอื่นที่เกี่ยวข้องกับ Application

High School Technology Services อธิบายว่า Firebase เป็นแพลตฟอร์มสำหรับพัฒนา Mobile และ Web Application ที่มีเครื่องมือและบริการหลายประเภทเพื่อช่วยสนับสนุนกระบวนการพัฒนา Application

Google LLC (2020) อธิบาย Firebase ในลักษณะ Platform as a Service หรือ PaaS สำหรับ Mobile Application โดยมี Software Development Kit หรือ SDK แบบรวม และมีบริการหลายประเภทที่สามารถใช้งานแยกจากกันหรือทำงานร่วมกันตามความต้องการของระบบ

บริการของ Firebase ครอบคลุมองค์ประกอบหลายด้าน เช่น Firebase Console, Firebase Hosting, Firebase Cloud Messaging, Firebase Realtime Database และบริการอื่น ๆ การมีบริการหลายประเภทภายใน Platform เดียวช่วยให้ Application สามารถเชื่อมโยงองค์ประกอบสำคัญ เช่น Authentication, Database และ Deployment เข้าด้วยกันได้

สำหรับหัวข้อนี้จะกล่าวถึงบริการที่สำคัญ 3 ส่วน ได้แก่ **Firebase Authentication, Cloud Firestore และ Firebase Hosting**



ภาพที่ 2.38 องค์ประกอบหลักของ Firebase Platform ที่ใช้ในการพัฒนา Application

Firebase Authentication

Firebase Authentication เป็นบริการสำหรับจัดการกระบวนการ Authentication หรือการยืนยันตัวตนของผู้ใช้งานก่อนเข้าสู่ระบบ โดย Application สามารถกำหนด Sign-in Method ได้หลายรูปแบบ

เอกสาร *Authentication with Firebase* ระบุว่า Firebase Authentication รองรับการเข้าสู่ระบบด้วย Email Address และ Password รวมถึง Federated Identity Provider เช่น Google Sign-In และ Facebook Login

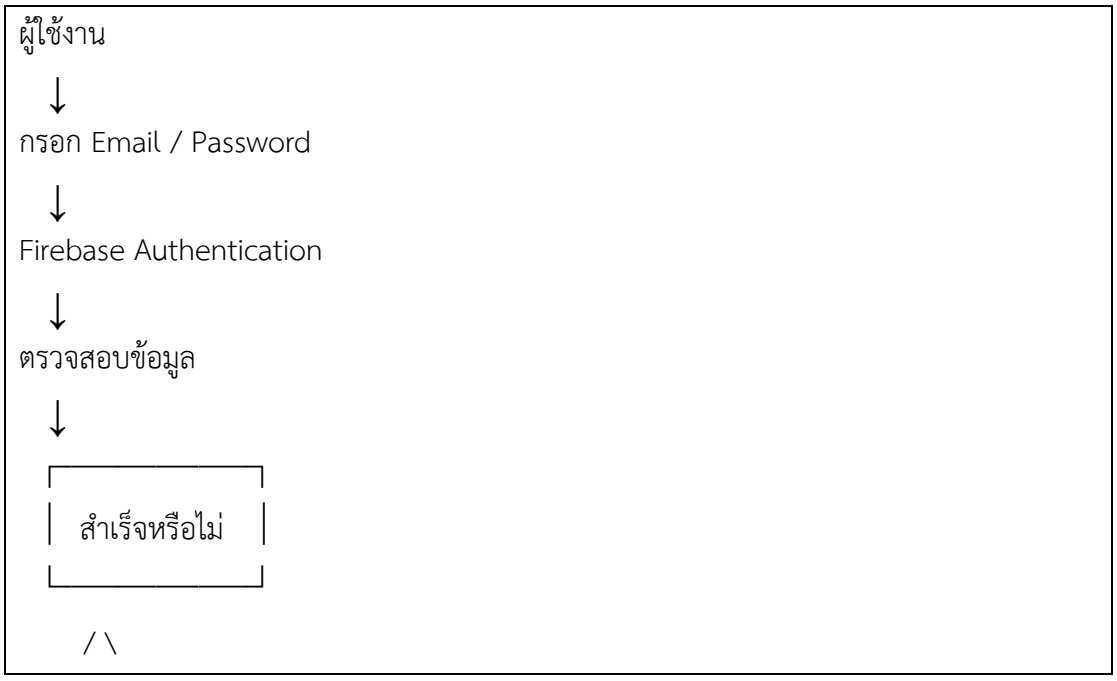
Authentication มีบทบาทสำคัญต่อ Application ที่มีข้อมูลเฉพาะของผู้ใช้งาน เนื่องจากระบบจำเป็นต้องสามารถระบุว่าผู้ที่กำลังใช้งานเป็นใครก่อนอนุญาตให้เข้าถึงข้อมูลหรือ Function ที่กำหนดไว้

กระบวนการ Authentication โดยทั่วไปสามารถแบ่งเป็น 2 ขั้นตอนหลัก ได้แก่ **การสมัครสมาชิก (Sign Up)** และ **การเข้าสู่ระบบ (Sign In)**

สำหรับการสร้างบัญชีใหม่ Firebase มี Method createUserWithEmailAndPassword ซึ่งรับข้อมูล Email Address และ Password ของผู้ใช้งาน เอกสารระบุว่าควรมีการตรวจสอบ Email และ Password ก่อนส่งข้อมูลเข้าสู่ Method ดังกล่าว

ส่วนผู้ใช้งานที่มีบัญชีอยู่แล้วสามารถเข้าสู่ระบบด้วย signInWithEmailAndPassword โดย Application รับ Email และ Password จาก Form และส่งข้อมูลให้ Firebase Authentication ตรวจสอบ

สามารถสรุปกระบวนการได้ดังนี้



```
    Yes  No
     ↓   ↓
เข้าสู่ระบบ   แสดงข้อผิดพลาด
```

ตัวอย่าง Firebase Authentication ด้วย Flutter

ตัวอย่างต่อไปนี้แสดงการเข้าสู่ระบบด้วย Email และ Password ผ่าน Package firebase_auth ใน Flutter โดยเป็นตัวอย่างที่ผู้จัดทำสามารถทดลองใช้งานได้

```dart
import 'package:firebase_auth/firebase_auth.dart';

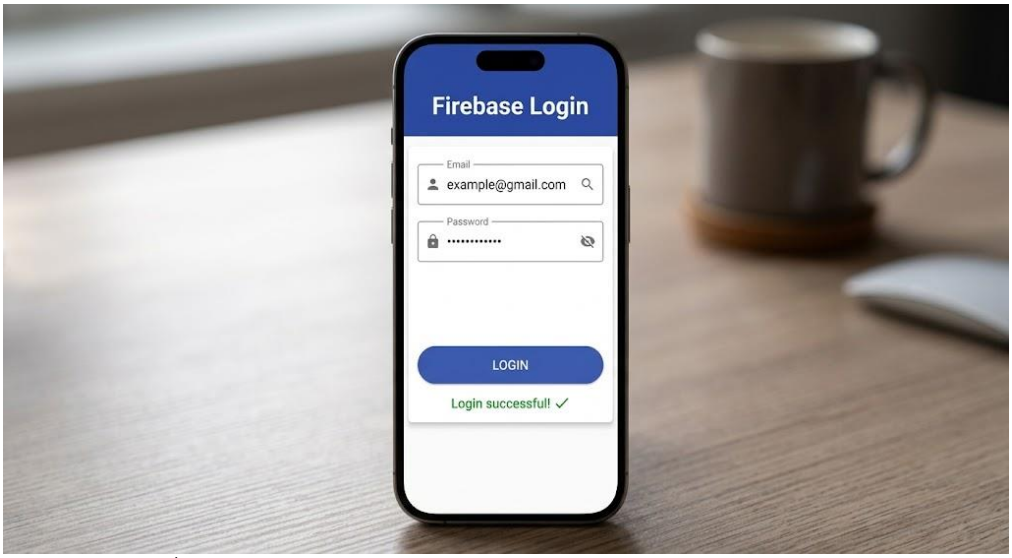
Future<void> login(
  String email,
  String password,
) async {
  try {
    final UserCredential userCredential =
        await FirebaseAuth.instance
            .signInWithEmailAndPassword(
      email: email,
      password: password,
    );

    print(
      'Login successful: '
      '${userCredential.user?.email}',
    );
  } on FirebaseAuthException catch (e) {
    print('Login failed: ${e.message}');
  }
}
```

หลักการของ Code ดังกล่าวสอดคล้องกับแนวคิดในเอกสารที่ใช้ Method signInWithEmailAndPassword สำหรับเข้าสู่ระบบด้วย Email และ Password

เมื่อ Authentication สำเร็จ จะสามารถอ่านข้อมูลของผู้ใช้ที่เข้าสู่ระบบผ่าน Object UserCredential ได้ หากเกิดข้อผิดพลาด เช่น Email หรือ Password ไม่ถูกต้อง สามารถใช้ FirebaseAuthException เพื่อรองรับ Error ที่เกิดขึ้น

ผลลัพธ์



ภาพที่ 2.39 ตัวอย่างหน้าจอเข้าสู่ระบบด้วย Firebase Authentication

Google Sign-In และการใช้ Identity Provider

นอกจาก Email และ Password แล้ว Firebase Authentication ยังสามารถทำงานร่วมกับ Identity Provider ภายนอกได้ โดยเอกสารระบุ Google Sign-In เป็นหนึ่งใน Sign-in Method ที่สามารถใช้ได้

การใช้ Identity Provider ช่วยให้ผู้ใช้งานสามารถใช้บัญชีที่มีอยู่แล้วเข้าสู่ Application แทนการสร้าง Username และ Password ใหม่ภายในระบบ

เชิงแนวคิดสามารถอธิบายได้ว่า

Google Account
↓
Google Sign-In
↓
Firebase Authentication
↓
User Identity
↓
Application

เมื่อ Authentication สำเร็จ Application สามารถใช้ User Identifier ที่ได้รับเพื่อเชื่อมโยงผู้ใช้กับข้อมูลอื่นภายในระบบได้ เช่น Document หรือข้อมูลที่ผู้ใช้งานสร้างขึ้น

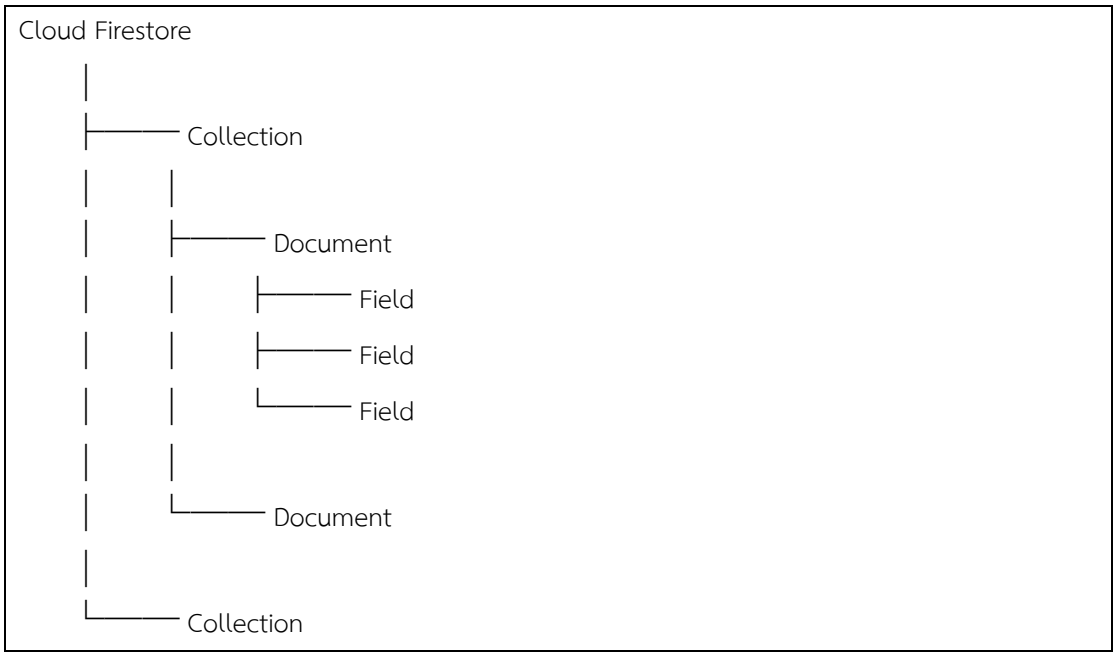
Cloud Firestore

Cloud Firestore เป็นฐานข้อมูลบน Cloud ที่สามารถใช้เป็น Backend สำหรับ Mobile และ Web Application โดยจัดเก็บข้อมูลในรูปแบบที่เหมาะกับ Application ที่ต้องการเข้าถึงข้อมูลผ่านเครือข่าย

Nadhifa et al. (2026) ใช้ Firebase Firestore เป็น Cloud Backend และระบุความสามารถที่สำคัญ ได้แก่ การจัดเก็บข้อมูลแบบ Real-time, การจัดการ Historical Log ด้วย Timestamp, การให้ API สำหรับเชื่อมต่อกับ Mobile Application และการรองรับการเข้าถึงแบบ Multi-platform โดยไม่จำเป็นต้องมี Physical Server เพิ่มเติม

Firestore ใช้รูปแบบโครงสร้างข้อมูลแบบ **Collection–Document** โดยงานของ Nadhifa et al. (2026) ระบุว่าข้อมูลสามารถจัดเก็บใน Collection และ Document พร้อม Timestamp เพื่อรองรับทั้งข้อมูลปัจจุบันและข้อมูลย้อนหลัง

โครงสร้างสามารถอธิบายเชิงแนวคิดได้ว่า



แนวทาง Collection–Document ทำให้สามารถจัดกลุ่มข้อมูลตามประเภทและกำหนด Field ภายใน Document ได้ตามความต้องการของ Application

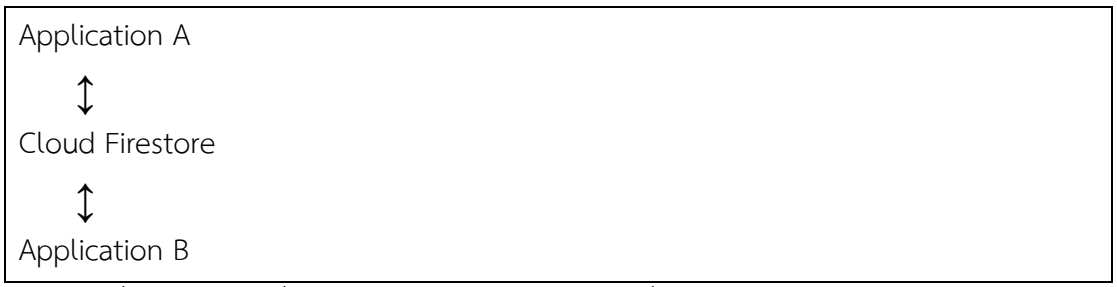
การทำงานแบบ Real-Time

คุณสมบัติสำคัญของ Firebase คือความสามารถในการ Synchronize ข้อมูลระหว่าง Database และ Client โดยเมื่อข้อมูลมีการเปลี่ยนแปลง Application สามารถรับข้อมูลใหม่และ Update การแสดงผลได้

Nadhifa et al. (2026) ระบุว่า Firestore รองรับ Real-time Synchronization ทำให้ข้อมูลล่าสุดสามารถแสดงบน Mobile Application ได้โดยไม่ต้อง Refresh ด้วยตนเอง

งานวิจัยเดียวกันยังอธิบายว่าข้อมูลที่จัดเก็บใน Firebase สามารถถูกเข้าถึงจาก Mobile Application ในลักษณะ Multi-user และ Cross-device โดย Firebase ทำหน้าที่เป็น Cloud Backend อยู่ระหว่าง Data Source และ Application

ลักษณะดังกล่าวสามารถแสดงได้ว่า



เมื่อข้อมูลถูกเปลี่ยนแปลงใน Database Client ที่กำลังรับฟังข้อมูลสามารถได้รับข้อมูลใหม่ตามกลไกของระบบ

ตัวอย่างการบันทึกข้อมูลลง Cloud Firestore ด้วย Flutter

ตัวอย่างต่อไปนี้เป็นการสร้าง Document ใหม่ภายใน Collection trips

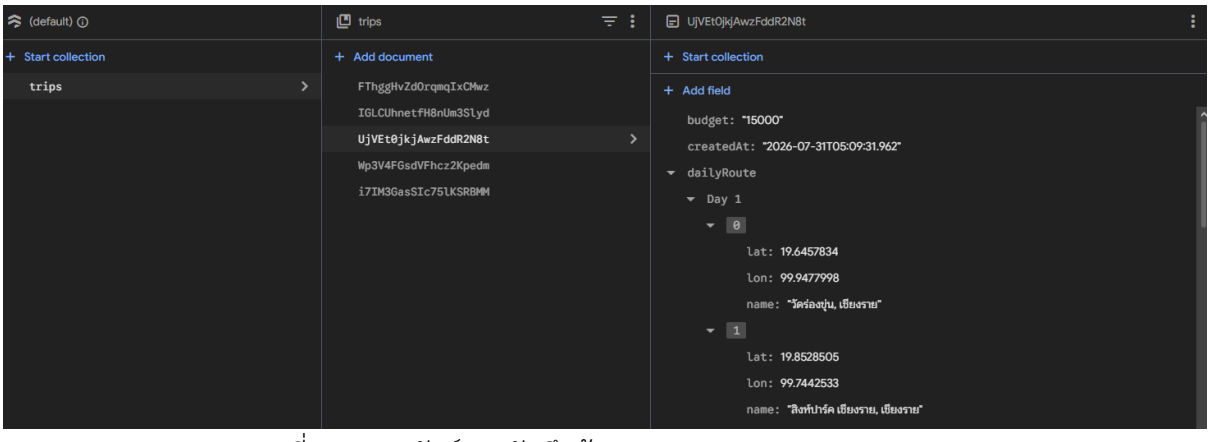
```
import 'package:cloud_firestore/cloud_firestore.dart';

Future<void> saveTrip() async {
  await FirebaseFirestore.instance
      .collection('trips')
      .add({
    'destination': 'Bangkok',
    'days': 3,
    'budget': 5000,
    'createdAt':
        FieldValue.serverTimestamp(),
  });

  print('Data saved successfully');
}
```

ในตัวอย่างนี้ collection('trips') ใช้ระบุ Collection ที่ต้องการจัดเก็บข้อมูล และ add() ใช้สร้าง Document ใหม่พร้อม Field เช่น Destination, Days, Budget และ Timestamp แนวคิดดังกล่าวสอดคล้องกับเอกสารที่ระบุว่า Firestore จัดข้อมูลแบบ Collection–Document และสามารถบันทึกข้อมูลพร้อม Timestamp เพื่อใช้จัดการ Historical Data ได้

ผลลัพธ์



ภาพที่ 2.40 ผลลัพธ์การบันทึกข้อมูลลง Cloud Firestore

ตัวอย่างการอ่านข้อมูลจาก Cloud Firestore

Flutter สามารถอ่าน Document ภายใน Collection ผ่าน cloud_firestore ได้ ตัวอย่างเช่น

```
Future<void> loadTrips() async {
 final QuerySnapshot result =
    await FirebaseFirestore.instance
        .collection('trips')
        .get();

 for (final doc in result.docs) {
   print(doc.data());
 }
}
```

เมื่อเรียก .get() ระบบจะ Query ข้อมูลภายใน Collection และคืนผลลัพธ์เป็นชุด Document จากนั้นสามารถอ่านข้อมูลแต่ละ Document และนำไปแสดงบน User Interface ได้

ในงานของ Nadhifa et al. (2026) Application ที่พัฒนาด้วย Flutter ใช้ firebase_core และ cloud_firestore เพื่อเชื่อมกับ Firebase และนำข้อมูล Firestore มาแสดงแบบ Real-time

หากต้องการให้ข้อมูลบนหน้าจอเปลี่ยนแปลงตาม Database สามารถใช้ Stream ร่วมกับ Firestore เช่น

```
StreamBuilder<QuerySnapshot>(
  stream: FirebaseFirestore.instance
      .collection('trips')
      .snapshots(),
  builder: (context, snapshot) {
    if (!snapshot.hasData) {
      return const CircularProgressIndicator();
    }

    return ListView(
      children: snapshot.data!.docs.map((doc) {
        return ListTile(
          title: Text(
            doc['destination'],
          ),
        );
      }).toList(),
    );
  },
)
```

ตัวอย่างนี้ใช้ .snapshots() เพื่อรับ Stream ของข้อมูลและ StreamBuilder เพื่อ Update User Interface เมื่อข้อมูลเปลี่ยนแปลง

Firebase Authentication และ Cloud Firestore ทำงานร่วมกันอย่างไร

Firebase Authentication และ Firestore สามารถทำงานร่วมกันโดยใช้ข้อมูลจาก Authentication เพื่อระบุเจ้าของข้อมูลใน Database

ตัวอย่างเชิงแนวคิดคือ

ผู้ใช้งาน

↓

Firebase Authentication

↓

User ID (UID)

↓

Cloud Firestore

↓

ข้อมูลของผู้ใช้งาน

หลังผู้ใช้งานเข้าสู่ระบบ Application สามารถอ่าน UID ของผู้ใช้จาก Firebase Authentication และนำค่าดังกล่าวไปเชื่อมโยงกับ Document ที่ผู้ใช้งานสร้างขึ้น

ตัวอย่าง Code:

```
final User? user =
    FirebaseAuth.instance.currentUser;

await FirebaseFirestore.instance
    .collection('trips')
    .add({
  'userId': user?.uid,
  'destination': 'Bangkok',
  'createdAt':
      FieldValue.serverTimestamp(),
});
```

เมื่อ Document มี userId กำกับไว้ ระบบสามารถ Query ข้อมูลตาม User ID เพื่อแยกข้อมูลของผู้ใช้งานแต่ละรายออกจากกันได้

Firebase Security และ Access Control

การจัดการ Authentication จำเป็นต้องทำงานร่วมกับ Access Control เพื่อกำหนดว่า User ใดสามารถอ่านหรือแก้ไขข้อมูลประเภทใดได้

เอกสาร Firebase Authentication ระบุว่าการเข้าถึง Database สามารถถูกกำหนดผ่าน Database Rules และโดยค่าเริ่มต้นสามารถจำกัด Read/Write เฉพาะผู้ใช้ที่ผ่าน Authentication ได้

Google LLC (2020) ยังระบุว่า Google Firebase มี Authentication และ Access Control สำหรับจำกัดการเข้าถึงระบบและข้อมูล และใช้ TLS Certificate เพื่อสนับสนุนการ Authentication และ Encryption สำหรับข้อมูลระหว่างการรับส่ง

ดังนั้นการใช้ Authentication ร่วมกับ Database Rule ช่วยให้ Application สามารถกำหนดระดับการเข้าถึงข้อมูลตาม User Identity ได้

ตัวอย่างแนวคิดของ Rule เช่น

User Login



Firebase Hosting

Firebase Hosting เป็นบริการสำหรับเผยแพร่ Web Application ผ่านโครงสร้างพื้นฐานของ Firebase

Google LLC (2020) อธิบายว่า Firebase Hosting เป็น Web Hosting ที่มุ่งเน้นสำหรับ Modern Front-end Web Application โดยสามารถ Deploy Web Application ที่รองรับ SSL รวมถึง Static Content และ Microservices ไปยัง Global Content Delivery Network ได้

การใช้ Hosting ช่วยให้ Application ที่พัฒนาเสร็จแล้วสามารถถูกนำขึ้นให้ผู้ใช้งานเข้าถึงผ่าน Internet ได้ โดยส่วนของ Application ที่ Build แล้วจะถูกเผยแพร่ไปยัง Hosting Infrastructure

Firebase Console ทำหน้าที่เป็น Web Interface ส่วนกลางสำหรับบริหาร Application และเปิดใช้งานหรือ Configuration Firebase Product ต่าง ๆ

กระบวนการ Deployment สามารถอธิบายได้ดังนี้



ผู้ใช้งานเปิดผ่าน Browser

ตัวอย่างขั้นตอนการ Deploy Flutter Web

หลังพัฒนา Flutter Web เสร็จแล้ว สามารถ Build Application ก่อนนำขึ้น Hosting ได้ ตัวอย่างคำสั่ง:

```
flutter build web
```

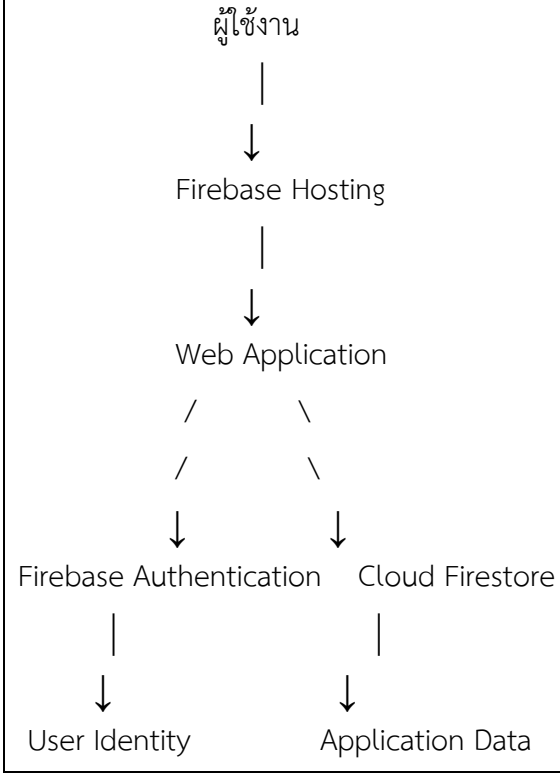
เมื่อ Build สำเร็จ Flutter จะสร้างไฟล์สำหรับ Web Application จากนั้นจึงสามารถ Deploy Resource ไปยัง Firebase Hosting ตาม Configuration ของ Project

การทำงานร่วมกันของ Firebase Platform

Firebase Authentication, Cloud Firestore และ Firebase Hosting มีหน้าที่แตกต่างกัน แต่สามารถทำงานร่วมกันภายใน Application เดียวกัน

Firebase Authentication ทำหน้าที่เกี่ยวกับ **Identity และการเข้าสู่ระบบ** Cloud Firestore ทำหน้าที่ **จัดเก็บและเรียกใช้ข้อมูล** ส่วน Firebase Hosting ทำหน้าที่ **เผยแพร่ Web Application ให้สามารถใช้งานผ่าน Internet**

ภาพรวมสามารถเขียนได้ว่า



Application ที่เผยแพร่ผ่าน Firebase Hosting สามารถเชื่อมต่อกับ Firebase Authentication เพื่อจัดการ Identity ของผู้ใช้งาน และใช้ Cloud Firestore เป็นระบบฐานข้อมูลสำหรับจัดเก็บและเรียกใช้ข้อมูล การใช้บริการหลายประเภทภายใน Platform เดียวช่วยให้องค์ประกอบด้าน Authentication, Data Management และ Deployment สามารถทำงานร่วมกันภายในโครงสร้างระบบเดียวกัน

ความสามารถด้าน Cloud และ Availability

Firebase ทำงานบน Cloud Infrastructure ของ Google โดย Google LLC (2020) ระบุว่าสามารถเข้าถึง Firebase ผ่าน Internet ได้จากหลาย Location และ Device

Infrastructure ของ Firebase ทำงานในลักษณะ Distributed Environment โดยข้อมูลถูกจัดเก็บบน Shared Infrastructure และ Distributed Database

รายงานยังระบุว่ามี Redundant Architecture ซึ่ง Replicate ข้อมูลแบบ Real-time ไปยัง Data Center ที่แยกกันทางภูมิศาสตร์อย่างน้อยสองแห่ง และใช้ Load Balancing เพื่อสนับสนุน High Availability

คุณลักษณะดังกล่าวเป็นส่วนหนึ่งของเหตุผลที่ Cloud Platform สามารถลดความจำเป็นในการติดตั้งและบริหาร Physical Server ของ Application เอง

ข้อดีของการใช้ Firebase Platform

ข้อดีประการสำคัญคือ Firebase รวมบริการหลายประเภทไว้ภายใน Platform เดียว ทำให้ Application สามารถใช้ Authentication, Database และ Hosting โดยไม่ต้องสร้าง Infrastructure แต่ละส่วนขึ้นใหม่ทั้งหมด

Cloud Firestore สนับสนุน Real-time Storage และ Synchronization รวมถึงการเข้าถึงแบบ Multi-platform ขณะที่ Firebase Authentication ช่วยจัดการ Sign-in Method สำหรับผู้ใช้งาน และ Firebase Hosting รองรับ SSL และ Global Content Delivery Network สำหรับ Web Application

การรวมบริการดังกล่าวช่วยลดภาระด้าน Infrastructure ในการพัฒนา Application และทำให้บริการต่าง ๆ สามารถใช้ข้อมูลหรือ Identity ร่วมกันภายใน Firebase Ecosystem ได้

งานของ Nadhifa et al. (2026) ยังรายงานว่า Firestore ช่วยลดความจำเป็นในการใช้ Server เพิ่มเติม รองรับ Real-time Synchronization และเหมาะกับการเชื่อมต่อ Mobile Application

ข้อควรพิจารณาในการใช้ Firebase Platform

แม้ Firebase จะช่วยลดภาระด้าน Infrastructure แต่ Application ยังคงจำเป็นต้องออกแบบ Authentication, Access Rule และ Data Structure อย่างเหมาะสม

การเปิด Database ให้ Public Access โดยไม่มี Authentication สามารถทำให้บุคคลภายนอกเข้าถึงข้อมูลได้ เอกสาร Authentication จึงเตือนว่าหากเปิด Rule เป็น Public ควรกลับมากำหนดข้อจำกัดหลังตั้งค่า Authentication แล้ว

ในด้านข้อมูล ผู้พัฒนาจำเป็นต้องออกแบบ Collection และ Document ให้เหมาะกับรูปแบบ Query และ Application Logic โดยเฉพาะเมื่อข้อมูลมีจำนวนเพิ่มขึ้น

นอกจากนี้ Cloud Service ยังต้องอาศัย Network Connection เพื่อสื่อสารกับบริการภายนอก หาก Connection มีปัญหาอาจส่งผลต่อการ Authentication หรือการอ่านและเขียนข้อมูล จึงควรมี Error Handling และ UI State รองรับกรณีที่ Service ไม่สามารถตอบสนองได้

สรุป Firebase Platform

Firebase เป็น Platform สำหรับสนับสนุนการพัฒนา Mobile และ Web Application โดยให้บริการหลายประเภทผ่าน Cloud Infrastructure และสามารถนำแต่ละ Service มาทำงานร่วมกันภายใน Application ได้

Firebase Authentication ทำหน้าที่จัดการการยืนยันตัวตนและรองรับ Sign-in Method เช่น Email/Password และ Google Sign-In

Cloud Firestore ทำหน้าที่เป็น Cloud Backend สำหรับจัดเก็บข้อมูลในรูปแบบ Collection–Document และสามารถรองรับการ Synchronize ข้อมูลแบบ Real-time ระหว่าง Database และ Application

Firebase Hosting ทำหน้าที่เผยแพร่ Front-end Web Application ผ่าน Hosting Infrastructure ที่รองรับ SSL และ Global Content Delivery Network

ดังนั้นบริการทั้งสามประเภทจึงสามารถใช้ร่วมกันเพื่อสนับสนุนกระบวนการตั้งแต่ **การเข้าสู่ระบบของผู้ใช้งาน การจัดเก็บข้อมูล ไปจนถึงการเผยแพร่ Web Application** ทำให้ Firebase สามารถทำหน้าที่เป็น Platform กลางสำหรับองค์ประกอบหลายส่วนของ Application ได้

2.2.5 Unsplash API

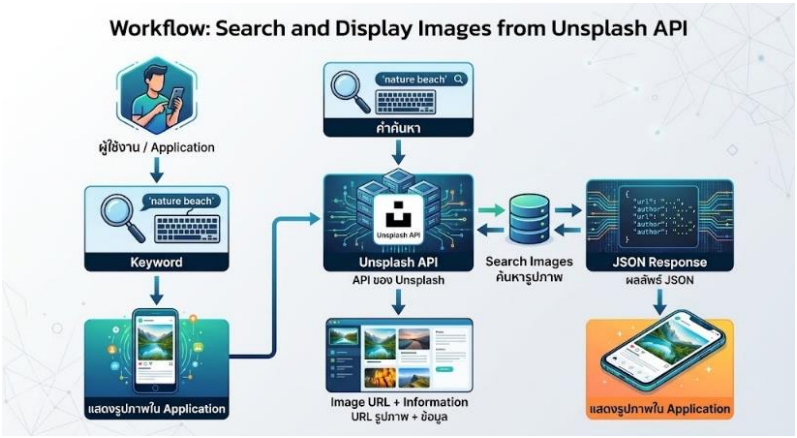
Unsplash API เป็นบริการที่เปิดให้นักพัฒนาสามารถเชื่อมต่อ Application เข้ากับแหล่งข้อมูลรูปภาพของ Unsplash เพื่อค้นหาและนำข้อมูลรูปภาพไปใช้ภายในระบบผ่าน API โดย

Application สามารถส่งคำค้นหรือ Request ไปยังบริการ จากนั้นรับข้อมูลรูปภาพกลับมาและนำผลลัพธ์ไปประมวลผลหรือแสดงบน User Interface

Paul (2026) อธิบายการพัฒนาระบบค้นหารูปภาพที่ทำงานผ่าน Unsplash API โดยผู้ใช้งานสามารถส่งคำค้นและรับรายการรูปภาพที่ตรงกับคำค้นกลับมา ระบบดังกล่าวใช้ Unsplash API เป็น External Service สำหรับการค้นหาและดึงข้อมูลรูปภาพคุณภาพสูงมาแสดงภายใน Web Application

ในอีกงานหนึ่ง Siranjeevi et al. (2026) นำ Unsplash API มาใช้ร่วมกับระบบสร้าง User Interface ด้วยปัญญาประดิษฐ์ โดย Unsplash API ทำหน้าที่จัดหารูปภาพจริงที่มีบริบทสัมพันธ์กับเนื้อหาของหน้าจอที่ถูกสร้างขึ้น ทำให้ Interface ที่ได้มีองค์ประกอบทางภาพที่เหมาะสมกับบริบทมากขึ้น

จากลักษณะดังกล่าว Unsplash API จึงสามารถทำหน้าที่เป็นแหล่งข้อมูลรูปภาพภายนอกสำหรับ Application ที่ต้องการค้นหารูปภาพตามข้อความหรือ Keyword โดยไม่จำเป็นต้องจัดเก็บไฟล์รูปภาพทั้งหมดไว้ภายในระบบตั้งแต่ต้น



ภาพที่ 2.41 กระบวนการค้นหารูปภาพผ่าน Unsplash API

จากรูปที่ 2.41 การใช้งาน Unsplash API เริ่มจาก Application สร้างคำค้นและส่ง Request ไปยัง External API จากนั้นบริการจะค้นหารูปภาพที่สัมพันธ์กับคำค้นและส่งข้อมูลกลับมายัง Application เพื่อใช้ในการแสดงผล ซึ่งระบบค้นหารูปภาพที่ศึกษาโดย Paul (2026) ใช้รูปแบบการทำงานในลักษณะเดียวกัน โดยเมื่อไม่พบข้อมูลใน Cache ระบบจะเรียก Unsplash API และรับ Response ในรูปแบบ JSON ก่อนส่งข้อมูลกลับไปยัง Frontend

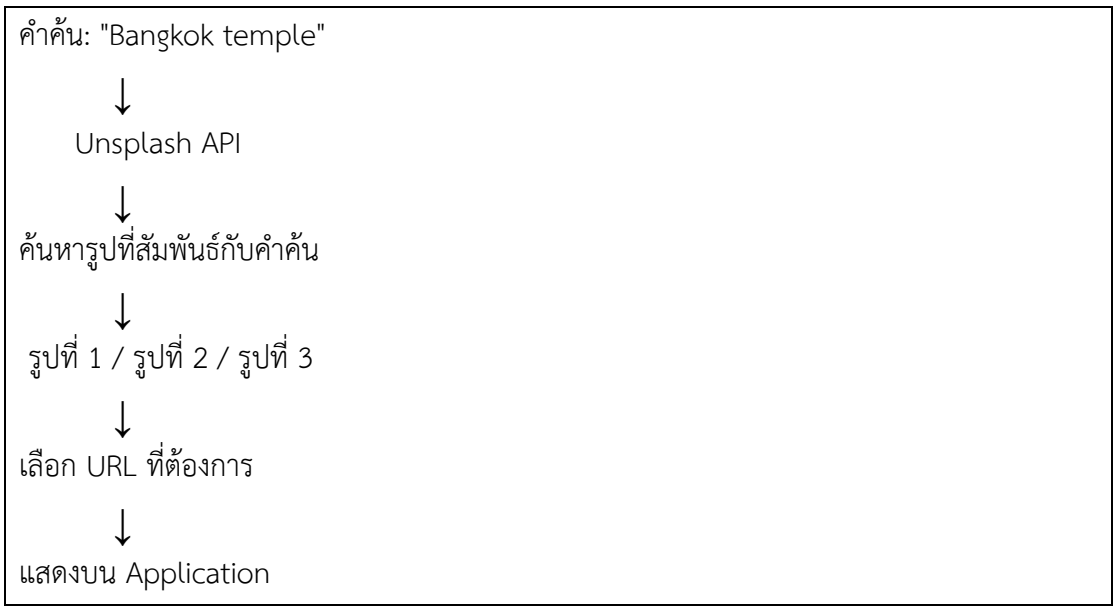
การค้นหารูปภาพด้วย Keyword

หนึ่งในรูปแบบการใช้งาน Unsplash API คือการค้นหารูปภาพตามข้อความ โดยผู้ใช้งานสามารถระบุ Keyword ที่อธิบายสิ่งที่ต้องการ เช่น ชื่อเมือง ธรรมชาติ อาคาร อาหาร หรือสถานที่ และ Application ส่งข้อความดังกล่าวไปใช้เป็น Query สำหรับค้นหารูปภาพที่เกี่ยวข้อง

Paul (2026) กำหนด Image Search Capability เป็นหนึ่งในความสามารถหลักของระบบ โดยผู้ใช้งานสามารถป้อน Search Term และรับรูปภาพที่ตรงกับข้อความค้นหาจาก Unsplash API

แนวคิดการใช้ข้อความเพื่อค้นหารูปภาพยังพบในงานของ Li et al. (2025) ซึ่งใช้ Unsplash เป็นแหล่งภาพสำหรับสร้าง Dataset โดยใช้ทั้ง Label ของรูปภาพและ Text Description เพื่อกรองรูปที่ตรงตามความต้องการ

ตัวอย่างกระบวนการสามารถแสดงได้ดังนี้



ลักษณะดังกล่าวทำให้ Application สามารถเปลี่ยนรูปภาพที่แสดงตามข้อมูลหรือบริบทที่ได้รับ โดยไม่จำเป็นต้องกำหนดรูปภาพแบบคงที่ให้กับทุกสถานที่ล่วงหน้า

ข้อมูลที่ได้รับจาก Unsplash API

เมื่อ Application ส่ง Request ไปยัง API ผลลัพธ์สามารถถูกส่งกลับในรูปแบบข้อมูลที่มีโครงสร้างเพื่อให้โปรแกรมสามารถนำไปประมวลผลต่อได้

ในระบบ Unsplash FindAWall เมื่อ Backend เรียก Unsplash API ระบบจะรับ Response ในรูป **JSON** จากนั้นทำการ Parse ข้อมูลและเตรียมเฉพาะส่วนที่จำเป็นก่อนส่งไปยัง Frontend

ข้อมูลที่ Application ต้องการใช้จาก Response โดยทั่วไปอาจประกอบด้วยข้อมูลเกี่ยวกับรูปภาพ เช่น ตัวระบุรูปภาพ ข้อมูลคำอธิบาย ผู้ถ่ายภาพ และ URL สำหรับโหลดรูปภาพในขนาดต่าง ๆ ทั้งนี้ Field ที่มีอยู่จริงขึ้นอยู่กับ Response Structure ของบริการและ Version ของ API ที่นำมาใช้

ในเชิงโครงสร้างสามารถอธิบายได้ว่า

```
JSON Response
    |
    ├──── Image information
    ├──── Description
    ├──── Photographer
    ├──── Image URL
    └──── Additional metadata
```

การรับข้อมูลในรูป JSON เหมาะกับ Mobile และ Web Application เนื่องจากสามารถนำไป Decode เป็น Object หรือ Data Structure ของ Programming Language แล้วเลือก Field ที่ต้องการมาแสดงผลได้

ตัวอย่างการค้นหารูปภาพจาก Unsplash API ด้วย Dart

ตัวอย่างต่อไปนี้เป็นตัวอย่างสำหรับการทดลองใช้งานใน Flutter/Dart โดยผู้จัดทำสามารถปรับให้ตรงกับ API Credential และรูปแบบ API ปัจจุบันได้

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<String?> searchImage(String keyword) async {
  const String accessKey = 'YOUR_UNSPLASH_ACCESS_KEY';

  final Uri url = Uri.https(
    'api.unsplash.com',
    '/search/photos',
    {
      'query': keyword,
      'per_page': '1',
    },
  );

  final response = await http.get(
    url,
    headers: {
```

```
      'Authorization': 'Client-ID $accessKey',
    },
  );

  if (response.statusCode == 200) {
    final Map<String, dynamic> data =
        jsonDecode(response.body);

    final List<dynamic> results = data['results'];

    if (results.isNotEmpty) {
      final String imageUrl =
          results[0]['urls']['regular'];

      print('ค้นหารูปภาพสำเร็จ');
      print('Keyword: $keyword');
      print('Image URL: $imageUrl');

      return imageUrl;
    }
  }

  print('ไม่พบรูปภาพ');
  return null;
}
```

ในตัวอย่างดังกล่าว Application รับ Keyword ผ่านตัวแปร keyword จากนั้นสร้าง HTTP Request ไปยังบริการค้นหารูปภาพ เมื่อได้รับ Response แล้ว jsonDecode() จะทำหน้าที่แปลง JSON ให้อยู่ในรูป Object ที่ Dart สามารถประมวลผลได้ และเลือก URL ของรูปภาพจากผลลัพธ์เพื่อนำไปใช้งานต่อ

แนวคิดดังกล่าวสอดคล้องกับสถาปัตยกรรมในงานของ Paul (2026) ซึ่งอธิบายว่าหลัง Unsplash API ส่ง JSON Response กลับมา ระบบจะ Parse ข้อมูลและส่งข้อมูลที่เตรียมแล้วไปยัง Frontend เพื่อ Rendering

ตัวอย่างผลลัพธ์

เมื่อทดลองค้นหา

Bangkok temple

ผลลัพธ์ที่ Console จะมีลักษณะประมาณ

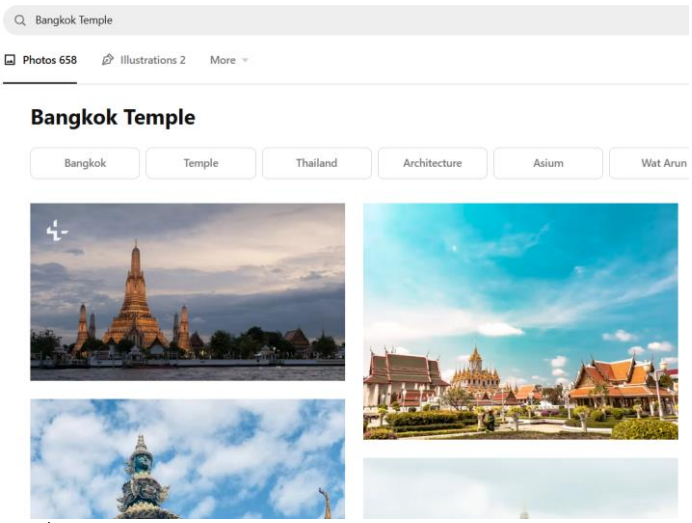
ค้นหารูปภาพสำเร็จ

```
Keyword: Bangkok temple
Image URL: https://images.unsplash.com/...
```

จากนั้น URL ที่ได้รับสามารถนำไปใช้กับ Flutter Image.network() ได้ เช่น

```dart
Image.network(
  imageUrl,
  width: 350,
  height: 220,
  fit: BoxFit.cover,
)
```

ผลลัพธ์บนหน้าจอจะมีลักษณะประมาณ



ภาพที่ 42 ผลลัพธ์การค้นหาและแสดงรูปภาพจาก Unsplash API

การนำ Unsplash API มาใช้ร่วมกับระบบอัตโนมัติ

Unsplash API ไม่จำเป็นต้องใช้เฉพาะกับ Search Box ที่ผู้ใช้งานกรอกคำค้นโดยตรง แต่สามารถนำมาใช้ในกระบวนการอัตโนมัติที่ระบบสร้าง Keyword แล้วค้นหารูปภาพให้เองได้

งานของ Siranjeevi et al. (2026) เป็นตัวอย่างที่ชัดเจน โดยระบบที่นำเสนอใช้ Google Gemini ร่วมกับ Unsplash API ในกระบวนการสร้าง Mobile UI เมื่อ AI ต้องการรูปภาพที่สัมพันธ์กับบริบท ระบบสามารถเรียก Unsplash เพื่อดึงภาพจริงมาใส่ในหน้าจอที่สร้างขึ้น

ในขั้นตอนการสร้างแต่ละหน้าจอ Gemini ยังได้รับอนุญาตให้เรียกเครื่องมือ Unsplash สำหรับ Contextual Photography และสามารถทำ Query ซ้ำได้ตามความเหมาะสมภายในขั้นตอนการสร้าง

ผลการประเมินของงานดังกล่าวระบุว่าการผสาน Unsplash ช่วยให้ระบบสามารถฝังภาพถ่ายที่สัมพันธ์กับบริบทลงใน Interface และเพิ่มคุณภาพด้านภาพของหน้าจอที่สร้างขึ้น

แนวคิดนี้สามารถสรุปเป็น

ข้อมูลหรือบริบท
↓
สร้าง Keyword
↓
Unsplash API
↓
ค้นหารูปที่สัมพันธ์กับ Keyword
↓
รับ Image URL
↓
แสดงรูปบน User Interface

การใช้ Unsplash API สำหรับรวบรวมข้อมูลรูปภาพ

Unsplash API ยังสามารถประยุกต์ใช้กับงานที่ต้องรวบรวมรูปภาพจำนวนมาก ไม่ได้จำกัดเพียงการนำภาพหนึ่งภาพไปแสดงบน Application

Li et al. (2025) ใช้ Unsplash ในการสร้าง Dataset สำหรับการทดลองด้าน Image Style Transfer โดยผู้วิจัยระบุว่าใช้ Full Gallery API ของ Unsplash เพื่อรวบรวมตัวอย่างรูปภาพตามข้อกำหนดของการทดลอง

ในขั้นตอนการสร้าง Dataset ผู้วิจัยใช้ Unsplash API เพื่อดึง Label ของรูปภาพก่อน จากนั้นใช้ Text Description เพื่อกรองภาพเพิ่มเติม และสุดท้ายใช้ CLIP Model ตรวจสอบความสัมพันธ์ระหว่าง Image และ Text เพื่อให้ข้อมูลที่เลือกมีความเกี่ยวข้องกับวัตถุประสงค์ของ Dataset

กระบวนการสามารถสรุปได้ดังนี้

Unsplash API
↓
ดึงข้อมูลรูปภาพและ Label
↓
กรองด้วย Keyword / Description

↓

ตรวจสอบความสัมพันธ์ของข้อมูล

↓

เลือกรูปภาพที่ตรงตามเงื่อนไข

ตัวอย่างดังกล่าวแสดงให้เห็นว่า API สำหรับรูปภาพสามารถใช้ได้ทั้งในระดับ User Interface และ Data Processing Pipeline



ภาพที่ 43 ตัวอย่างชุดข้อมูลรูปภาพที่รวบรวมจาก Unsplash

การรักษาความปลอดภัยของ API Key

หนึ่งในประเด็นสำคัญของการเรียก External API คือการรักษาความปลอดภัยของ Credential โดยเฉพาะ API Key ซึ่งใช้ระบุ Application ที่กำลังเรียก Service

Paul (2026) อธิบายว่าการใส่ API Key ไว้ใน Frontend มีความเสี่ยง เนื่องจากข้อมูลดังกล่าวสามารถถูกตรวจสอบและดึงออกจาก Browser ได้ จึงเสนอให้เก็บ API Key ไว้ใน Environment Variable และใช้ Backend Proxy เป็นตัวกลางสำหรับการเรียก External API

รูปแบบที่ปลอดภัยกว่าจึงสามารถอธิบายได้ว่า

Frontend

↓

Search Keyword

↓

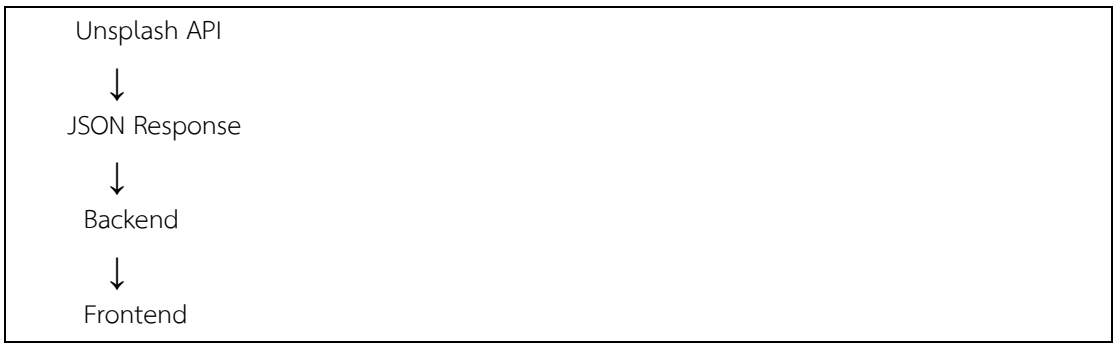
Backend Server

|

├──── อ่าน API Key

| จาก Environment

↓



ในสถาปัตยกรรมดังกล่าว Client ไม่จำเป็นต้องทราบ API Key โดยตรง Backend ทำหน้าที่เพิ่ม Credential ให้กับ Request และรับ Response ก่อนส่งข้อมูลที่จำเป็นกลับให้ Client

Paul (2026) ระบุว่าระบบที่ศึกษาใช้ Backend Proxy เพื่อ Validate Input, ปกป้อง API Key และเป็นตัวกลางระหว่าง Frontend กับ Unsplash API

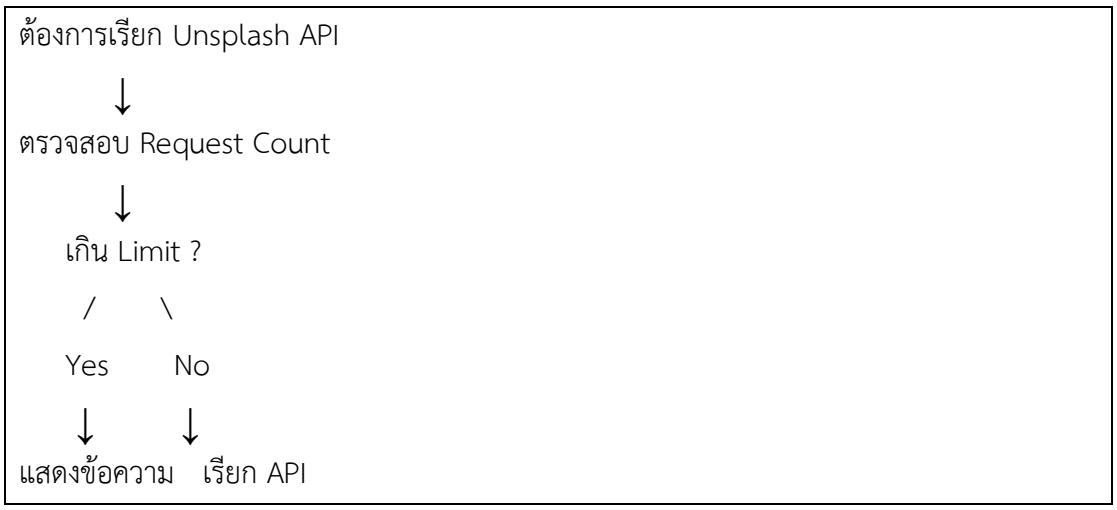
Rate Limit

บริการ API ภายนอกมักกำหนดจำนวน Request ที่สามารถส่งได้ภายในช่วงเวลาหนึ่ง เพื่อควบคุมการใช้ทรัพยากรและป้องกันการใช้งานที่มากเกินไป

Paul (2026) อธิบายว่า Unsplash API มีข้อจำกัดด้าน Request และระบบที่เรียกใช้งานควรมีการติดตามจำนวน Request รวมถึงรองรับกรณีที่กำลังเข้าใกล้หรือเกิน Limit เพื่อหลีกเลี่ยงการหยุดให้บริการ

งานดังกล่าวใช้แนวคิด Request Counter และ Time Window เพื่อตรวจสอบจำนวน API Call หากจำนวน Request ถึง Limit ระบบจะหยุดหรือคืน Controlled Response แทนที่จะปล่อยให้ระบบล้มเหลว

สามารถแสดงได้ว่า



```
  ↓      ↓
รอ Reset   +1 Request
```

ดังนั้น Application ที่เรียก External API ควรมี Error Handling สำหรับกรณี API Limit, Network Error และ Service Error เพื่อป้องกันไม่ให้ปัญหาจาก External Service ส่งผลให้ Application ทำงานผิดพลาดทั้งหมด

Caching

Caching เป็นอีกแนวทางหนึ่งสำหรับช่วยลดจำนวน Request ที่ต้องส่งไปยัง Unsplash API โดยเฉพาะเมื่อผู้ใช้งานค้นหาคำเดิมหลายครั้ง

ในระบบของ Paul (2026) มีการใช้ LRU หรือ Least Recently Used Cache เพื่อจัดเก็บผลลัพธ์ที่ถูกเรียกใช้บ่อย หากคำค้นมีอยู่ใน Cache ระบบสามารถคืนผลลัพธ์ให้ผู้ใช้งานได้โดยไม่ต้องส่ง Request ใหม่ไปยัง Unsplash API

สามารถอธิบายได้ดังนี้

```
Search Keyword
    ↓
ตรวจสอบ Cache
  /     \
พบ        ไม่พบ
↓          ↓
คืนข้อมูล   Unsplash API
           ↓
        เก็บใน Cache
           ↓
        คืนข้อมูล
```

แนวทางดังกล่าวช่วยลด Network Traffic และลดโอกาสใช้ API Request เกินความจำเป็น ในผลการทดสอบระบบที่ศึกษา ผู้วิจัยรายงานว่าหลังเพิ่ม LRU Cache จำนวน API Call ลดลงประมาณ 48% ภายใต้เงื่อนไขการทดลองของระบบดังกล่าว

อย่างไรก็ตาม ตัวเลขดังกล่าวเป็นผลของระบบและ Workload เฉพาะในงานวิจัย ไม่ควรนำไปตีความว่า Application ทุกประเภทจะสามารถลด API Call ได้ในอัตราเดียวกัน

Lazy Loading

Application ที่แสดงรูปภาพจำนวนมากอาจใช้เวลาในการโหลดสูงหากดาวน์โหลดรูปทั้งหมดพร้อมกัน ดังนั้น Lazy Loading เป็นเทคนิคที่สามารถนำมาใช้เพื่อลดการโหลด Resource ที่ยังไม่จำเป็น

Paul (2026) อธิบาย Lazy Loading ว่าเป็นแนวทางที่โหลดรูปภาพเมื่อรูปกำลังเข้าสู่พื้นที่ที่ผู้ใช้งานสามารถมองเห็นแทนการโหลดรูปทั้งหมดตั้งแต่เริ่มต้น ซึ่งช่วยลด Initial Loading Time และปริมาณข้อมูลที่ต้องรับส่ง

หลักการสามารถสรุปได้ว่า

เปิดหน้ารายการรูป

↓

โหลดเฉพาะรูปที่เห็น

↓

ผู้ใช้ Scroll

↓

รูปใหม่เข้าสู่ Viewport

↓

จึงโหลดรูปเพิ่ม

สำหรับ Mobile Application แนวคิดนี้ช่วยลดการใช้ Bandwidth และ Memory เมื่อมีรายการรูปจำนวนมาก โดยเฉพาะเมื่อผู้ใช้งานอาจไม่ได้ Scroll ดูรูปทั้งหมด

Responsive Image Rendering

การเลือกขนาดรูปให้เหมาะสมกับอุปกรณ์ก็เป็นอีกปัจจัยหนึ่งของประสิทธิภาพ เนื่องจากการส่งรูปขนาดใหญ่ให้กับอุปกรณ์ที่มีหน้าจอขนาดเล็กสามารถทำให้ใช้ Network Bandwidth มากกว่าที่จำเป็น

Paul (2026) อธิบาย Responsive Image Rendering ว่าระบบสามารถเลือก Image Size ให้เหมาะสมกับขนาดหน้าจอหรือ Device แทนการส่งรูปขนาดเดียวให้ทุกอุปกรณ์ ซึ่งช่วยลดปริมาณข้อมูลและเวลาการโหลด

แนวคิดนี้มีความสำคัญโดยเฉพาะกับ Web และ Mobile Application ที่ต้องรองรับ Device หลายขนาด เนื่องจากภาพเดียวกันอาจไม่จำเป็นต้องใช้ Resolution เท่ากันบน Smartphone, Tablet และ Desktop

ข้อดีของ Unsplash API

Unsplash API ช่วยให้ Application สามารถค้นหาและนำรูปภาพจาก External Image Service มาใช้งานตาม Keyword โดยไม่จำเป็นต้องสร้างคลังรูปภาพขนาดใหญ่ภายในระบบเอง

จากตัวอย่างการใช้งานในงานวิจัย Unsplash API สามารถใช้ได้ตั้งแต่การสร้างระบบค้นหารูปภาพ การนำรูปภาพที่สัมพันธ์กับบริบทมาใช้ภายใน Interface ไปจนถึงการรวบรวมข้อมูลรูปภาพสำหรับ Dataset

API ยังช่วยให้รูปภาพสามารถถูกเลือกตามข้อมูลหรือ Context ของ Application แทนการกำหนดรูปแบบตายตัว ทำให้ Content ที่แสดงมีความสัมพันธ์กับข้อมูลที่เกิดขึ้นในขณะใช้งาน

อย่างไรก็ตาม การนำ External API มาใช้จำเป็นต้องพิจารณา Security, Rate Limit, Network Availability และ Performance เนื่องจาก Application ต้องพึ่งพาบริการภายนอกในการดึงข้อมูล

ข้อจำกัดและข้อควรพิจารณา

การใช้งาน Unsplash API มีข้อควรพิจารณาหลายด้าน ประการแรกคือ Application ต้องอาศัย Network Connection และ External Service หากเกิดปัญหาการเชื่อมต่อหรือ Service ไม่พร้อมใช้งาน รูปภาพอาจไม่สามารถโหลดได้

ประการต่อมาคือ API มี Rate Limit จึงไม่ควรเรียก API ใหม่โดยไม่จำเป็น การใช้ Cache สามารถช่วยลดจำนวน Request ได้ ขณะที่ Lazy Loading ช่วยลดจำนวนรูปที่ถูกโหลดในช่วงเริ่มต้น

API Key เป็นข้อมูลที่ควรได้รับการปกป้องและไม่ควรเปิดเผยผ่าน Frontend Source Code การใช้ Environment Variable และ Backend Proxy เป็นแนวทางที่งานของ Paul (2026) เสนอเพื่อช่วยลดความเสี่ยงจากการเปิดเผย Credential

นอกจากนี้ระบบควรมี Error Handling สำหรับกรณีไม่พบรูปภาพ API ตอบกลับผิดพลาด Network ขัดข้อง หรือ Request เกิน Limit เพื่อให้ Application ยังคงสามารถทำงานส่วนอื่นได้โดยไม่หยุดทั้งหมด

Siranjeevi et al. (2026) ก็ระบุว่าการพึ่งพา External APIs เช่น Gemini และ Unsplash ทำให้ต้องพิจารณาทั้งเรื่อง Availability และ Cost สำหรับการใช้งานระดับ Production

สรุป Unsplash API

Unsplash API เป็นบริการสำหรับเชื่อมต่อ Application กับข้อมูลรูปภาพบน Unsplash โดยสามารถนำไปใช้สร้างระบบค้นหารูปภาพตาม Keyword และนำข้อมูลที่ได้กลับมาแสดงภายใน User Interface

การใช้งานสามารถเริ่มจาก Application ส่ง Search Query ไปยัง Unsplash API จากนั้น API ส่ง Response กลับในรูปแบบข้อมูล เช่น JSON ก่อนที่ Application จะเลือกข้อมูลรูปภาพที่ต้องการและนำ Image URL ไปใช้แสดงผล

Unsplash API สามารถประยุกต์ใช้ในหลายรูปแบบ ตั้งแต่ระบบ Image Search, การเลือก Contextual Image สำหรับ User Interface ไปจนถึงการรวบรวมรูปภาพสำหรับ Dataset โดยงานของ Siranjeevi et al. (2026) ใช้ Unsplash สำหรับเพิ่มภาพจริงที่สัมพันธ์กับ UI ที่สร้างโดย AI ขณะที่ Li et al. (2025) ใช้ API สำหรับรวบรวมและกรองข้อมูลรูปภาพตาม Label และ Text Description

อย่างไรก็ตาม การใช้งาน External Image API ควรคำนึงถึงการรักษาความปลอดภัยของ API Key การควบคุม Rate Limit การใช้ Cache การจัดการ Network Error และการโหลดรูปอย่างมีประสิทธิภาพ เพื่อให้ระบบสามารถให้บริการได้อย่างต่อเนื่องและลดการเรียก Resource ที่ไม่จำเป็น(Qiming Li, Mengcheng Wu และ Daozheng Chen ,2025) และ (Sougat Paul,2026)

2.3 งานวิจัยและผลงานที่เกี่ยวข้องทั้งในและต่างประเทศ

กิตติโชค คุ้มครอง และคณะ (2568) ได้พัฒนาเว็บแอปพลิเคชันสำหรับให้บริการแนะนำสถานที่ท่องเที่ยวด้วยปัญญาประดิษฐ์ร่วมกับระบบบริหารจัดการข้อมูลบนแผนที่ภูมิศาสตร์ โดยมีวัตถุประสงค์เพื่อพัฒนาระบบที่สามารถช่วยแนะนำสถานที่ท่องเที่ยวให้เหมาะสมกับผู้ใช้งาน และช่วยให้ผู้ดูแลระบบสามารถจัดการข้อมูลสถานที่ท่องเที่ยวได้อย่างมีประสิทธิภาพ ระบบถูกพัฒนาในรูปแบบเว็บแอปพลิเคชันโดยใช้ภาษา PHP, HTML, JavaScript และ CSS ร่วมกับฐานข้อมูล MySQL และใช้เทคนิคปัญญาประดิษฐ์แบบ Decision Tree ในการวิเคราะห์ข้อมูลความต้องการของผู้ใช้ โดยทำการฝึกโมเดลด้วยโปรแกรม WEKA ซึ่งมีความแม่นยำในการแนะนำสถานที่ท่องเที่ยวประมาณ 90% ผลการประเมินระบบโดยผู้เชี่ยวชาญพบว่า ระบบมีประสิทธิภาพอยู่ในระดับดีมาก โดยผู้ใช้งานสามารถดูรายละเอียดสถานที่ท่องเที่ยว แสดงความคิดเห็น ให้คะแนน และใช้ระบบปัญญาประดิษฐ์ในการแนะนำสถานที่ท่องเที่ยวที่เหมาะสมกับความต้องการได้

งานวิจัยดังกล่าวแสดงให้เห็นว่าการนำเทคโนโลยีปัญญาประดิษฐ์มาประยุกต์ใช้ร่วมกับระบบแผนที่ภูมิศาสตร์สามารถช่วยเพิ่มประสิทธิภาพในการให้บริการข้อมูลด้านการท่องเที่ยวและช่วยให้นักท่องเที่ยวสามารถตัดสินใจเลือกสถานที่ท่องเที่ยวได้ง่ายขึ้น ซึ่งแนวคิดดังกล่าวสามารถนำมา

ประยุกต์ใช้กับโครงงาน ระบบออกแบบทริปการท่องเที่ยวด้วย AI เพื่อช่วยวิเคราะห์ความสนใจของผู้ใช้งานและแนะนำแผนการเดินทางที่เหมาะสมได้

สุมนา บุษบก และคณะ (2568) ได้พัฒนาแชทบอทปัญญาประดิษฐ์เพื่อแนะนำข้อมูลการท่องเที่ยวในจังหวัดภูเก็ต ประเทศไทย โดยระบบถูกพัฒนาบนแพลตฟอร์ม LINE เพื่อให้ผู้ใช้งานสามารถค้นหาข้อมูลเกี่ยวกับสถานที่ท่องเที่ยว ร้านอาหาร ที่พัก และโปรแกรมการท่องเที่ยวแบบรายวันได้อย่างสะดวก นอกจากนี้ระบบยังสามารถเชื่อมโยงกับแผนที่เพื่อแสดงตำแหน่งของสถานที่ท่องเที่ยวได้อย่างชัดเจน

ผลการประเมินระบบจากผู้เชี่ยวชาญจำนวน 7 คน และนักท่องเที่ยวจำนวน 120 คน พบว่าระบบมีประสิทธิภาพในระดับดีมาก โดยมีความเร็วในการตอบสนองของระบบอยู่ในระดับสูง และผู้ใช้งานมีความพึงพอใจโดยรวมอยู่ในระดับสูง ซึ่งแสดงให้เห็นว่าการนำเทคโนโลยีปัญญาประดิษฐ์มาใช้ในการให้ข้อมูลด้านการท่องเที่ยวสามารถช่วยเพิ่มประสิทธิภาพในการให้บริการข้อมูลและอำนวยความสะดวกให้แก่นักท่องเที่ยวได้อย่างมีประสิทธิภาพ

กมลลักษณ์ ชัยดี และคณะ (2559) ได้ทำการวิจัยเรื่องการพัฒนาแอพพลิเคชันสำหรับสื่อการท่องเที่ยวบนระบบปฏิบัติการแอนดรอยด์ โดยมีวัตถุประสงค์เพื่อพัฒนาแอพพลิเคชันบนสมาร์ทโฟนสำหรับประชาสัมพันธ์และนำเสนอข้อมูลแหล่งท่องเที่ยวพระธาตุ 9 จอม จังหวัดเชียงราย รวมถึงช่วยอำนวยความสะดวกในการเข้าถึงข้อมูลและการเดินทางของนักท่องเที่ยว งานวิจัยได้ทำการรวบรวมข้อมูลแหล่งท่องเที่ยว วิเคราะห์ความต้องการของผู้ใช้งาน และออกแบบระบบแอพพลิเคชันให้สามารถแสดงข้อมูลในรูปแบบข้อความ ภาพ และมัลติมีเดีย

ระบบที่พัฒนาขึ้นสามารถทำงานบนโทรศัพท์สมาร์ทโฟนระบบปฏิบัติการแอนดรอยด์ และมีการเชื่อมต่อกับบริการแผนที่ Google Maps และระบบระบุตำแหน่งด้วย GPS เพื่อแสดงตำแหน่งของสถานที่ท่องเที่ยว รวมถึงสามารถแนะนำเส้นทางการเดินทางจากตำแหน่งปัจจุบันของผู้ใช้งานไปยังสถานที่ท่องเที่ยวที่ต้องการได้

ผลการทดลองใช้งานแอพพลิเคชันกับกลุ่มตัวอย่างนักท่องเที่ยวจำนวน 70 คน พบว่าผู้ใช้งานมีความพึงพอใจต่อการใช้งานระบบในระดับดีมาก โดยมีค่าเฉลี่ยความพึงพอใจโดยรวมเท่ากับ 4.68 แสดงให้เห็นว่าโมบายแอพพลิเคชันสามารถช่วยสนับสนุนการเข้าถึงข้อมูลการท่องเที่ยวและช่วยอำนวยความสะดวกในการวางแผนการเดินทางของนักท่องเที่ยวได้อย่างมีประสิทธิภาพ แนวคิดดังกล่าวสามารถนำมาประยุกต์ใช้กับโครงงานระบบออกแบบทริปการท่องเที่ยวด้วย AI เพื่อช่วยให้ผู้ใช้งานสามารถค้นหาข้อมูลสถานที่ท่องเที่ยวและวางแผนการเดินทางได้สะดวกยิ่งขึ้น

สุนิสา ทัศนะ และคณะ (2561) ได้ทำการวิจัยเรื่องการพัฒนาระบบสารสนเทศสำหรับแนะนำสถานที่ท่องเที่ยวยอดนิยมในภูมิภาคตะวันตกของประเทศไทย โดยมีวัตถุประสงค์เพื่อพัฒนาระบบสำหรับรวบรวมและแนะนำข้อมูลสถานที่ท่องเที่ยว รวมถึงศึกษาความพึงพอใจของผู้ใช้งานระบบ ระบบที่พัฒนาขึ้นได้รวบรวมข้อมูลสถานที่ท่องเที่ยวจาก 5 จังหวัดในภูมิภาคตะวันตก ได้แก่ จังหวัดตาก กาญจนบุรี ราชบุรี เพชรบุรี และประจวบคีรีขันธ์ โดยมีสถานที่ท่องเที่ยวรวมทั้งสิ้น 539 แห่ง และแบ่งประเภทสถานที่ท่องเที่ยวออกเป็น 13 ประเภท เช่น ธรรมชาติ ประวัติศาสตร์ นันทนาการ น้ำตก ถ้ำ เกาะ ร้านอาหาร และที่พัก

ระบบถูกพัฒนาในรูปแบบเว็บแอปพลิเคชัน โดยใช้ภาษา PHP และ HTML ร่วมกับระบบจัดการฐานข้อมูล MySQL และมีการเชื่อมต่อกับ Google Maps API เพื่อแสดงตำแหน่งของสถานที่ท่องเที่ยวบนแผนที่ดิจิทัล ทำให้ผู้ใช้งานสามารถค้นหาข้อมูลสถานที่ท่องเที่ยวและดูตำแหน่งของสถานที่ต่าง ๆ ได้อย่างสะดวก

จากการประเมินประสิทธิภาพของระบบโดยผู้ใช้งานจำนวน 25 คน พบว่าผู้ใช้งานมีความพึงพอใจต่อระบบในระดับมาก โดยมีค่าเฉลี่ยความพึงพอใจเท่ากับ 4.20 แสดงให้เห็นว่าระบบสารสนเทศที่พัฒนาขึ้นสามารถช่วยอำนวยความสะดวกในการค้นหาข้อมูลและแนะนำสถานที่ท่องเที่ยวให้กับผู้ใช้งานได้อย่างมีประสิทธิภาพ

แนวคิดดังกล่าวสามารถนำมาประยุกต์ใช้กับโครงงาน ระบบออกแบบทริปการท่องเที่ยวด้วย AI เพื่อช่วยรวบรวมข้อมูลสถานที่ท่องเที่ยวและใช้เป็นฐานข้อมูลสำหรับการวิเคราะห์และสร้างคำแนะนำแผนการเดินทางที่เหมาะสมสำหรับผู้ใช้งาน

ภัทร สูตรสุวรรณ และคณะ (2564) ได้ทำการศึกษาระบบแนะนำสถานที่ท่องเที่ยวในกรุงเทพฯ และปริมณฑลสำหรับนักท่องเที่ยวต่างชาติ โดยมีวัตถุประสงค์เพื่อพัฒนาระบบแนะนำสถานที่ท่องเที่ยวตามความสนใจของผู้ใช้งาน และเปรียบเทียบประสิทธิภาพของอัลกอริทึมที่ใช้ในการสร้างระบบแนะนำ ได้แก่ วิธีแนะนำตามความนิยม (Popularity-based), วิธีพิจารณาจากเนื้อหาของข้อมูล (Content-based), วิธีการกรองร่วมกัน (Collaborative filtering) และวิธีการแบบผสม (Hybrid model)

งานวิจัยนี้ใช้ข้อมูลประวัติการท่องเที่ยวและการให้คะแนนสถานที่ท่องเที่ยวของนักท่องเที่ยวต่างชาติจากเว็บไซต์ Tripadvisor ซึ่งเป็นเว็บไซต์รีวิวการท่องเที่ยวที่มีข้อมูลจำนวนมาก โดยนำข้อมูลดังกล่าวมาทำการวิเคราะห์และแบ่งออกเป็นข้อมูลสำหรับฝึกโมเดลและข้อมูลสำหรับทดสอบประสิทธิภาพของระบบ จากนั้นทำการพัฒนาระบบแนะนำสถานที่ท่องเที่ยวด้วยอัลกอริทึมต่าง ๆ และนำผลลัพธ์มาเปรียบเทียบกัน

ผลการศึกษาพบว่าวิธีการแบบผสม (Hybrid model) ซึ่งเป็นการรวมผลลัพธ์จากหลายอัลกอริทึมเข้าด้วยกันให้ผลลัพธ์ที่ดีที่สุด โดยมีค่าความแม่นยำในการแนะนำสถานที่ท่องเที่ยวสูงกว่าวิธีการอื่น ๆ แสดงให้เห็นว่าการใช้เทคนิคการรวมอัลกอริทึมสามารถช่วยเพิ่มประสิทธิภาพของระบบแนะนำสถานที่ท่องเที่ยวได้

2.3.1 การประยุกต์และสังเคราะห์งานวิจัยที่เกี่ยวข้องต่อการพัฒนาระบบออกแบบทริปการท่องเที่ยวด้วย AI

การทบทวนวรรณกรรมและงานวิจัยที่เกี่ยวข้องถือเป็นขั้นตอนสำคัญในการพัฒนาระบบสารสนเทศ เนื่องจากช่วยให้ผู้พัฒนาระบบสามารถเข้าใจแนวคิด เทคโนโลยี และแนวทางการออกแบบระบบที่มีอยู่ในปัจจุบัน เพื่อนำมาประยุกต์ใช้เป็นกรอบแนวคิดในการพัฒนาระบบใหม่ให้มีประสิทธิภาพมากยิ่งขึ้น สำหรับโครงงานระบบออกแบบทริปการท่องเที่ยวด้วย AI (AI-Powered Travel Trip Design System) ได้มีการศึกษางานวิจัยที่เกี่ยวข้องทั้งในด้านระบบแนะนำข้อมูล การพัฒนาโมบายแอปพลิเคชันด้านการท่องเที่ยว การใช้ระบบแผนที่ดิจิทัล และการประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ในการวิเคราะห์ข้อมูลผู้ใช้งาน ซึ่งองค์ความรู้จากงานวิจัยเหล่านี้สามารถนำมาบูรณาการร่วมกันเพื่อใช้ในการออกแบบโครงสร้างระบบและโมดูลการทำงานต่าง ๆ ของระบบได้อย่างเหมาะสม

งานวิจัยของ กิตติโชค คุ้มครอง และคณะ (2568) ที่ได้พัฒนาเว็บแอปพลิเคชันสำหรับแนะนำสถานที่ท่องเที่ยวด้วยปัญญาประดิษฐ์ร่วมกับระบบแผนที่ภูมิศาสตร์ ได้แสดงให้เห็นถึงบทบาทสำคัญของเทคโนโลยีปัญญาประดิษฐ์ในการวิเคราะห์ข้อมูลและสร้างคำแนะนำสำหรับผู้ใช้งาน โดยระบบดังกล่าวใช้เทคนิค Decision Tree ในการวิเคราะห์ข้อมูลความต้องการของผู้ใช้งาน และสามารถให้คำแนะนำสถานที่ท่องเที่ยวได้อย่างแม่นยำ แนวคิดดังกล่าวถูกนำมาประยุกต์ใช้ในการออกแบบโมดูลการวิเคราะห์ข้อมูลผู้ใช้งานของระบบในโครงงาน เพื่อช่วยให้ระบบสามารถวิเคราะห์ข้อมูลความสนใจของผู้ใช้งานและนำข้อมูลดังกล่าวมาใช้ในการสร้างแผนการเดินทางที่เหมาะสมกับผู้ใช้งานแต่ละบุคคลได้อย่างมีประสิทธิภาพ

ในด้านการให้บริการข้อมูลการท่องเที่ยวผ่านเทคโนโลยีปัญญาประดิษฐ์ งานวิจัยของ สุมนา บุษบก และคณะ (2568) ที่ได้พัฒนาแชทบอทเพื่อแนะนำข้อมูลการท่องเที่ยวในจังหวัดภูเก็ต ได้แสดงให้เห็นว่าการนำเทคโนโลยี AI มาประยุกต์ใช้สามารถช่วยเพิ่มประสิทธิภาพในการให้บริการข้อมูลแก่นักท่องเที่ยวได้อย่างมาก ระบบดังกล่าวสามารถให้ข้อมูลเกี่ยวกับสถานที่ท่องเที่ยว ร้านอาหาร และที่พัก รวมถึงสามารถเชื่อมต่อกับระบบแผนที่เพื่อแสดงตำแหน่งของสถานที่ต่าง ๆ ได้อย่างสะดวก แนวคิดจากงานวิจัยนี้จึงถูกนำมาใช้เป็นแนวทางในการออกแบบระบบแนะนำข้อมูลภายในแอปพลิเค

ชันของโครงงาน เพื่อช่วยให้ผู้ใช้งานสามารถเข้าถึงข้อมูลสถานที่ท่องเที่ยวได้อย่างรวดเร็วและสามารถใช้ข้อมูลดังกล่าวในการวางแผนการเดินทางได้อย่างเหมาะสม

สำหรับการพัฒนาโมบายแอปพลิเคชันด้านการท่องเที่ยว งานวิจัยของ กมลลักษณ์ ชัยดี และคณะ (2559) ได้แสดงให้เห็นถึงแนวทางในการพัฒนาแอปพลิเคชันบนสมาร์ตโฟนเพื่อใช้ในการนำเสนอข้อมูลสถานที่ท่องเที่ยวและอำนวยความสะดวกในการเดินทางของนักท่องเที่ยว โดยระบบสามารถแสดงข้อมูลสถานที่ท่องเที่ยวในรูปแบบข้อความ ภาพ และมัลติมีเดีย รวมถึงสามารถเชื่อมต่อกับ Google Maps และระบบ GPS เพื่อแสดงตำแหน่งของสถานที่และแนะนำเส้นทางการเดินทาง แนวคิดดังกล่าวถูกนำมาประยุกต์ใช้ในการออกแบบส่วนติดต่อผู้ใช้ของโมบายแอปพลิเคชันในโครงงาน เพื่อให้ผู้ใช้งานสามารถค้นหาข้อมูลสถานที่ท่องเที่ยว ดูตำแหน่งของสถานที่ และวางแผนเส้นทางการเดินทางผ่านแผนที่ดิจิทัลได้อย่างสะดวก

นอกจากนี้ งานวิจัยของ สุนิสา ทัศนะ และคณะ (2561) ที่ได้พัฒนาระบบสารสนเทศสำหรับแนะนำสถานที่ท่องเที่ยวยอดนิยมในภูมิภาคตะวันตกของประเทศไทย ยังแสดงให้เห็นถึงความสำคัญของการจัดการฐานข้อมูลสถานที่ท่องเที่ยวและการใช้ระบบแผนที่ดิจิทัลในการแสดงตำแหน่งของสถานที่ต่าง ๆ ระบบดังกล่าวได้รวบรวมข้อมูลสถานที่ท่องเที่ยวจำนวนมากและแบ่งประเภทของสถานที่ท่องเที่ยวออกเป็นหมวดหมู่ที่ชัดเจน ซึ่งแนวคิดนี้สามารถนำมาประยุกต์ใช้ในการออกแบบโครงสร้างฐานข้อมูลของโครงงาน เพื่อให้สามารถจัดเก็บข้อมูลสถานที่ท่องเที่ยวอย่างเป็นระบบ และสามารถนำข้อมูลเหล่านี้มาใช้ในการวิเคราะห์และสร้างคำแนะนำแผนการเดินทางได้อย่างมีประสิทธิภาพ

ในด้านระบบแนะนำข้อมูล (Recommendation System) งานวิจัยของ ภัทร สูตรสุวรรณ และคณะ (2564) ได้ศึกษาการพัฒนาระบบแนะนำสถานที่ท่องเที่ยวโดยใช้เทคนิคการวิเคราะห์ข้อมูลหลายรูปแบบ เช่น Popularity-based, Content-based, Collaborative filtering และ Hybrid model โดยผลการศึกษาพบว่าวิธีการแบบ Hybrid model ซึ่งเป็นการรวมผลลัพธ์จากหลายอัลกอริทึมเข้าด้วยกันสามารถให้ผลลัพธ์ที่มีความแม่นยำสูงกว่าวิธีการอื่น แนวคิดดังกล่าวจึงถูกนำมาใช้เป็นแนวทางในการออกแบบโมดูลระบบแนะนำแผนการเดินทางของโครงงาน เพื่อช่วยให้ระบบสามารถวิเคราะห์ข้อมูลความสนใจของผู้ใช้งานและสร้างคำแนะนำสถานที่ท่องเที่ยวที่เหมาะสมกับผู้ใช้งานแต่ละคนได้อย่างมีประสิทธิภาพ

จากการสังเคราะห์งานวิจัยที่เกี่ยวข้องทั้งหมด พบว่าเทคโนโลยีปัญญาประดิษฐ์ ระบบแนะนำข้อมูล ระบบสารสนเทศภูมิศาสตร์ และโมบายแอปพลิเคชัน มีบทบาทสำคัญในการพัฒนาระบบสารสนเทศด้านการท่องเที่ยว โดยองค์ความรู้จากงานวิจัยเหล่านี้สามารถนำมาบูรณาการร่วมกันเพื่อใช้ในการออกแบบสถาปัตยกรรมของระบบออกแบบทริปการท่องเที่ยวด้วย AI ซึ่งประกอบด้วยโมดูลการจัดการข้อมูลสถานที่ท่องเที่ยว โมดูลการวิเคราะห์ความสนใจของผู้ใช้งาน

โมดูลการแนะนำสถานที่ท่องเที่ยว และโมดูลการแสดง เส้นทางการเดินทางผ่านแผนที่ดิจิทัล การนำแนวคิดจากงานวิจัยเหล่านี้มาประยุกต์ใช้จึงช่วยให้ระบบที่พัฒนาขึ้นสามารถตอบสนองต่อความต้องการของผู้ใช้งานได้อย่างมีประสิทธิภาพ และช่วยให้ผู้ใช้งานสามารถวางแผนการเดินทางได้สะดวกและเหมาะสมกับความต้องการของตนเองมากยิ่งขึ้น

2.4 การสังเคราะห์งานที่เกี่ยวข้องและช่องว่างของโครงงาน

องค์ประกอบ	กิตติโชค (2568)	สุมนา (2568)	กมลลักษณ์ (2559)	สุนิสา (2561)	ภัทร (2564)	ช่องว่างที่พบ
ระบบแนะนำสถานที่ท่องเที่ยว	✓	✓	✓	✓	✓	ยังไม่มีระบบที่รวม “แนะนำ + วางแผนทริปครบ”
การใช้ AI	✓	✓	–	✓	–	ยังไม่มี AI ที่สร้างแผนทริปอัตโนมัติแบบครบวงจร
Recommendation System เชิงลึก	–	✓	–	✓	–	ยังไม่มีการผสานหลายเทคนิค (Hybrid + Context)
การวิเคราะห์ผู้ใช้งาน	–	✓	✓	✓	–	ยังไม่วิเคราะห์พฤติกรรมแบบ Real-time
Route Optimization (จัดเส้นทาง)	–	–	–	–	–	ยังไม่มีระบบจัดลำดับเส้นทางอัตโนมัติ
การใช้ GIS / แผนที่	✓	✓	✓	–	✓	ยังไม่มีการใช้ร่วมกับ AI อย่างเต็มรูปแบบ
Mobile Application	–	–	✓	–	–	ยังไม่มี Mobile + AI + Route ครบระบบ
Web	-	✓	–	✓	✓	ยังไม่มีระบบที่รวมหลาย Platform
การสร้างแผนทริปอัตโนมัติ	✓	–	–	–	–	ยังไม่มีระบบที่สร้าง “Trip Plan อัจฉริยะ”

บทที่ 3
วิธีการดำเนินโครงงาน

3.1 การวิเคราะห์ความต้องการของระบบ

การวิเคราะห์ความต้องการของระบบ (System Requirement Analysis) เป็นขั้นตอนสำคัญในกระบวนการพัฒนาระบบสารสนเทศ เนื่องจากเป็นขั้นตอนที่ใช้ในการศึกษาปัญหา ความต้องการ และลักษณะการใช้งานของผู้ใช้ เพื่อกำหนดขอบเขตและความสามารถของระบบให้มีความชัดเจน การวิเคราะห์ความต้องการของระบบช่วยให้ผู้พัฒนาสามารถออกแบบระบบที่ตอบสนองต่อความต้องการของผู้ใช้งานได้อย่างเหมาะสม

ในปัจจุบันการวางแผนการท่องเที่ยวจำเป็นต้องค้นหาข้อมูลจากหลายแหล่ง เช่น ข้อมูลสถานที่ท่องเที่ยว แผนที่ รูปภาพ รายละเอียดของสถานที่ ตลอดจนการจัดลำดับกิจกรรมในแต่ละวัน ส่งผลให้ผู้ใช้งานต้องใช้เวลาในการค้นหาและรวบรวมข้อมูลจากหลายแหล่งก่อนที่จะสามารถจัดทำแผนการเดินทางได้ โดยเฉพาะในกรณีที่ต้องการเดินทางไปยังหลายสถานที่ภายในระยะเวลาที่กำหนด

จากปัญหาดังกล่าวจึงเกิดแนวคิดในการพัฒนาระบบออกแบบทริปการท่องเที่ยวด้วยปัญญาประดิษฐ์ (AI-Powered Travel Trip Design System) เพื่อช่วยสนับสนุนกระบวนการวางแผนการเดินทางให้มีความสะดวกและรวดเร็วยิ่งขึ้น โดยระบบสามารถรับข้อมูลความต้องการของผู้ใช้งาน เช่น จุดหมายปลายทาง จำนวนวัน งบประมาณ จำนวนผู้เดินทาง และรูปแบบการท่องเที่ยวที่สนใจ

ผู้ใช้งานสามารถสมัครสมาชิกและเข้าสู่ระบบเพื่อใช้งานระบบ กำหนดรายละเอียดที่เกี่ยวข้องกับการเดินทาง และสร้างแผนการเดินทางผ่านระบบปัญญาประดิษฐ์ เมื่อระบบสร้างแผนการเดินทางเรียบร้อยแล้ว ผู้ใช้งานสามารถดูรายละเอียดกิจกรรมและสถานที่ในแต่ละวัน รวมถึงดูตำแหน่งของสถานที่บนแผนที่และรูปภาพประกอบของสถานที่ เพื่อช่วยให้สามารถทำความเข้าใจแผนการเดินทางได้ง่ายขึ้น

นอกจากนี้ ระบบยังรองรับการบันทึกแผนการเดินทางไว้ในบัญชีของผู้ใช้งาน ทำให้ผู้ใช้งานสามารถกลับมาเรียกดูแผนการเดินทางที่เคยบันทึกไว้ได้ในภายหลัง รวมถึงสามารถลบข้อมูลแผนการเดินทางที่ไม่ต้องการออกจากระบบได้ โดยข้อมูลที่เกี่ยวข้องกับผู้ใช้งานและแผนการเดินทางจะถูกจัดเก็บไว้ในฐานข้อมูลบนระบบ Cloud

ระบบยังมีการเชื่อมต่อกับบริการภายนอกเพื่อสนับสนุนการทำงานในด้านต่าง ๆ ได้แก่ บริการปัญญาประดิษฐ์สำหรับสร้างแผนการเดินทาง บริการ Geocoding สำหรับค้นหาค่าพิกัดของสถานที่ บริการแผนที่สำหรับแสดงตำแหน่งทางภูมิศาสตร์ และบริการค้นหารูปภาพสำหรับนำรูปภาพที่เกี่ยวข้องกับสถานที่มาใช้ประกอบการแสดงผลภายในระบบ

3.2 สถาปัตยกรรมของระบบ

สถาปัตยกรรมของระบบ (System Architecture) เป็นโครงสร้างหลักที่ใช้ในการกำหนดรูปแบบการทำงานของระบบโดยรวม โดยแสดงถึงองค์ประกอบต่าง ๆ ของระบบ และความสัมพันธ์ระหว่างองค์ประกอบเหล่านั้น เพื่ออธิบายให้เห็นว่าระบบมีการรับข้อมูล ประมวลผล จัดเก็บข้อมูล และแสดงผลอย่างไร การออกแบบสถาปัตยกรรมของระบบที่เหมาะสมจะช่วยให้ระบบสามารถทำงานได้อย่างมีประสิทธิภาพ รองรับการขยายตัวในอนาคต และช่วยให้การพัฒนาและบำรุงรักษาระบบทำได้ง่ายขึ้น

โครงงานนี้ใช้สถาปัตยกรรมแบบ **Client–Server Architecture** โดยแบ่งองค์ประกอบหลักออกเป็น 3 ส่วน ได้แก่ **ส่วนผู้ใช้งาน (User)**, **ส่วนโปรแกรมประยุกต์ฝั่งผู้ใช้ (Client Side)** และ **ส่วนบริการเบื้องหลังและบริการภายนอก (Backend Services and External APIs)** ผู้ใช้งานสามารถเข้าถึงระบบผ่านเว็บแอปพลิเคชันที่พัฒนาด้วย Flutter และใช้งานผ่านเว็บเบราว์เซอร์ โดยระบบจะทำหน้าที่รับข้อมูลที่ผู้ใช้งานกรอก ส่งข้อมูลไปประมวลผลกับบริการที่เกี่ยวข้อง และแสดงผลลัพธ์กลับมายังหน้าจอของผู้ใช้งาน

ในส่วนของ **Client Side** ระบบถูกพัฒนาในรูปแบบ **Web Application** ด้วยเทคโนโลยี **Flutter** ซึ่งทำหน้าที่เป็นส่วนติดต่อผู้ใช้งานสำหรับการสมัครสมาชิก การเข้าสู่ระบบ การกรอกข้อมูลสำหรับสร้างแผนการเดินทาง การแสดงผลแผนการเดินทาง การแสดงตำแหน่งสถานที่บนแผนที่ และการแสดงรูปภาพประกอบสถานที่ท่องเที่ยว โดยฝั่ง Client จะเป็นตัวกลางในการรับข้อมูลจากผู้ใช้งานและส่งคำขอไปยังบริการต่าง ๆ ผ่านเครือข่ายอินเทอร์เน็ต

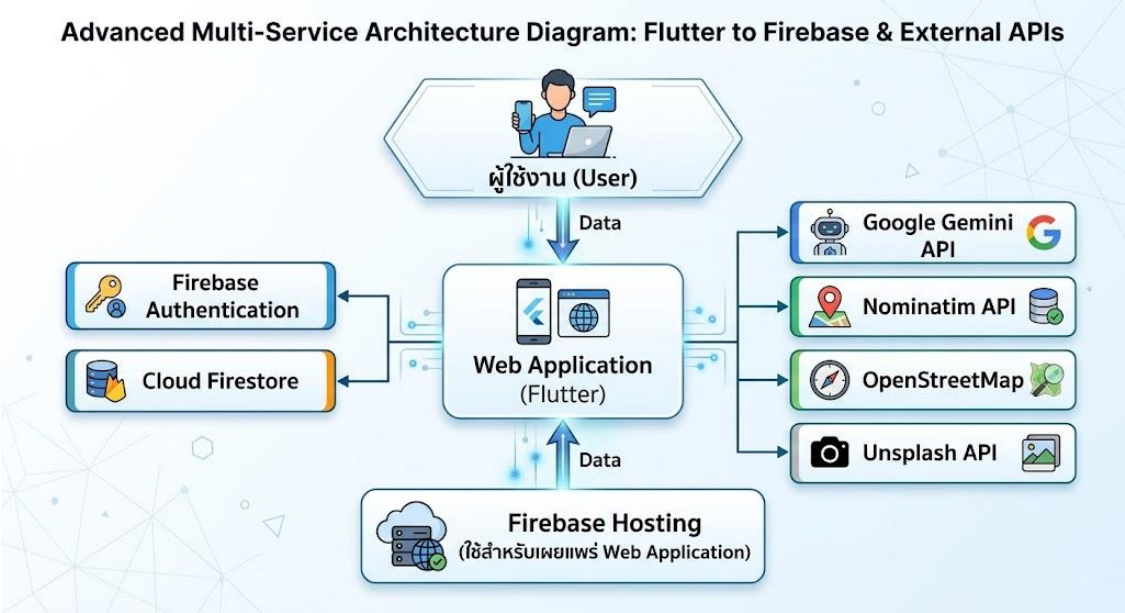
ในส่วนของ **Backend Services** ระบบใช้บริการจาก **Firebase Platform** เป็นโครงสร้างสนับสนุนการทำงานของระบบ โดยประกอบด้วย **Firebase Authentication** สำหรับจัดการการสมัครสมาชิกและการเข้าสู่ระบบของผู้ใช้งาน, **Cloud Firestore** สำหรับจัดเก็บข้อมูลผู้ใช้งานและข้อมูลแผนการเดินทางที่ถูกบันทึกไว้ และ **Firebase Hosting** สำหรับเผยแพร่และให้บริการเว็บแอปพลิเคชันผ่านอินเทอร์เน็ต

นอกจากนี้ ระบบยังเชื่อมต่อกับ **External APIs** เพื่อสนับสนุนการทำงานเฉพาะด้าน โดย **Google Gemini API** ทำหน้าที่ประมวลผลข้อมูลที่ผู้ใช้งานระบุและสร้างแผนการเดินทางด้วยปัญญาประดิษฐ์, **Nominatim API** ใช้สำหรับค้นหาค่าพิกัดละติจูดและลองจิจูดของสถานที่จากชื่อสถานที่, **OpenStreetMap** ใช้เป็นแหล่งข้อมูลแผนที่สำหรับแสดงตำแหน่งของสถานที่ และ **Unsplash API** ใช้สำหรับค้นหารูปภาพสถานที่ที่เกี่ยวข้องเพื่อนำมาแสดงประกอบผลลัพธ์ภายในระบบ

ลำดับการทำงานของระบบเริ่มต้นจากผู้ใช้งานกรอกข้อมูลความต้องการเกี่ยวกับการเดินทางผ่านเว็บแอปพลิเคชัน จากนั้นระบบจะนำข้อมูลดังกล่าวไปสร้างคำสั่งหรือ Prompt และส่งต่อไป

ยัง Google Gemini API เพื่อสร้างแผนการเดินทาง เมื่อได้รับผลลัพธ์แล้ว ระบบจะนำชื่อสถานที่ที่อยู่ในแผนไปค้นหาค่าพิกัดผ่าน Nominatim API และนำพิกัดที่ได้ไปแสดงผลบนแผนที่ OpenStreetMap พร้อมทั้งค้นหารูปภาพสถานที่ผ่าน Unsplash API เพื่อใช้ประกอบการแสดงผล หากผู้ใช้งานต้องการบันทึกแผนการเดินทาง ระบบจะจัดเก็บข้อมูลดังกล่าวไว้ใน Cloud Firestore ภายใต้บัญชีของผู้ใช้งานที่ผ่านการยืนยันตัวตนด้วย Firebase Authentication

จากสถาปัตยกรรมของระบบดังกล่าว ทำให้ระบบสามารถทำงานร่วมกันระหว่างส่วนติดต่อผู้ใช้งาน ระบบจัดการบัญชีผู้ใช้ ฐานข้อมูลบน Cloud และบริการภายนอกด้าน AI แผนที่ และรูปภาพได้อย่างเป็นระบบ ส่งผลให้ผู้ใช้งานสามารถใช้งานระบบวางแผนการเดินทางผ่านเว็บแอปพลิเคชันได้อย่างสะดวก รวดเร็ว และมีประสิทธิภาพ



ภาพที่ 3.1 สถาปัตยกรรมของระบบ

3.3 เครื่องมือและเทคโนโลยีที่ใช้

ในการพัฒนาระบบออกแบบทริปการท่องเที่ยวด้วย AI มีการนำเทคโนโลยีและเครื่องมือหลายประเภทมาใช้ร่วมกัน เพื่อสนับสนุนการทำงานตั้งแต่การพัฒนาส่วนติดต่อผู้ใช้งาน การประมวลผลด้วยปัญญาประดิษฐ์ การแสดงข้อมูลสถานที่บนแผนที่ การจัดเก็บข้อมูลบนระบบ Cloud ตลอดจนการค้นหารูปภาพประกอบสถานที่ โดยเทคโนโลยีและเครื่องมือที่นำมาใช้มีดังต่อไปนี้

3.3.1 Flutter and Dart

Flutter เป็น Framework สำหรับพัฒนาแอปพลิเคชันแบบข้ามแพลตฟอร์ม (Cross-Platform) ที่พัฒนาโดย Google และใช้ภาษา Dart เป็นภาษาโปรแกรมหลักในการพัฒนา โดยสามารถนำ Source Code ชุดเดียวกันไปใช้พัฒนาแอปพลิเคชันสำหรับหลายแพลตฟอร์ม เช่น Web, Android และ iOS ได้

ในโครงงานนี้ Flutter และ Dart ถูกนำมาใช้เป็นเทคโนโลยีหลักในการพัฒนา Web Application โดยใช้สำหรับสร้างส่วนติดต่อผู้ใช้งาน การจัดวางองค์ประกอบของหน้าจอ การรับข้อมูลความต้องการในการเดินทาง การแสดงแผนการเดินทาง การจัดการ Navigation และการเชื่อมต่อกับบริการภายนอกต่าง ๆ ผ่าน API

3.3.2 OpenStreetMap and Geocoding

OpenStreetMap (OSM) เป็นแหล่งข้อมูลแผนที่แบบเปิดที่สามารถนำข้อมูลไปประยุกต์ใช้ร่วมกับระบบหรือแอปพลิเคชันได้ ส่วน Geocoding เป็นกระบวนการแปลงข้อมูลชื่อสถานที่หรือที่อยู่ให้เป็นค่าพิกัด Latitude และ Longitude เพื่อนำไปใช้ระบุตำแหน่งบนแผนที่

ในโครงงานนี้ OpenStreetMap ถูกนำมาใช้สำหรับแสดงข้อมูลแผนที่ภายใน Application โดยใช้ร่วมกับ flutter_map สำหรับแสดงผลแผนที่ และใช้ Nominatim API สำหรับค้นหาค่าพิกัดของสถานที่จากชื่อสถานที่ที่ปรากฏในแผนการเดินทาง จากนั้นนำ Latitude และ Longitude ที่ได้รับมาใช้แสดงตำแหน่งของสถานที่บนแผนที่

3.3.3 Google Gemini API

Google Gemini API เป็นบริการปัญญาประดิษฐ์ที่เปิดให้นักพัฒนาสามารถนำความสามารถของโมเดล Gemini มาเชื่อมต่อกับ Application ผ่าน API โดยสามารถรับคำสั่งหรือข้อมูลในรูป Prompt และสร้างผลลัพธ์กลับมาในรูปแบบข้อความหรือข้อมูลที่มีโครงสร้าง

ในโครงงานนี้ Google Gemini API ถูกนำมาใช้เป็นส่วนหลักในการสร้างแผนการเดินทาง โดยระบบจะนำข้อมูลที่ผู้ใช้งานกำหนด เช่น จุดหมายปลายทาง จำนวนวัน งบประมาณ จำนวนผู้เดินทาง และรูปแบบการท่องเที่ยว มาสร้างเป็น Prompt และส่งให้ Gemini ประมวลผล จากนั้นนำผลลัพธ์ที่ได้รับมาจัดรูปแบบและแสดงเป็นแผนการเดินทางให้แก่ผู้ใช้งาน

3.3.4 Firebase Platform

Firebase เป็น Platform สำหรับสนับสนุนการพัฒนา Web และ Mobile Application โดยมีบริการหลายประเภทที่สามารถนำมาใช้งานร่วมกัน เช่น ระบบยืนยันตัวตน ฐานข้อมูลบน Cloud และบริการ Hosting

ในโครงงานนี้มีการใช้ **Firebase Authentication** สำหรับจัดการการสมัครสมาชิกและการเข้าสู่ระบบด้วย Email/Password และ Google Sign-In ใช้ **Cloud Firestore** สำหรับจัดเก็บข้อมูลที่เกี่ยวข้องกับผู้ใช้งานและแผนการเดินทางที่บันทึกไว้ และใช้ **Firebase Hosting** สำหรับเผยแพร่ Web Application เพื่อให้ผู้ใช้งานสามารถเข้าถึงระบบผ่านเว็บเบราว์เซอร์ได้

3.3.5 Unsplash API

Unsplash API เป็นบริการสำหรับค้นหาและเข้าถึงข้อมูลรูปภาพจาก Unsplash ผ่าน API โดยสามารถส่ง Keyword เพื่อค้นหารูปภาพที่มีเนื้อหาสัมพันธ์กับคำค้น และนำข้อมูลหรือ URL ของรูปภาพที่ได้รับกลับมาแสดงภายใน Application ได้

ในโครงงานนี้ Unsplash API ถูกนำมาใช้สำหรับค้นหารูปภาพที่เกี่ยวข้องกับสถานที่ในแผนการเดินทาง โดยระบบสามารถนำชื่อสถานที่หรือข้อมูลที่เกี่ยวข้องมาใช้เป็นคำค้น จากนั้นรับ URL ของรูปภาพที่ได้จาก Unsplash และนำมาแสดงเป็นรูปภาพประกอบสถานที่ภายใน Web Application เพื่อช่วยให้ผู้ใช้งานสามารถเห็นภาพของสถานที่และเข้าใจข้อมูลแผนการเดินทางได้ชัดเจนมากยิ่งขึ้น

3.4 การออกแบบระบบ

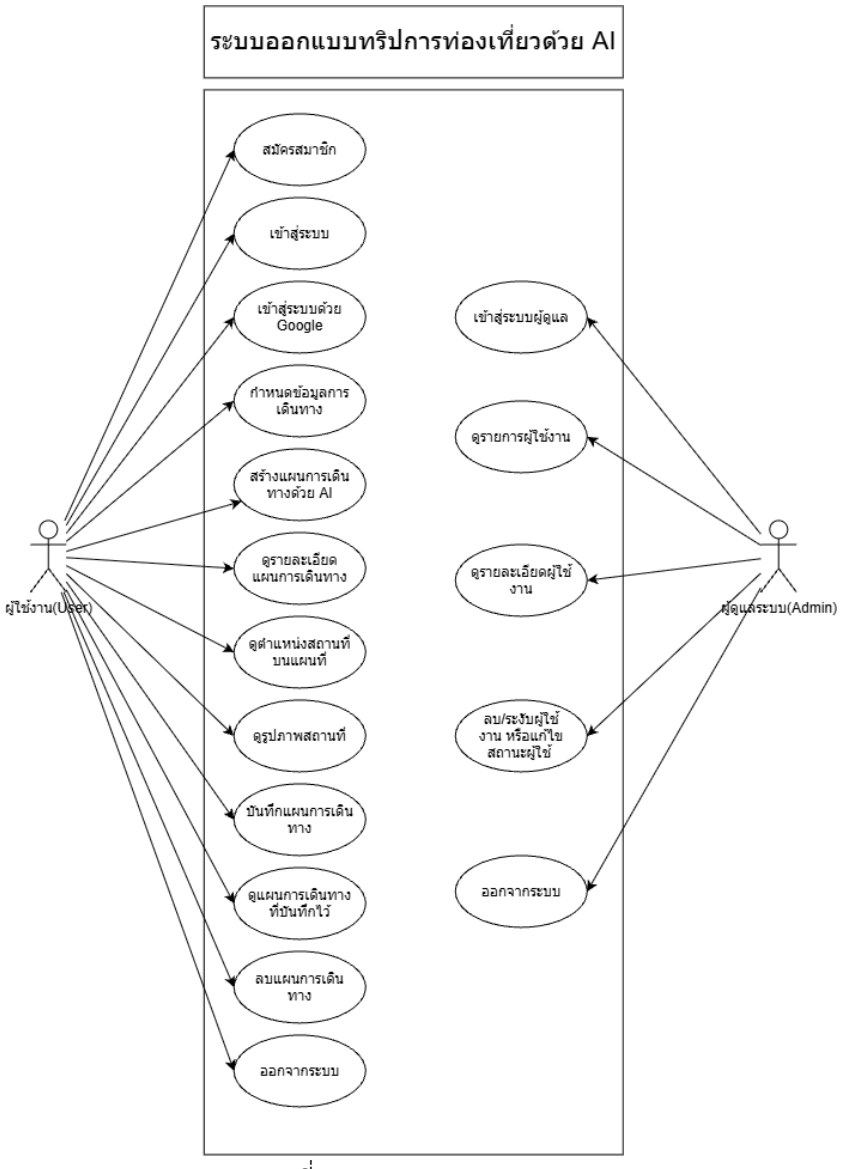
การออกแบบระบบเป็นขั้นตอนสำคัญในการพัฒนาระบบสารสนเทศ โดยมีวัตถุประสงค์เพื่อกำหนดโครงสร้างและรูปแบบการทำงานของระบบให้มีความชัดเจนก่อนการพัฒนาโปรแกรมจริง การออกแบบระบบช่วยให้สามารถมองเห็นภาพรวมของการทำงานของระบบ รวมถึงความสัมพันธ์ระหว่างองค์ประกอบต่าง ๆ ของระบบ ซึ่งช่วยลดความผิดพลาดในขั้นตอนการพัฒนาและทำให้ระบบสามารถทำงานได้อย่างมีประสิทธิภาพ

ในการพัฒนาระบบออกแบบทริปการท่องเที่ยวด้วย AI ผู้พัฒนาได้ทำการออกแบบระบบในหลายส่วน ได้แก่ การออกแบบ Use Case เพื่อแสดงการทำงานของระบบและความสัมพันธ์ระหว่างผู้ใช้งานกับระบบ การออกแบบโครงสร้างฐานข้อมูลเพื่อจัดเก็บข้อมูลที่เกี่ยวข้องกับระบบ และการออกแบบส่วนติดต่อผู้ใช้เพื่อให้ผู้ใช้งานสามารถใช้งานระบบได้อย่างสะดวกและเข้าใจง่าย โดยรายละเอียดของการออกแบบระบบมีดังนี้

3.4.1 การออกแบบ Use Case

Use Case Diagram เป็นแผนภาพที่ใช้แสดงความสัมพันธ์ระหว่างผู้ใช้งาน (Actor) กับฟังก์ชันการทำงานของระบบ โดยช่วยให้สามารถมองเห็นภาพรวมของการทำงานของระบบได้อย่างชัดเจน ในระบบออกแบบทริปการท่องเที่ยวด้วย AI ผู้ใช้งานของระบบแบ่งออกเป็น 2 ประเภท ได้แก่ ผู้ใช้งานทั่วไป (User) และผู้ดูแลระบบ (Admin)

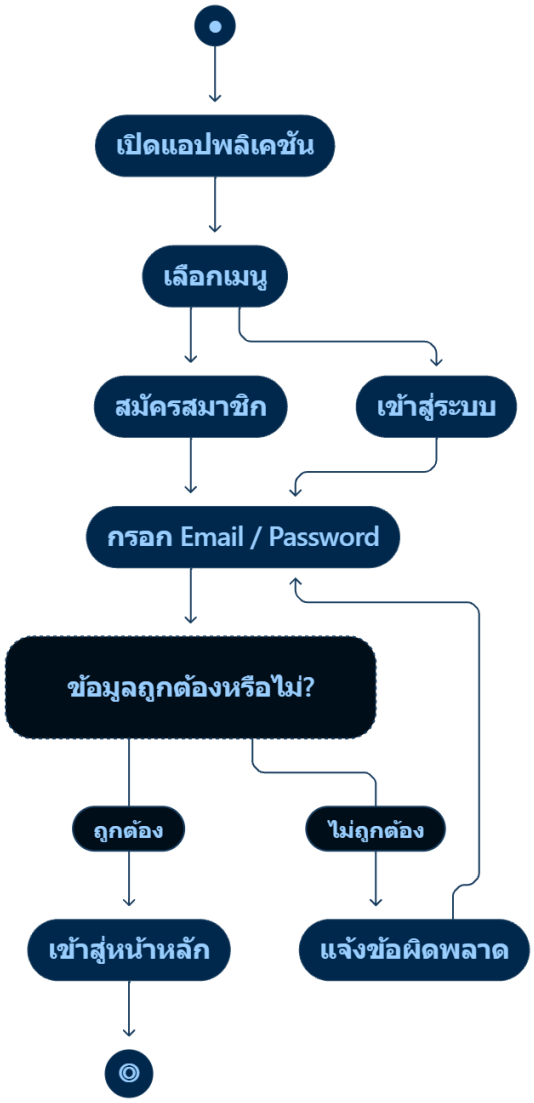
ผู้ใช้งานทั่วไปสามารถใช้งานโมบายแอปพลิเคชันเพื่อค้นหาสถานที่ท่องเที่ยว ดูรายละเอียดของสถานที่ วางแผนการเดินทาง และดูเส้นทางการเดินทางผ่านแผนที่ดิจิทัล ส่วนผู้ดูแลระบบสามารถจัดการข้อมูลสถานที่ท่องเที่ยว เช่น การเพิ่ม แก้ไข หรือลบข้อมูลสถานที่ รวมถึงการจัดการหมวดหมู่ของสถานที่ภายในระบบ



ภาพที่ 3.1 Use Case Diagram

3.4.2 การออกแบบแผนภาพกิจกรรม (Activity Diagram)

1) Activity Diagram: การสมัครสมาชิกและเข้าสู่ระบบ



ภาพที่ 3.2 Activity Diagram: การสมัครสมาชิกและเข้าสู่ระบบ

จากภาพที่ 3.2 กระบวนการสมัครสมาชิกและเข้าสู่ระบบเริ่มต้นจากผู้ใช้งานเปิดเว็บแอปพลิเคชัน จากนั้นเลือกว่าจะสมัครสมาชิกหรือเข้าสู่ระบบ หากผู้ใช้งานเลือกสมัครสมาชิกหรือเข้าสู่ระบบ ระบบจะให้กรอกข้อมูล Email และ Password เพื่อใช้ในการยืนยันตัวตน เมื่อผู้ใช้งานกรอกข้อมูลเรียบร้อยแล้ว ระบบจะทำการตรวจสอบความถูกต้องของข้อมูล หากข้อมูลถูกต้อง ระบบจะนำผู้ใช้งานเข้าสู่หน้าหลักของแอปพลิเคชัน แต่หากข้อมูลไม่ถูกต้อง ระบบจะแสดงข้อความแจ้งข้อผิดพลาดและให้ผู้ใช้งานกรอกข้อมูลใหม่อีกครั้งจนกว่าจะสามารถเข้าสู่ระบบได้สำเร็จ

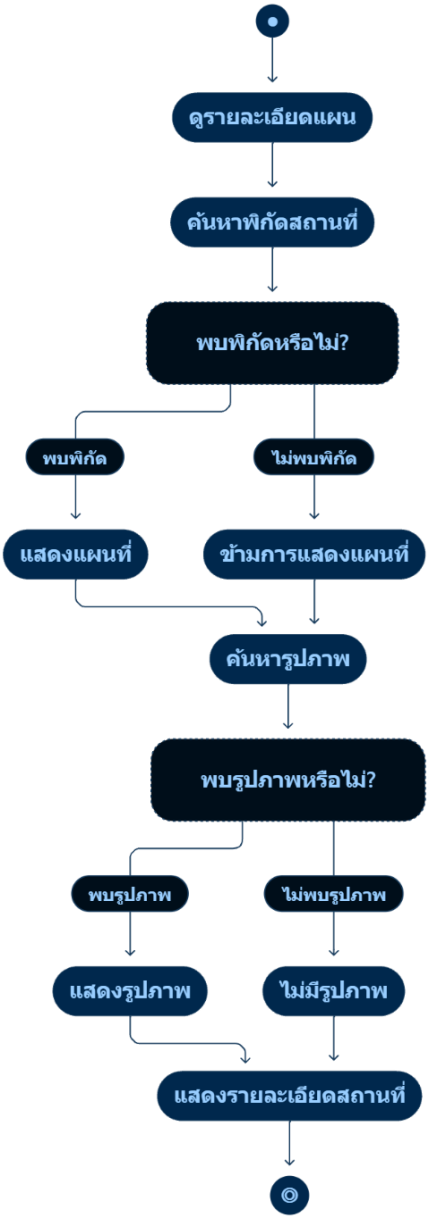
2) Activity Diagram: การสร้างแผนการเดินทางด้วย AI



ภาพที่ 3.3 Activity Diagram: การสร้างแผนการเดินทางด้วย AI

จากภาพที่ 3.3 กระบวนการสร้างแผนการเดินทางด้วย AI เริ่มต้นจากผู้ใช้งานเข้าสู่หน้าสำหรับสร้างแผนการเดินทาง จากนั้นผู้ใช้งานกรอกข้อมูลที่เกี่ยวข้องกับการเดินทางตามที่ระบบกำหนด เมื่อกรอกข้อมูลเรียบร้อยแล้ว ระบบจะตรวจสอบความครบถ้วนของข้อมูล หากพบว่าข้อมูลไม่ครบถ้วน ระบบจะแสดงข้อความแจ้งเตือนและให้ผู้ใช้งานกรอกข้อมูลใหม่ แต่หากข้อมูลครบถ้วน ระบบจะสร้างคำสั่งจากข้อมูลที่ผู้ใช้งานกรอกและส่งไปยัง Gemini API เพื่อประมวลผล หลังจาก Gemini API ประมวลผลเสร็จแล้ว ระบบจะรับผลลัพธ์ที่ได้มาจัดรูปแบบให้อยู่ในลักษณะที่เหมาะสมและแสดงแผนการเดินทางให้ผู้ใช้งานสามารถดูรายละเอียดได้

3) Activity Diagram: การแสดงตำแหน่งสถานที่และรูปภาพ



ภาพที่ 3.4 Activity Diagram: การแสดงตำแหน่งสถานที่และรูปภาพ

จากภาพที่ 3.4 กระบวนการแสดงตำแหน่งสถานที่และรูปภาพเริ่มต้นจากผู้ใช้งานเลือกดูรายละเอียดของแผนการเดินทาง จากนั้นระบบจะค้นหาพิกัดของสถานที่ที่เกี่ยวข้อง หากระบบสามารถค้นหาพิกัดได้ จะนำข้อมูลพิกัดมาใช้ในการแสดงตำแหน่งของสถานที่บนแผนที่ แต่หากไม่พบพิกัด ระบบจะข้ามขั้นตอนการแสดงแผนที่และดำเนินการในขั้นตอนถัดไป หลังจากนั้นระบบจะค้นหารูปภาพของสถานที่ หากพบรูปภาพ ระบบจะแสดงรูปภาพประกอบร่วมกับข้อมูลสถานที่ แต่หากไม่พบรูปภาพ ระบบจะแสดงเฉพาะรายละเอียดของสถานที่ สุดท้ายระบบจะแสดงข้อมูลรายละเอียดของสถานที่ให้ผู้ใช้งานสามารถดูได้อย่างครบถ้วนตามข้อมูลที่สามารถค้นหาได้

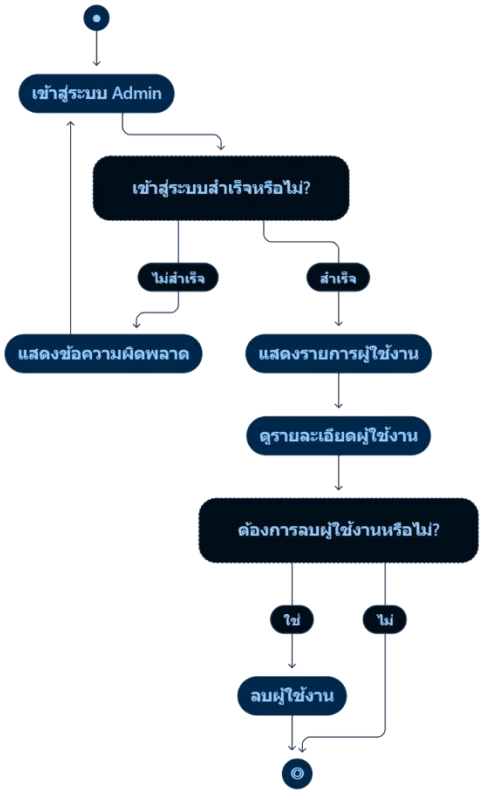
4) Activity Diagram: การบันทึกและลบแผนการเดินทาง



ภาพที่ 3.5 Activity Diagram: การบันทึกและลบแผนการเดินทาง

จากภาพที่ 3.5 กระบวนการบันทึกและลบแผนการเดินทางเริ่มต้นจากผู้ใช้งานเปิดดูแผนการเดินทางและเลือกบันทึกแผนการเดินทาง จากนั้นระบบจะตรวจสอบว่าผู้ใช้งานได้เข้าสู่ระบบแล้วหรือไม่ หากผู้ใช้งานยังไม่ได้เข้าสู่ระบบ ระบบจะแจ้งให้เข้าสู่ระบบก่อนจึงจะสามารถบันทึกข้อมูลได้ แต่หากเข้าสู่ระบบแล้ว ระบบจะบันทึกข้อมูลแผนการเดินทางลงใน Cloud Firestore เมื่อบันทึกข้อมูลเรียบร้อยแล้ว ผู้ใช้งานสามารถเปิดดูรายการแผนการเดินทางที่บันทึกไว้ และเลือกแผนการเดินทางที่ต้องการลบได้ เมื่อผู้ใช้งานเลือกที่จะลบ ระบบจะแสดงข้อความเพื่อยืนยันการลบข้อมูล หากผู้ใช้งานยืนยัน ระบบจะลบข้อมูลแผนการเดินทางออกจาก Cloud Firestore แต่หากผู้ใช้งานยกเลิก ระบบจะกลับไปยังหน้ารายการแผนการเดินทางที่บันทึกไว้ตามเดิม

5) Activity Diagram: การจัดการผู้ใช้งานของผู้ดูแลระบบ



ภาพที่ 3.6 Activity Diagram: การจัดการผู้ใช้งานของผู้ดูแลระบบ

จากภาพที่ 3.6 กระบวนการจัดการผู้ใช้งานของผู้ดูแลระบบเริ่มต้นจากผู้ดูแลระบบเข้าสู่หน้า Admin Web และทำการเข้าสู่ระบบ จากนั้นระบบจะตรวจสอบข้อมูลการเข้าสู่ระบบว่าถูกต้องหรือไม่ หากข้อมูลถูกต้อง ระบบจะแสดงรายการผู้ใช้งานทั้งหมดที่อยู่ในระบบ เพื่อให้ผู้ดูแลระบบสามารถเลือกดูรายละเอียดของผู้ใช้งานแต่ละรายได้ หลังจากนั้นผู้ดูแลระบบสามารถเลือกว่าจะลบผู้ใช้งานหรือไม่ หากผู้ดูแลระบบยืนยันการลบ ระบบจะดำเนินการลบข้อมูลผู้ใช้งานออกจากระบบ แต่หากไม่ต้องการลบ ระบบจะกลับไปยังหน้ารายการผู้ใช้งานเพื่อดำเนินการจัดการข้อมูลต่อไป

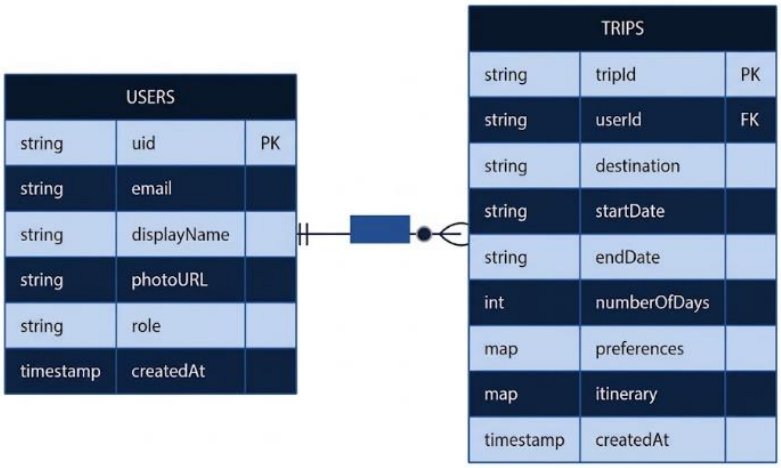
3.4.3 การออกแบบโครงสร้างข้อมูล/ฐานข้อมูล

การออกแบบโครงสร้างฐานข้อมูลเป็นขั้นตอนสำคัญในการพัฒนาระบบออกแบบทริปการท่องเที่ยวด้วยปัญญาประดิษฐ์ TripAI เนื่องจากฐานข้อมูลเป็นส่วนที่ใช้สำหรับจัดเก็บและบริหารจัดการข้อมูลที่เกี่ยวข้องกับการทำงานของระบบ เช่น ข้อมูลผู้ใช้งาน และข้อมูลแผนการเดินทางที่ผู้ใช้งานสร้างและบันทึกไว้ การออกแบบโครงสร้างข้อมูลที่เหมาะสมช่วยให้ระบบสามารถจัดเก็บค้นหา และเรียกใช้ข้อมูลได้อย่างเป็นระบบ รวมถึงช่วยสนับสนุนการทำงานร่วมกันระหว่างเว็บแอปพลิเคชันและบริการต่าง ๆ ภายในระบบ

โครงงานนี้เลือกใช้ **Cloud Firestore** ซึ่งเป็นฐานข้อมูลแบบ **NoSQL Document Database** ของ Firebase โดยข้อมูลถูกจัดเก็บในรูปแบบ Collection และ Document ซึ่งแต่ละ Document สามารถประกอบด้วย Field ที่มีชนิดข้อมูลแตกต่างกันได้ โครงสร้างดังกล่าวมีความยืดหยุ่นและเหมาะสมกับการพัฒนาเว็บแอปพลิเคชันด้วย Flutter ที่มีข้อมูลแผนการเดินทางซึ่งประกอบด้วยข้อมูลหลายระดับ

ฐานข้อมูลของระบบ TripAI ประกอบด้วย Collection หลักจำนวน 2 Collection ได้แก่ users สำหรับจัดเก็บข้อมูลผู้ใช้งาน และ trips สำหรับจัดเก็บข้อมูลแผนการเดินทาง โดยใช้รหัสผู้ใช้งาน uid ที่ได้รับจาก Firebase Authentication เชื่อมโยงกับ Field userId ภายใน Collection trips ทำให้สามารถระบุเจ้าของแผนการเดินทางแต่ละรายการได้

แผนภาพโครงสร้างฐานข้อมูล



ภาพที่ 3.7 แผนภาพโครงสร้างฐานข้อมูล

จากภาพที่ 3.7 แสดงความสัมพันธ์ของโครงสร้างข้อมูลภายในระบบ TripAI ซึ่งประกอบด้วยเอนทิตีหลัก 2 เอนทิตี ได้แก่ USERS และ TRIPS โดยเอนทิตี USERS ใช้สำหรับจัดเก็บข้อมูลผู้ใช้งานของระบบ เช่น อีเมล ชื่อที่แสดง รูปโปรไฟล์ บทบาท และวันที่สร้างบัญชี ส่วนเอนทิตี TRIPS ใช้สำหรับจัดเก็บข้อมูลแผนการเดินทางที่ผู้ใช้งานสร้างขึ้น เช่น จุดหมายปลายทาง วันเริ่มต้น วันสิ้นสุด จำนวนวัน ความต้องการของผู้ใช้งาน และรายละเอียดแผนการเดินทางที่สร้างโดย AI

ความสัมพันธ์ระหว่างเอนทิตีทั้งสองเป็นแบบหนึ่งต่อหลาย (One-to-Many) กล่าวคือ ผู้ใช้งานหนึ่งคนสามารถมีแผนการเดินทางได้หลายรายการ ขณะที่แผนการเดินทางแต่ละรายการจะเป็นของผู้ใช้งานเพียงหนึ่งคน โดยเชื่อมโยงกันผ่าน uid ของผู้ใช้งานและ userId ในเอนทิตี TRIPS

1. Collection users

Collection users ใช้สำหรับจัดเก็บข้อมูลพื้นฐานของผู้ใช้งานในระบบ โดย Document ของผู้ใช้งานแต่ละรายใช้ uid ที่ได้รับจาก Firebase Authentication เป็น Document ID เพื่อให้ข้อมูลใน Cloud Firestore สามารถเชื่อมโยงกับบัญชีที่ใช้ในการยืนยันตัวตนได้อย่างถูกต้อง

ตารางที่ 3.1 โครงสร้าง Collection users

Field	Type	Key	Description
uid	String	PK	รหัสประจำตัวผู้ใช้งานจาก Firebase Authentication
email	String	-	อีเมลของผู้ใช้งาน
displayName	String	-	ชื่อที่ใช้แสดงของผู้ใช้งาน
photoURL	String	-	URL รูปภาพโปรไฟล์ของผู้ใช้งาน
role	String	-	บทบาทของผู้ใช้งาน เช่น user หรือ admin
createdAt	Timestamp	-	วันที่และเวลาที่สร้างข้อมูลผู้ใช้งาน

ทั้งนี้ ระบบไม่ได้จัดเก็บรหัสผ่านของผู้ใช้งานไว้ใน Cloud Firestore เนื่องจากกระบวนการสมัครสมาชิกและเข้าสู่ระบบดำเนินการผ่าน **Firebase Authentication** ซึ่งทำหน้าที่จัดการข้อมูลการยืนยันตัวตนแยกออกจากข้อมูลภายในฐานข้อมูลของระบบ

2. Collection trips

Collection trips ใช้สำหรับจัดเก็บข้อมูลแผนการเดินทางที่ผู้ใช้งานสร้างด้วย Google Gemini API และเลือกบันทึกไว้ในระบบ โดยแต่ละ Document แทนแผนการเดินทางหนึ่งรายการ และมี userId สำหรับเชื่อมโยงแผนการเดินทางกับเจ้าของข้อมูลใน Collection users

ตารางที่ 3.2 โครงสร้าง Collection trips

Field	Type	Key	Description
tripId	String	PK	รหัสประจำแผนการเดินทาง
userId	String	FK	รหัสผู้ใช้งานเจ้าของแผนการเดินทาง
destination	String	-	จุดหมายปลายทางของการเดินทาง
startDate	Timestamp / String	-	วันที่เริ่มต้นการเดินทาง

Field	Type	Key	Description
endDate	Timestamp / String	-	วันที่สิ้นสุดการเดินทาง
numberOfDays	Number	-	จำนวนวันของการเดินทาง
preferences	Map / Array	-	ข้อมูลความสนใจหรือเงื่อนไขที่ผู้ใช้งานกำหนด
itinerary	Array / Map	-	รายละเอียดแผนการเดินทางที่สร้างโดย AI
createdAt	Timestamp	-	วันที่และเวลาที่บันทึกแผนการเดินทาง

Field itinerary ใช้สำหรับจัดเก็บรายละเอียดแผนการเดินทางในแต่ละวัน โดยสามารถประกอบด้วยข้อมูลของสถานที่ เช่น ชื่อสถานที่ รายละเอียดกิจกรรม ช่วงเวลา พิกัดละติจูดและลองจิจูด รวมถึง URL รูปภาพสถานที่ที่ระบบนำมาใช้ในการแสดงผล ซึ่งโครงสร้างข้อมูลแบบ Array และ Map ของ Cloud Firestore ช่วยให้สามารถจัดเก็บข้อมูลที่มีโครงสร้างหลายระดับได้อย่างยืดหยุ่น

3.4.4 การออกแบบส่วนติดต่อผู้ใช้ (UI/UX)

การออกแบบส่วนติดต่อผู้ใช้ (User Interface: UI) และประสบการณ์ผู้ใช้ (User Experience: UX) เป็นส่วนสำคัญในการพัฒนาระบบ TripAI เนื่องจากเป็นส่วนที่ผู้ใช้งานต้องติดต่อและโต้ตอบกับระบบโดยตรง การออกแบบส่วนติดต่อที่เหมาะสมช่วยให้ผู้ใช้งานสามารถเข้าถึงฟังก์ชันต่าง ๆ ได้อย่างสะดวก เข้าใจง่าย และลดความซับซ้อนในการใช้งาน

ในการพัฒนาระบบ TripAI ได้ออกแบบส่วนติดต่อในรูปแบบ **Web Application** โดยคำนึงถึงความเรียบง่าย ความชัดเจนของข้อมูล และความสะดวกในการใช้งาน โดยฟังก์ชันหลักประกอบด้วย การสมัครสมาชิกและเข้าสู่ระบบ การสร้างแผนการเดินทางด้วย AI การแสดงรายละเอียดแผนการเดินทาง การแสดงตำแหน่งสถานที่บนแผนที่ การแสดงรูปภาพสถานที่ และการจัดการแผนการเดินทางที่ผู้ใช้งานบันทึกไว้

นอกจากนี้ ระบบยังมีส่วนของ **Admin Web** สำหรับผู้ดูแลระบบ โดยออกแบบให้ผู้ดูแลสามารถเข้าสู่ระบบ ตรวจสอบ และจัดการข้อมูลผู้ใช้งานภายในระบบได้ตามสิทธิ์ที่กำหนด

หน้า UI ของแอพ TRIP AI

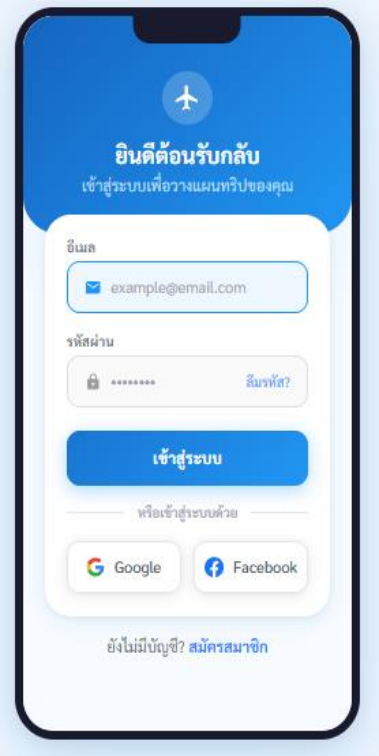
หน้าที่ 1 หน้าเริ่มต้นใช้งาน



ภาพที่ 3.8 หน้าเริ่มต้นการใช้งานของระบบ TripAI

จากภาพที่ 3.8 หน้าเริ่มต้นการใช้งานของระบบ TripAI เป็นหน้าจอแรกที่ผู้ใช้งานพบเมื่อเข้าสู่ระบบ โดยออกแบบให้มีลักษณะเรียบง่าย ทันสมัย และสื่อถึงการท่องเที่ยวด้วยโทนสีฟ้า ซึ่งช่วยสร้างความรู้สึกเป็นมิตรและสบายตา ภายในหน้าจอประกอบด้วยชื่อระบบ “TripAI” พร้อมข้อความอธิบายว่าเป็นระบบวางแผนการท่องเที่ยวด้วย AI เพื่อให้ผู้ใช้งานเข้าใจวัตถุประสงค์ของระบบได้ทันที

หน้าที่ 2 หน้าเข้าสู่ระบบ

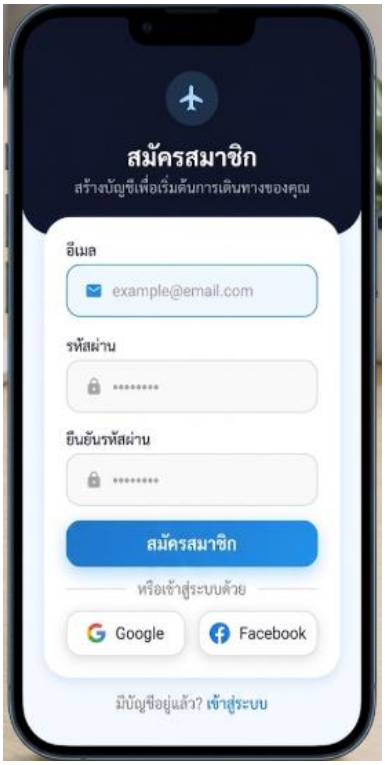


ภาพที่ 3.9 หน้าเข้าสู่ระบบของระบบ TripAI

จากภาพที่ 3.9 หน้าเข้าสู่ระบบของระบบ TripAI เป็นหน้าสำหรับให้ผู้ใช้งานยืนยันตัวตนก่อนเข้าใช้งานฟังก์ชันต่าง ๆ ภายในระบบ โดยผู้ใช้งานสามารถกรอกอีเมลและรหัสผ่าน จากนั้นกดปุ่ม **“เข้าสู่ระบบ”** เพื่อดำเนินการเข้าสู่ระบบ

นอกจากนี้ หน้าจอยังมีฟังก์ชัน **“ลืมรหัส?”** สำหรับกรณีที่ผู้ใช้งานไม่สามารถจดจำรหัสผ่านได้ รวมถึงรองรับการเข้าสู่ระบบด้วยบัญชีภายนอก ได้แก่ **Google** และ **Facebook** เพื่อเพิ่มความสะดวกในการใช้งาน ภายในส่วนล่างของหน้าจอยังมีลิงก์ **“สมัครสมาชิก”** สำหรับผู้ใช้งานที่ยังไม่มีบัญชี เพื่อเข้าสู่ขั้นตอนการลงทะเบียนใช้งานระบบต่อไป

หน้าที่ 3 หน้าสมัครสมาชิก



ภาพที่ 3.10 หน้าสมัครสมาชิกของระบบ TripAI

จากภาพที่ 3.10 หน้าสมัครสมาชิกของระบบ TripAI เป็นหน้าสำหรับให้ผู้ใช้งานใหม่สร้างบัญชีเพื่อเริ่มต้นใช้งานระบบ โดยผู้ใช้งานจะต้องกรอกข้อมูล ได้แก่ อีเมล รหัสผ่าน และยืนยันรหัสผ่าน จากนั้นกดปุ่ม **“สมัครสมาชิก”** เพื่อดำเนินการสร้างบัญชีผู้ใช้งาน

นอกจากนี้ ระบบยังรองรับการสมัครหรือเข้าสู่ระบบผ่านบัญชีภายนอก ได้แก่ **Google** และ **Facebook** เพื่อเพิ่มความสะดวกในการเข้าถึงระบบ และบริเวณส่วนล่างของหน้าจอมีลิงก์ **“เข้าสู่ระบบ”** สำหรับผู้ใช้งานที่มีบัญชีอยู่แล้ว ทำให้สามารถเปลี่ยนไปยังหน้าเข้าสู่ระบบได้อย่างรวดเร็ว

หน้าที่ 4 หน้าหลักของระบบ

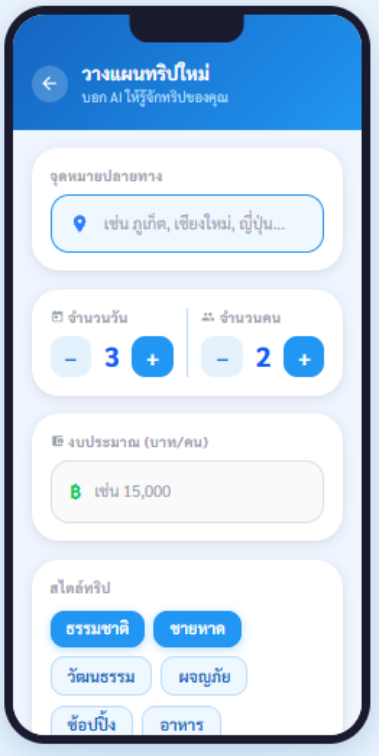


ภาพที่ 3.11 หน้าหลักของระบบ TripAI

จากภาพที่ 3.11 หน้าหลักของระบบ TripAI เป็นหน้าสำหรับแสดงภาพรวมการใช้งานหลังจากผู้ใช้งานเข้าสู่ระบบ โดยบริเวณด้านบนจะแสดงข้อความต้อนรับและชื่อผู้ใช้งาน พร้อมช่องค้นหาจุดหมายปลายทางเพื่อช่วยให้ผู้ใช้งานสามารถค้นหาสถานที่ที่ต้องการเดินทางได้อย่างสะดวก

บริเวณส่วนกลางของหน้าจอมีปุ่ม **“วางแผนทริปใหม่ด้วย AI”** ซึ่งเป็นฟังก์ชันหลักสำหรับเข้าสู่กระบวนการสร้างแผนการเดินทางด้วยปัญญาประดิษฐ์ นอกจากนี้ ระบบยังแสดงรายการ **ทริปล่าสุดของผู้ใช้งาน** ในรูปแบบการ์ด โดยแสดงข้อมูลสำคัญ เช่น ชื่อทริป สถานที่ท่องเที่ยว จำนวนวัน จำนวนผู้เดินทาง ลักษณะการเดินทาง และคะแนนของทริป ทำให้ผู้ใช้งานสามารถตรวจสอบและเข้าถึงแผนการเดินทางที่ผ่านมาได้อย่างสะดวกและรวดเร็ว

หน้าที่ 5 หน้าวางแผนทริปใหม่



ภาพที่ 3.12 หน้าวางแผนทริปใหม่ของระบบ TripAI

จากภาพที่ 3.12 หน้าวางแผนทริปใหม่ของระบบ TripAI เป็นหน้าสำหรับให้ผู้ใช้งานกำหนดข้อมูลเบื้องต้นก่อนสร้างแผนการเดินทางด้วย AI โดยผู้ใช้งานสามารถระบุจุดหมายปลายทาง จำนวนวัน จำนวนผู้ร่วมเดินทาง และงบประมาณที่ต้องการใช้ต่อคนได้ตามความต้องการ

นอกจากนี้ ระบบยังเปิดให้ผู้ใช้งานเลือก **สไตล์ของทริป** เช่น ธรรมชาติ ชายหาด วัฒนธรรม ผจญภัย ช้อปปิ้ง และอาหาร เพื่อให้ระบบสามารถนำข้อมูลดังกล่าวไปใช้เป็นเงื่อนไขในการสร้างแผนการเดินทางที่เหมาะสมกับความสนใจของผู้ใช้งานแต่ละราย โดยการออกแบบหน้าจอแบ่งข้อมูลออกเป็นส่วนอย่างชัดเจน ช่วยให้ผู้ใช้งานสามารถกรอกและปรับค่าต่าง ๆ ได้สะดวกก่อนเข้าสู่ขั้นตอนการสร้างแผนการเดินทางด้วย AI

หน้าที่ 6 หน้ากำลังสร้างแผนทริป

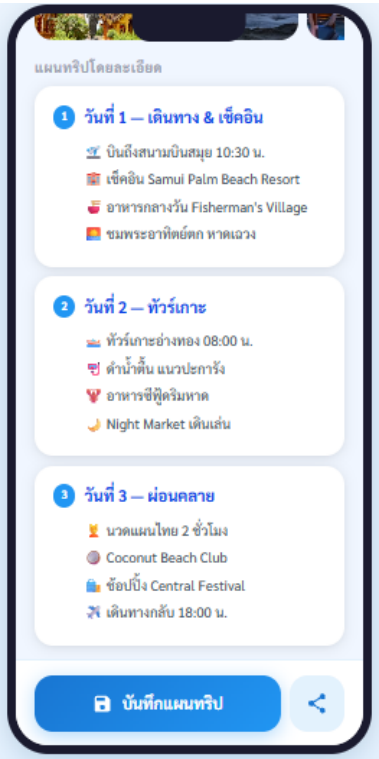


ภาพที่ 3.13 หน้ากำลังสร้างแผนทริปของระบบ TripAI

จากภาพที่ 3.13 หน้ากำลังสร้างแผนทริปของระบบ TripAI เป็นหน้าสำหรับแสดงสถานะระหว่างที่ระบบกำลังประมวลผลข้อมูลและสร้างแผนการเดินทางด้วย AI โดยระบบจะแสดงลำดับขั้นตอนการทำงาน เช่น การวิเคราะห์ความต้องการของผู้ใช้งาน การค้นหาสถานที่ที่น่าสนใจ การวางแผนเส้นทาง และการคำนวณงบประมาณรวมถึงที่พัก เพื่อให้ผู้ใช้งานทราบความคืบหน้าของกระบวนการสร้างแผนทริป

ภายในหน้าจอมีการใช้สัญลักษณ์สถานะและเอฟเฟกต์การทำงานเพื่อช่วยสื่อว่าระบบกำลังประมวลผลอยู่ พร้อมแสดงข้อความประมาณเวลาที่ใช้ในการสร้างแผนการเดินทาง ซึ่งช่วยลดความสับสนและทำให้ผู้ใช้งานสามารถรับรู้สถานะของระบบได้อย่างชัดเจนจนกว่าการสร้างแผนทริปจะเสร็จสมบูรณ์

หน้าที่ 7 หน้าแสดงรายละเอียดแผนทริป



ภาพที่ 3.14 หน้าแสดงรายละเอียดแผนทริปของระบบ TripAI

จากภาพที่ 3.14 หน้าแสดงรายละเอียดแผนทริปของระบบ TripAI เป็นหน้าสำหรับแสดงผลลัพธ์ของแผนการเดินทางที่ระบบสร้างขึ้น โดยจัดแบ่งรายละเอียดตามวันอย่างชัดเจน เช่น วันเดินทาง สถานที่ที่แนะนำ กิจกรรมที่ควรทำ ช่วงเวลา และข้อมูลที่เกี่ยวข้องกับการเดินทางในแต่ละวัน ทำให้ผู้ใช้งานสามารถตรวจสอบแผนการเดินทางได้อย่างเป็นลำดับและเข้าใจง่าย

บริเวณด้านล่างของหน้าจอมีปุ่ม **“บันทึกแผนทริป”** สำหรับให้ผู้ใช้งานบันทึกแผนการเดินทางลงในระบบ เพื่อสามารถเรียกดูได้ภายหลัง และมีปุ่มสำหรับแชร์แผนการเดินทางให้กับผู้อื่น ซึ่งช่วยเพิ่มความสะดวกในการนำแผนทริปไปใช้งานหรือแบ่งปันต่อไป

หน้าที่ 8 หน้าทริปล่าสุดหลังจากบันทึกแผนทริป



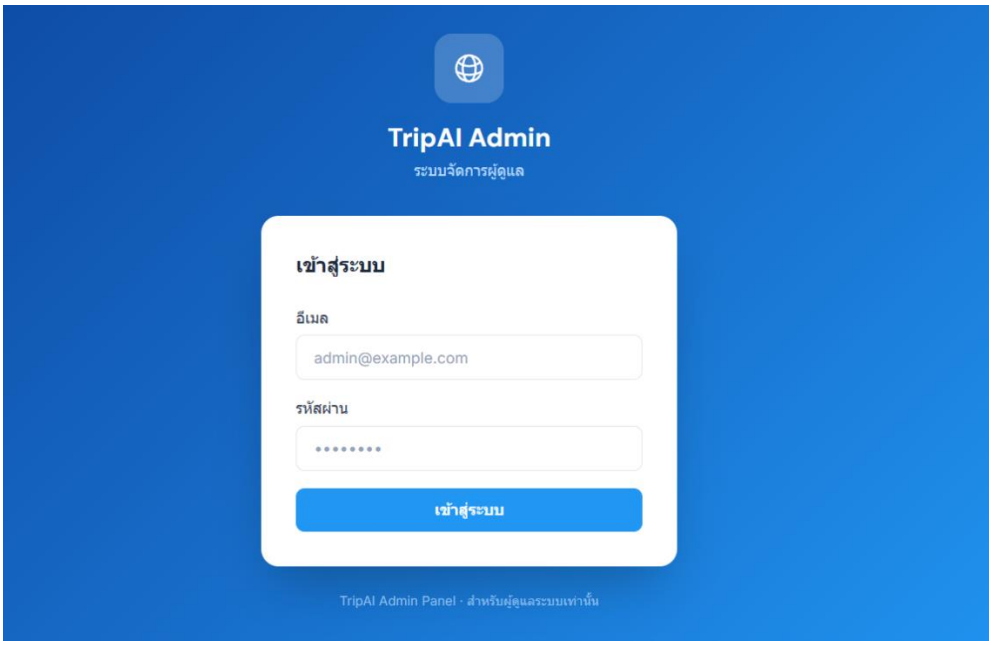
ภาพที่ 3.15 หน้าทริปล่าสุดหลังจากบันทึกแผนทริปของระบบ TripAI

จากภาพที่ 3.15 หลังจากผู้ใช้งานกดปุ่ม **“บันทึกแผนทริป”** ระบบจะบันทึกข้อมูลแผนการเดินทางลงใน Cloud Firestore และนำผู้ใช้งานกลับมายังหน้าหลัก โดยแผนการเดินทางที่บันทึกไว้จะแสดงในส่วน **“ทริปล่าสุดของคุณ”** ในรูปแบบการ์ดรายการทริป เพื่อให้ผู้ใช้งานสามารถตรวจสอบและเข้าถึงแผนการเดินทางที่บันทึกไว้ได้อย่างสะดวก

ในการ์ดของแต่ละทริปจะแสดงข้อมูลสำคัญ เช่น ชื่อทริป รูปภาพสถานที่ จำนวนวัน จำนวนผู้ร่วมเดินทาง ประเภทหรือสไตล์ของทริป และข้อมูลประกอบอื่น ๆ ที่เกี่ยวข้อง นอกจากนี้ บริเวณด้านล่างของหน้าจอยังมีแถบนำทางสำหรับเข้าสู่หน้าหลัก หน้าทริป หน้าวางแผน และหน้าโปรไฟล์ ทำให้ผู้ใช้งานสามารถเปลี่ยนไปยังส่วนต่าง ๆ ของระบบได้อย่างรวดเร็วและเป็นระเบียบ

หน้า UI ของ Web Admin

หน้าที่ 1 หน้าเข้าสู่ระบบผู้ดูแลระบบ

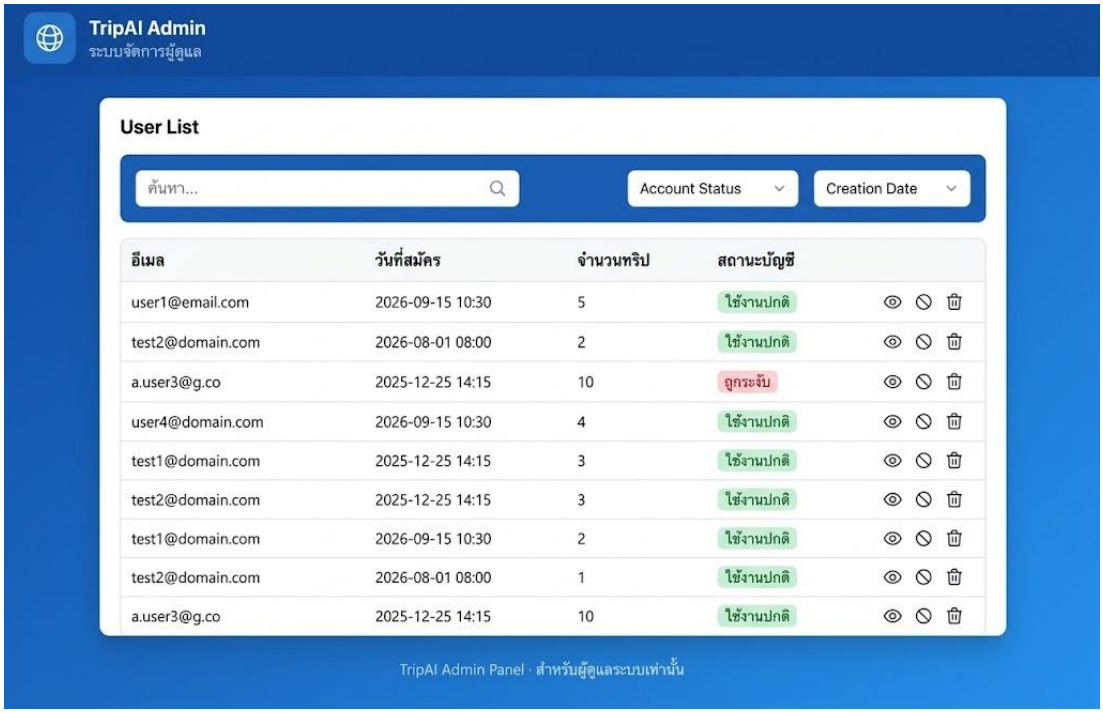


ภาพที่ 3.16 หน้าเข้าสู่ระบบผู้ดูแลระบบ TripAI Admin

จากภาพที่ 3.16 หน้าเข้าสู่ระบบผู้ดูแลระบบเป็นหน้าสำหรับให้ผู้ดูแลระบบยืนยันตัวตนก่อนเข้าสู่ส่วนบริหารจัดการของ TripAI โดยผู้ดูแลระบบจะต้องกรอกอีเมลและรหัสผ่านให้ถูกต้อง จากนั้นกดปุ่ม **“เข้าสู่ระบบ”** เพื่อดำเนินการตรวจสอบข้อมูลและสิทธิ์การใช้งาน

เมื่อข้อมูลถูกต้องและบัญชีมีสิทธิ์เป็นผู้ดูแลระบบ ระบบจะอนุญาตให้เข้าสู่หน้า Admin Web เพื่อจัดการข้อมูลผู้ใช้งานภายในระบบ แต่หากข้อมูลไม่ถูกต้องหรือไม่มีสิทธิ์ผู้ดูแล ระบบจะไม่อนุญาตให้เข้าสู่ส่วนจัดการ ทั้งนี้ หน้าจอถูกออกแบบให้แยกจากส่วนของผู้ใช้งานทั่วไปอย่างชัดเจน เพื่อให้เหมาะสมกับการบริหารจัดการระบบและลดความสับสนในการใช้งาน

หน้าที่ 2 หน้าจัดการข้อมูลผู้ใช้งาน

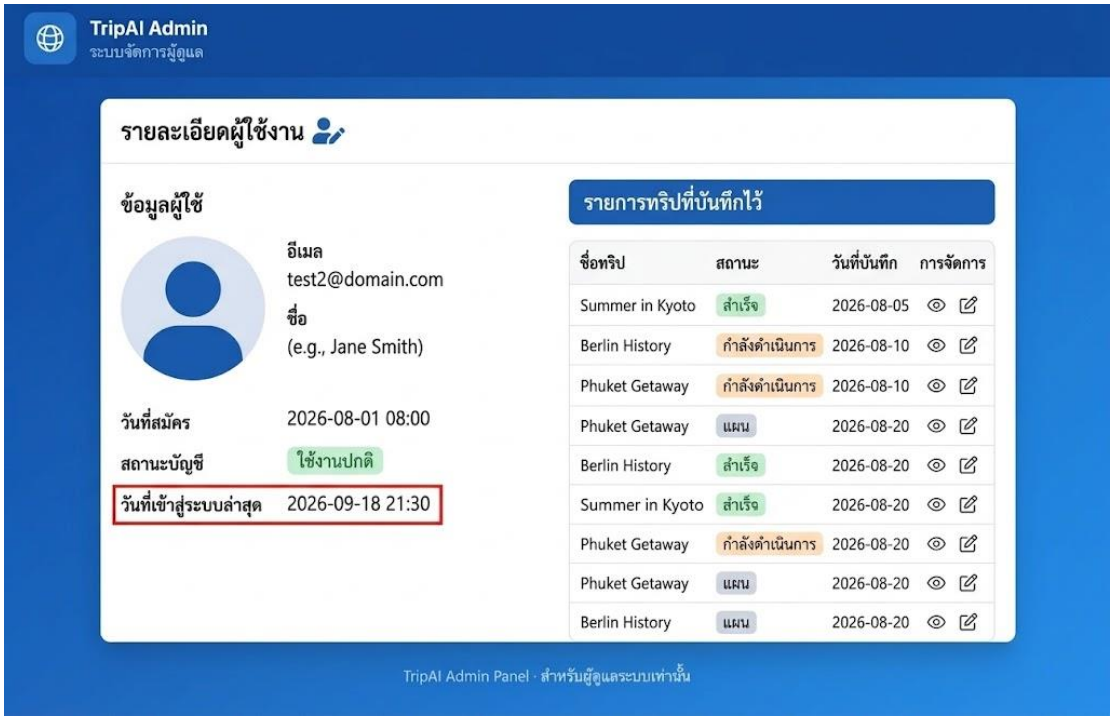


ภาพที่ 3.17 หน้าจัดการข้อมูลผู้ใช้งานของระบบ TripAI Admin

จากภาพที่ 3.17 หน้าจัดการข้อมูลผู้ใช้งานเป็นหน้าสำหรับให้ผู้ดูแลระบบตรวจสอบและบริหารจัดการบัญชีผู้ใช้งานภายในระบบ TripAI โดยแสดงข้อมูลผู้ใช้งานในรูปแบบตาราง ประกอบด้วยข้อมูลสำคัญ ได้แก่ อีเมล วันที่สมัคร จำนวนทริป และสถานะบัญชีของผู้ใช้งานแต่ละราย

บริเวณด้านบนมีช่องค้นหาสำหรับค้นหาผู้ใช้งาน และมีตัวกรองตามสถานะบัญชีและวันที่สร้างบัญชี เพื่อช่วยให้ผู้ดูแลสามารถค้นหาและตรวจสอบข้อมูลได้สะดวกยิ่งขึ้น นอกจากนี้ ในแต่ละรายการยังมีเครื่องมือสำหรับดูรายละเอียด ระงับการใช้งาน และลบบัญชีผู้ใช้งาน ทำให้ผู้ดูแลระบบสามารถจัดการข้อมูลผู้ใช้งานได้อย่างเป็นระบบและเหมาะสมตามสิทธิ์ที่กำหนดไว้

หน้าที่ 3 หน้ารายละเอียดผู้ใช้งาน

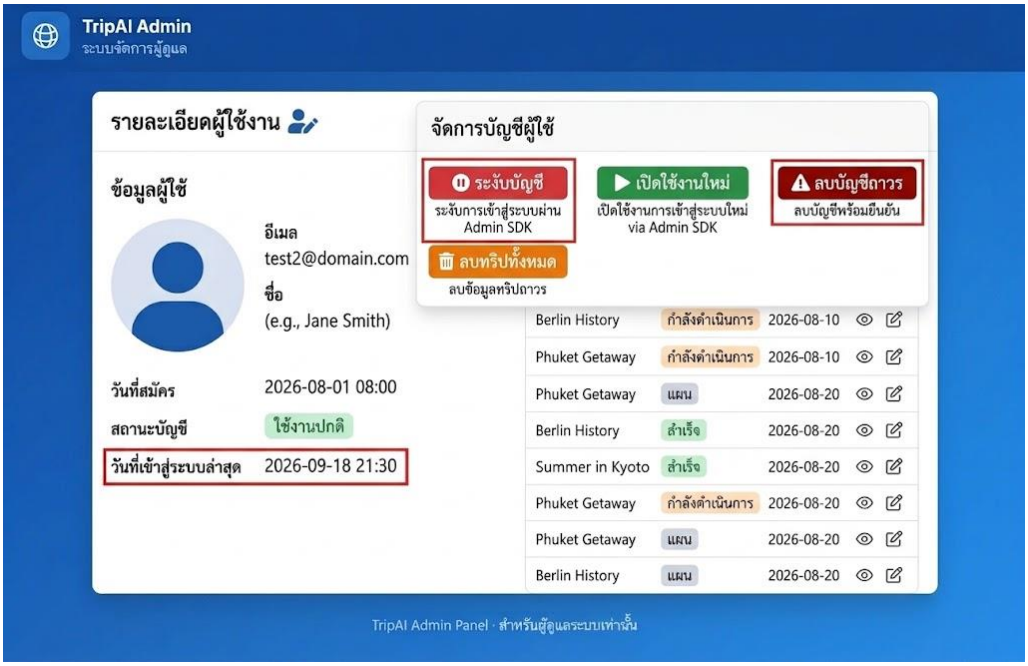


ภาพที่ 3.18 หน้ารายละเอียดผู้ใช้งานของระบบ TripAI Admin

จากภาพที่ 3.18 หน้ารายละเอียดผู้ใช้งานเป็นหน้าสำหรับให้ผู้ดูแลระบบตรวจสอบข้อมูลของผู้ใช้งานแต่ละราย โดยแสดงข้อมูลสำคัญ ได้แก่ อีเมล ชื่อ วันที่สมัคร สถานะบัญชี และวันที่เข้าสู่ระบบล่าสุด เพื่อช่วยให้ผู้ดูแลสามารถตรวจสอบข้อมูลและสถานะการใช้งานของบัญชีได้อย่างชัดเจน

นอกจากนี้ ระบบยังแสดงรายการทริปที่ผู้ใช้งานบันทึกไว้ เช่น ชื่อทริป สถานะ และวันที่บันทึก เพื่อใช้เป็นข้อมูลประกอบในการตรวจสอบการใช้งานของผู้ใช้แต่ละราย โดยผู้ดูแลสามารถเลือกดูรายละเอียดของรายการที่เกี่ยวข้องได้จากหน้าจอดังกล่าว

หน้าที่ 4 หน้าจัดการบัญชีผู้ใช้งาน

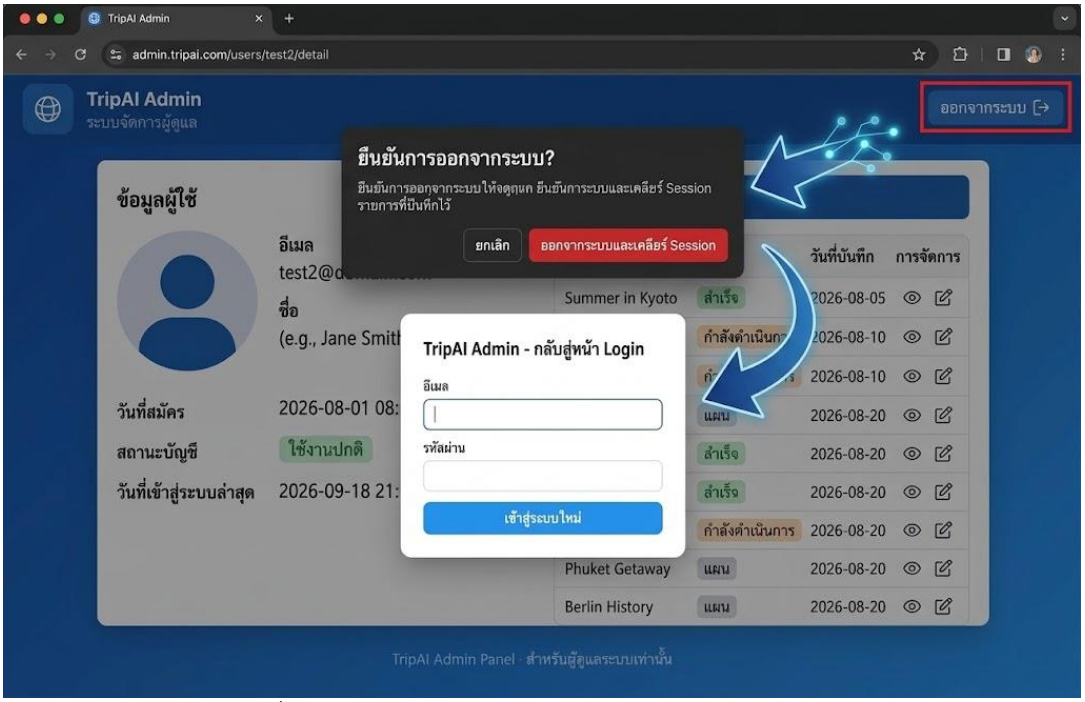


ภาพที่ 3.19 หน้าจัดการบัญชีผู้ใช้งานของระบบ TripAI Admin

จากภาพที่ 3.19 หน้าจัดการบัญชีผู้ใช้งานเป็นหน้าสำหรับให้ผู้ดูแลระบบดำเนินการกับบัญชีของผู้ใช้งานแต่ละราย โดยสามารถเลือก **ระงับบัญชี** เพื่อไม่อนุญาตให้ผู้ใช้งานเข้าสู่ระบบชั่วคราว และสามารถ **เปิดใช้งานบัญชีใหม่** เมื่อต้องการคืนสิทธิ์การเข้าใช้งานให้แก่ผู้ใช้งาน

นอกจากนี้ ผู้ดูแลระบบสามารถเลือก **ลบบัญชีถาวร** สำหรับบัญชีที่ไม่ต้องการให้มีอยู่ในระบบอีกต่อไป รวมถึงสามารถลบข้อมูลทริปทั้งหมดที่เชื่อมโยงกับผู้ใช้งานในกรณีที่ต้องการจัดการข้อมูลประกอบของบัญชีนั้น การออกแบบหน้าจอได้แยกคำสั่งแต่ละประเภทอย่างชัดเจน และใช้ข้อความกำกับเพื่อช่วยลดความผิดพลาดในการดำเนินการที่มีผลต่อข้อมูลของผู้ใช้งาน

หน้าที่ 5 หน้าออกจากระบบผู้ดูแลระบบ



ภาพที่ 3.20 หน้าออกจากระบบผู้ดูแลระบบ TripAI Admin

จากภาพที่ 3.20 หน้าออกจากระบบผู้ดูแลระบบเป็นหน้าสำหรับยืนยันการออกจากระบบก่อนสิ้นสุดการใช้งาน โดยเมื่อผู้ดูแลกดปุ่ม **“ออกจากระบบ”** ระบบจะแสดงหน้าต่างยืนยัน เพื่อป้องกันการออกจากระบบโดยไม่ได้ตั้งใจ ผู้ดูแลสามารถเลือกยกเลิกหรือยืนยันการออกจากระบบได้

เมื่อยืนยันการออกจากระบบ ระบบจะสิ้นสุด Session ของผู้ดูแลและนำกลับไปยังหน้าเข้าสู่ระบบของ TripAI Admin เพื่อให้ต้องทำการยืนยันตัวตนใหม่ก่อนเข้าใช้งานอีกครั้ง กระบวนการดังกล่าวช่วยเพิ่มความปลอดภัยในการใช้งานระบบ โดยเฉพาะในกรณีที่มีการใช้งานผ่านอุปกรณ์ร่วมกับบุคคลอื่น

3.4.5 การวางแผนและออกแบบส่วนเชื่อมต่อโปรแกรมประยุกต์ (API Planning and Design)

การออกแบบส่วนเชื่อมต่อโปรแกรมประยุกต์ (Application Programming Interface: API) เป็นส่วนสำคัญในการเชื่อมต่อการทำงานระหว่างเว็บแอปพลิเคชัน TripAI กับบริการภายนอกที่ใช้ในการประมวลผลและสนับสนุนการวางแผนการเดินทาง โดยระบบพัฒนาด้วย Flutter Web และภาษา Dart และใช้บริการของ Firebase สำหรับการยืนยันตัวตนและจัดเก็บข้อมูลของระบบ

เว็บแอปพลิเคชัน TripAI จะส่งคำขอไปยังบริการต่าง ๆ ผ่านเครือข่ายอินเทอร์เน็ตตามลักษณะของการทำงาน โดยข้อมูลที่มีการแลกเปลี่ยนกับ API ภายนอกส่วนใหญ่ใช้รูปแบบ JSON เพื่อให้ระบบสามารถรับข้อมูล ประมวลผล และนำผลลัพธ์ไปแสดงบนส่วนติดต่อผู้ใช้งานได้อย่างเหมาะสม

โครงสร้างการเชื่อมต่อของระบบสามารถอธิบายได้ดังนี้

TripAI Web Application ⟶ API / Firebase Services ⟶ ประมวลผลข้อมูล ⟶ ส่งผลลัพธ์กลับมายัง TripAI Web Application

โดยส่วนเชื่อมต่อแต่ละบริการจะทำหน้าที่แตกต่างกัน เช่น การสร้างแผนการเดินทางด้วยปัญญาประดิษฐ์ การค้นหาพิกัดของสถานที่ การค้นหารูปภาพประกอบ การยืนยันตัวตนของผู้ใช้งาน และการบันทึกข้อมูลแผนการเดินทาง

API และบริการที่สำคัญซึ่งใช้ในการทำงานของระบบมีดังนี้

API / Service	หน้าที่
Google Gemini API	รับข้อมูลความต้องการของผู้ใช้งานและสร้างแผนการเดินทางด้วยปัญญาประดิษฐ์
Nominatim API	ค้นหาสถานที่และแปลงชื่อสถานที่เป็นค่าพิกัดละติจูดและลองจิจูด
Unsplash API	ค้นหารูปภาพที่เกี่ยวข้องกับสถานที่เพื่อนำมาแสดงประกอบแผนการเดินทาง
Firebase Authentication	จัดการการสมัครสมาชิกและเข้าสู่ระบบด้วย Email/Password และ Google Sign-In
Cloud Firestore	จัดเก็บ เรียกดู และลบข้อมูลผู้ใช้งานและข้อมูลแผนการเดินทาง
OpenStreetMap	ให้ข้อมูลแผนที่สำหรับนำมาแสดงผลผ่านไลบรารี flutter_map

ในการสร้างแผนการเดินทาง ระบบกำหนดลำดับการทำงานของการเชื่อมต่อบริการไว้อย่างชัดเจน ได้แก่

กรอกข้อมูลทริป ⟶ Google Gemini API ⟶ สร้างแผนการเดินทาง ⟶ Nominatim API ⟶ ค้นหาพิกัดสถานที่ ⟶ Unsplash API ⟶ ค้นหารูปภาพ ⟶ แสดงผลแผนทริป

เมื่อผู้ใช้งานกรอกข้อมูลสำหรับสร้างแผนการเดินทาง เช่น จุดหมายปลายทาง จำนวนวัน จำนวนผู้เดินทาง งบประมาณ และรูปแบบการท่องเที่ยว ระบบจะรวบรวมข้อมูลและส่งไปยัง Google Gemini API เพื่อประมวลผล จากนั้นผลลัพธ์ที่ได้รับจะถูกนำมาจัดรูปแบบเป็นรายละเอียดแผนการเดินทางในแต่ละวัน

สำหรับสถานที่ที่ปรากฏในแผนการเดินทาง ระบบจะนำชื่อสถานที่ไปค้นหาผ่าน Nominatim API เพื่อรับค่าพิกัด Latitude และ Longitude แล้วนำข้อมูลพิกัดดังกล่าวไปใช้ในการแสดงตำแหน่งผ่าน flutter_map ร่วมกับ OpenStreetMap ส่วน Unsplash API จะใช้สำหรับค้นหารูปภาพที่เกี่ยวข้องกับสถานที่ เพื่อใช้ประกอบการแสดงผลของแผนการเดินทาง

ในส่วนของข้อมูลผู้ใช้งาน ระบบใช้ Firebase Authentication สำหรับการสมัครสมาชิก การเข้าสู่ระบบ และการยืนยันตัวตน ขณะที่ Cloud Firestore ใช้สำหรับจัดเก็บข้อมูลผู้ใช้งานและแผนการเดินทาง เมื่อผู้ใช้งานกดบันทึกแผนทริป ระบบจะบันทึกข้อมูลแผนการเดินทางพร้อม userId เพื่อเชื่อมโยงแผนการเดินทางกับผู้ใช้งานแต่ละราย และสามารถนำข้อมูลดังกล่าวกลับมาแสดงในรายการทริปที่บันทึกไว้ได้

การออกแบบส่วนเชื่อมต่อโปรแกรมประยุกต์ในลักษณะดังกล่าวช่วยแบ่งหน้าที่ของแต่ละบริการออกจากกันอย่างชัดเจน โดย Google Gemini API รับผิดชอบการสร้างแผนการเดินทาง Nominatim API และ OpenStreetMap รับผิดชอบข้อมูลตำแหน่งและแผนที่ Unsplash API รับผิดชอบข้อมูลรูปภาพ ส่วน Firebase Authentication และ Cloud Firestore รับผิดชอบด้านการยืนยันตัวตนและการจัดเก็บข้อมูล ทำให้ระบบสามารถพัฒนา แก้ไข และเพิ่มเติมฟังก์ชันการทำงานในอนาคตได้สะดวกยิ่งขึ้น

3.4.6 การวางแผนและออกแบบโครงสร้างส่วนหน้าด้วย Flutter Web (Frontend Application Planning)

การพัฒนาส่วนหน้าของระบบออกแบบทริปการท่องเที่ยวด้วยปัญญาประดิษฐ์ TripAI ใช้ **Flutter Web** และภาษา **Dart** ในการพัฒนาเว็บแอปพลิเคชัน เนื่องจาก Flutter สามารถออกแบบส่วนติดต่อผู้ใช้งานให้มีรูปแบบที่เป็นระบบ สามารถแบ่งส่วนประกอบของหน้าจอออกเป็น Widget และรองรับการพัฒนาแอปพลิเคชันในรูปแบบเว็บได้

โครงสร้างส่วนหน้าของระบบถูกออกแบบให้ผู้ใช้งานสามารถดำเนินการตั้งแต่การสมัครสมาชิกและเข้าสู่ระบบ การกำหนดข้อมูลสำหรับวางแผนการเดินทาง การสร้างแผนการเดินทางด้วย

AI การแสดงรายละเอียดสถานที่และตำแหน่งบนแผนที่ ตลอดจนการบันทึกและเรียกดูแผนการเดินทางของตนเอง

นอกจากนี้ ส่วนหน้าของระบบยังมีการเชื่อมต่อกับ Firebase Authentication สำหรับจัดการการยืนยันตัวตน ใช้ Cloud Firestore สำหรับจัดเก็บและเรียกดูข้อมูลแผนการเดินทาง เชื่อมต่อ Google Gemini API สำหรับสร้างแผนการเดินทาง ใช้ Nominatim API สำหรับค้นหาพิกัดสถานที่ และใช้ flutter_map ร่วมกับ OpenStreetMap สำหรับแสดงข้อมูลตำแหน่งบนแผนที่

1) โครงสร้างหน้าจอสำหรับผู้ใช้งาน (User Application)

หน้าจอ	หน้าที่
Welcome Page	แสดงหน้าเริ่มต้นของระบบและนำผู้ใช้งานไปยังหน้าเข้าสู่ระบบหรือสมัครสมาชิก
Login Page	ใช้สำหรับเข้าสู่ระบบด้วย Email/Password หรือบัญชี Google
Register Page	ใช้สำหรับสร้างบัญชีผู้ใช้งานใหม่
Home Page	แสดงหน้าหลักของระบบ ช่องค้นหาจุดหมาย และรายการทริปล่าสุดของผู้ใช้งาน
Plan Trip Page	ใช้สำหรับกำหนดจุดหมาย จำนวนวัน จำนวนผู้เดินทาง งบประมาณ และสไตล์การท่องเที่ยว
Generating Trip Page	แสดงสถานะการประมวลผลขณะที่ AI กำลังสร้างแผนการเดินทาง
Trip Detail Page	แสดงรายละเอียดแผนการเดินทางที่ AI สร้างขึ้น โดยแบ่งรายละเอียดตามวัน
Saved Trips Page	แสดงรายการแผนการเดินทางที่ผู้ใช้งานบันทึกไว้
Map Page	แสดงตำแหน่งสถานที่ในแผนการเดินทางบนแผนที่
Profile Page	แสดงข้อมูลบัญชีและข้อมูลส่วนตัวของผู้ใช้งาน

กระบวนการทำงานหลักของส่วนผู้ใช้งานสามารถแสดงได้ดังนี้

Welcome ⟶ Login / Register ⟶ Home ⟶ Plan Trip ⟶ Generate Trip ⟶ Trip Detail ⟶ Save Trip ⟶ Saved Trips

เมื่อผู้ใช้งานเข้าสู่ระบบสำเร็จ ระบบจะนำเข้าสู่หน้า Home ซึ่งแสดงข้อมูลทริปล่าสุดและเมนูสำหรับเข้าสู่ฟังก์ชันหลัก ผู้ใช้งานสามารถเลือกสร้างแผนการเดินทางใหม่ โดยกำหนดจุดหมายปลายทาง จำนวนวัน จำนวนผู้เดินทาง งบประมาณ และลักษณะการท่องเที่ยวที่ต้องการ

หลังจากผู้ใช้งานยืนยันข้อมูล ระบบจะเข้าสู่หน้ากำลังสร้างแผนทริป และส่งข้อมูลไปยัง Google Gemini API เพื่อสร้างรายละเอียดแผนการเดินทาง เมื่อประมวลผลเสร็จสิ้น ระบบจะแสดงผลในหน้า Trip Detail โดยจัดแบ่งกิจกรรมและสถานที่ตามแต่ละวัน ผู้ใช้งานสามารถบันทึกแผนการเดินทางไว้ใน Cloud Firestore เพื่อเรียกดูในภายหลังได้

2) การจัดการการนำทางระหว่างหน้าจอ

ระบบออกแบบการนำทางระหว่างหน้าจอให้สอดคล้องกับลำดับการใช้งาน โดยข้อมูลที่จำเป็นสามารถส่งต่อระหว่างหน้าจอ เช่น จุดหมายปลายทาง จำนวนวัน จำนวนผู้เดินทาง งบประมาณ รูปแบบการท่องเที่ยว และข้อมูลแผนการเดินทางที่สร้างขึ้นจาก AI

นอกจากนี้ บริเวณส่วนล่างของหน้าหลักมีเมนูนำทางสำหรับเข้าสู่ส่วนต่าง ๆ ได้แก่ หน้าหลัก ทริปฉัน วางแผน และโปรไฟล์ เพื่อช่วยให้ผู้ใช้งานสามารถเปลี่ยนไปยังฟังก์ชันหลักของระบบได้อย่างสะดวก

การแบ่งหน้าจอตามหน้าที่ช่วยให้โครงสร้างของโปรแกรมมีความชัดเจน สามารถตรวจสอบและปรับปรุงแต่ละส่วนได้ง่าย รวมทั้งช่วยลดความซับซ้อนในการพัฒนาเมื่อมีการเพิ่มเติมฟังก์ชันในอนาคต

3) การจัดการข้อมูลและสถานะภายในแอปพลิเคชัน

ระบบมีการจัดการข้อมูลและสถานะที่เกิดขึ้นระหว่างการใช้งาน เช่น สถานะการเข้าสู่ระบบ ข้อมูลผู้ใช้งาน ข้อมูลที่ใช้สำหรับวางแผนทริป สถานะการสร้างแผนการเดินทาง และข้อมูลผลลัพธ์จาก Google Gemini API เพื่อให้ข้อมูลที่แสดงในแต่ละหน้าจอมีความสอดคล้องกัน

ข้อมูลบัญชีผู้ใช้งานจะเชื่อมโยงกับ Firebase Authentication ขณะที่ข้อมูลแผนการเดินทางที่ผู้ใช้งานเลือกบันทึกจะถูกจัดเก็บไว้ใน Cloud Firestore โดยใช้ uid หรือ userId เพื่อระบุเจ้าของแผนการเดินทาง ทำให้ระบบสามารถเรียกดูเฉพาะข้อมูลของผู้ใช้งานแต่ละรายได้

4) การเชื่อมต่อระบบแผนที่และตำแหน่ง

ส่วนหน้าที่เกี่ยวข้องกับข้อมูลสถานที่มีการใช้งาน flutter_map ร่วมกับ OpenStreetMap สำหรับแสดงแผนที่ โดยระบบใช้ Nominatim API ในการค้นหาและแปลงชื่อสถานที่จากแผนการเดินทางให้เป็นค่าพิกัดละติจูดและลองจิจูด ก่อนนำพิกัดดังกล่าวมาแสดงตำแหน่งบนแผนที่

นอกจากนี้ ระบบยังเชื่อมต่อกับ Unsplash API เพื่อค้นหารูปภาพที่เกี่ยวข้องกับสถานที่และนำมาแสดงประกอบข้อมูลทริป ช่วยให้ผู้ใช้งานสามารถเห็นรายละเอียดของจุดหมายปลายทางได้ชัดเจนยิ่งขึ้น

5) การเชื่อมต่อบริการปัญญาประดิษฐ์

ระบบ TripAI เชื่อมต่อกับ Google Gemini API เพื่อใช้เป็นส่วนหลักในการสร้างแผนการเดินทาง โดยข้อมูลที่ผู้ใช้งานกำหนดในหน้าวางแผนทริปจะถูกนำมาจัดเตรียมและส่งไปยัง API เพื่อประมวลผล จากนั้นระบบจะรับผลลัพธ์กลับมาและจัดรูปแบบสำหรับแสดงในหน้าแผนการเดินทาง

การออกแบบโครงสร้างส่วนหน้าในลักษณะดังกล่าวทำให้ระบบ TripAI สามารถแบ่งหน้าที่ของแต่ละหน้าจอได้อย่างชัดเจน เชื่อมโยงการทำงานของ Flutter Web เข้ากับ Firebase และบริการ API ภายนอกได้อย่างเป็นระบบ รวมถึงรองรับการปรับปรุงหรือเพิ่มเติมฟังก์ชันการทำงานของระบบในอนาคตได้อย่างเหมาะสม

3.4.7 การวางแผนและออกแบบส่วนติดต่อผู้ใช้งานสำหรับผู้ดูแลระบบ (Admin Web Dashboard Planning)

นอกจากส่วนของเว็บแอปพลิเคชันสำหรับผู้ใช้งานทั่วไปแล้ว ระบบ TripAI ยังมีการพัฒนาส่วนสำหรับผู้ดูแลระบบในรูปแบบเว็บแอปพลิเคชัน (Admin Web) เพื่อใช้สำหรับตรวจสอบและจัดการข้อมูลบัญชีผู้ใช้งานภายในระบบ โดยผู้ดูแลระบบจะต้องเข้าสู่ระบบและผ่านการตรวจสอบสิทธิ์ก่อนจึงจะสามารถเข้าถึงส่วนจัดการข้อมูลได้

ระบบ Admin Web เชื่อมต่อกับ **Firebase Authentication** สำหรับการตรวจสอบบัญชีและการยืนยันตัวตน และเชื่อมต่อกับ **Cloud Firestore** สำหรับเรียกดูข้อมูลผู้ใช้งานและข้อมูลที่เกี่ยวข้องกับผู้ใช้งาน การออกแบบส่วนผู้ดูแลระบบของ TripAI จำกัดขอบเขตการทำงานไว้ที่การจัดการข้อมูลผู้ใช้งานเป็นหลัก เพื่อให้สอดคล้องกับขอบเขตของโครงงาน

การออกแบบ Admin Web แบ่งออกเป็นส่วนสำคัญ ดังนี้

1) หน้าสำหรับเข้าสู่ระบบผู้ดูแล

หน้า Admin Login ใช้สำหรับตรวจสอบตัวตนของผู้ดูแลระบบ โดยผู้ดูแลจะต้องกรอกอีเมลและรหัสผ่านให้ถูกต้องก่อนเข้าสู่ระบบ เมื่อระบบตรวจสอบข้อมูลและสิทธิ์ของบัญชีแล้ว หากบัญชีมีสิทธิ์เป็นผู้ดูแลระบบ ระบบจะอนุญาตให้เข้าสู่ส่วนจัดการข้อมูลผู้ใช้งาน แต่หากข้อมูลไม่ถูกต้องหรือไม่มีสิทธิ์ ระบบจะปฏิเสธการเข้าสู่ส่วน Admin Web

การแยกหน้าเข้าสู่ระบบของผู้ดูแลออกจากส่วนของผู้ใช้งานทั่วไป ช่วยให้การควบคุมสิทธิ์และการเข้าถึงส่วนจัดการระบบมีความชัดเจนและเหมาะสมกับลักษณะการใช้งาน

2) หน้ารายการผู้ใช้งาน

หน้ารายการผู้ใช้งาน (User List) เป็นหน้าหลักหลังจากผู้ดูแลเข้าสู่ระบบสำเร็จ ใช้สำหรับแสดงรายการบัญชีผู้ใช้งานภายในระบบ TripAI ในรูปแบบตาราง โดยข้อมูลสำคัญที่แสดงประกอบด้วย

- อีเมลของผู้ใช้งาน
- วันที่สมัครสมาชิก
- จำนวนทริปที่บันทึกไว้
- สถานะของบัญชี

นอกจากนี้ หน้ารายการผู้ใช้งานยังมีช่องสำหรับค้นหาผู้ใช้งาน รวมถึงตัวกรองตามสถานะบัญชีและวันที่สร้างบัญชี เพื่อช่วยให้ผู้ดูแลสามารถค้นหาและตรวจสอบข้อมูลได้สะดวกรวดเร็วยิ่งขึ้น ผู้ดูแลสามารถเลือกดูรายละเอียด ระงับบัญชี หรือดำเนินการลบบัญชีของผู้ใช้งานแต่ละรายได้ตามสิทธิ์ที่กำหนด

3) หน้ารายละเอียดผู้ใช้งาน

หน้ารายละเอียดผู้ใช้งานใช้สำหรับแสดงข้อมูลของผู้ใช้งานที่ผู้ดูแลเลือกจากหน้ารายการ โดยแสดงข้อมูลสำคัญ เช่น อีเมล ชื่อ วันที่สมัคร สถานะบัญชี และวันที่เข้าสู่ระบบล่าสุด เพื่อให้ผู้ดูแลสามารถตรวจสอบข้อมูลของบัญชีแต่ละรายได้อย่างชัดเจน

นอกจากนี้ หน้าดังกล่าวยังสามารถแสดงรายการทริปที่ผู้ใช้งานบันทึกไว้ เพื่อใช้เป็นข้อมูลประกอบในการตรวจสอบการใช้งานของบัญชี โดยข้อมูลทริปที่แสดงอาจประกอบด้วยชื่อทริป สถานะ และวันที่บันทึก ทั้งนี้ ส่วนของผู้ดูแลใช้ข้อมูลดังกล่าวเพื่อการตรวจสอบเท่านั้น โดยไม่เน้นการแก้ไขเนื้อหาแผนการเดินทางของผู้ใช้งาน

4) ส่วนจัดการบัญชีผู้ใช้งาน

ส่วนจัดการบัญชีผู้ใช้งานใช้สำหรับดำเนินการเกี่ยวกับสถานะและข้อมูลบัญชีของผู้ใช้งาน โดยมีการทำงานหลัก ได้แก่

- **ระงับบัญชี** สำหรับป้องกันไม่ให้ผู้ใช้งานเข้าสู่ระบบชั่วคราว
- **เปิดใช้งานใหม่** สำหรับคืนสิทธิ์การเข้าสู่ระบบให้แก่บัญชีที่ถูกระงับ
- **ลบบัญชีถาวร** สำหรับลบบัญชีผู้ใช้งานออกจากระบบ

- **ลบทริปทั้งหมด** สำหรับลบข้อมูลแผนการเดินทางที่เชื่อมโยงกับผู้ใช้งานในกรณีที่จำเป็น

สำหรับคำสั่งที่มีผลต่อบัญชีหรือข้อมูลของผู้ใช้งาน ระบบควรมีขั้นตอนการยืนยันก่อนดำเนินการ เพื่อช่วยลดความผิดพลาดจากการกดคำสั่งโดยไม่ได้ตั้งใจ

5) การออกจากระบบผู้ดูแล

ระบบมีฟังก์ชันออกจากระบบสำหรับสิ้นสุดการใช้งาน Admin Web โดยเมื่อผู้ดูแลเลือกออกจากระบบ ระบบจะแสดงหน้าต่างยืนยันก่อนดำเนินการ เมื่อผู้ดูแลยืนยัน ระบบจะสิ้นสุดสถานะการเข้าสู่ระบบและนำกลับไปยังหน้า Admin Login

การดำเนินการดังกล่าวช่วยป้องกันไม่ให้บุคคลอื่นสามารถเข้าใช้งานส่วนผู้ดูแลต่อจากบัญชีเดิมได้ โดยเฉพาะกรณีที่มีการใช้งานผ่านเครื่องคอมพิวเตอร์ร่วมกัน

6) การออกแบบส่วนติดต่อของ Admin Web

ส่วนติดต่อของ Admin Web ถูกออกแบบให้มีรูปแบบเรียบง่ายและสอดคล้องกับรูปแบบของระบบ TripAI โดยใช้โทนสีฟ้าเป็นสีหลัก เพื่อสร้างความต่อเนื่องกับส่วนของผู้ใช้งานทั่วไป ข้อมูลผู้ใช้งานถูกแสดงในรูปแบบตารางเพื่อให้สามารถตรวจสอบข้อมูลหลายรายการได้สะดวก พร้อมใช้สีและป้ายสถานะ (Status Badge) เพื่อแยกสถานะบัญชี เช่น **ใช้งานปกติ** และ **ถูกระงับ** ให้สามารถสังเกตได้อย่างชัดเจน

โครงสร้างการใช้งานหลักของ Admin Web สามารถแสดงได้ดังนี้

Admin Login ⟶ User List ⟶ User Detail ⟶ Manage User Account ⟶ Logout

การออกแบบส่วนผู้ดูแลระบบในลักษณะดังกล่าวช่วยให้ผู้ดูแลสามารถตรวจสอบและจัดการข้อมูลผู้ใช้งานของ TripAI ได้อย่างเป็นระบบ โดยจำกัดการเข้าถึงเฉพาะผู้ที่ได้รับสิทธิ์ และแยกหน้าที่ของส่วนผู้ดูแลออกจากการใช้งานของผู้ใช้งานทั่วไปอย่างชัดเจน ซึ่งช่วยเพิ่มความเป็นระเบียบและความปลอดภัยในการบริหารจัดการข้อมูลภายในระบบ

3.5 การหาประสิทธิภาพของระบบออกแบบทริปการท่องเที่ยวด้วย AI

ภายหลังจากการพัฒนาระบบออกแบบทริปการท่องเที่ยวด้วย AI เสร็จสมบูรณ์ ผู้จัดทำได้กำหนดแนวทางและขั้นตอนในการทดสอบเพื่อประเมินประสิทธิภาพของระบบ โดยมุ่งเน้นการประเมินการทำงานของฟังก์ชันหลัก ความรวดเร็วในการประมวลผล ความเหมาะสมของแผนการเดินทางที่สร้างขึ้น และความสะดวกในการใช้งานของผู้ใช้

การประเมินครอบคลุมการทำงานของระบบตั้งแต่การสมัครสมาชิกและเข้าสู่ระบบ การสร้างแผนการเดินทางด้วย Google Gemini API การค้นหาพิกัดสถานที่ด้วย Nominatim API การแสดง

ตำแหน่งสถานที่ผ่าน OpenStreetMap การแสดงรูปภาพสถานที่จาก Unsplash API ตลอดจนการบันทึกและเรียกดูข้อมูลแผนการเดินทางผ่าน Cloud Firestore

การประเมินผลครอบคลุมทั้งด้านการทำงานของระบบ ความถูกต้องของผลลัพธ์ ระยะเวลาในการประมวลผล และประสบการณ์ของผู้ใช้งาน เพื่อให้มั่นใจว่าระบบสามารถนำไปใช้งานได้อย่างมีประสิทธิภาพ

3.5.1 รูปแบบและวิธีการทดสอบระบบ (Testing Methodology)

ผู้จัดทำกำหนดรูปแบบการทดสอบในสภาพแวดล้อมการใช้งานจริง โดยจำลองการใช้งานของผู้ใช้ตั้งแต่การเข้าสู่ระบบ การกำหนดข้อมูลสำหรับการเดินทาง การสร้างแผนทริปด้วย AI จนถึงการบันทึกและเรียกดูข้อมูลแผนการเดินทาง

การทดสอบแบ่งออกเป็น 2 ส่วนหลัก ได้แก่

1) การทดสอบการทำงานของระบบ (Black-Box Testing)

เป็นการทดสอบการทำงานของระบบโดยไม่พิจารณาโค้ดหรือกระบวนการภายใน โดยตรวจสอบความถูกต้องจากข้อมูลที่นำเข้าและผลลัพธ์ที่ระบบแสดงออกมา เช่น

- การสมัครสมาชิกและเข้าสู่ระบบของผู้ใช้งาน
- การเข้าสู่ระบบด้วย Email/Password และ Google Sign-In
- การกรอกข้อมูลเพื่อสร้างแผนการเดินทาง
- การสร้างแผนการเดินทางด้วย Google Gemini API
- การแสดงรายละเอียดแผนการเดินทางที่สร้างขึ้น
- การค้นหาพิกัดสถานที่ด้วย Nominatim API
- การแสดงตำแหน่งสถานที่ผ่าน OpenStreetMap
- การแสดงรูปภาพสถานที่จาก Unsplash API
- การบันทึกและเรียกดูแผนการเดินทาง
- การลบแผนการเดินทาง
- การเข้าสู่ระบบและจัดการข้อมูลผู้ใช้งานในส่วน Admin Web

2) การทดสอบประสิทธิภาพของระบบ (System Performance Testing)

ทำการทดสอบโดยจำลองสถานการณ์การใช้งานที่หลากหลาย เช่น

- การเลือกประเทศหรือจุดหมายปลายทางที่แตกต่างกัน
- การกำหนดจำนวนวันเดินทางที่แตกต่างกัน

- การกำหนดจำนวนผู้เดินทางที่แตกต่างกัน
- การกำหนดงบประมาณที่แตกต่างกัน
- การเลือกรูปแบบหรือสไตล์การท่องเที่ยวที่แตกต่างกัน

เพื่อประเมินว่าระบบสามารถ

- สร้างแผนการเดินทางได้ภายในระยะเวลาที่เหมาะสม
- สร้างแผนการเดินทางที่สอดคล้องกับข้อมูลที่ผู้ใช้กำหนด
- แสดงข้อมูลสถานที่และพิกัดได้อย่างเหมาะสม
- แสดงข้อมูลรูปภาพประกอบสถานที่ได้
- บันทึกและเรียกดูข้อมูลแผนการเดินทางได้อย่างถูกต้อง

3.5.2 เครื่องมือที่ใช้ในการประเมินประสิทธิภาพ (Evaluation Tools)

เพื่อให้ผลการทดสอบมีความน่าเชื่อถือ ผู้จัดทำได้ใช้เครื่องมือในการเก็บข้อมูล 2 รูปแบบ ได้แก่

ข้อมูลบันทึกการทำงานของระบบ (System Logs)

เป็นข้อมูลที่ระบบบันทึกโดยอัตโนมัติ เพื่อใช้ตรวจสอบและวิเคราะห์การทำงานของระบบ เช่น

- เวลาในการเริ่มและสิ้นสุดการสร้างแผนทริป
- ระยะเวลาในการประมวลผลของระบบ
- ผลการเรียกใช้งาน Google Gemini API
- ผลการค้นหาพิกัดจาก Nominatim API
- ผลการเรียกข้อมูลรูปภาพจาก Unsplash API
- จำนวนสถานที่ที่ระบบสร้างในแต่ละแผนการเดินทาง
- สถานะการบันทึกข้อมูลลงใน Cloud Firestore
- ข้อผิดพลาดที่เกิดขึ้นระหว่างการทำงานของระบบ

3.5.3 ตัวชี้วัดประสิทธิภาพ (Key Performance Indicators: KPIs)

การประเมินประสิทธิภาพของระบบแบ่งออกเป็น 2 มิติหลัก ได้แก่

1) มิติด้านประสิทธิภาพเชิงเวลา (Time Efficiency)

- **ระยะเวลาในการสร้างแผนทริป (Trip Generation Time)**

  วัดเวลาตั้งแต่ผู้ใช้งานส่งข้อมูลสำหรับสร้างแผนการเดินทาง จนกระทั่งระบบได้รับผลลัพธ์จาก Google Gemini API และสามารถแสดงแผนการเดินทางให้ผู้ใช้งานได้

- **ระยะเวลาในการตอบสนองของระบบ (Response Time)**

  วัดความเร็วในการตอบสนองต่อคำสั่งของผู้ใช้งาน เช่น การเรียกดูข้อมูล การค้นหาพิกัดสถานที่ การโหลดรูปภาพ และการแสดงข้อมูลบนหน้าจอ

2) มิติด้านคุณภาพของผลลัพธ์ (Output Quality)

- **ความสอดคล้องของการแนะนำ (Recommendation Relevance)**

  ประเมินว่าแผนการเดินทางที่ระบบสร้างขึ้นมีความสอดคล้องกับข้อมูลและความต้องการที่ผู้ใช้งานกำหนด เช่น จุดหมายปลายทาง จำนวนวัน งบประมาณ จำนวนผู้เดินทาง และสไตล์การท่องเที่ยว

- **ความเหมาะสมของแผนการเดินทาง (Trip Suitability)**

  ประเมินความเหมาะสมของสถานที่และกิจกรรมที่ถูกจัดไว้ในแต่ละวัน รวมถึงความสมเหตุสมผลของลำดับกิจกรรม

- **คุณภาพของแผนการเดินทาง (Trip Plan Quality)**

  ประเมินความเหมาะสมของการจัดลำดับสถานที่ ความครบถ้วนของข้อมูล และความชัดเจนของรายละเอียดในแต่ละวัน

- **ความถูกต้องของพิกัดสถานที่ (Location Accuracy)**

  ประเมินว่าพิกัดสถานที่ที่ค้นหาโดย Nominatim API และแสดงบนแผนที่ตรงกับสถานที่ที่ปรากฏในแผนการเดินทางหรือไม่

3.5.4 การวิเคราะห์ข้อมูลและสถิติ (Data Analysis and Statistics)

ข้อมูลที่ได้จาก System Logs และแบบสอบถามจะถูกนำมาวิเคราะห์โดยใช้สถิติพื้นฐาน ได้แก่

- **ค่าเฉลี่ย (Mean)**

  ใช้ในการคำนวณค่าเฉลี่ยของระยะเวลาในการทำงานของระบบ รวมถึงคะแนนจากแบบประเมินประสิทธิภาพ

$$\bar{X} = \frac{\sum X}{N}$$

โดยที่

$\bar{X}$ หมายถึง ค่าเฉลี่ย

$\sum X$ หมายถึง ผลรวมของคะแนนทั้งหมด

N หมายถึง จำนวนข้อมูลหรือจำนวนผู้ประเมิน

บทที่ 4
ผลการพัฒนาและการทดสอบระบบ

บทนี้นำเสนอผลการพัฒนาและการทดสอบระบบ **TripAI ระบบออกแบบทริปการท่องเที่ยวด้วยปัญญาประดิษฐ์** ซึ่งพัฒนาด้วย Flutter และภาษา Dart โดยนำ Google Gemini API มาใช้ในการสร้างแผนการเดินทางตามข้อมูลและความต้องการของผู้ใช้งาน พร้อมทั้งเชื่อมต่อกับ Firebase Authentication สำหรับการยืนยันตัวตน Cloud Firestore สำหรับจัดเก็บข้อมูล OpenStreetMap ร่วมกับ flutter_map สำหรับแสดงแผนที่ Nominatim API สำหรับค้นหาพิกัดสถานที่ และ Unsplash API สำหรับแสดงรูปภาพประกอบสถานที่

การนำเสนอผลการพัฒนาแบ่งออกเป็นผลการพัฒนาแอปพลิเคชัน ฟังก์ชันการทำงานหลัก ผลการทดสอบการทำงานของระบบ การประเมินประสิทธิภาพและความถูกต้อง ตลอดจนผลการประเมินความพึงพอใจของผู้ใช้งาน เพื่อแสดงให้เห็นว่าระบบที่พัฒนาขึ้นสามารถทำงานได้ตามวัตถุประสงค์และขอบเขตที่กำหนดไว้

4.1 ผลการพัฒนาโมบายแอปพลิเคชัน

จากการวิเคราะห์และออกแบบระบบในบทที่ 3 ผู้จัดทำได้พัฒนาระบบ TripAI เพื่อช่วยให้ผู้ใช้งานสามารถสร้างแผนการเดินทางตามความต้องการได้อย่างสะดวก โดยระบบรองรับการทำงานผ่านส่วนติดต่อที่พัฒนาด้วย Flutter ซึ่งสามารถแสดงผลและใช้งานบนอุปกรณ์ที่รองรับได้

ระบบถูกออกแบบให้ผู้ใช้งานสามารถดำเนินการตั้งแต่การสมัครสมาชิกและเข้าสู่ระบบ การกำหนดข้อมูลสำหรับวางแผนการเดินทาง เช่น จุดหมายปลายทาง จำนวนวัน จำนวนผู้เดินทาง งบประมาณ และสไตล์การท่องเที่ยว จากนั้นระบบจะนำข้อมูลดังกล่าวไปประมวลผลด้วย Google Gemini API เพื่อสร้างแผนการเดินทางที่เหมาะสมกับเงื่อนไขของผู้ใช้งาน

ผลการพัฒนาส่วนติดต่อผู้ใช้งานประกอบด้วยหน้าจอหลัก ได้แก่ หน้าเริ่มต้น หน้าเข้าสู่ระบบ หน้าสมัครสมาชิก หน้าหลัก หน้าวางแผนทริป หน้ากำลังสร้างแผนการเดินทาง หน้าแสดงรายละเอียดแผนทริป หน้ารายการทริปที่บันทึก และส่วนแสดงข้อมูลสถานที่บนแผนที่ โดยแต่ละหน้าถูกออกแบบให้มีรูปแบบและลำดับการใช้งานที่ต่อเนื่องกัน

นอกจากนี้ ระบบยังมีส่วน **Admin Web** สำหรับผู้ดูแลระบบ โดยผู้ดูแลสามารถเข้าสู่ระบบเพื่อตรวจสอบข้อมูลผู้ใช้งาน ดูรายละเอียดบัญชี ระงับหรือเปิดใช้งานบัญชีใหม่ และลบบัญชีผู้ใช้งานตามสิทธิ์ที่กำหนด ทั้งนี้ การจัดการของผู้ดูแลระบบมุ่งเน้นเฉพาะข้อมูลผู้ใช้งานตามขอบเขตของระบบที่กำหนดไว้

ผลจากการพัฒนาพบว่าระบบสามารถเชื่อมต่อกับบริการต่าง ๆ ที่ใช้ในโครงการได้ ได้แก่ Google Gemini API, Firebase Authentication, Cloud Firestore, Nominatim API, OpenStreetMap และ Unsplash API ทำให้สามารถดำเนินกระบวนการตั้งแต่การรับข้อมูลจากผู้ใช้งาน การสร้างแผนการเดินทาง การค้นหาพิกัดและรูปภาพสถานที่ ตลอดจนการบันทึกข้อมูลแผนการเดินทางได้ตามการออกแบบของระบบ

หน้าแอพพลิเคชั่น

4.1.1 ผลการติดตั้งแอปพลิเคชัน



ภาพที่ 4.1 ผลการติดตั้งแอปพลิเคชัน TripAI บนอุปกรณ์ Android

จากภาพที่ 4.1 แสดงผลการติดตั้งแอปพลิเคชัน TripAI บนอุปกรณ์ Android โดยแอปพลิเคชันสามารถติดตั้งและแสดงไอคอนของระบบบนหน้าจอหลักของอุปกรณ์ได้อย่างถูกต้อง ผู้ใช้งานสามารถเลือกไอคอน TripAI เพื่อเปิดเข้าสู่ระบบและใช้งานฟังก์ชันต่าง ๆ ของแอปพลิเคชันได้

ภาพดังกล่าวแสดงให้เห็นว่าระบบที่พัฒนาด้วย Flutter สามารถนำมาทำงานบนระบบปฏิบัติการ Android ได้ และสามารถใช้งานผ่านอุปกรณ์เคลื่อนที่ตามรูปแบบที่ได้ออกแบบไว้

4.1.2 หน้าเริ่มต้นใช้งานระบบ



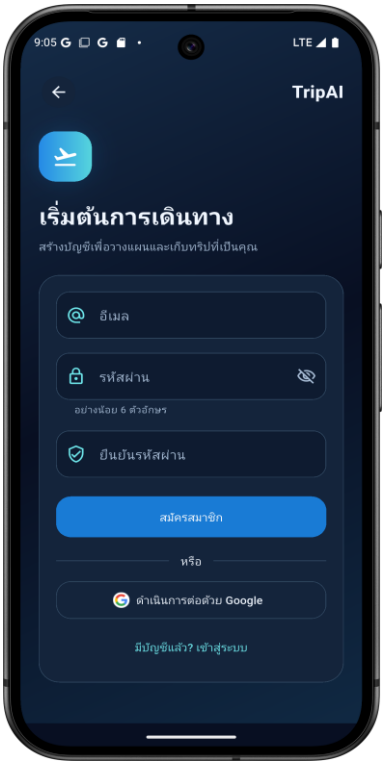
ภาพที่ 4.2 หน้าเริ่มต้นใช้งานของระบบ TripAI

จากภาพที่ 4.2 แสดงหน้าเริ่มต้นใช้งานของระบบ TripAI ซึ่งออกแบบให้สื่อถึงลักษณะของระบบวางแผนการเดินทางด้วยปัญญาประดิษฐ์ โดยภายในหน้าจอประกอบด้วยโลโก้และชื่อระบบ TripAI พร้อมข้อความแนะนำคุณสมบัติหลัก เช่น การใช้ AI ช่วยวางแผนการเดินทาง การปรับแผนตามความต้องการของผู้ใช้งาน และการบันทึกแผนการเดินทางไว้สำหรับเรียกดูภายหลัง

บริเวณส่วนกลางของหน้าจอมีข้อความ “โลกกว้างแค่ไหน ก็ไปได้ในแบบคุณ” เพื่อสื่อถึงแนวคิดของระบบในการสร้างแผนการเดินทางให้เหมาะสมกับความต้องการของผู้ใช้งานแต่ละคน พร้อมแสดงคำอธิบายเกี่ยวกับการเลือกจุดหมายและการให้ AI ช่วยจัดแผนการเดินทาง

บริเวณด้านล่างมีปุ่ม “เริ่มต้นใช้งาน” สำหรับเข้าสู่ขั้นตอนการใช้งานระบบ และมีลิงก์ “เข้าสู่ระบบ” สำหรับผู้ใช้งานที่มีบัญชีอยู่แล้ว ทำให้ผู้ใช้งานสามารถเลือกดำเนินการต่อได้ตามสถานะของบัญชีตนเองอย่างชัดเจน

4.1.3 หน้าสมัครสมาชิก



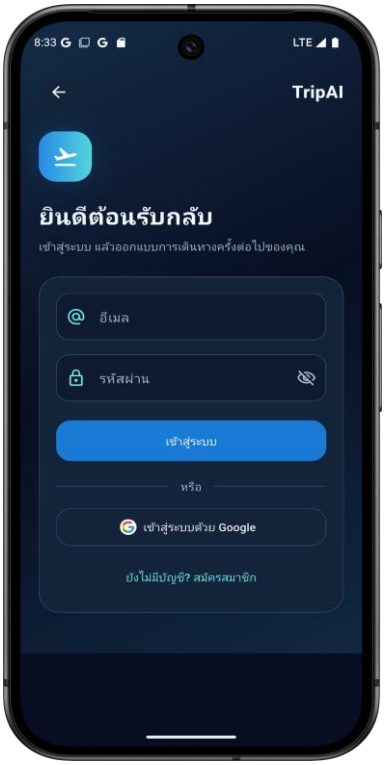
ภาพที่ 4.3 หน้าสมัครสมาชิกของระบบ TripAI

จากภาพที่ 4.4 แสดงหน้าสมัครสมาชิกของระบบ TripAI สำหรับผู้ใช้งานที่ยังไม่มีบัญชี โดยผู้ใช้งานจะต้องกรอก **อีเมล รหัสผ่าน และยืนยันรหัสผ่าน** เพื่อใช้ในการสร้างบัญชีใหม่ภายในระบบ

ระบบกำหนดให้รหัสผ่านมีความยาวอย่างน้อย **6 ตัวอักษร** และผู้ใช้งานต้องกรอกรหัสผ่านยืนยันให้ตรงกับรหัสผ่านที่กำหนดไว้ ก่อนกดปุ่ม **“สมัครสมาชิก”** เพื่อดำเนินการสร้างบัญชีผ่าน Firebase Authentication

บริเวณด้านล่างมีลิงก์ **“กลับไปเข้าสู่ระบบ”** สำหรับผู้ใช้งานที่มีบัญชีอยู่แล้ว ช่วยให้สามารถสลับไปยังหน้าเข้าสู่ระบบได้อย่างสะดวก โดยรูปแบบของหน้าจอยังคงใช้โทนสีและองค์ประกอบเดียวกับส่วนอื่นของ TripAI เพื่อให้การใช้งานมีความต่อเนื่องและเป็นไปในทิศทางเดียวกัน

4.1.4 หน้าเข้าสู่ระบบ



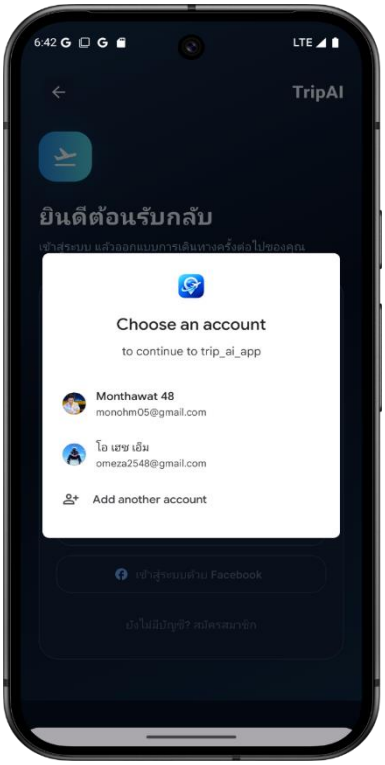
ภาพที่ 4.4 หน้าเข้าสู่ระบบของระบบ TripAI

จากภาพที่ 4.3 แสดงหน้าเข้าสู่ระบบของ TripAI ซึ่งใช้สำหรับยืนยันตัวตนของผู้ใช้งานก่อนเข้าสู่ฟังก์ชันหลักของระบบ โดยผู้ใช้งานสามารถกรอก **อีเมล** และ **รหัสผ่าน** จากนั้นกดปุ่ม **“เข้าสู่ระบบ”** เพื่อให้ระบบตรวจสอบข้อมูลบัญชีผู้ใช้งาน

นอกจากนี้ ระบบยังรองรับการเข้าสู่ระบบด้วยบัญชี **Google** เพื่อเพิ่มความสะดวกในการใช้งาน และมีลิงก์ **“สมัครสมาชิก”** สำหรับผู้ใช้งานที่ยังไม่มีบัญชี โดยภายในช่องรหัสผ่านมีฟังก์ชันสำหรับแสดงหรือซ่อนรหัสผ่าน เพื่อช่วยให้ผู้ใช้งานสามารถตรวจสอบข้อมูลก่อนเข้าสู่ระบบได้สะดวกยิ่งขึ้น

หน้าจอถูกออกแบบให้มีรูปแบบสอดคล้องกับเอกลักษณ์ของ TripAI โดยใช้โทนสีน้ำเงินเข้มและสีฟ้า พร้อมจัดวางช่องกรอกข้อมูลและปุ่มดำเนินการอย่างชัดเจน ทำให้ผู้ใช้งานสามารถเข้าใจขั้นตอนการเข้าสู่ระบบได้ง่าย

4.1.5 หน้าเข้าสู่ระบบด้วยบัญชี Google



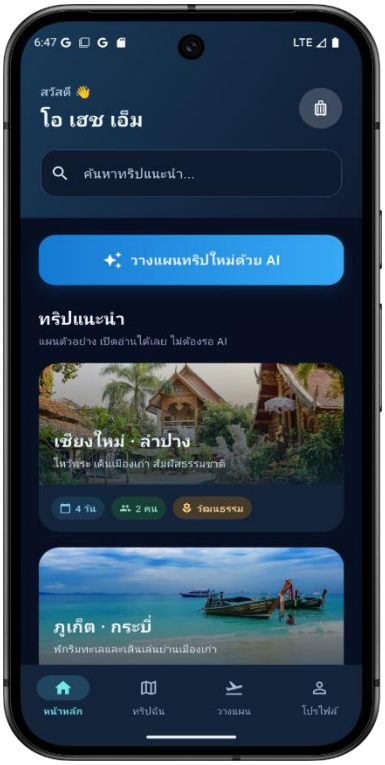
ภาพที่ 4.5 หน้าเลือกบัญชี Google สำหรับเข้าสู่ระบบ TripAI

จากภาพที่ 4.5 แสดงขั้นตอนการเข้าสู่ระบบด้วยบัญชี Google ของระบบ TripAI โดยเมื่อผู้ใช้งานเลือกคำสั่ง **“เข้าสู่ระบบด้วย Google”** ระบบจะแสดงหน้าต่างสำหรับเลือกบัญชี Google ที่มีอยู่บนอุปกรณ์ หรือสามารถเลือก **“Add another account”** เพื่อเพิ่มบัญชีใหม่สำหรับใช้ในการเข้าสู่ระบบ

เมื่อผู้ใช้งานเลือกบัญชีที่ต้องการ ระบบจะดำเนินการยืนยันตัวตนผ่าน Google Sign-In และ Firebase Authentication จากนั้นจึงนำผู้ใช้งานเข้าสู่ส่วนหลักของ TripAI โดยไม่จำเป็นต้องกรอกอีเมลและรหัสผ่านภายในแอปพลิเคชันอีกครั้ง

ฟังก์ชันดังกล่าวช่วยเพิ่มความสะดวกในการเข้าสู่ระบบ และลดขั้นตอนในการยืนยันตัวตนสำหรับผู้ใช้งานที่มีบัญชี Google อยู่แล้ว

4.1.6 หน้าหลักของระบบ



ภาพที่ 4.6 หน้าหลักของระบบ TripAI

จากภาพที่ 4.6 แสดงหน้าหลักของระบบ TripAI หลังจากผู้ใช้งานเข้าสู่ระบบสำเร็จ โดยระบบจะแสดงชื่อผู้ใช้งาน ช่องค้นหาทริปแนะนำ และปุ่ม **“วางแผนทริปใหม่ด้วย AI”** สำหรับเข้าสู่ขั้นตอนการสร้างแผนการเดินทางใหม่

บริเวณส่วนกลางของหน้าจอแสดง **ทริปแนะนำ** ในรูปแบบการ์ด ซึ่งประกอบด้วยรูปภาพ ชื่อจุดหมายปลายทาง คำอธิบาย จำนวนวัน จำนวนผู้เดินทาง และสไตล์การท่องเที่ยว เพื่อให้ผู้ใช้งานสามารถเลือกดูตัวอย่างแผนการเดินทางได้อย่างสะดวก

บริเวณด้านล่างมีแถบนำทางหลัก ประกอบด้วย **หน้าหลัก ทริปฉัน วางแผน และโปรไฟล์** เพื่อให้ผู้ใช้งานสามารถเปลี่ยนไปยังฟังก์ชันต่าง ๆ ของระบบได้อย่างรวดเร็วและเป็นระเบียบ.

4.1.7 หน้ารายละเอียดทริปแนะนำ



ภาพที่ 4.7 หน้าแสดงรายละเอียดทริปแนะนำของระบบ TripAI

จากภาพที่ 4.7 แสดงหน้ารายละเอียดของทริปแนะนำที่ผู้ใช้งานเลือกจากหน้าหลัก โดยระบบจะแสดงแผนการเดินทางตัวอย่างตามจุดหมายปลายทาง เช่น **เชียงใหม่–ลำปาง** พร้อมรายละเอียดกิจกรรมในแต่ละวันอย่างเป็นลำดับ

ภายในแผนจะแบ่งรายละเอียดตามวันและช่วงเวลา ได้แก่ **ช่วงเช้า ช่วงบ่าย และช่วงเย็น** พร้อมแนะนำสถานที่ กิจกรรม และแนวทางการเดินทาง เพื่อช่วยให้ผู้ใช้งานสามารถใช้เป็นแนวคิดประกอบการวางแผนทริปของตนเองได้

หน้าทริปแนะนำมีวัตถุประสงค์เพื่อแสดงตัวอย่างรูปแบบแผนการเดินทางให้ผู้ใช้งานสามารถเปิดอ่านได้ทันที โดยไม่จำเป็นต้องรอการประมวลผลจาก AI และสามารถนำข้อมูลดังกล่าวไปประยุกต์ใช้กับการวางแผนการเดินทางของตนเองได้ตามความเหมาะสม

4.1.8 ค้นหาทริปแนะนำ



ภาพที่ 4.8 หน้าค้นหาทริปแนะนำของระบบ TripAI

จากภาพที่ 4.8 แสดงการใช้งานฟังก์ชันค้นหาทริปแนะนำภายในหน้าหลักของระบบ TripAI โดยผู้ใช้งานสามารถกรอกชื่อจังหวัดหรือจุดหมายปลายทางที่ต้องการในช่องค้นหา เช่น **กระบี่** เพื่อให้ระบบกรองรายการทริปแนะนำที่เกี่ยวข้องกับคำค้นดังกล่าว

เมื่อมีการกรอกคำค้น ระบบจะแสดงเฉพาะรายการทริปที่ตรงหรือสัมพันธ์กับคำค้น เช่น **ภูเก็ต–กระบี่** พร้อมข้อมูลประกอบ ได้แก่ รูปภาพ ชื่อทริป คำอธิบาย จำนวนวัน จำนวนผู้เดินทาง และสไตล์การท่องเที่ยว ทำให้ผู้ใช้งานสามารถค้นหาและเลือกดูทริปแนะนำที่สนใจได้อย่างสะดวกและรวดเร็วยิ่งขึ้น

ฟังก์ชันค้นหาดังกล่าวช่วยลดเวลาในการค้นหารายการทริปจากข้อมูลทั้งหมด และช่วยให้ผู้ใช้งานเข้าถึงทริปที่ตรงกับจุดหมายปลายทางที่ต้องการได้ง่ายมากขึ้น

4.1.9 หน้าทริปฉัน

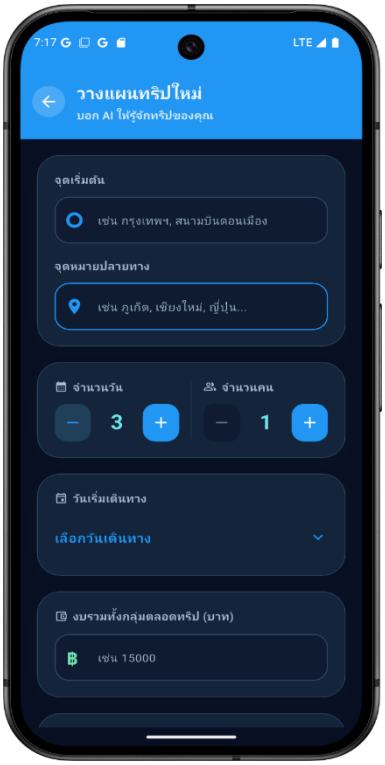


ภาพที่ 4.9 หน้าทริปฉันของระบบ TripAI

จากภาพที่ 4.9 แสดงหน้าทริปฉันของระบบ TripAI ซึ่งใช้สำหรับแสดงรายการแผนการเดินทางที่ผู้ใช้งานได้บันทึกไว้ภายในระบบ โดยในกรณีที่ผู้ใช้งานยังไม่มีทริปที่บันทึก ระบบจะแสดงข้อความ **“ยังไม่มีทริปที่บันทึกไว้ เริ่มสร้างทริปแรกของคุณกันเลย!”** เพื่อแจ้งสถานะให้ผู้ใช้งานทราบอย่างชัดเจน

บริเวณด้านล่างของหน้าจอมีปุ่ม **“สร้างทริปใหม่”** สำหรับนำผู้ใช้งานไปยังขั้นตอนการสร้างแผนการเดินทางด้วย AI ได้โดยตรง นอกจากนี้ ยังมีแถบนำทางหลักสำหรับเข้าสู่ **หน้าหลัก ทริปฉัน วางแผน และโปรไฟล์** ทำให้ผู้ใช้งานสามารถเข้าถึงฟังก์ชันต่าง ๆ ของระบบได้อย่างสะดวกและต่อเนื่อง

4.1.10 หน้าวางแผนทริปใหม่

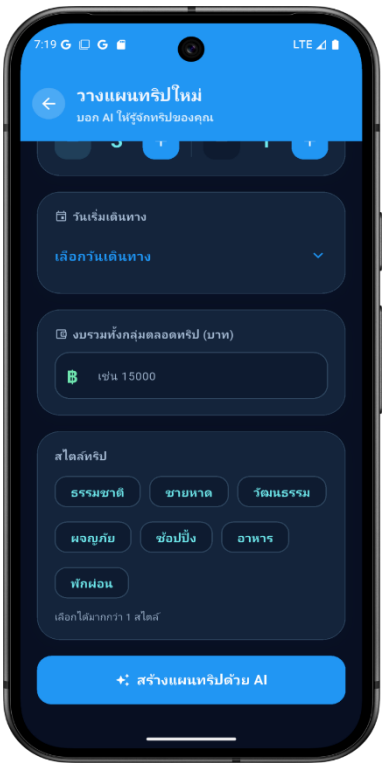


ภาพที่ 4.10 หน้าวางแผนทริปใหม่ของระบบ TripAI

จากภาพที่ 4.10 แสดงหน้าวางแผนทริปใหม่ของระบบ TripAI ซึ่งใช้สำหรับให้ผู้ใช้งานกำหนดข้อมูลเบื้องต้นก่อนสร้างแผนการเดินทางด้วย AI โดยสามารถระบุ **จุดเริ่มต้น จุดหมายปลายทาง จำนวนวัน จำนวนผู้เดินทาง วันเริ่มเดินทาง และงบประมาณรวมของกลุ่มตลอดทริป** ได้ตามความต้องการ

ระบบจัดแบ่งข้อมูลออกเป็นส่วนอย่างชัดเจน และมีปุ่มสำหรับเพิ่มหรือลดจำนวนวันและจำนวนผู้เดินทาง เพื่อช่วยให้ผู้ใช้งานปรับข้อมูลได้สะดวก นอกจากนี้ ผู้ใช้งานยังสามารถเลือกวันเริ่มเดินทางและกำหนดงบประมาณ เพื่อให้ระบบนำข้อมูลดังกล่าวไปใช้เป็นเงื่อนไขในการสร้างแผนการเดินทางให้เหมาะสมกับทริป

เมื่อผู้ใช้งานกรอกข้อมูลที่จำเป็นครบถ้วนแล้ว ระบบจะนำข้อมูลทั้งหมดไปใช้ประกอบการประมวลผลในขั้นตอนการสร้างแผนการเดินทางด้วย AI ต่อไป



ภาพที่ 4.11 ส่วนกำหนดงบประมาณและสไตล์การท่องเที่ยวของระบบ TripAI

จากภาพที่ 4.11 แสดงส่วนต่อเนื่องของหน้าวางแผนทริปใหม่ โดยผู้ใช้งานสามารถกำหนด **วันเริ่มเดินทาง งบประมาณรวมของกลุ่มตลอดทริป และสไตล์การท่องเที่ยว** เพื่อใช้เป็นเงื่อนไขในการสร้างแผนการเดินทางด้วย AI

ในส่วนของสไตล์การท่องเที่ยว ผู้ใช้งานสามารถเลือกได้มากกว่า 1 รูปแบบ ได้แก่ **ธรรมชาติ ชายหาด วัฒนธรรม ผจญภัย ช้อปปิ้ง อาหาร และพักผ่อน** เพื่อให้ระบบสามารถนำความสนใจของผู้ใช้งานไปประกอบการสร้างแผนการเดินทางให้เหมาะสมมากยิ่งขึ้น

เมื่อกำหนดข้อมูลทั้งหมดครบถ้วนแล้ว ผู้ใช้งานสามารถกดปุ่ม **“สร้างแผนทริปด้วย AI”** เพื่อส่งข้อมูลไปยังขั้นตอนการประมวลผลและสร้างแผนการเดินทางต่อไป

4.1.11 หน้ากำลังสร้างแผนการเดินทาง



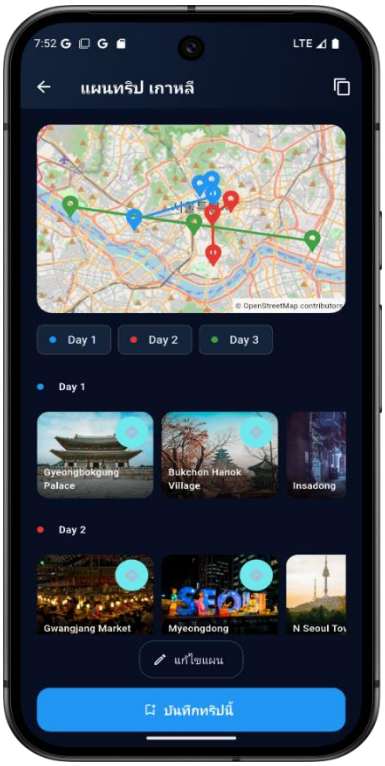
ภาพที่ 4.12 หน้ากำลังสร้างแผนการเดินทางด้วย AI ของระบบ TripAI

จากภาพที่ 4.12 แสดงหน้าสถานะระหว่างการสร้างแผนการเดินทางของระบบ TripAI หลังจากผู้ใช้งานกรอกข้อมูลและกดปุ่ม **“สร้างแผนทริปด้วย AI”** แล้ว ระบบจะนำข้อมูลที่กำหนดไว้ไปประมวลผลเพื่อสร้างแผนการเดินทางตามความต้องการของผู้ใช้งาน

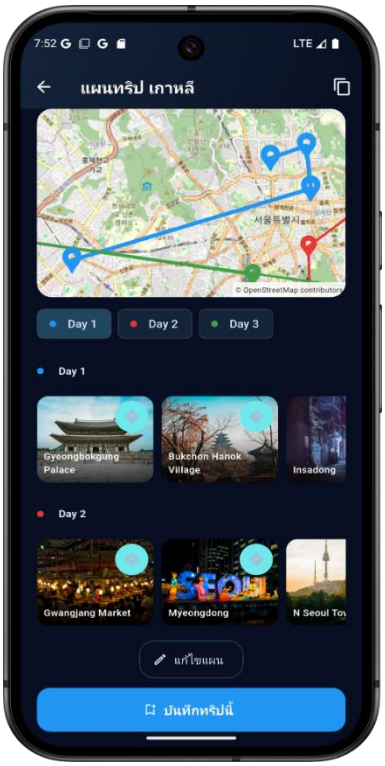
หน้าจอจะแสดงจุดหมายปลายทางและจำนวนวันที่กำหนด พร้อมแสดงความคืบหน้าของการทำงาน เช่น **การรับข้อมูลทริป การวางแผนสถานที่และที่พัก และการจับคู่สถานที่บนแผนที่** เพื่อให้ผู้ใช้งานทราบว่าระบบกำลังดำเนินการในขั้นตอนใด

เมื่อการประมวลผลเสร็จสมบูรณ์ ระบบจะแสดงผลลัพธ์เป็นรายละเอียดแผนการเดินทาง ซึ่งผู้ใช้งานสามารถตรวจสอบและปรับแก้แผนก่อนบันทึกเป็นทริปของตนเองได้

4.1.12 หน้าแสดงผลแผนการเดินทาง



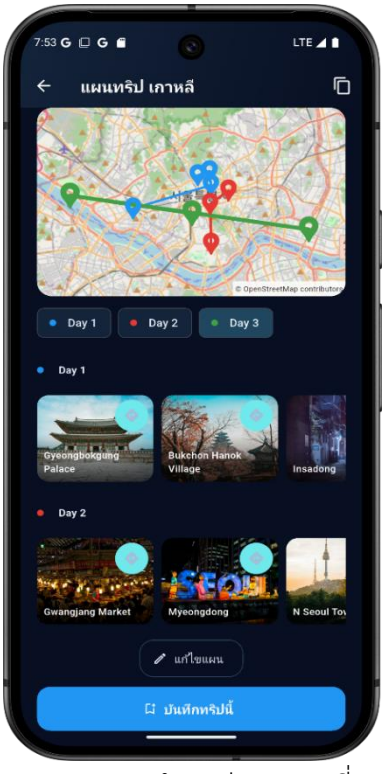
ภาพที่ 4.13 หน้าแสดงผลแผนการเดินทางของระบบ TripAI



ภาพที่ 4.14 การแสดงตำแหน่งบนแผนที่ของ Day 1



ภาพที่ 4.15 การแสดงตำแหน่งบนแผนที่ของ Day 2



ภาพที่ 4.16 การแสดงตำแหน่งบนแผนที่ของ Day 3



ภาพที่ 4.17 หน้าแสดงรายละเอียดแผนการเดินทางหลังบันทึก

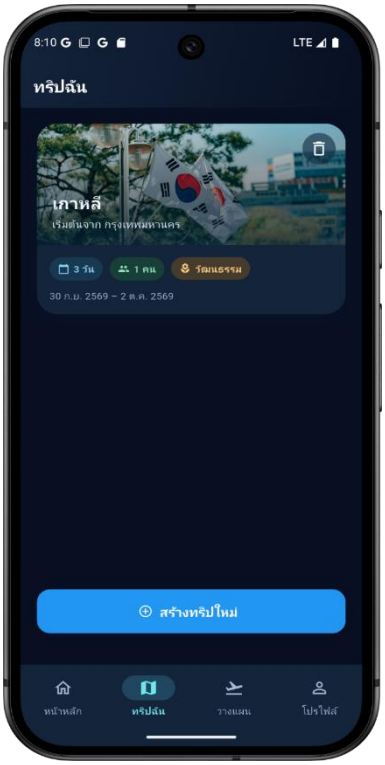
จากภาพที่ 4.13 เป็นหน้าแสดงผลแผนการเดินทางที่ระบบ TripAI สร้างขึ้น โดยส่วนบนของหน้าจอจะแสดงแผนที่จาก OpenStreetMap พร้อมหมุดตำแหน่งสถานที่ท่องเที่ยวและเส้นเชื่อมเส้นทางการเดินทางในแต่ละวัน ซึ่งใช้สีที่แตกต่างกันเพื่อแบ่งแผนการเดินทางรายวัน เช่น Day 1, Day 2 และ Day 3 ทำให้ผู้ใช้งานสามารถมองเห็นภาพรวมของทริปได้อย่างชัดเจน

ด้านล่างของแผนที่ ระบบจะแสดงปุ่มเลือกวัน ได้แก่ Day 1, Day 2 และ Day 3 เพื่อให้ผู้ใช้งานสามารถเลือกดูรายละเอียดของแผนในแต่ละวันได้ เมื่อผู้ใช้งานกดเลือกวันที่ต้องการ ระบบจะเลื่อนหรือปรับตำแหน่งแผนที่ไปยังจุดหมายและเส้นทางของวันนั้นโดยอัตโนมัติ ดังแสดงในภาพที่ 4.14, 4.15 และ 4.16 ซึ่งช่วยให้ผู้ใช้งานสามารถดูตำแหน่งสถานที่ของแต่ละวันได้สะดวกมากยิ่งขึ้น

นอกจากนี้ ในแต่ละวันยังมีการแสดงสถานที่ท่องเที่ยวในรูปแบบการ์ด โดยมีรูปภาพและชื่อสถานที่ประกอบ เช่น Gyeongbokgung Palace, Bukchon Hanok Village, Insadong, Gwangjang Market, Myeongdong และ N Seoul Tower ผู้ใช้งานสามารถกดที่รูปภาพของสถานที่แต่ละแห่งได้ ซึ่งเมื่อกดแล้ว ระบบจะเชื่อมโยงไปยังแผนที่เพื่อแสดงเส้นทางการเดินทางไปยังสถานที่นั้นโดยตรง ช่วยให้ผู้ใช้งานสามารถดูเส้นทางและนำทางได้ทันที

บริเวณด้านล่างของหน้าจอยังมีปุ่ม “แก้ไขแผน” สำหรับปรับเปลี่ยนรายละเอียดของแผนการเดินทาง และปุ่ม “บันทึกทริปนี้” สำหรับบันทึกแผนการเดินทางไว้ในระบบ เมื่อบันทึกสำเร็จ ระบบจะแสดงสถานะ “บันทึกแล้ว” เพื่อยืนยันการบันทึกข้อมูลให้แก่ผู้ใช้งาน

4.1.13 หน้าทริปฉันหลังบันทึกแผนการเดินทาง



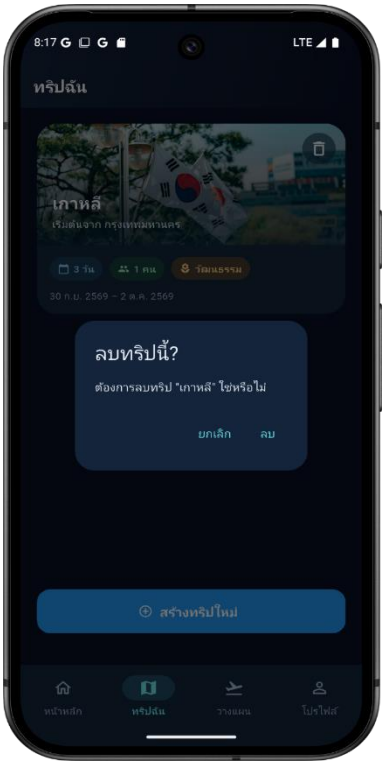
ภาพที่ 4.17 หน้าแสดงทริปที่บันทึกไว้ของระบบ TripAI

จากภาพที่ 4.17 แสดงหน้าทริปฉันหลังจากผู้ใช้งานบันทึกแผนการเดินทางเรียบร้อยแล้ว โดยระบบจะแสดงแผนการเดินทางที่บันทึกไว้ในรูปแบบการ์ด พร้อมข้อมูลสำคัญ ได้แก่ ชื่อจุดหมายปลายทาง จุดเริ่มต้น จำนวนวัน จำนวนผู้เดินทาง สไตล์การท่องเที่ยว และช่วงวันที่เดินทาง

ผู้ใช้งานสามารถเลือกการ์ดทริปเพื่อเปิดดูรายละเอียดของแผนการเดินทางที่บันทึกไว้ได้อีกครั้ง และสามารถกดไอคอน **ลบ** บริเวณมุมของการ์ดเพื่อลบแผนการเดินทางที่ไม่ต้องการออกจากรายการ นอกจากนี้ ยังมีปุ่ม **“สร้างทริปใหม่”** สำหรับเข้าสู่ขั้นตอนการวางแผนการเดินทางใหม่ได้โดยตรง

หน้าดังกล่าวช่วยให้ผู้ใช้งานสามารถจัดเก็บ เรียกดู และจัดการแผนการเดินทางของตนเองได้อย่างสะดวก โดยข้อมูลทริปจะเชื่อมโยงกับบัญชีผู้ใช้งานและจัดเก็บไว้ในระบบ Cloud Firestore

4.1.14 หน้ายืนยันการลบแผนการเดินทาง



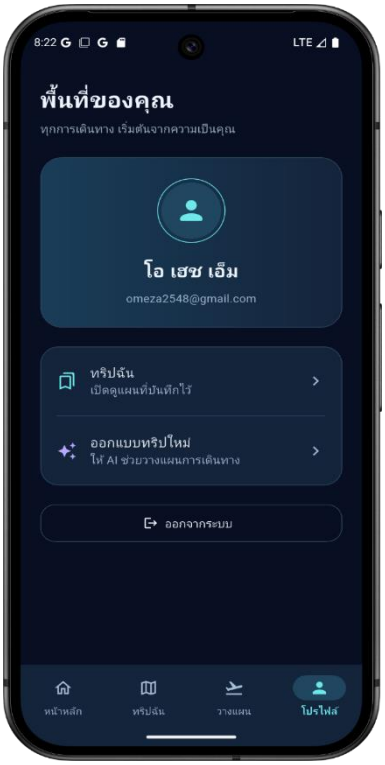
ภาพที่ 4.18 หน้ายืนยันการลบแผนการเดินทางของระบบ TripAI

จากภาพที่ 4.18 แสดงขั้นตอนการลบแผนการเดินทางที่ผู้ใช้งานบันทึกไว้ โดยเมื่อผู้ใช้งานกดไอคอนลบบนการ์ดทริป ระบบจะแสดงหน้าต่างยืนยันพร้อมชื่อทริปที่ต้องการลบ เพื่อให้ผู้ใช้งานตรวจสอบก่อนดำเนินการจริง

ภายในหน้าต่างยืนยันมีตัวเลือก **“ยกเลิก”** และ **“ลบ”** หากผู้ใช้งานเลือกยกเลิก ระบบจะปิดหน้าต่างและเก็บข้อมูลทริปไว้ตามเดิม แต่หากเลือก **“ลบ”** ระบบจะลบข้อมูลแผนการเดินทางดังกล่าวออกจากรายการทริปที่บันทึกไว้

การเพิ่มขั้นตอนยืนยันก่อนลบช่วยลดความผิดพลาดจากการกดลบโดยไม่ได้ตั้งใจ และช่วยให้การจัดการข้อมูลแผนการเดินทางของผู้ใช้งานมีความปลอดภัยและชัดเจนมากยิ่งขึ้น

4.1.15 หน้าโปรไฟล์ผู้ใช้งาน



ภาพที่ 4.19 หน้าโปรไฟล์ผู้ใช้งานของระบบ TripAI

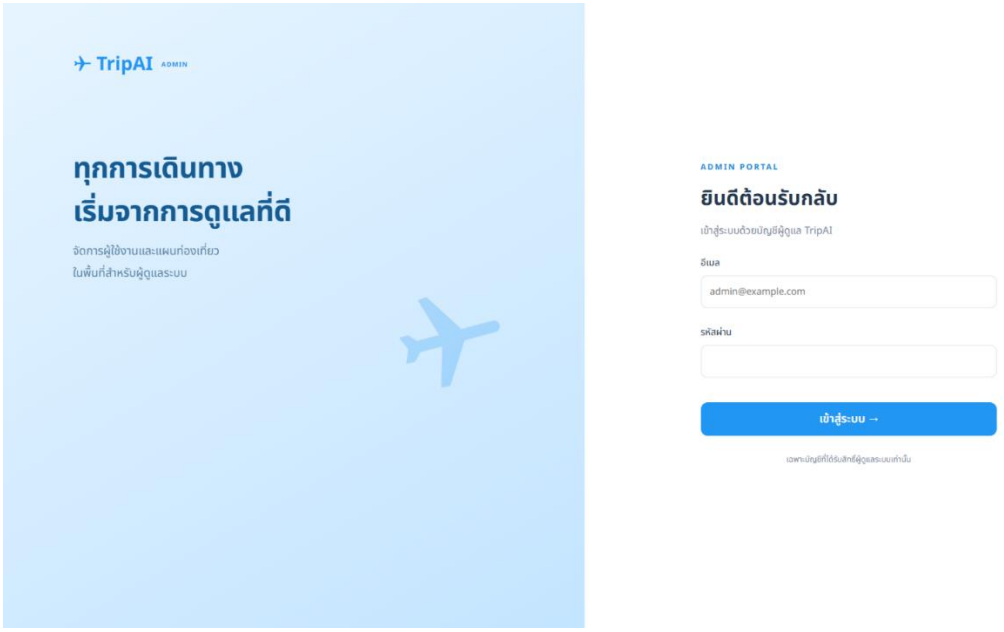
จากภาพที่ 4.19 แสดงหน้าโปรไฟล์ของผู้ใช้งานภายในระบบ TripAI โดยจะแสดงข้อมูลพื้นฐานของบัญชี ได้แก่ **ชื่อผู้ใช้งานและอีเมล** พร้อมส่วนเมนูสำหรับเข้าถึงฟังก์ชันที่เกี่ยวข้องกับบัญชีและการเดินทาง

ผู้ใช้งานสามารถเลือกเมนู **“ทริปฉัน”** เพื่อดูแผนการเดินทางที่บันทึกไว้ หรือเลือก **“ออกแบบทริปใหม่”** เพื่อเข้าสู่หน้าสร้างแผนการเดินทางด้วย AI ได้โดยตรง นอกจากนี้ ยังมีปุ่ม **“ออกจากระบบ”** สำหรับสิ้นสุดการใช้งานบัญชีปัจจุบัน

บริเวณด้านล่างของหน้าจอยังคงแสดงแถบนำทางหลักของระบบ ได้แก่ **หน้าหลัก ทริปฉัน วางแผน และโปรไฟล์** ซึ่งช่วยให้ผู้ใช้งานสามารถเปลี่ยนไปยังส่วนต่าง ๆ ของแอปพลิเคชันได้อย่างสะดวกและต่อเนื่อง

หน้า Web Admin

4.1.16 หน้าเข้าสู่ระบบผู้ดูแลระบบ



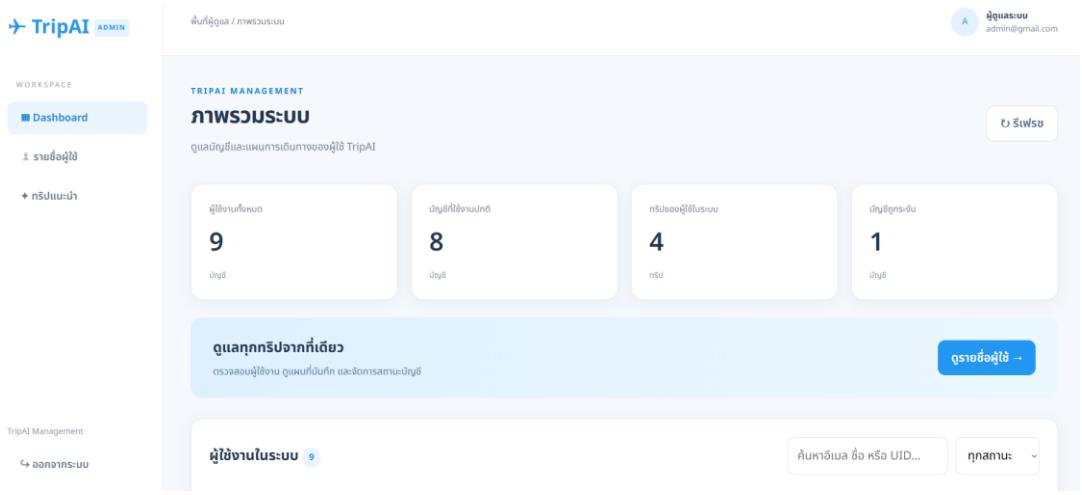
ภาพที่ 4.20 หน้าเข้าสู่ระบบผู้ดูแลระบบ TripAI Admin

จากภาพที่ 4.20 แสดงหน้าเข้าสู่ระบบสำหรับผู้ดูแลระบบของ TripAI ซึ่งพัฒนาขึ้นในรูปแบบเว็บแอปพลิเคชัน โดยผู้ดูแลระบบจะต้องกรอก **อีเมล** และ **รหัสผ่าน** เพื่อยืนยันตัวตนก่อนเข้าสู่ส่วนจัดการระบบ

หน้าจอถูกออกแบบให้แยกส่วนของผู้ดูแลระบบออกจากส่วนของผู้ใช้งานทั่วไปอย่างชัดเจน พร้อมแสดงชื่อ **TripAI Admin** และข้อความอธิบายเกี่ยวกับพื้นที่สำหรับจัดการข้อมูลผู้ใช้งานและแผนท่องเที่ยว เมื่อกรอกข้อมูลครบถ้วนแล้ว ผู้ดูแลสามารถกดปุ่ม **“เข้าสู่ระบบ”** เพื่อให้ระบบตรวจสอบข้อมูลและสิทธิ์ของบัญชี

หากข้อมูลถูกต้องและบัญชีมีสิทธิ์เป็นผู้ดูแลระบบ ระบบจะอนุญาตให้เข้าสู่หน้าจัดการข้อมูลภายใน Admin Web แต่หากข้อมูลไม่ถูกต้องหรือไม่มีสิทธิ์ ระบบจะไม่อนุญาตให้เข้าถึงส่วนดังกล่าว เพื่อช่วยควบคุมการเข้าถึงข้อมูลภายในระบบให้เหมาะสมตามสิทธิ์ของผู้ใช้งาน

4.1.17 หน้า Dashboard ผู้ดูแลระบบ



ภาพที่ 4.21 หน้า Dashboard ของระบบ TripAI Admin

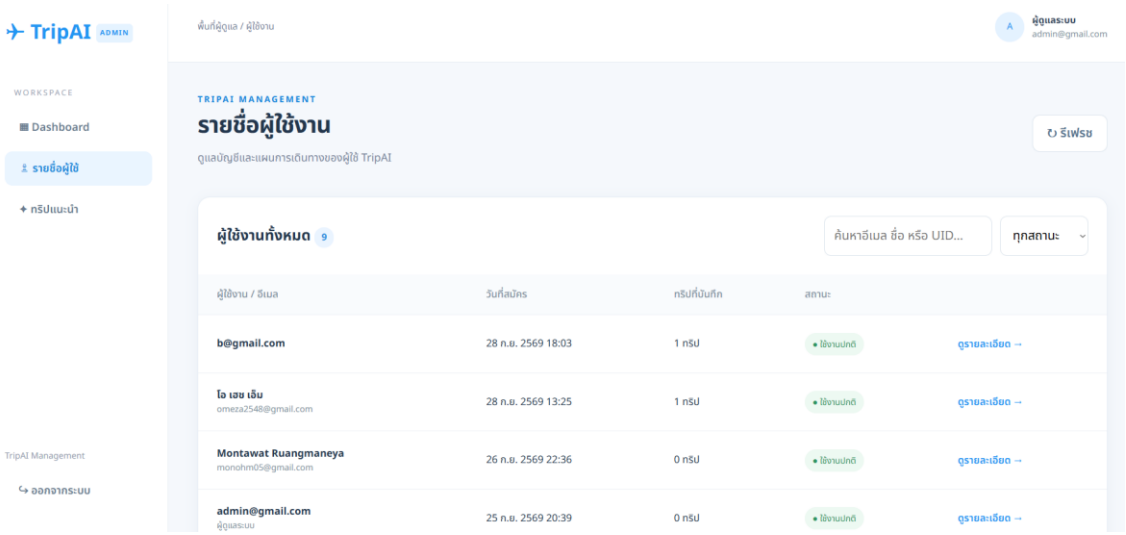
จากภาพที่ 4.21 แสดงหน้า Dashboard ของระบบ TripAI Admin หลังจากผู้ดูแลระบบเข้าสู่ระบบสำเร็จ โดยหน้าจอใช้สำหรับแสดงข้อมูลภาพรวมของระบบ เพื่อช่วยให้ผู้ดูแลสามารถตรวจสอบสถานะการใช้งานได้อย่างรวดเร็ว

บริเวณส่วนบนของหน้าแสดงข้อมูลสรุป ได้แก่ **จำนวนผู้ใช้งานทั้งหมด จำนวนบัญชีที่ใช้งานปกติ จำนวนทริปของผู้ใช้ในระบบ และจำนวนบัญชีที่ถูกระงับ** พร้อมปุ่ม **“รีเฟรช”** สำหรับปรับปรุงข้อมูลให้เป็นปัจจุบัน

ด้านซ้ายมีเมนูสำหรับเข้าสู่ส่วนต่าง ๆ ของระบบ ได้แก่ **Dashboard รายชื่อผู้ใช้ และทริปแนะนำ** ส่วนบริเวณด้านล่างแสดงรายชื่อผู้ใช้งานในระบบ พร้อมช่องค้นหาจากอีเมล ชื่อ หรือ UID และตัวกรองสถานะบัญชี เพื่อช่วยให้ผู้ดูแลสามารถตรวจสอบและจัดการข้อมูลผู้ใช้งานได้สะดวกยิ่งขึ้น

หน้า Dashboard จึงทำหน้าที่เป็นศูนย์กลางสำหรับติดตามข้อมูลภาพรวมและเข้าถึงฟังก์ชันการจัดการต่าง ๆ ของ TripAI Admin

4.1.18 หน้ารายชื่อผู้ใช้งาน



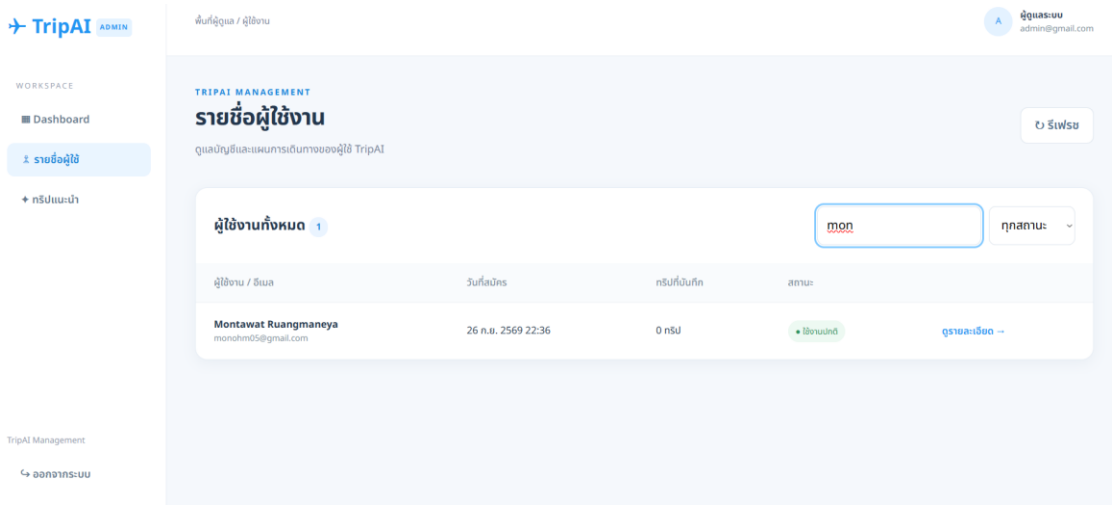
ภาพที่ 4.22 หน้ารายชื่อผู้ใช้งานของระบบ TripAI Admin

จากภาพที่ 4.22 แสดงหน้ารายชื่อผู้ใช้งานในส่วน TripAI Admin ซึ่งใช้สำหรับให้ผู้ดูแลระบบตรวจสอบข้อมูลบัญชีผู้ใช้งานทั้งหมดภายในระบบ โดยแสดงข้อมูลในรูปแบบตาราง ประกอบด้วยข้อมูลสำคัญ ได้แก่ ชื่อหรืออีเมลของผู้ใช้งาน วันที่สมัคร จำนวนทริปที่บันทึก และสถานะของบัญชี

บริเวณด้านบนของตารางมีช่องค้นหาสำหรับค้นหาผู้ใช้งานจากอีเมล ชื่อ หรือ UID และมีตัวกรองสถานะบัญชี เพื่อช่วยให้ผู้ดูแลสามารถค้นหาและตรวจสอบข้อมูลผู้ใช้งานได้สะดวกยิ่งขึ้น

ในแต่ละรายการมีคำสั่ง **“ดูรายละเอียด”** สำหรับเข้าสู่หน้าข้อมูลของผู้ใช้งานแต่ละราย ซึ่งช่วยให้ผู้ดูแลสามารถตรวจสอบข้อมูลบัญชีและดำเนินการจัดการตามสิทธิ์ที่กำหนดไว้ได้อย่างเป็นระบบ

4.1.19 หน้าค้นหาผู้ใช้งาน



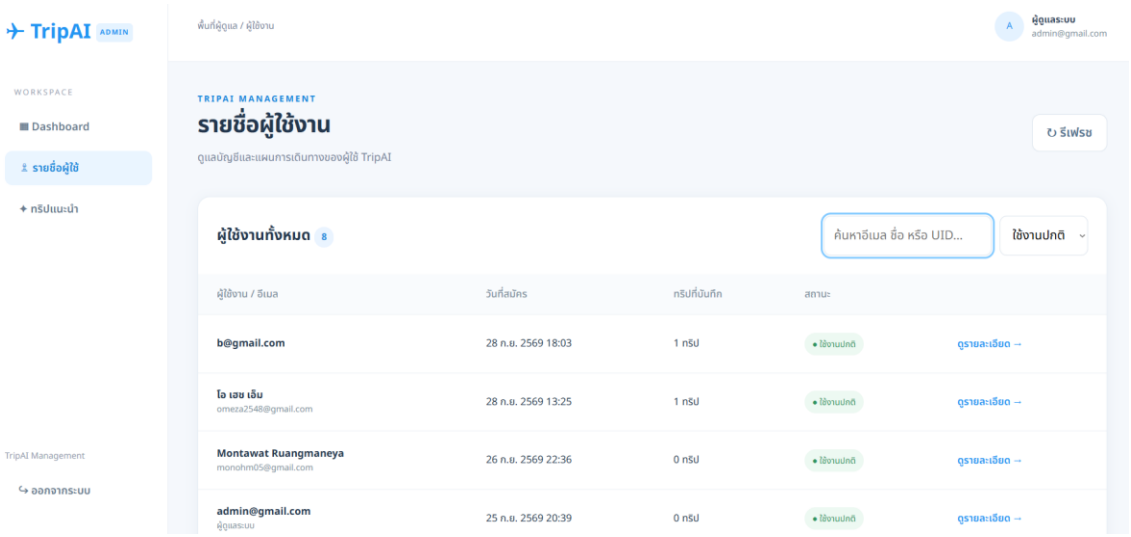
ภาพที่ 4.23 หน้าค้นหาผู้ใช้งานของระบบ TripAI Admin

จากภาพที่ 4.23 แสดงการใช้งานฟังก์ชันค้นหาผู้ใช้งานภายในระบบ TripAI Admin โดยผู้ดูแลระบบสามารถกรอกคำค้นในช่องค้นหา เช่น ชื่อ อีเมล หรือ UID ของผู้ใช้งาน เพื่อให้ระบบกรองและแสดงเฉพาะข้อมูลที่ตรงกับคำค้นดังกล่าว

เมื่อมีการกรอกคำค้น ระบบจะแสดงผลลัพธ์ที่เกี่ยวข้องในตารางทันที พร้อมข้อมูลสำคัญของผู้ใช้งาน ได้แก่ ชื่อหรืออีเมล วันที่สมัคร จำนวนทริปที่บันทึก และสถานะของบัญชี นอกจากนี้ ผู้ดูแลยังสามารถใช้ตัวกรองสถานะร่วมกับการค้นหา เพื่อช่วยให้การตรวจสอบบัญชีผู้ใช้งานทำได้สะดวกและรวดเร็วยิ่งขึ้น

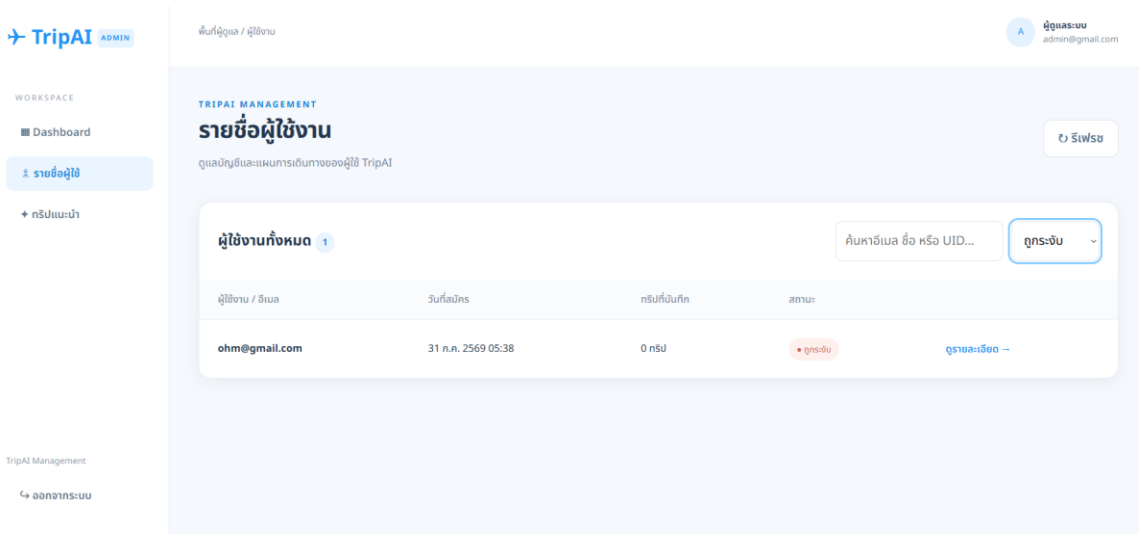
ฟังก์ชันดังกล่าวช่วยลดระยะเวลาในการค้นหาข้อมูล โดยเฉพาะในกรณีที่ระบบมีผู้ใช้งานจำนวนมาก และช่วยให้ผู้ดูแลสามารถเข้าสู่หน้ารายละเอียดของบัญชีที่ต้องการตรวจสอบได้โดยตรง

4.1.20 หน้ากรองสถานะผู้ใช้งาน



ภาพที่ 4.24 หน้ากรองผู้ใช้งานที่มีสถานะใช้งานปกติของระบบ TripAI Admin

จากภาพที่ 4.24 แสดงการทำงานของฟังก์ชันกรองข้อมูลผู้ใช้งานตามสถานะบัญชี โดยเมื่อผู้ดูแลระบบเลือกสถานะ **“ใช้งานปกติ”** ระบบจะแสดงเฉพาะบัญชีผู้ใช้งานที่สามารถเข้าใช้งานระบบได้ตามปกติ พร้อมข้อมูลประกอบ ได้แก่ ชื่อหรืออีเมล วันที่สมัคร จำนวนทริปที่บันทึก และสถานะบัญชี

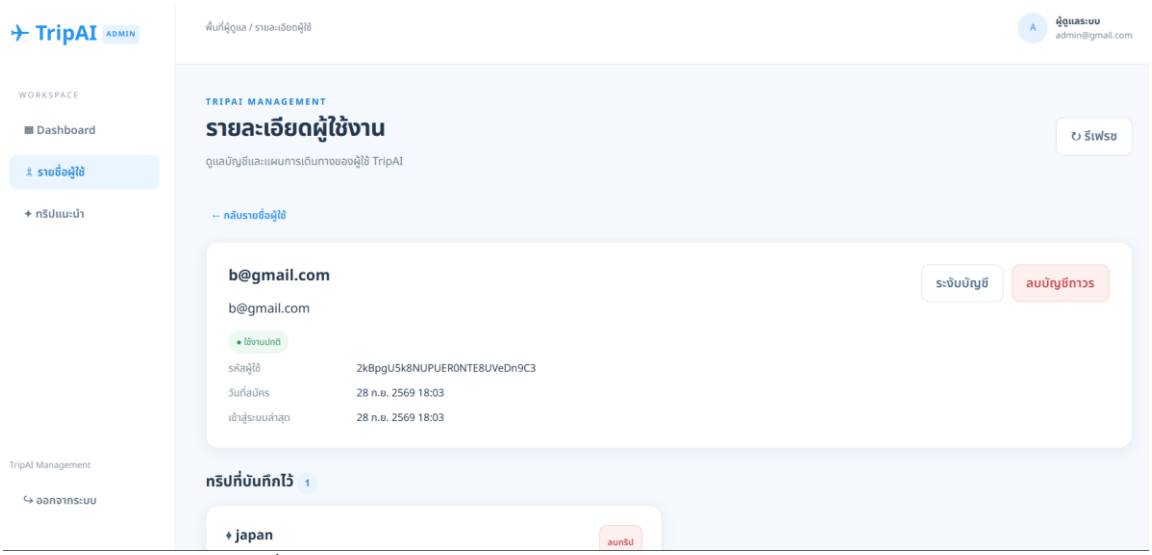


ภาพที่ 4.25 หน้ากรองผู้ใช้งานที่มีสถานะถูกระงับของระบบ TripAI Admin

จากภาพที่ 4.25 เมื่อผู้ดูแลระบบเลือกสถานะ **“ถูกระงับ”** ระบบจะแสดงเฉพาะบัญชีที่ถูกระงับการใช้งาน โดยสถานะจะแสดงด้วยป้ายกำกับที่แตกต่างจากบัญชีปกติอย่างชัดเจน ผู้ดูแลสามารถเลือกคำสั่ง **“ดูรายละเอียด”** เพื่อเข้าสู่หน้าข้อมูลของบัญชีและดำเนินการจัดการตามสิทธิ์ที่กำหนดไว้

ฟังก์ชันการกรองสถานะช่วยให้ผู้ดูแลสามารถแยกตรวจสอบบัญชีที่ใช้งานปกติและบัญชีที่ถูกระงับได้สะดวก โดยไม่จำเป็นต้องค้นหาจากรายการผู้ใช้งานทั้งหมด ทำให้การบริหารจัดการบัญชีผู้ใช้งานมีความรวดเร็วและเป็นระบบมากยิ่งขึ้น

4.1.21 หน้ารายละเอียดผู้ใช้งาน



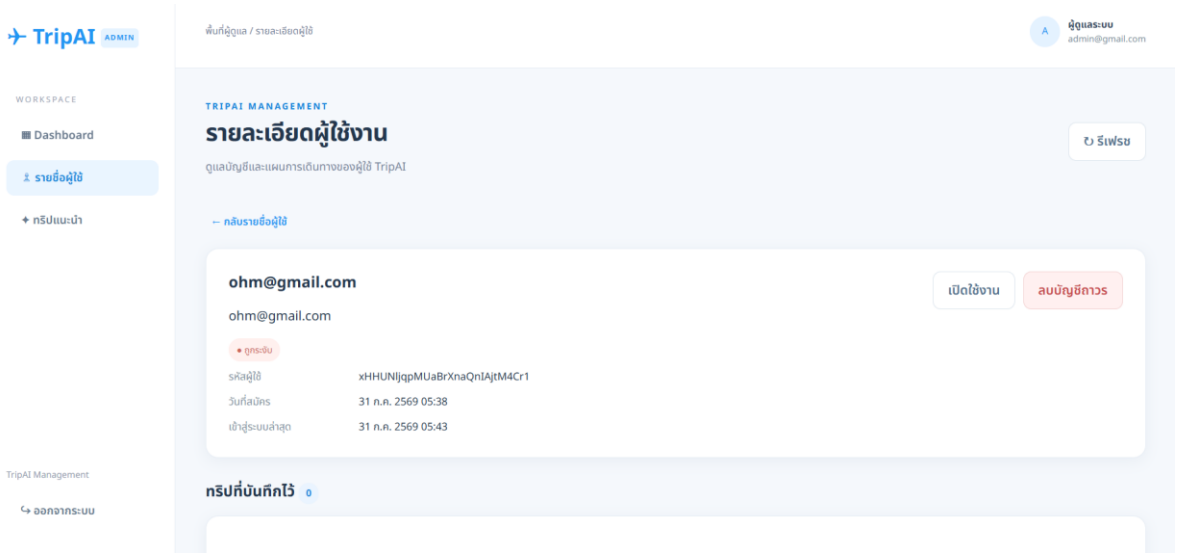
ภาพที่ 4.26 หน้ารายละเอียดผู้ใช้งานของระบบ TripAI Admin

จากภาพที่ 4.26 แสดงหน้ารายละเอียดผู้ใช้งานสำหรับให้ผู้ดูแลระบบตรวจสอบข้อมูลของบัญชีแต่ละราย โดยระบบแสดงข้อมูลสำคัญ ได้แก่ ชื่อผู้ใช้งาน อีเมล สถานะบัญชี รหัสผู้ใช้ (UID) วันที่สมัครสมาชิก และวันที่เข้าสู่ระบบล่าสุด เพื่อช่วยให้ผู้ดูแลสามารถตรวจสอบข้อมูลและสถานะของบัญชีได้อย่างชัดเจน

บริเวณด้านขวาของข้อมูลผู้ใช้งานมีคำสั่งสำหรับ **ระงับบัญชี** และ **ลบบัญชีถาวร** เพื่อใช้จัดการบัญชีตามสิทธิ์ของผู้ดูแลระบบ นอกจากนี้ ส่วนด้านล่างยังแสดงรายการทริปที่เชื่อมโยงกับผู้ใช้งาน โดยแสดงข้อมูล เช่น ชื่อทริป จำนวนวัน จำนวนผู้เดินทาง งบประมาณ และวันเดินทาง เพื่อใช้ประกอบการตรวจสอบข้อมูลการใช้งานของบัญชีดังกล่าว

จุดที่ควรแก้ก่อนนำภาพลงรายงาน: ด้านบนของภาพมีข้อความ To exit full screen, press and hold Esc ซึ่งเป็นข้อความของเบราว์เซอร์ ควรถ่ายภาพใหม่หลังข้อความดังกล่าวหายไป

4.1.22 หน้ารายละเอียดผู้ใช้งานที่ถูกระงับ



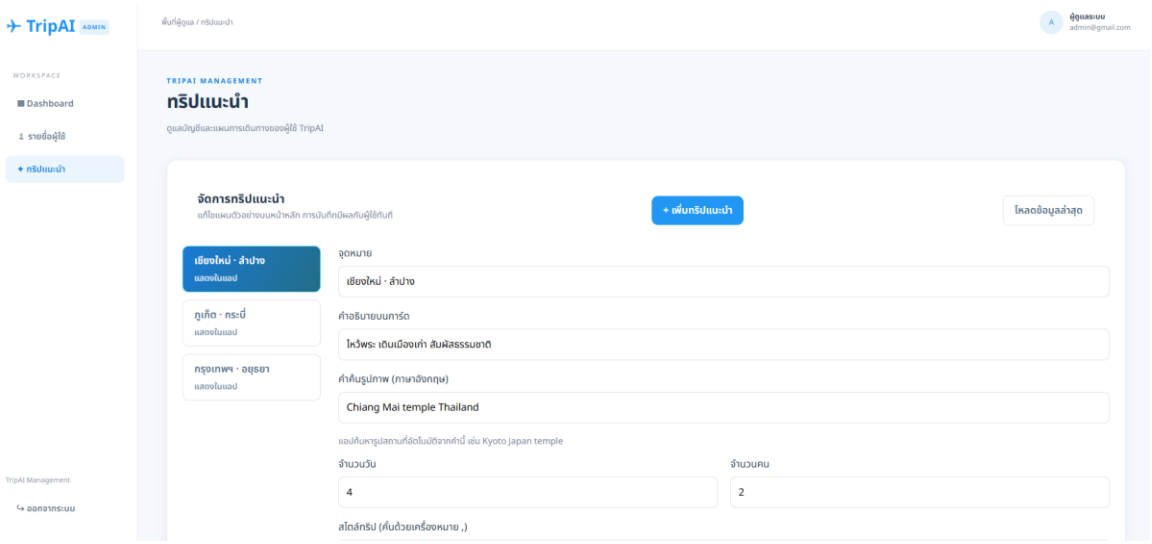
ภาพที่ 4.27 หน้ารายละเอียดผู้ใช้งานที่ถูกระงับของระบบ TripAI Admin

จากภาพที่ 4.27 แสดงหน้ารายละเอียดของบัญชีผู้ใช้งานที่มีสถานะ **“ถูกระงับ”** โดยระบบจะแสดงข้อมูลสำคัญของบัญชี ได้แก่ ชื่อผู้ใช้งาน อีเมล รหัสผู้ใช้ (UID) วันที่สมัคร และวันที่เข้าสู่ระบบล่าสุด พร้อมแสดงสถานะบัญชีด้วยป้ายกำกับที่ช่วยให้ผู้ดูแลสามารถตรวจสอบสถานะได้อย่างชัดเจน

สำหรับบัญชีที่ถูกระงับ ระบบจะแสดงปุ่ม **“เปิดใช้งาน”** เพื่อให้ผู้ดูแลสามารถคืนสิทธิ์การเข้าใช้งานแก่ผู้ใช้ได้อีกครั้ง และมีปุ่ม **“ลบบัญชีถาวร”** สำหรับกรณีที่ต้องการลบบัญชีออกจากระบบอย่างถาวร

บริเวณด้านล่างของหน้าจอจะแสดงจำนวนทริปที่ผู้ใช้งานบันทึกไว้ โดยในกรณีที่ไม่มีข้อมูลระบบจะแสดงข้อความ **“ยังไม่มีทริปที่บันทึกไว้”** เพื่อให้ผู้ดูแลสามารถตรวจสอบข้อมูลของบัญชีได้ครบถ้วนในหน้าเดียว

4.1.23 หน้าจัดการทริปแนะนำ

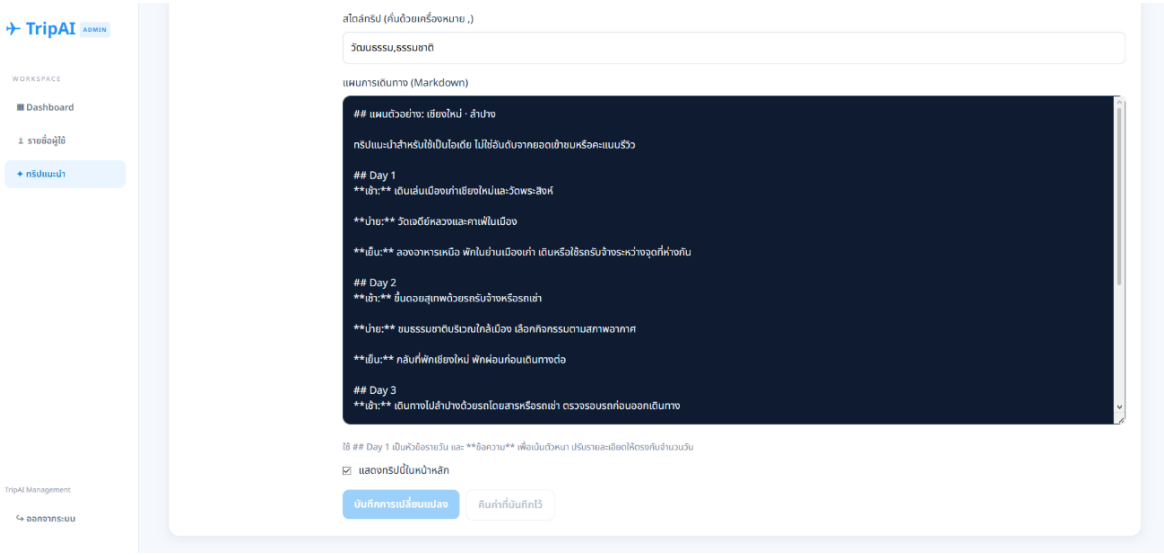


ภาพที่ 4.28 หน้าจัดการข้อมูลทริปแนะนำของระบบ TripAI Admin

จากภาพที่ 4.28 แสดงหน้าจัดการทริปแนะนำสำหรับผู้ดูแลระบบ ซึ่งใช้สำหรับเพิ่มและปรับปรุงข้อมูลทริปตัวอย่างที่นำไปแสดงในหน้าหลักของแอปพลิเคชัน TripAI โดยผู้ดูแลสามารถเลือกทริปที่มีอยู่จากรายการด้านซ้าย เช่น เชียงใหม่–ลำปาง ภูเก็ต–กระบี่ และกรุงเทพฯ–อยุธยา เพื่อแก้ไขรายละเอียดของแต่ละทริป

ระบบเปิดให้แก้ไขข้อมูลสำคัญ ได้แก่ **จุดหมาย คำอธิบายบนการ์ด คำค้นรูปภาพ จำนวนวัน จำนวนผู้เดินทาง และสไตล์การท่องเที่ยว** รวมทั้งมีปุ่ม **“เพิ่มทริปแนะนำ”** สำหรับสร้างทริปตัวอย่างใหม่เข้าสู่ระบบ และปุ่ม **“โหลดข้อมูลล่าสุด”** สำหรับเรียกข้อมูลปัจจุบันมาแสดงใหม่

การจัดการข้อมูลผ่านหน้า Admin Web ช่วยให้ผู้ดูแลสามารถปรับปรุงทริปแนะนำที่แสดงแก่ผู้ใช้งานได้โดยตรง โดยไม่จำเป็นต้องแก้ไขข้อมูลจากส่วนของแอปพลิเคชันผู้ใช้งาน



ภาพที่ 4.29 หน้าจัดการรายละเอียดแผนการเดินทางและการแสดงผลทริปแนะนำ

จากภาพที่ 4.29 แสดงส่วนสำหรับจัดการรายละเอียดของทริปแนะนำ โดยผู้ดูแลระบบสามารถแก้ไขข้อมูลแผนการเดินทางในรูปแบบ Markdown ซึ่งใช้กำหนดรายละเอียดกิจกรรมในแต่ละวัน เช่น ช่วงเช้า ช่วงบ่าย ช่วงเย็น และคำแนะนำก่อนออกเดินทาง เพื่อให้เนื้อหาที่แสดงในแอปพลิเคชันมีความครบถ้วนและสามารถปรับปรุงได้สะดวก

บริเวณด้านล่างมีตัวเลือก **“แสดงทริปนี้ในหน้าหลัก”** สำหรับกำหนดว่าทริปดังกล่าวจะถูกแสดงในส่วนทริปแนะนำของผู้ใช้งานหรือไม่ พร้อมปุ่ม **“บันทึกการเปลี่ยนแปลง”** สำหรับจัดเก็บข้อมูลที่แก้ไข และปุ่ม **“คืนค่าที่บันทึกไว้”** สำหรับย้อนกลับไปใช้ข้อมูลเดิมที่บันทึกไว้ก่อนหน้า

นอกจากนี้ ยังมีปุ่ม **“ลบทริปแนะนำ”** สำหรับนำทริปที่ไม่ต้องการออกจากรายการ โดยการจัดการในส่วนนี้ช่วยให้ผู้ดูแลสามารถควบคุมและปรับปรุงเนื้อหาทริปแนะนำที่แสดงแก่ผู้ใช้งานได้จากระบบ TripAI Admin โดยตรง

4.2 ฟังก์ชันการทำงานหลักของระบบ

ระบบ TripAI ได้รับการพัฒนาให้รองรับการทำงานหลักที่เกี่ยวข้องกับการวางแผนการเดินทางด้วยปัญญาประดิษฐ์ ตั้งแต่การยืนยันตัวตนของผู้ใช้งาน การกำหนดข้อมูลสำหรับสร้างทริป การประมวลผลแผนการเดินทาง การแสดงข้อมูลสถานที่และแผนที่ ตลอดจนการบันทึกและจัดการข้อมูลทริป โดยสามารถสรุปฟังก์ชันการทำงานหลักของระบบได้ดังนี้

4.2.1 ฟังก์ชันการสมัครสมาชิกและเข้าสู่ระบบ

ระบบรองรับการสมัครสมาชิกและเข้าสู่ระบบของผู้ใช้งาน โดยใช้ Firebase Authentication เป็นบริการสำหรับยืนยันตัวตน ผู้ใช้งานสามารถเข้าสู่ระบบด้วยอีเมลและรหัสผ่าน รวมถึง Google Sign-In เมื่อเข้าสู่ระบบสำเร็จ ระบบจะนำผู้ใช้งานเข้าสู่ส่วนการใช้งานหลักของ TripAI และเชื่อมโยงข้อมูลต่าง ๆ กับบัญชีของผู้ใช้งานผ่านรหัสผู้ใช้ (UID)

4.2.2 ฟังก์ชันการสร้างแผนการเดินทางด้วย AI

ผู้ใช้งานสามารถกำหนดข้อมูลสำหรับสร้างแผนการเดินทาง ได้แก่ จุดหมายปลายทาง วันเริ่มเดินทาง จำนวนวัน งบประมาณ จำนวนผู้เดินทาง และสไตล์การท่องเที่ยว เช่น ธรรมชาติ วัฒนธรรม อาหาร ช้อปปิ้ง ผจญภัย และพักผ่อน

เมื่อผู้ใช้งานยืนยันข้อมูล ระบบจะนำข้อมูลดังกล่าวไปใช้ในการสร้างแผนการเดินทางผ่าน Google Gemini API โดยผลลัพธ์ที่ได้จะประกอบด้วยรายละเอียดสถานที่ กิจกรรม ช่วงเวลา และค่าใช้จ่ายโดยประมาณในแต่ละวัน เพื่อให้สอดคล้องกับเงื่อนไขที่ผู้ใช้งานกำหนด

4.2.3 ฟังก์ชันการแสดงรายละเอียดแผนการเดินทาง

หลังจากระบบสร้างแผนการเดินทางเสร็จสิ้น ระบบจะแสดงรายละเอียดแผนทริปโดยแบ่งข้อมูลตามวัน เพื่อให้ผู้ใช้งานสามารถตรวจสอบกิจกรรมและสถานที่ในแต่ละช่วงของการเดินทางได้อย่างเป็นลำดับ

ข้อมูลที่แสดงประกอบด้วยชื่อสถานที่ กิจกรรมที่แนะนำ ช่วงเวลา ร้านอาหารหรือสถานที่ที่เกี่ยวข้อง และค่าใช้จ่ายโดยประมาณ พร้อมทั้งแสดงรูปภาพประกอบสถานที่เพื่อช่วยให้ผู้ใช้งานเข้าใจข้อมูลได้ชัดเจนยิ่งขึ้น

4.2.4 ฟังก์ชันการแสดงตำแหน่งสถานที่บนแผนที่

ระบบใช้ Nominatim API สำหรับค้นหาพิกัดละติจูดและลองจิจูดจากชื่อสถานที่ที่ปรากฏในแผนการเดินทาง จากนั้นนำพิกัดที่ได้มาแสดงผ่าน flutter_map ร่วมกับข้อมูลแผนที่จาก OpenStreetMap

ระบบสามารถแสดงตำแหน่งของสถานที่ในแต่ละวันด้วยสัญลักษณ์บนแผนที่ ทำให้ผู้ใช้งานสามารถตรวจสอบภาพรวมของตำแหน่งสถานที่ต่าง ๆ ในแผนการเดินทางได้สะดวก

นอกจากนี้ เมื่อผู้ใช้งานเลือกสถานที่จากแผนทริป ระบบยังสามารถเปิดแอปพลิเคชันแผนที่ภายนอกบนอุปกรณ์เพื่อใช้ในการแสดงเส้นทางจากตำแหน่งปัจจุบันไปยังสถานที่ที่เลือกได้

4.2.5 ฟังก์ชันการแสดงรูปภาพสถานที่

ระบบเชื่อมต่อกับ Unsplash API เพื่อค้นหารูปภาพที่เกี่ยวข้องกับสถานที่ในแผนการเดินทาง โดยนำชื่อสถานที่ไปใช้เป็นคำค้นหา และนำรูปภาพที่ได้รับมาแสดงประกอบในหน้ารายละเอียดแผนทริป

ฟังก์ชันดังกล่าวช่วยให้ผู้ใช้งานสามารถเห็นภาพตัวอย่างของสถานที่ก่อนเดินทาง และช่วยเพิ่มความชัดเจนในการนำเสนอข้อมูลของแผนการเดินทาง

4.2.6 ฟังก์ชันการบันทึกและเรียกดูแผนการเดินทาง

ผู้ใช้งานสามารถบันทึกแผนการเดินทางที่สร้างขึ้นลงใน Cloud Firestore โดยข้อมูลแต่ละทริปจะเชื่อมโยงกับ UID หรือ userId ของผู้ใช้งาน เพื่อระบุเจ้าของข้อมูล

หลังจากบันทึกแล้ว ผู้ใช้งานสามารถเรียกดูรายการทริปของตนเองผ่านหน้าทริปของฉัน โดยระบบจะแสดงข้อมูลสำคัญ เช่น ชื่อจุดหมาย จำนวนวัน จำนวนผู้เดินทาง วันที่เดินทาง และสไตล์การท่องเที่ยวที่เลือกไว้

4.2.7 ฟังก์ชันการลบแผนการเดินทาง

ระบบรองรับการลบแผนการเดินทางที่ผู้ใช้งานไม่ต้องการเก็บไว้ โดยเมื่อเลือกคำสั่งลบ ระบบจะแสดงหน้าต่างยืนยันก่อนดำเนินการ เพื่อช่วยลดความผิดพลาดจากการลบข้อมูลโดยไม่ได้ตั้งใจ

เมื่อผู้ใช้งานยืนยันการลบ ระบบจะนำข้อมูลทริปดังกล่าวออกจาก Cloud Firestore และปรับปรุงรายการทริปของผู้ใช้งานให้เป็นข้อมูลล่าสุด

4.2.8 ฟังก์ชันสำหรับผู้ดูแลระบบ

ระบบ TripAI มีส่วน Admin Web สำหรับผู้ดูแลระบบ โดยผู้ดูแลต้องเข้าสู่ระบบด้วยบัญชีที่ได้รับสิทธิ์ก่อนจึงจะสามารถเข้าถึงส่วนจัดการได้

ฟังก์ชันหลักของผู้ดูแลระบบประกอบด้วย การตรวจสอบภาพรวมของระบบ การเรียกดูรายชื่อผู้ใช้งาน การค้นหาผู้ใช้งาน การกรองตามสถานะบัญชี การดูรายละเอียดบัญชี การระงับบัญชี การเปิดใช้งานบัญชีที่ถูกระงับ และการลบบัญชีผู้ใช้งานอย่างถาวร

4.3 ผลการทดสอบการทำงานของระบบ

การทดสอบการทำงานของระบบ TripAI ใช้วิธีการทดสอบแบบ **Black-Box Testing** โดยพิจารณาข้อมูลนำเข้า ขั้นตอนการทำงาน และผลลัพธ์ที่ระบบแสดงออกมา โดยไม่พิจารณาโครงสร้างหรือชุดคำสั่งภายในโปรแกรม การทดสอบครอบคลุมฟังก์ชันหลักของผู้ใช้งานและส่วนผู้ดูแลระบบเพื่อประเมินว่าระบบสามารถทำงานได้ตรงตามความต้องการที่กำหนดไว้

ตารางที่ 4.1 ผลการทดสอบการทำงานของระบบ TripAI แบบ Black-Box Testing

ลำดับ	รายการทดสอบ	ข้อมูลนำเข้า/ขั้นตอนการทดสอบ	ผลที่คาดหวัง	ผลการทดสอบจริง
1	การเข้าสู่ระบบด้วยอีเมลและรหัสผ่าน	กรอกอีเมลและรหัสผ่านที่ลงทะเบียนไว้ แล้วกดเข้าสู่ระบบ	ระบบตรวจสอบข้อมูลและอนุญาตให้เข้าสู่ระบบ	สามารถเข้าสู่ระบบและเข้าสู่หน้าทริปของฉันได้
2	การเข้าสู่ระบบด้วย Google	เลือกเข้าสู่ระบบด้วยบัญชี Google	ระบบยืนยันตัวตนและเข้าสู่ระบบด้วยบัญชี Google	สามารถยืนยันตัวตนและเข้าสู่ระบบได้
3	การแสดงรายการทริปกรณียังไม่มีข้อมูล	เข้าสู่หน้าทริปของฉันด้วยบัญชีที่ยังไม่มีทริป	ระบบแสดงข้อความแจ้งว่ายังไม่มีทริปที่บันทึกไว้	ระบบแสดงสถานะไม่มีทริปและปุ่มสร้างทริปใหม่
4	การกรอกข้อมูลสร้างทริป	ระบุจุดหมาย วันเริ่มเดินทาง จำนวนวัน งบประมาณ จำนวน	ระบบรับข้อมูลที่ผู้ใช้งานกำหนดและพร้อมสร้างแผนทริป	ระบบสามารถรับและนำข้อมูลไปใช้สร้างแผนทริปได้

ลำดับ	รายการทดสอบ	ข้อมูลนำเข้า/ขั้นตอนการทดสอบ	ผลที่คาดหวัง	ผลการทดสอบจริง
		ผู้เดินทาง และสไตล์การท่องเที่ยว		
5	การสร้างแผนทริปด้วย AI	กดปุ่ม “สร้างแผนทริปด้วย AI” หลังกรอกข้อมูลครบถ้วน	ระบบส่งข้อมูลไปประมวลผลและสร้างแผนการเดินทาง	ระบบสามารถสร้างและแสดงแผนการเดินทางตามข้อมูลที่กำหนด
6	การแสดงรายละเอียดแผนทริป	เปิดผลลัพธ์หลัง AI ประมวลผลเสร็จ	ระบบแสดงกิจกรรมและรายละเอียดการเดินทางแยกตามวัน	ระบบแสดงรายละเอียดสถานที่ กิจกรรม ช่วงเวลา และค่าใช้จ่ายโดยประมาณ
7	การแสดงรูปภาพสถานที่	เปิดหน้ารายละเอียดแผนทริป	ระบบค้นหาและแสดงรูปภาพที่เกี่ยวข้องกับสถานที่	ระบบสามารถแสดงรูปภาพประกอบสถานที่ในแผนทริปได้
8	การแสดงตำแหน่งบนแผนที่	เปิดแผนทริปที่มีข้อมูลสถานที่	ระบบค้นหาพิกัดและแสดงตำแหน่งสถานที่บนแผนที่	ระบบสามารถแสดงตำแหน่งสถานที่แยกตามวันบนแผนที่ได้
9	การเปิดเส้นทางในแอปแผนที่ภายนอก	กดเลือกภาพหรือคำสั่งนำทางของสถานที่	ระบบเปิดแอปแผนที่และกำหนดสถานที่ที่เลือกเป็นจุดหมาย	ระบบสามารถเปิดแอปแผนที่ภายนอกและแสดงเส้นทางไปยังจุดหมายได้
10	การบันทึกแผนการเดินทาง	กดปุ่ม “บันทึกทริปนี้”	ระบบบันทึกข้อมูลทริปและแจ้งสถานะว่าบันทึกสำเร็จ	ระบบบันทึกข้อมูลและเปลี่ยนสถานะเป็น “บันทึกแล้ว”
11	การเรียกดูทริปที่บันทึก	เปิดหน้าทริปของฉันหลังบันทึกแผน	ระบบแสดงรายการทริปที่	ระบบแสดงชื่อทริป จำนวนวัน จำนวนผู้

ลำดับ	รายการทดสอบ	ข้อมูลนำเข้า/ขั้นตอนการทดสอบ	ผลที่คาดหวัง	ผลการทดสอบจริง
			เชื่อมโยงกับบัญชีผู้ใช้งาน	เดินทาง วันที่ และสไตล์การท่องเที่ยว
12	การลบแผนการเดินทาง	เลือกลบทริปจากรายการและยืนยันการลบ	ระบบลบข้อมูลทริปและนำรายการดังกล่าวออกจากหน้าทริปของฉัน	ระบบสามารถลบทริปและปรับปรุงรายการได้
13	การเข้าสู่ระบบ Admin Web	กรอกอีเมลและรหัสผ่านของผู้ดูแลระบบ	ระบบตรวจสอบสิทธิ์และเข้าสู่หน้า Admin Web	ผู้ดูแลสามารถเข้าสู่หน้า Dashboard ได้
14	การแสดงข้อมูล Dashboard	เข้าสู่หน้า Dashboard หลังเข้าสู่ระบบ	ระบบแสดงจำนวนผู้ใช้ บัญชีปกติ จำนวนทริป และบัญชีที่ถูกระงับ	ระบบสามารถแสดงข้อมูลภาพรวมของระบบได้
15	การค้นหาผู้ใช้งาน	กรอกชื่อ อีเมล หรือ UID ในช่องค้นหา	ระบบแสดงเฉพาะผู้ใช้งานที่ตรงกับคำค้น	ระบบสามารถกรองและแสดงข้อมูลที่ตรงกับคำค้นได้
16	การกรองสถานะผู้ใช้งาน	เลือกสถานะ “ใช้งานปกติ” หรือ “ถูกระงับ”	ระบบแสดงเฉพาะบัญชีที่มีสถานะตรงกับตัวกรอง	ระบบสามารถแยกรายการผู้ใช้งานตามสถานะได้
17	การดูรายละเอียดผู้ใช้งาน	กด “ดูรายละเอียด” จากรายชื่อผู้ใช้	ระบบแสดงข้อมูลบัญชีและข้อมูลที่เกี่ยวข้องของผู้ใช้รายนั้น	ระบบแสดงชื่อ อีเมล UID วันที่สมัคร วันที่เข้าสู่ระบบล่าสุด และจำนวนทริปได้
18	การระงับบัญชีผู้ใช้งาน	ผู้ดูแลเลือกคำสั่งระงับบัญชี	ระบบเปลี่ยนสถานะบัญชีเป็นถูกระงับ	ระบบสามารถเปลี่ยนสถานะบัญชีเป็น “ถูกระงับ” ได้

ลำดับ	รายการทดสอบ	ข้อมูลนำเข้า/ขั้นตอนการทดสอบ	ผลที่คาดหวัง	ผลการทดสอบจริง
19	การเปิดใช้งานบัญชีที่ถูกระงับ	ผู้ดูแลเลือกคำสั่ง “เปิดใช้งาน”	ระบบคืนสถานะบัญชีเป็นใช้งานปกติ	ระบบสามารถเปลี่ยนบัญชีกลับเป็นสถานะใช้งานปกติได้
20	การลบบัญชีผู้ใช้งาน	ผู้ดูแลเลือกคำสั่งลบบัญชีถาวรและยืนยัน	ระบบลบบัญชีผู้ใช้งานออกจากระบบ	ระบบสามารถดำเนินการลบบัญชีตามคำสั่งของผู้ดูแลได้

จากผลการทดสอบในตารางที่ 4.1 พบว่าฟังก์ชันหลักของระบบ TripAI สามารถทำงานได้ตามผลที่คาดหวัง โดยครอบคลุมตั้งแต่การยืนยันตัวตน การสร้างแผนการเดินทางด้วย AI การแสดงข้อมูลสถานที่และแผนที่ การบันทึกและลบแผนการเดินทาง ตลอดจนการจัดการข้อมูลบัญชีผู้ใช้งานในส่วน Admin Web

จากกรณีทดสอบทั้งหมด **20 กรณี ผ่านการทดสอบจำนวน 20 กรณี คิดเป็นร้อยละ 100** แสดงให้เห็นว่าฟังก์ชันหลักที่นำมาทดสอบสามารถทำงานได้สอดคล้องกับขอบเขตและความต้องการของระบบที่กำหนดไว้

4.4 การประเมินประสิทธิภาพและความถูกต้อง

การประเมินประสิทธิภาพของระบบ TripAI ดำเนินการตามตัวชี้วัดที่กำหนดไว้ในบทที่ 3 โดยแบ่งการประเมินออกเป็นด้านประสิทธิภาพเชิงเวลาและด้านคุณภาพของผลลัพธ์ เพื่อพิจารณาความรวดเร็วในการทำงานของระบบ ตลอดจนความเหมาะสมและความถูกต้องของแผนการเดินทางที่ระบบสร้างขึ้น

4.4.1 ผลการประเมินประสิทธิภาพเชิงเวลา (Time Efficiency)

การประเมินประสิทธิภาพเชิงเวลาแบ่งออกเป็นการวัดระยะเวลาในการสร้างแผนการเดินทางและระยะเวลาในการตอบสนองของฟังก์ชันต่าง ๆ ภายในระบบ

1) ระยะเวลาในการสร้างแผนทริป (Trip Generation Time)

ทำการทดสอบโดยกำหนดข้อมูลการเดินทางที่แตกต่างกัน จำนวน 5 ครั้ง และวัดระยะเวลาตั้งแต่ผู้ใช้งานกดปุ่ม “สร้างแผนทริปด้วย AI” จนกระทั่งระบบสามารถแสดงผลแผนการเดินทางได้สำเร็จ

ตารางที่ 4.2 ผลการทดสอบระยะเวลาในการสร้างแผนการเดินทาง

ครั้งที่	จุดหมายปลายทาง	จำนวนวัน	จำนวนผู้เดินทาง	งบประมาณ (บาท)	สไตล์การท่องเที่ยว	ระยะเวลา (วินาที)
1	Japan	5	2	15,000	อาหาร, ช้อปปิ้ง	29.90
2	Thailand	3	1	20,000	ธรรมชาติ,วัฒนธรรม	19.67
3	Korea	4	3	30,000	อาหาร,ช้อปปิ้ง,พักผ่อน	22.59
4	Laos	7	2	50,000	ธรรมชาติ,อาหาร	37.41
5	China	6	5	55,000	ธรรมชาติ,วัฒนธรรม,อาหาร,ช้อปปิ้,งผจญภัย,พักผ่อน	29.82
เฉลี่ย						27.88

จากตารางที่ 4.2 พบว่าระบบใช้เวลาในการสร้างแผนการเดินทางเฉลี่ย **27.88 วินาที** โดยใช้เวลาน้อยที่สุด **19.67 วินาที** และใช้เวลามากที่สุด **37.41 วินาที** ทั้งนี้ ระยะเวลาในการสร้างแผนการเดินทางอาจแตกต่างกันตามรายละเอียดของข้อมูลที่ผู้ใช้งานกำหนดและระยะเวลาในการตอบสนองของบริการภายนอกโดยทั้งหมดเริ่มต้นจาก Thailand

2) ระยะเวลาในการตอบสนองของระบบ (Response Time)

การทดสอบระยะเวลาในการตอบสนองของระบบเป็นการวัดเวลาที่ใช้ในการดำเนินการของฟังก์ชันหลัก เช่น การเรียกดูข้อมูล การค้นหาพิกัด การแสดงรูปภาพ และการบันทึกข้อมูล

ตารางที่ 4.3 ผลการทดสอบระยะเวลาในการตอบสนองของระบบ

รายการทดสอบ	ครั้งที่ 1	ครั้งที่ 2	ครั้งที่ 3	ครั้งที่ 4	ครั้งที่ 5	ค่าเฉลี่ย (วินาที)
เรียกดูข้อมูลทริป	0.30	0.46	0.46	0.41	0.31	0.39
บันทึกแผนการเดินทาง	0.55	0.74	0.58	0.83	0.63	0.67
ค้นหาพิกัดสถานที่	12.98	6.05	7.03	5.05	6.10	7.44
โหลดรูปภาพสถานที่	2.80	3.60	3.15	5.95	4.05	3.91

จากตารางที่ 4.3 พบว่า ฟังก์ชันที่ใช้เวลาในการตอบสนองน้อยที่สุดคือ **การเรียกดูข้อมูลทริป** โดยมีค่าเฉลี่ยประมาณ **0.39 วินาที** รองลงมาคือ **การบันทึกแผนการเดินทาง** มีค่าเฉลี่ย **0.67 วินาที** และ **การโหลดรูปภาพสถานที่** มีค่าเฉลี่ย **3.91 วินาที** ส่วนฟังก์ชันที่ใช้เวลาในการตอบสนองมากที่สุดคือ **การค้นหาพิกัดสถานที่** ซึ่งมีค่าเฉลี่ย **7.44 วินาที** เนื่องจากเป็นการเรียกใช้บริการภายนอกเพื่อค้นหาพิกัดของสถานที่ก่อนนำข้อมูลไปแสดงผลบนแผนที่

โดยรวม ระบบสามารถตอบสนองต่อการใช้งานได้ในระดับที่เหมาะสม และสามารถดำเนินการในฟังก์ชันหลักได้อย่างครบถ้วนตามที่กำหนดไว้

4.4.2 ผลการประเมินคุณภาพของผลลัพธ์ (Output Quality)

ตามที่กำหนดไว้ในหัวข้อ 3.5.3 การประเมินคุณภาพของผลลัพธ์พิจารณาจาก 3 ด้าน ได้แก่ **ความสอดคล้องของการแนะนำ ความเหมาะสมของแผนการเดินทาง และคุณภาพของแผนการเดินทาง**

ตารางที่ 4.4 ผลการประเมินคุณภาพของแผนการเดินทาง

ครั้งที่	ความสอดคล้องกับความต้องการ	ความเหมาะสมของแผน	ความครบถ้วนของแผน	ผลการประเมิน
1	ผ่าน	ผ่าน	ผ่าน	ผ่าน
2	ผ่าน	ผ่าน	ผ่าน	ผ่าน
3	ผ่าน	ผ่าน	ผ่าน	ผ่าน
4	ผ่าน	ผ่าน	ผ่าน	ผ่าน

ครั้งที่	ความสอดคล้องกับความต้องการ	ความเหมาะสมของแผน	ความครบถ้วนของแผน	ผลการประเมิน
5	ผ่าน	ผ่าน	ผ่าน	ผ่าน

จากตารางที่ 4.4 ผลการประเมินคุณภาพของแผนการเดินทางจำนวน 5 ครั้ง พบว่าแผนการเดินทางที่ระบบ TripAI สร้างขึ้น **ผ่านเกณฑ์การประเมินทั้ง 5 ครั้ง** โดยมีความสอดคล้องกับความต้องการของผู้ใช้งาน มีความเหมาะสมของแผนการเดินทาง และมีรายละเอียดของแผนการเดินทางครบถ้วนตามเงื่อนไขที่กำหนด คิดเป็น **ร้อยละ 100 ของกรณีที่ทำการทดสอบ** แสดงให้เห็นว่าระบบสามารถสร้างแผนการเดินทางที่สอดคล้องกับข้อมูลนำเข้าของผู้ใช้งานได้ตามขอบเขตการทดสอบที่กำหนด

4.4.3 ผลการตรวจสอบความถูกต้องของพิกัดสถานที่

เนื่องจากระบบ TripAI ใช้ Nominatim API ในการค้นหาพิกัดของสถานที่ จึงทำการตรวจสอบว่าสถานที่ที่ระบบค้นหาและแสดงบนแผนที่ตรงกับสถานที่ที่ระบุไว้ในแผนการเดินทางหรือไม่

ตารางที่ 4.5 ผลการตรวจสอบความถูกต้องของพิกัดสถานที่

ลำดับ	ชื่อสถานที่	พบพิกัด	ตำแหน่งตรงกับสถานที่	ผลการตรวจสอบ
1	Jinli Ancient Street	✓	✓	ถูกต้อง
2	Chunxi Road	✓	✓	ถูกต้อง
3	Jiuzhaigou Valley	✓	✓	ถูกต้อง
4	Shuzheng Valley	✓	✓	ถูกต้อง
5	Rize Valley	✓	✓	ถูกต้อง
6	Zechawa Valley	✓	✓	ถูกต้อง
7	Huanglong National Park	✓	✓	ถูกต้อง
8	Kuanzhai Alley	✓	✓	ถูกต้อง
9	Chengdu Research Base of Giant Panda Breeding	✓	✓	ถูกต้อง
10	Temple of Wu Hou	✓	✓	ถูกต้อง

จากตารางที่ 4.5 ผลการตรวจสอบความถูกต้องของพิกัดสถานที่จำนวน 10 แห่ง พบว่าระบบสามารถค้นหาพิกัดของสถานที่ได้ครบทุกแห่ง และตำแหน่งที่แสดงบนแผนที่ตรงกับสถานที่ที่ระบุในแผนการเดินทางทั้ง 10 แห่ง คิดเป็น **ร้อยละ 100** ของสถานที่ที่นำมาทดสอบ แสดงให้เห็นว่าการค้นหาพิกัดสถานที่ด้วย Nominatim API สามารถให้ผลลัพธ์ที่สอดคล้องกับข้อมูลสถานที่ในแผนการเดินทางภายใต้กรณีทดสอบที่กำหนด

บทที่ 5
สรุปผล อภิปรายผล และข้อเสนอแนะ

บทนี้เป็นการสรุปผลการดำเนินโครงงาน TripAI ระบบออกแบบทริปการท่องเที่ยวด้วยปัญญาประดิษฐ์ ตั้งแต่กระบวนการวิเคราะห์ ออกแบบ พัฒนา และทดสอบระบบ รวมถึงการอภิปรายผลที่ได้จากการพัฒนา ข้อจำกัดที่พบระหว่างการดำเนินงาน และข้อเสนอแนะสำหรับการพัฒนาระบบในอนาคต เพื่อให้ระบบสามารถรองรับการใช้งานได้อย่างมีประสิทธิภาพมากยิ่งขึ้น

5.1 สรุปผลการดำเนินโครงงาน

โครงงาน TripAI มีวัตถุประสงค์เพื่อพัฒนาระบบสำหรับช่วยผู้ใช้งานวางแผนการเดินทางด้วยปัญญาประดิษฐ์ โดยระบบเปิดให้ผู้ใช้งานกำหนดข้อมูลที่เกี่ยวข้องกับการเดินทาง ได้แก่ จุดหมายปลายทาง วันเริ่มเดินทาง จำนวนวัน จำนวนผู้เดินทาง งบประมาณ และสไตล์การท่องเที่ยว จากนั้นนำข้อมูลดังกล่าวไปใช้ในการสร้างแผนการเดินทางที่เหมาะสมกับความต้องการของผู้ใช้งาน

ระบบพัฒนาด้วย **Flutter และภาษา Dart** โดยใช้ **Google Gemini API** สำหรับสร้างแผนการเดินทาง ใช้ **Firebase Authentication** สำหรับการสมัครสมาชิกและเข้าสู่ระบบ ใช้ **Cloud Firestore** สำหรับจัดเก็บข้อมูลผู้ใช้งานและแผนการเดินทาง ใช้ **OpenStreetMap ร่วมกับ flutter_map** สำหรับแสดงข้อมูลแผนที่ ใช้ **Nominatim API** สำหรับค้นหาพิกัดสถานที่ และใช้ **Unsplash API** สำหรับค้นหารูปภาพประกอบสถานที่

ผลการพัฒนาพบว่าระบบสามารถรองรับฟังก์ชันหลัก ได้แก่ การเข้าสู่ระบบ การสร้างแผนการเดินทางด้วย AI การแสดงรายละเอียดกิจกรรมและสถานที่ การแสดงตำแหน่งสถานที่บนแผนที่ การเชื่อมโยงไปยังแอปพลิเคชันแผนที่ภายนอก การบันทึกและลบแผนการเดินทาง รวมถึงการเรียกดูรายการทริปที่บันทึกไว้

นอกจากนี้ ยังได้พัฒนาส่วน **Admin Web** สำหรับผู้ดูแลระบบ เพื่อใช้ในการตรวจสอบข้อมูลผู้ใช้งาน ค้นหาและกรองบัญชีผู้ใช้งาน ดูรายละเอียด ระงับบัญชี เปิดใช้งานบัญชีที่ถูกระงับ และลบบัญชีผู้ใช้งานตามสิทธิ์ที่กำหนด

จากการทดสอบการทำงานแบบ Black-Box Testing พบว่าฟังก์ชันหลักสามารถทำงานได้ตามขอบเขตที่กำหนด และผลการประเมินประสิทธิภาพพบว่าระบบใช้เวลาในการสร้างแผนการเดินทางเฉลี่ย **27.88 วินาที** รวมถึงสามารถค้นหาและแสดงพิกัดสถานที่ที่นำมาทดสอบได้ถูกต้องครบถ้วนตามกรณีทดสอบที่กำหนด

5.2 อภิปรายผลการพัฒนา

จากผลการพัฒนาและทดสอบระบบ TripAI พบว่า ระบบสามารถทำงานตามขอบเขตที่กำหนดไว้ โดยครอบคลุมตั้งแต่การสมัครสมาชิกและเข้าสู่ระบบ การกำหนดข้อมูลสำหรับการวางแผนการเดินทาง การสร้างแผนการเดินทางด้วยปัญญาประดิษฐ์ การแสดงรายละเอียดสถานที่และตำแหน่งบนแผนที่ ตลอดจนการบันทึกและจัดการแผนการเดินทางของผู้ใช้งาน ทั้งนี้ ระบบได้นำเทคโนโลยีหลายส่วนมาทำงานร่วมกัน ได้แก่ Google Gemini API สำหรับสร้างแผนการเดินทาง Firebase Authentication และ Cloud Firestore สำหรับการยืนยันตัวตนและจัดเก็บข้อมูล Nominatim API สำหรับค้นหาพิกัด OpenStreetMap สำหรับแสดงแผนที่ และ Unsplash API สำหรับแสดงรูปภาพประกอบสถานที่ ผล

ผลการทดสอบฟังก์ชันหลักของระบบจำนวน 20 กรณี พบว่าสามารถทำงานได้ตามผลที่คาดหวังทั้งหมด คิดเป็นร้อยละ 100 ภายใต้กรณีทดสอบที่กำหนดไว้ แสดงให้เห็นว่าการเชื่อมโยงองค์ประกอบต่าง ๆ ของระบบสามารถรองรับกระบวนการวางแผนการเดินทางได้ตั้งแต่การรับข้อมูลจากผู้ใช้งานจนถึงการนำเสนอและจัดเก็บผลลัพธ์ อย่างไรก็ตาม ผลดังกล่าวเป็นการยืนยันความถูกต้องของการทำงานตามกรณีทดสอบที่กำหนด มิได้หมายความว่าระบบจะปราศจากข้อผิดพลาดในทุกสถานการณ์การใช้งานจริง

ผลการพัฒนาดังกล่าวสอดคล้องกับแนวคิดที่ศึกษาไว้ในบทที่ 2 ซึ่งอธิบายว่าการพัฒนา Web และ Mobile Application ในปัจจุบันไม่ได้พิจารณาเพียงการทำงานของโปรแกรมภายในระบบเท่านั้น แต่ยังมีการเชื่อมโยงกับบริการด้าน Cloud Computing, Artificial Intelligence และบริการภายนอกผ่านเครือข่าย เพื่อเพิ่มความสามารถในการประมวลผลและให้บริการแก่ผู้ใช้งาน บท 2 นอกจากนี้ แนวคิดด้าน User Interface และ User Experience ยังระบุว่าระบบควรให้ความสำคัญกับความง่ายในการใช้งาน ความต่อเนื่องของขั้นตอน และความสามารถในการเข้าถึงฟังก์ชันที่ต้องการอย่างเหมาะสม บท 2

เมื่อพิจารณาร่วมกับงานวิจัยที่เกี่ยวข้องในบทที่ 2 พบว่า เทคโนโลยีปัญญาประดิษฐ์ ระบบแนะนำข้อมูล ระบบสารสนเทศภูมิศาสตร์ และโมบายแอปพลิเคชันสามารถนำมาบูรณาการร่วมกันเพื่อสนับสนุนการวางแผนการเดินทางได้ โดยแนวคิดจากงานเหล่านี้ถูกนำมาใช้เป็นพื้นฐานในการออกแบบ TripAI ให้สามารถรับข้อมูลความต้องการของผู้ใช้งาน ประมวลผลคำแนะนำ และแสดงข้อมูลสถานที่ร่วมกับแผนที่ภายในระบบเดียวกัน ซึ่งสอดคล้องกับการสังเคราะห์งานวิจัยในบทที่ 2 ที่มุ่งนำองค์ความรู้จากหลายด้านมารวมกันเพื่อช่วยให้การวางแผนการเดินทางมีความสะดวกและเหมาะสมกับผู้ใช้งานมากขึ้น

5.2.1 การสร้างแผนการเดินทางด้วย AI และความสอดคล้องกับความต้องการของผู้ใช้งาน

จากผลการพัฒนาระบบ TripAI พบว่า ผู้ใช้งานสามารถกำหนดข้อมูลที่เกี่ยวข้องกับการเดินทาง ได้แก่ จุดเริ่มต้น จุดหมายปลายทาง จำนวนวัน จำนวนผู้เดินทาง วันเริ่มเดินทาง งบประมาณ และสไตล์การท่องเที่ยว จากนั้นระบบจะนำข้อมูลดังกล่าวไปใช้เป็นเงื่อนไขในการสร้างแผนการเดินทางด้วย Google Gemini API เพื่อให้ได้แผนที่สอดคล้องกับข้อมูลที่ผู้ใช้งานกำหนดไว้ โดยระบบยังเปิดโอกาสให้ผู้ใช้งานเลือกสไตล์การท่องเที่ยวได้มากกว่าหนึ่งรูปแบบ เช่น ธรรมชาติ ชายหาด วัฒนธรรม ผจญภัย ช้อปปิ้ง อาหาร และพักผ่อน ซึ่งช่วยให้ข้อมูลนำเข้ามีความละเอียดมากขึ้นและสามารถสะท้อนความต้องการของผู้ใช้งานได้หลายด้าน ผล ผล

ผลการประเมินคุณภาพของแผนการเดินทางจำนวน 5 ครั้ง พบว่าแผนที่ระบบสร้างขึ้นผ่านเกณฑ์การประเมินทั้งด้านความสอดคล้องกับความต้องการของผู้ใช้งาน ความเหมาะสมของแผน และความครบถ้วนของรายละเอียดทุกกรณี คิดเป็นร้อยละ 100 ของกรณีที่นำมาทดสอบ ผลดังกล่าวแสดงให้เห็นว่า ภายใต้เงื่อนไขที่ใช้ในการทดสอบ ระบบสามารถนำข้อมูลความต้องการของผู้ใช้งานมาใช้ประกอบการสร้างแผนการเดินทางได้ตามวัตถุประสงค์ของระบบ อย่างไรก็ตาม ผลดังกล่าวควรตีความภายใต้จำนวนกรณีทดสอบที่กำหนดไว้ และยังไม่อาจสรุปได้ว่าผลลัพธ์จะมีความเหมาะสมในทุกลักษณะการเดินทางหรือทุกกลุ่มผู้ใช้งาน

ผลที่ได้สอดคล้องกับแนวคิดด้านระบบแนะนำข้อมูลที่ศึกษาไว้ในบทที่ 2 โดยเฉพาะงานของ **ภัทร สูตรสุวรรณ และคณะ (2564)** ซึ่งศึกษาการพัฒนาระบบแนะนำสถานที่ท่องเที่ยวโดยใช้เทคนิคหลายรูปแบบ ได้แก่ Popularity-based, Content-based, Collaborative Filtering และ Hybrid Model และพบว่าวิธีแบบ Hybrid Model สามารถให้ผลลัพธ์ที่มีความแม่นยำสูงกว่าวิธีอื่น แนวคิดดังกล่าวสะท้อนให้เห็นว่าการนำข้อมูลจากหลายปัจจัยมาประกอบการประมวลผลสามารถช่วยเพิ่มความเหมาะสมของคำแนะนำที่ระบบสร้างให้แก่ผู้ใช้งานได้

แม้ TripAI จะไม่ได้ใช้ Hybrid Model ในลักษณะเดียวกับงานดังกล่าวโดยตรง แต่มีลักษณะที่สอดคล้องในแง่ของการนำข้อมูลหลายเงื่อนไขของผู้ใช้งานมาใช้ร่วมกันในการสร้างผลลัพธ์ เช่น จุดหมาย ระยะเวลา งบประมาณ จำนวนผู้เดินทาง และสไตล์การท่องเที่ยว ซึ่งทำให้ระบบสามารถสร้างแผนการเดินทางที่แตกต่างกันตามข้อมูลของแต่ละทริปได้ นอกจากนี้ TripAI ยังแตกต่างจากงานแนะนำสถานที่ท่องเที่ยวทั่วไปตรงที่ไม่ได้ให้ผลลัพธ์เฉพาะรายการสถานที่ แต่ใช้ Generative AI เพื่อสร้างแผนการเดินทางในลักษณะเป็นรายวันและจัดรายละเอียดกิจกรรมไว้ในแผนเดียวกัน

ดังนั้น ผลการพัฒนาในส่วนนี้จึงแสดงให้เห็นว่า การประยุกต์ใช้ปัญญาประดิษฐ์ร่วมกับข้อมูลความต้องการของผู้ใช้งานสามารถนำมาใช้สนับสนุนการสร้างแผนการเดินทางที่มีลักษณะเฉพาะตามเงื่อนไขของแต่ละทริปได้ และสอดคล้องกับแนวคิดในบทที่ 2 ที่ให้ความสำคัญกับการใช้ข้อมูลผู้ใช้งานเป็นพื้นฐานในการสร้างคำแนะนำที่เหมาะสมมากขึ้น

5.2.2 คุณภาพและความครบถ้วนของแผนการเดินทาง

จากผลการประเมินคุณภาพของผลลัพธ์ของระบบ TripAI โดยพิจารณา 3 ด้าน ได้แก่ ความสอดคล้องกับความต้องการของผู้ใช้งาน ความเหมาะสมของแผนการเดินทาง และความครบถ้วนของรายละเอียด พบว่าแผนการเดินทางที่สร้างขึ้นจำนวน 5 ครั้งผ่านเกณฑ์การประเมินทุกครั้ง คิดเป็นร้อยละ 100 ของกรณีที่ทำการทดสอบ แสดงให้เห็นว่า ภายใต้เงื่อนไขการทดสอบที่กำหนด ระบบสามารถสร้างแผนการเดินทางที่มีรายละเอียดครบถ้วนและสอดคล้องกับข้อมูลนำเข้าของผู้ใช้งานได้ผล

ผลดังกล่าวมีความสอดคล้องกับแนวคิดของระบบแนะนำข้อมูลที่ศึกษาในบทที่ 2 ซึ่งให้ความสำคัญกับการวิเคราะห์ข้อมูลและความสนใจของผู้ใช้งานเพื่อนำไปสร้างคำแนะนำที่เหมาะสม โดยงานวิจัยของ **ภัทร สูตรสุวรรณ และคณะ (2564)** ได้ศึกษาระบบแนะนำสถานที่ท่องเที่ยวโดยใช้เทคนิค Popularity-based, Content-based, Collaborative Filtering และ Hybrid Model และพบว่าวิธีแบบ Hybrid Model สามารถให้ผลลัพธ์ที่มีความแม่นยำสูงกว่าวิธีการอื่น งานดังกล่าวจึงแสดงให้เห็นว่าการนำข้อมูลหลายส่วนมาประกอบการวิเคราะห์มีส่วนช่วยให้ระบบสามารถสร้างคำแนะนำที่เหมาะสมกับผู้ใช้งานมากขึ้น บท 2

สำหรับ TripAI แม้จะไม่ได้ใช้อัลกอริทึม Hybrid Model ในรูปแบบเดียวกับงานวิจัยดังกล่าว แต่ระบบได้นำข้อมูลหลายเงื่อนไข ได้แก่ จุดหมายปลายทาง จำนวนวัน จำนวนผู้เดินทาง งบประมาณ และสไตล์การท่องเที่ยว มาใช้ร่วมกันเป็นข้อมูลสำหรับการสร้างแผนการเดินทาง จึงมีลักษณะที่สอดคล้องกับแนวคิดของการใช้ข้อมูลผู้ใช้งานหลายด้านเพื่อสร้างผลลัพธ์ที่เหมาะสมกับความต้องการเฉพาะบุคคล

นอกจากนี้ การสังเคราะห์งานวิจัยในบทที่ 2 ยังระบุว่า การบูรณาการเทคโนโลยีปัญญาประดิษฐ์ ระบบแนะนำข้อมูล ระบบสารสนเทศภูมิศาสตร์ และโมบายแอปพลิเคชัน สามารถนำมาใช้ร่วมกันในการพัฒนาระบบวางแผนการเดินทางที่ตอบสนองต่อความต้องการของผู้ใช้งานได้อย่างมีประสิทธิภาพ บท 2 ซึ่งสอดคล้องกับผลของ TripAI ที่ไม่ได้แสดงเพียงรายการสถานที่ท่องเที่ยว แต่สร้างรายละเอียดแผนการเดินทางเป็นลำดับตามวัน และนำเสนอข้อมูลสถานที่ภายในแผนเดียวกัน อย่างไรก็ตาม ผลการประเมินคุณภาพของแผนการเดินทางในโครงงานนี้มาจากกรณีทดสอบจำนวน 5 ครั้ง จึงควรตีความผลเฉพาะภายใต้ข้อมูลและเงื่อนไขที่ใช้ในการทดสอบ ยังไม่สามารถสรุปได้ว่าแผนการเดินทางที่ระบบสร้างขึ้นจะมีความเหมาะสมและครบถ้วนในทุกจุดหมายหรือทุกสถานการณ์การเดินทาง ดังนั้น การเพิ่มจำนวนกรณีทดสอบและความหลากหลายของเงื่อนไขการเดินทางจะช่วยให้สามารถประเมินคุณภาพของผลลัพธ์ได้ครอบคลุมมากขึ้น

5.2.3 การบูรณาการแผนการเดินทางกับข้อมูลพิกัดและแผนที่

จากผลการพัฒนาระบบ TripAI พบว่า ระบบสามารถนำสถานที่ที่ปรากฏในแผนการเดินทางไปค้นหาพิกัดด้วย Nominatim API และนำตำแหน่งดังกล่าวมาแสดงบนแผนที่ OpenStreetMap ได้ โดยผลการตรวจสอบสถานที่จำนวน 10 แห่ง พบว่าสามารถค้นหาพิกัดได้ครบทุกแห่ง และตำแหน่งที่แสดงบนแผนที่ตรงกับสถานที่ที่ระบุในแผนการเดินทางทุกแห่ง คิดเป็นร้อยละ 100 ของสถานที่ที่นำมาทดสอบภายใต้กรณีทดสอบที่กำหนดไว้ ผล

ผลดังกล่าวสอดคล้องกับงานวิจัยที่ศึกษาในบทที่ 2 โดยงานของ **กมลลักษณ์ ชัยดี และคณะ (2559)** ได้พัฒนาแอปพลิเคชันบนสมาร์ตโฟนสำหรับนำเสนอข้อมูลสถานที่ท่องเที่ยว และมีการเชื่อมต่อกับ Google Maps และระบบ GPS เพื่อแสดงตำแหน่งของสถานที่และสนับสนุนการวางแผนเส้นทางการเดินทาง แนวคิดดังกล่าวถูกนำมาใช้เป็นพื้นฐานในการออกแบบ TripAI ให้สามารถเชื่อมโยงข้อมูลสถานที่ท่องเที่ยวกับแผนที่ดิจิทัล เพื่อให้ผู้ใช้งานสามารถตรวจสอบตำแหน่งของสถานที่ในแผนการเดินทางได้สะดวกมากขึ้น บท 2

นอกจากนี้ งานวิจัยของ **สุนิสา ทัศนะ และคณะ (2561)** ยังให้ความสำคัญกับการจัดเก็บข้อมูลสถานที่ท่องเที่ยวอย่างเป็นระบบและการใช้แผนที่ดิจิทัลเพื่อแสดงตำแหน่งของสถานที่ต่าง ๆ ซึ่งแนวคิดดังกล่าวสอดคล้องกับ TripAI ที่นำข้อมูลสถานที่จากแผนการเดินทางมาเชื่อมโยงกับข้อมูลเชิงพิกัดและแสดงผลบนแผนที่ ทำให้ผู้ใช้งานไม่ได้รับเพียงข้อมูลข้อความของสถานที่ แต่ยังสามารถมองเห็นตำแหน่งเชิงพื้นที่ประกอบการวางแผนการเดินทางได้ด้วย บท 2

เมื่อพิจารณาร่วมกับการสังเคราะห์งานวิจัยในบทที่ 2 พบว่า ระบบสารสนเทศภูมิศาสตร์และแผนที่ดิจิทัลเป็นองค์ประกอบสำคัญของระบบสารสนเทศด้านการท่องเที่ยว และสามารถนำมาบูรณาการร่วมกับเทคโนโลยีปัญญาประดิษฐ์และระบบแนะนำข้อมูลเพื่อสนับสนุนการวางแผนการเดินทางได้ บท 2 ดังนั้น การที่ TripAI สามารถนำผลลัพธ์จากการสร้างแผนด้วย AI ไปเชื่อมโยงกับพิกัดและแสดงบนแผนที่ได้ จึงเป็นการนำแนวคิดจากงานเดิมมาประยุกต์ใช้ในลักษณะที่เชื่อมต่อกระบวนการสร้างแผนการเดินทางกับข้อมูลเชิงพื้นที่ภายในระบบเดียวกัน

อย่างไรก็ตาม ผลการตรวจสอบพิกัดในโครงงานนี้ครอบคลุมสถานที่จำนวน 10 แห่ง จึงยังไม่สามารถสรุปได้ว่าการค้นหาพิกัดจะถูกต้องในทุกสถานที่ โดยเฉพาะกรณีที่ชื่อสถานที่มีความคล้ายคลึงกัน มีชื่อหลายภาษา หรือข้อมูลในบริการภายนอกไม่สมบูรณ์ ดังนั้น หากมีการเพิ่มจำนวนสถานที่และความหลากหลายของกรณีทดสอบ จะช่วยให้สามารถประเมินความถูกต้องของการเชื่อมโยงข้อมูลสถานที่กับแผนที่ได้ครอบคลุมมากยิ่งขึ้น

5.2.4 ประสิทธิภาพในการตอบสนองของระบบ

จากผลการทดสอบประสิทธิภาพเชิงเวลาของระบบ TripAI พบว่า ระบบใช้เวลาในการสร้างแผนการเดินทางเฉลี่ย 27.88 วินาที โดยใช้เวลาน้อยที่สุด 19.67 วินาที และมากที่สุด 37.41 วินาที ซึ่งระยะเวลาที่แตกต่างกันอาจขึ้นอยู่กับรายละเอียดของข้อมูลที่ผู้ใช้งานกำหนด รวมถึงระยะเวลาในการตอบสนองของบริการภายนอกที่ระบบเรียกใช้งาน ผล

เมื่อพิจารณาระยะเวลาในการตอบสนองของฟังก์ชันหลัก พบว่าการเรียกดูข้อมูลทริปใช้เวลาเฉลี่ย 0.39 วินาที การบันทึกแผนการเดินทางใช้เวลาเฉลี่ย 0.67 วินาที การโหลดรูปภาพสถานที่ใช้เวลาเฉลี่ย 3.91 วินาที และการค้นหาพิกัดสถานที่ใช้เวลาเฉลี่ย 7.44 วินาที โดยการค้นหาพิกัดเป็นฟังก์ชันที่ใช้เวลามากที่สุด เนื่องจากต้องเรียกใช้บริการภายนอกเพื่อค้นหาพิกัดก่อนนำข้อมูลไปแสดงบนแผนที่ ผล ผลดังกล่าวแสดงให้เห็นว่าฟังก์ชันที่เกี่ยวข้องกับการประมวลผลหรือเข้าถึงข้อมูลภายในระบบสามารถตอบสนองได้เร็วกว่าฟังก์ชันที่ต้องติดต่อกับบริการภายนอกผ่านเครือข่าย

ผลที่เกิดขึ้นสามารถอธิบายได้ด้วยแนวคิด **Client-Server Architecture** ที่ศึกษาในบทที่ 2 ซึ่งอธิบายว่า Client จะส่งคำขอหรือ Request ผ่านเครือข่ายไปยัง Server จากนั้น Server จะรับคำขอ ประมวลผล และส่งผลลัพธ์หรือ Response กลับมายัง Client กระบวนการดังกล่าวจึงประกอบด้วยทั้งการรับส่งข้อมูลผ่านเครือข่ายและการประมวลผลของระบบที่ให้บริการ บท 2 นอกจากนี้ ในกรณีที่ระบบต้องเรียกใช้ข้อมูลหรือบริการอื่นเพิ่มเติม Server อาจต้องติดต่อกับ Database หรือ Server อื่นก่อนที่จะสามารถสร้างผลลัพธ์และส่งกลับมายัง Client ได้ ซึ่งทำให้กระบวนการตอบสนองมีหลายขั้นตอนมากขึ้น บท 2

ในด้านการใช้งาน API ผลการทดสอบยังสอดคล้องกับแนวคิด **Application Programming Interface (API)** ในบทที่ 2 ซึ่งอธิบายว่าระบบหนึ่งสามารถส่ง Request ไปยังระบบผู้ให้บริการผ่าน API Endpoint จากนั้นระบบผู้ให้บริการจะดำเนินการตามคำขอและส่ง Response กลับมายังระบบที่เรียกใช้ บท 2 สำหรับ TripAI มีการเรียกใช้บริการภายนอกหลายส่วน เช่น Google Gemini API สำหรับสร้างแผนการเดินทาง Nominatim API สำหรับค้นหาพิกัด และ Unsplash API สำหรับรูปภาพสถานที่ ดังนั้น ระยะเวลาในการตอบสนองของบางฟังก์ชันจึงไม่ได้ขึ้นอยู่กับประสิทธิภาพของตัวแอปพลิเคชันเพียงอย่างเดียว แต่ยังเกี่ยวข้องกับกระบวนการ Request–Response ของบริการภายนอกที่ระบบใช้งานร่วมด้วย

เมื่อพิจารณาจากผลการทดสอบ จึงเห็นได้ว่าการเชื่อมต่อบริการภายนอกช่วยเพิ่มความสามารถให้ TripAI สามารถสร้างแผนการเดินทาง ค้นหาพิกัด และแสดงรูปภาพสถานที่ได้ภายในระบบเดียว แต่ในขณะเดียวกันก็ทำให้ระยะเวลาในการตอบสนองของบางกระบวนการสูงกว่าการทำงานที่อาศัยข้อมูลภายในระบบโดยตรง โดยเฉพาะการสร้างแผนการเดินทางและการค้นหา

พิกัดสถานที่ ดังนั้น ประสิทธิภาพของระบบในส่วนดังกล่าวจึงมีความสัมพันธ์กับทั้งกระบวนการประมวลผลภายในและการตอบสนองของ API ภายนอกตามลักษณะสถาปัตยกรรมที่นำมาใช้

5.2.5 ภาพรวมเมื่อเปรียบเทียบกับงานวิจัยที่เกี่ยวข้อง

เมื่อพิจารณาผลการพัฒนาระบบ TripAI เปรียบเทียบกับงานวิจัยที่ศึกษาในบทที่ 2 พบว่า งานวิจัยเดิมส่วนใหญ่ให้ความสำคัญกับองค์ประกอบเฉพาะด้านของระบบการท่องเที่ยว เช่น การใช้ปัญญาประดิษฐ์เพื่อให้ข้อมูลหรือแนะนำสถานที่ การพัฒนาโมบายแอปพลิเคชันสำหรับเข้าถึงข้อมูลการท่องเที่ยว การใช้แผนที่ดิจิทัลเพื่อแสดงตำแหน่งสถานที่ และการใช้ระบบแนะนำข้อมูลเพื่อสร้างคำแนะนำที่เหมาะสมกับความสนใจของผู้ใช้งาน ส่วน TripAI ได้นำแนวคิดจากหลายด้านดังกล่าวมาประยุกต์ใช้ร่วมกันภายในกระบวนการวางแผนการเดินทางเดียว

ในด้านการประยุกต์ใช้ปัญญาประดิษฐ์ งานวิจัยของ **สุมนา บุษบก และคณะ (2568)** ได้นำ AI มาใช้ในรูปแบบแชทบอทเพื่อให้ข้อมูลเกี่ยวกับสถานที่ท่องเที่ยว ร้านอาหาร และที่พัก รวมถึงเชื่อมโยงกับระบบแผนที่ ซึ่งแสดงให้เห็นว่า AI สามารถนำมาใช้สนับสนุนการให้ข้อมูลแก่นักท่องเที่ยวได้ บท 2 ขณะที่ TripAI นำปัญญาประดิษฐ์มาใช้ในขั้นตอนการสร้างแผนการเดินทาง โดยผู้ใช้งานสามารถกำหนดเงื่อนไขของทริปก่อนให้ระบบสร้างรายละเอียดการเดินทางตามข้อมูลที่กำหนด จึงเป็นการนำ AI มาใช้สนับสนุนกระบวนการวางแผนมากกว่าการให้ข้อมูลสถานที่เพียงอย่างเดียว

ในด้านการแสดงข้อมูลตำแหน่ง งานวิจัยของ **กมลลักษณ์ ชัยดี และคณะ (2559)** มีการเชื่อมต่อแอปพลิเคชันกับ Google Maps และ GPS เพื่อแสดงตำแหน่งและช่วยในการวางแผนเส้นทาง บท 2 ขณะที่ TripAI ใช้ Nominatim API ร่วมกับ OpenStreetMap เพื่อเชื่อมโยงสถานที่ที่ปรากฏในแผนการเดินทางกับตำแหน่งบนแผนที่ จากการตรวจสอบพิกัดสถานที่ในการทดสอบพบว่าระบบสามารถค้นหาและแสดงตำแหน่งตรงกับสถานที่ที่กำหนดไว้ในกรณีทดสอบที่ดำเนินการ จึงแสดงให้เห็นถึงการนำข้อมูลเชิงพื้นที่มาใช้ร่วมกับผลลัพธ์ของแผนการเดินทางที่สร้างขึ้นโดยระบบ

นอกจากนี้ งานของ **ภัทร สูตรสุวรรณ และคณะ (2564)** ให้ความสำคัญกับ Recommendation System โดยเปรียบเทียบวิธี Popularity-based, Content-based, Collaborative Filtering และ Hybrid Model และพบว่าวิธีแบบ Hybrid Model ให้ความแม่นยำในการแนะนำสูงกว่าวิธีอื่น บท 2 แม้ TripAI จะไม่ได้ใช้วิธี Hybrid Model ดังกล่าวโดยตรง แต่ได้นำแนวคิดเรื่องการพิจารณาข้อมูลผู้ใช้งานหลายด้านมาใช้ โดยระบบรับข้อมูล เช่น จุดหมาย จำนวนวัน จำนวนผู้เดินทาง งบประมาณ และสไตล์การท่องเที่ยว เพื่อใช้เป็นเงื่อนไขในการสร้างแผนการเดินทางให้แตกต่างกันตามข้อมูลของแต่ละทริป

เมื่อพิจารณาผลการทดสอบของ TripAI ร่วมกับงานที่เกี่ยวข้อง พบว่า ระบบสามารถรวมกระบวนการรับข้อมูลความต้องการ การสร้างแผนการเดินทางด้วย AI การแสดงข้อมูลสถานที่บนแผน

ที่ และการจัดเก็บแผนการเดินทางไว้ภายในระบบเดียว โดยผลการประเมินคุณภาพของแผนจำนวน 5 กรณีผ่านเกณฑ์ด้านความสอดคล้อง ความเหมาะสม และความครบถ้วนทั้งหมดตามขอบเขตการทดสอบ ผล ลักษณะดังกล่าวสอดคล้องกับการสังเคราะห์งานวิจัยในบทที่ 2 ที่ระบุว่าสามารถนำเทคโนโลยีปัญญาประดิษฐ์ ระบบแนะนำข้อมูล ระบบสารสนเทศภูมิศาสตร์ และโมบายแอปพลิเคชันมาบูรณาการร่วมกันเพื่อสนับสนุนระบบสารสนเทศด้านการท่องเที่ยวได้ บท 2

ดังนั้น TripAI จึงมีลักษณะเป็นการนำแนวคิดและองค์ประกอบจากงานที่เกี่ยวข้องหลายด้านมาประยุกต์ใช้ร่วมกัน เพื่อสนับสนุนกระบวนการวางแผนการเดินทางตั้งแต่การกำหนดความต้องการของผู้ใช้งาน การสร้างแผนด้วย AI ไปจนถึงการแสดงตำแหน่งสถานที่บนแผนที่ อย่างไรก็ตาม ผลที่ได้ยังอยู่ภายใต้ขอบเขตและจำนวนกรณีทดสอบของโครงงาน จึงควรมีการทดสอบกับข้อมูล จุดหมาย และรูปแบบการเดินทางที่หลากหลายมากขึ้น เพื่อประเมินความสามารถของระบบในสถานการณ์การใช้งานที่กว้างขึ้น

5.3 ข้อจำกัดของโครงงาน

จากการพัฒนาและทดสอบระบบ TripAI พบว่ายังมีข้อจำกัดบางประการที่อาจส่งผลต่อการใช้งานระบบ ดังนี้

1. **การสร้างแผนการเดินทางขึ้นอยู่กับ Google Gemini API**

   เนื่องจากระบบใช้ Google Gemini API เป็นส่วนหลักในการสร้างแผนการเดินทาง หากบริการดังกล่าวเกิดความล่าช้า ไม่สามารถเชื่อมต่อได้ หรือมีข้อจำกัดในการเรียกใช้งาน อาจส่งผลให้ระบบไม่สามารถสร้างแผนการเดินทางได้ตามปกติ

2. **ระยะเวลาในการประมวลผลอาจแตกต่างกันในแต่ละครั้ง**

   ระยะเวลาในการสร้างแผนการเดินทางและเรียกใช้บริการต่าง ๆ ขึ้นอยู่กับหลายปัจจัย เช่น จำนวนวันของทริป รายละเอียดข้อมูลที่ผู้ใช้งานกำหนด ความเร็วของเครือข่าย และระยะเวลาในการตอบสนองของ API ภายนอก

3. **ข้อมูลที่สร้างโดยปัญญาประดิษฐ์อาจมีความคลาดเคลื่อน**

   ข้อมูลสถานที่ กิจกรรม ค่าใช้จ่าย หรือรายละเอียดอื่น ๆ ที่สร้างขึ้นโดย AI อาจไม่ตรงกับข้อมูลจริงทั้งหมด ผู้ใช้งานจึงควรตรวจสอบข้อมูลสำคัญเพิ่มเติมก่อนนำแผนไปใช้ในการเดินทางจริง

4. **การค้นหาพิกัดสถานที่ขึ้นอยู่กับข้อมูลของ Nominatim API**

ในกรณีที่ชื่อสถานที่มีความคล้ายคลึงกัน หรือข้อมูลสถานที่ไม่ครบถ้วน อาจทำให้ระบบค้นหาพิกัดผิดตำแหน่งหรือไม่พบพิกัดของสถานที่นั้นได้

5. **การแสดงรูปภาพสถานที่ขึ้นอยู่กับ Unsplash API**

รูปภาพที่ได้จากการค้นหาอาจไม่ตรงกับสถานที่ที่ต้องการในบางกรณี หรืออาจไม่พบรูปภาพสำหรับสถานที่บางแห่ง เนื่องจากขึ้นอยู่กับข้อมูลที่มีอยู่ในบริการ Unsplash

6. **ระบบยังไม่มีข้อมูลแบบเรียลไทม์บางประเภท**

ระบบยังไม่ได้เชื่อมต่อข้อมูลแบบเรียลไทม์ เช่น ราคาตั๋วเครื่องบิน ราคาที่พัก สภาพการจราจร เวลาเปิด–ปิดสถานที่ หรือสภาพอากาศ ดังนั้นข้อมูลค่าใช้จ่ายและรายละเอียดบางส่วนในแผนการเดินทางจึงเป็นข้อมูลโดยประมาณ

7. **ขอบเขตของ Admin Web จำกัดเฉพาะการจัดการผู้ใช้งาน**

ผู้ดูแลระบบสามารถตรวจสอบ ค้นหา ระงับ เปิดใช้งาน และลบบัญชีผู้ใช้งานได้ แต่ยังไม่ได้รองรับการบริหารจัดการข้อมูลสถานที่หรือข้อมูลอื่น ๆ ภายในระบบโดยตรง

8. **การทดสอบระบบยังอยู่ภายใต้กรณีทดสอบที่กำหนด**

ผลการทดสอบประสิทธิภาพและความถูกต้องของระบบเป็นผลจากชุดข้อมูลและสถานการณ์ที่ผู้จัดทำกำหนดขึ้น จึงอาจยังไม่ครอบคลุมรูปแบบการใช้งานและสถานที่ทั้งหมดที่สามารถเกิดขึ้นได้ในการใช้งานจริง

ข้อจำกัดดังกล่าวสามารถนำไปใช้เป็นแนวทางในการพัฒนาระบบ TripAI ในอนาคต เพื่อเพิ่มความถูกต้อง ความรวดเร็ว และความสามารถในการรองรับการใช้งานที่หลากหลายมากยิ่งขึ้น.

5.4 ข้อเสนอแนะและแนวทางการพัฒนาต่อยอด

จากผลการพัฒนาและข้อจำกัดของระบบ TripAI ผู้จัดทำมีข้อเสนอแนะสำหรับการพัฒนาระบบในอนาคต เพื่อเพิ่มประสิทธิภาพ ความถูกต้อง และความสะดวกในการใช้งาน ดังนี้

1. **เพิ่มการเชื่อมต่อข้อมูลการเดินทางแบบเรียลไทม์**

ควรพัฒนาระบบให้สามารถเชื่อมต่อข้อมูลที่เกี่ยวข้องกับการเดินทาง เช่น ราคาตั๋วเครื่องบิน

ราคาที่พัก สภาพอากาศ เวลาเปิด–ปิดของสถานที่ และข้อมูลการเดินทาง เพื่อช่วยให้แผนที่สร้างขึ้นมีความใกล้เคียงกับสถานการณ์จริงมากยิ่งขึ้น

2. **เพิ่มความสามารถในการปรับแก้แผนการเดินทางด้วย AI**

   ควรเพิ่มฟังก์ชันให้ผู้ใช้งานสามารถระบุความต้องการเพิ่มเติมหรือแก้ไขเฉพาะบางส่วนของแผน เช่น เปลี่ยนสถานที่ เพิ่มกิจกรรม หรือลดค่าใช้จ่าย โดยไม่จำเป็นต้องสร้างแผนการเดินทางใหม่ทั้งหมด

3. **เพิ่มระบบตรวจสอบความถูกต้องของข้อมูลสถานที่**

   ควรนำข้อมูลจากหลายแหล่งมาใช้ประกอบการตรวจสอบชื่อสถานที่ พิกัด เวลาเปิด–ปิด และรายละเอียดที่เกี่ยวข้อง เพื่อลดความคลาดเคลื่อนของข้อมูลที่ AI สร้างขึ้น

4. **พัฒนาการค้นหาและแสดงรูปภาพสถานที่ให้แม่นยำมากขึ้น**

   ควรปรับปรุงวิธีสร้างคำค้นสำหรับ Unsplash API หรือเพิ่มแหล่งข้อมูลรูปภาพอื่น เพื่อให้รูปภาพที่นำมาแสดงมีความสัมพันธ์กับสถานที่จริงมากขึ้น และลดกรณีที่ไม่พบรูปภาพ

5. **เพิ่มระบบจัดการงบประมาณโดยละเอียด**

   ควรพัฒนาให้ระบบสามารถแยกค่าใช้จ่ายออกเป็นหมวดหมู่ เช่น ค่าเดินทาง ค่าที่พัก ค่าอาหาร ค่าเข้าชม และค่าใช้จ่ายอื่น ๆ เพื่อช่วยให้ผู้ใช้งานสามารถวางแผนงบประมาณได้ละเอียดและเหมาะสมมากขึ้น

6. **เพิ่มการจัดเส้นทางระหว่างสถานที่**

   ในอนาคตสามารถพัฒนาระบบให้คำนวณลำดับการเดินทางระหว่างสถานที่โดยพิจารณาจากระยะทาง เวลาเดินทาง และรูปแบบการเดินทาง เพื่อช่วยลดเวลาและระยะทางที่ไม่จำเป็น

7. **เพิ่มฟังก์ชันแชร์แผนการเดินทาง**

   ควรเพิ่มความสามารถในการแชร์แผนการเดินทางผ่านลิงก์หรือแอปพลิเคชันอื่น รวมถึงรองรับการเดินทางร่วมกันของผู้ใช้หลายคน เพื่อเพิ่มความสะดวกในการวางแผนทริปแบบกลุ่ม

8. **เพิ่มความสามารถของ Admin Web**

สามารถพัฒนาส่วนผู้ดูแลระบบให้มีรายงานสรุปการใช้งาน เช่น จำนวนผู้ใช้งาน จำนวนทริปที่สร้าง จุดหมายปลายทางที่ได้รับความนิยม และสถิติการใช้งาน เพื่อช่วยในการตรวจสอบและวิเคราะห์การใช้งานของระบบในภาพรวม

9. **ขยายการรองรับแพลตฟอร์มและอุปกรณ์**

ควรทดสอบและปรับปรุงระบบให้รองรับอุปกรณ์ ขนาดหน้าจอ และระบบปฏิบัติการที่หลากหลายมากขึ้น เพื่อให้ผู้ใช้งานสามารถเข้าถึง TripAI ได้อย่างสะดวกในหลายแพลตฟอร์ม

แนวทางการพัฒนาดังกล่าวสามารถนำไปใช้ต่อยอดระบบ TripAI ให้มีความสามารถครอบคลุมการวางแผนการเดินทางมากยิ่งขึ้น รวมถึงเพิ่มความแม่นยำของข้อมูล ลดข้อจำกัดจากบริการภายนอก และช่วยให้ระบบสามารถตอบสนองต่อความต้องการของผู้ใช้งานได้ดียิ่งขึ้นในอนาคต

บรรณานุกรม

กิตติโชค คุ้มครอง, อุบลรัตน์ ศิริสุขโภคา, และ ไพศาล สิมาเลาเต่า. (2568). การพัฒนาเว็บแอปพลิเคชันให้บริการแนะนำสถานที่ท่องเที่ยวด้วยปัญญาประดิษฐ์ร่วมกับระบบบริหารจัดการข้อมูลบนแผนที่ภูมิศาสตร์ กรณีศึกษาจังหวัดราชบุรี. *วารสารวิทยสารสนเทศและเทคโนโลยี, 6*(1), 43–55.

กมลลักษณ์ ชัยดี, วิวัฒน์ วงค์ไชย, ศุภกร ภมร, และ สุรัสวดี ปรีดี. (2559). การพัฒนาแอพพลิเคชันสำหรับสื่อการท่องเที่ยวบนแอนดรอยด์. *วารสารวิชาการปทุมวัน, 6*(16), 29–44.

ชโลธร ชูทอง, อรวรรณ อิ่มสมบัติ, และ จิรวรรณ เจริญสุข. (ม.ป.ป.). *บทที่ 1: บทนำสู่ NoSQL และ MongoDB* [เอกสารประกอบการอบรมหลักสูตร NoSQL with MongoDB]. มหาวิทยาลัยเกษตรศาสตร์.

ดวงชัย มณีโชติ. (2564, 17 มิถุนายน). *บทบาทของ AI ในการปฏิรูป “อุตสาหกรรมท่องเที่ยว”*. NBTC e-Library. https://lib.nbtc.go.th/news/detail/279

ภัทร สูตรสุวรรณ, เดือนเพ็ญ ธีรวรรณวิวัฒน์, และ พาชิตชนัต ศิริพานิช. (2564). การศึกษาระบบแนะนำสถานที่ท่องเที่ยวในกรุงเทพฯ และปริมณฑลสำหรับนักท่องเที่ยวต่างชาติ. *วารสารสมาคมนักวิจัย, 26*(3), 121–139.

สุมนา บุษบก, กิตติยา ปัญญาเยาว์, ศศิประภา บุญเลิศ, และ ปวันนพัสตร์ ศรีทรงเมือง. (2568). การพัฒนาแชทบอทปัญญาประดิษฐ์เพื่อแนะนำข้อมูลการท่องเที่ยวในจังหวัดภูเก็ต ประเทศไทย. *วารสารแอนิเมชัน เกม ดิจิทัลมีเดียและเทคโนโลยี, 2*(2), 68–85.

สุนิสา ทัศนะ, ไก้รุ่ง เฮงพระพรหม, สุวิมล มรรควิบูลย์ชัย, สุพจน์ เฮงพระพรหม, และ เดชธรรมศิริ. (2561). ระบบสารสนเทศสำหรับแนะนำสถานที่ท่องเที่ยวยอดนิยมในภูมิภาคตะวันตก. ใน *การประชุมวิชาการระดับชาติ ครั้งที่ 10 มหาวิทยาลัยราชภัฏนครปฐม* (น. 320–330). มหาวิทยาลัยราชภัฏนครปฐม.

สุวนีย์ กุลกรินีธรรม และ วีริยา สุภาณิชย์. (2567). แอปพลิเคชันแพลตฟอร์มแนะนำการท่องเที่ยวรายบุคคลแบบปรับตัว และการบริหารจัดการการท่องเที่ยวโดยชุมชน ตำบลตะเคียนเตี้ย อำเภอบางละมุง จังหวัดชลบุรี. *วารสารเทคโนโลยีสารสนเทศ, 20*(1), 86–97. https://doi.org/10.14416/j.it.2024.v1.008

อนุชิต ชโลธร. (ม.ป.ป.). *สูตรลัด Flutter: เขียน Flutter ง่ายๆ จากโค้ดตัวอย่าง* (พิมพ์ครั้งที่ 3). สำนักพิมพ์ก๊อปวาง.

Clemens, K. (2015). Geocoding with OpenStreetMap data. In *GEOProcessing 2015: The Seventh International Conference on Advanced Geographic Information Systems, Applications, and Services* (pp. 1–2). IARIA.

Google LLC. (2020, December 17). *System and Organization Controls (SOC) 3 report over the Google Firebase system: Relevant to security, availability, and confidentiality for the period 1 November 2019 to 31 October 2020*.

High School Technology Services. (n.d.). *Authentication with Firebase* [Presentation slides].

Kelly, R. (2026, April 2). *L10 – Map integration* [Lecture slides]. ISE369/POL369.

Kilic, B., Hacar, M., & Gülgen, F. (2023). Effects of reverse geocoding on OpenStreetMap tag quality assessment. *Transactions in GIS, 27*, 1599–1613. https://doi.org/10.1111/tgis.13089

Kim, J. (2025). *gemini.R: Interface for 'Google Gemini' API* (Version 0.17.2) [R package]. CRAN. https://github.com/jhk0530/gemini.R

Lahtinen, A. (2024). *Developing a cross-platform mobile application using Flutter* [Bachelor's degree project, Universitat Politècnica de València].

Li, Q., Wu, M., & Chen, D. (2025). PhotoGAN: A novel style transfer model for digital photographs. *Computers, Materials & Continua, 83*(3), 4477–4494. https://doi.org/10.32604/cmc.2025.062969

maxbox4. (2025, January 20). *Google Gemini API*. Code Blog. https://softwareschule.code.blog/2025/01/20/google-gemini-api/

Nadhifa, A. R. F., Eliskar, Y., & Fatoni, S. B. P. (2026). Implementation of Firebase as a cloud backend for Internet of Things based compost monitoring system on mobile application. *Journal of Physics: Conference Series, 3188*, 012018. https://doi.org/10.1088/1742-6596/3188/1/012018

Neu, W. A., Vlasceanu, V., Oram, A., & Alapati, S. (2019). *An introduction to cloud databases: A guide for administrators*. O'Reilly Media.

Office of Systems Engineering and Architecture, Office of the Under Secretary of War for Research and Engineering. (2026, March). *Application Programming Interface (API) technical guidance: Minimum Viable Capability Release (MVCR) 3*. https://www.cto.mil/sea

Pakistan Software Export Board. (n.d.). *Web and mobile applications*. Tech Destination Pakistan. https://techdestination.com

Paul, S. (2026). Unsplash FindAWall: Secure, scalable and responsive image-search platform. *International Journal of Technology & Emerging Research, 2*(5), 104–116. https://doi.org/10.64823/ijter.2605007

Siranjeevi, K., Vinoth Kumar, K., Pathmesh, G., & Ponnarasu, A. (2026). SketchMorph.ai: An AI-powered mobile application design agent using large language models, agentic workflows, and a real-time interactive canvas. *Journal of Advance and Future Research, 4*(4), 728–734.

Universal Service Administrative Co. (n.d.). *Geolocation methods: A guide to successfully collecting broadband deployment data*.

Yadav, S. C., & Singh, S. K. (2009). An introduction to client/server computing. New Age International (P) Limited.

Zhu, G., Ren, G., Li, X., Li, X., & Ti, Y. (2020). IP geolocation accuracy evaluation based on crowdsourcing. Journal of Physics: Conference Series, 1624, 032028. https://doi.org/10.1088/1742-6596/1624/3/032028